

Baze de Date – Oracle Database
Facultatea de Matematica si Informatica – An I
Curs Introductiv
Sistemul de Notare

EDUCATION
is not the learning of facts,
but the TRAINING of the
MIND TO THINK.
Albert Einstein

Lect. Univ. Dr. Cezara Benegui – cezara.benegui@fmi.unibuc.ro

Cursuri si Laboratoare - <https://www.cezarabenegui.com/>

Cuprins

Ce vom studia.....	3
Ce este necesar pentru a incepe cursul.....	3
Structura cursului	3
Sistemul de notare	4
Cerinte proiect.....	5
Criterii de promovabilitate	5
Criterii minime	5
Criterii complete	6
Deadline-uri	6
Conditii de promovare	7
Restanta / reexaminare/ marire de nota.....	8
Cum se obtine bonusul la laborator	8
Cum se obtin punctele pentru activitatea la curs	8

Ce vom studia

- SQL (Structured Query Language)
- Proiectarea bazelor de date
- Optimizarea bazelor de date
- Implementarea bazelor de date folosind ca limbaj de programare SQL
- Interogarea si prelucrarea bazelor de date

Ce este necesar pentru a incepe cursul

- **Laptop**
- **Oracle SQL Developer** – IDE (Integrated Development Environment)
- **Oracle Database** (RDBMS – Relational Database Management System)
- **Scriptul diagramei HR** (Human Resources)

Structura cursului

- Saptamanile 1-12 **predare** (pana in saptamana 15-19 Mai)
- Saptamanile 13-14 **evaluare** (saptamanile 22-26 Mai si 29.05-02.06)

Sistemul de notare

NOTA FINALA = $(N_{lab} + N_{examen}) / 2$, unde

- N_{lab} = nota obtinuta din **testul de laborator** (N_{test}) și **bonus** pentru activitatea de la laborator (***Bonus_lab***) conform formulei:

$$N_{lab} = \min (1 \text{ punct oficiu} + N_{test} + \textit{Bonus_lab}, 10)$$

unde

- N_{test} = maximum 9 puncte (la care se adauga punctul din oficiu)
- ***Bonus_lab*** = maximum 1 punct

- N_{examen} = nota obtinuta din proiect ($N_{proiect}$) si activitatea la curs ($N_{activitate_curs}$) conform formulei:

$$N_{examen} = 1 \text{ punct oficiu} + N_{1_proiect} + N_{2_proiect} + N_{activitate_curs}$$

unde:

- $N_{1_proiect}$ = maximum 4 puncte, obtinute din rezolvarea cerintelor 1-13 ale proiectului, indiferent de complexitate
- $N_{2_proiect}$ = maximum 3 puncte, obtinute din standardele de complexitate ale proiectului si raspunsurile din cadrul examinarii din sesiune
- $N_{activitate_curs}$ = maximum 2 puncte, obtinute din activitatile organizate in cadrul cursurilor (intrebări, exercitii interactive, quiz-uri, etc).

Cerinte proiect

VEZI → documentul Cerinte_Proiect

Criterii de promovabilitate

Criterii minime

- Obținerea notei 5 la testul de laborator ($\Rightarrow 1 \text{ punct oficiu} + N_{\text{test}} \geq 5$).
- Rezolvarea integrală a punctelor 1-13 din fisierul ce conține cerințele proiectului ($\Rightarrow 1 \text{ punct oficiu} + N_{1_proiect} = 5$).

! OBSERVATII:

- Obținerea notei 5 la testul de laborator este **condiție eliminatorie** pentru intrarea în examen. Nepromovarea testului de laborator **implică nepromovarea examenului**
- Îndeplinirea celor 2 criterii minime nu implică promovarea examenului, aceasta fiind decisă în urma susținerii examenului în sesiune
- Nu se acordă punctaj pentru complexitate ($N_{2_proiect} = 0$) dacă sunt rezolvate doar cerințele 1-13 (acestea fiind cerințele pentru criteriile minime)

Criterii complete

- Îndeplinirea obligatorie a criteriilor minimale

Doar dacă toate criteriile minimale sunt îndeplinite, atunci:

- Se vor putea rezolva suplimentar punctele 14-17 din cerințele proiectului (acestea contribuie la obținerea punctajului pentru complexitate)
- Se poate obține maximum 1 punct bonus din activitatea de laborator ($0 \leq \text{Bonus_lab} \leq 1$).
- Se pot obține maximum 2 puncte din activitatea de curs ($\Rightarrow 0 \leq N_{\text{activitate_curs}} \leq 2$).
- Se pot obține maximum 3 puncte în funcție de complexitatea proiectului și răspunsurile date la examinarea din cadrul sesiunii ($\Rightarrow 0 \leq N_{2_proiect} \leq 3$)

! OBSERVATII:

- Neîndeplinirea standardului minim implică nepromovarea examenului
- Complexitatea rezolvării cerințelor 1-13 este luată în considerare doar dacă sunt abordate și cerințele 14-17 (cel puțin una)
- Punctajul maxim la proiect este condiționat de complexitatea tratării tuturor cerințelor
- Nota finală va fi decisă în urma examinării din sesiune

Deadline-uri

- **Tema pentru proiect** se alege până pe **24.03.2023**, inclusiv
- **Testul de laborator** se susține în săptămâna 13 (Sap 22.05 – 26.05), în cadrul laboratorului (sau cursului – discutăm și alegem varianta potrivită)
- **Proiectul final** se trimite până pe data **26.05.2023**, inclusiv

- In saptamana 14, in cadrul laboratorului (30/31 Mai) o sa aiba loc validarea proiectelor, validare care include **verificarea integralitatii proiectului** si **verificarea partii de SQL** (asigurare ca tot codul ruleaza)
- Prezentarea integrala a proiectului o sa aiba loc in sesiune, fiind obligatorie. Data sustinerii examenului o sa fie aleasa impreuna

Conditii de promovare

Nepromovarea testului de laborator implica nepromovarea examenului.

Studentii ale caror proiecte nu vor fi validate in cadrul laboratorului, nu pot intra in examen, ducand la nepromovarea examenului.

Prezentarea proiectului la laborator si neprezentarea la examen implica nepromovarea examenului.

Fiecare student prezinta proiectul si sustine examenul in cadrul semigrupului din care face parte tot semestrul.

Este obligatorie respectarea conditiilor de etica si profesionalism ale facultatii. Temele trimise si proiectul trebuie sa fie originale, reprezentand rezultatul muncii studentului.

Restanta / reexaminare/ marire de nota

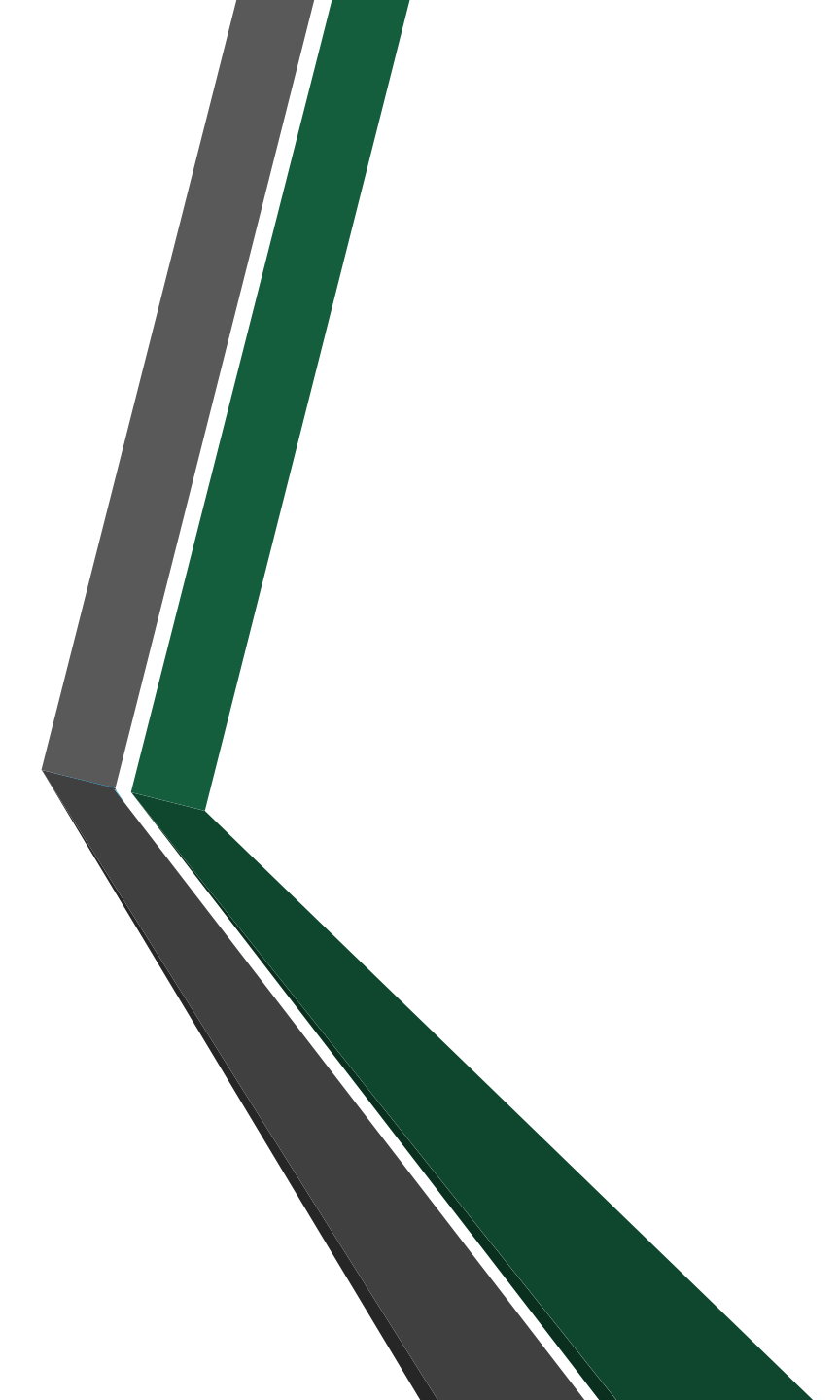
- Se aplica aceeasi modalitate de notare, cu modificarea datelor pentru testul de laborator, incarcarea proiectului si examinare
- Se pastreaza punctele pentru activitate obtinute pe parcursul semestrului

Cum se obtine bonusul la laborator

- **Bonus_lab** = maximum 1 punct
- Bonusul pentru activitatea la laborator se obtine pe baza raspunsurilor corecte date in cadrul laboratorului si a rezolvarilor de probleme propuse in timpul laboratorului
- Bonusul se adauga la nota obtinuta la testul de laborator doar daca aceasta, incluzand punctul din oficiu, este ≥ 5
- Bonusul suplimentar (care implică depasirea notei 10 la laborator) nu se transfera la curs

Cum se obtin punctele pentru activitatea la curs

- $N_{activitate_curs}$ = maximum 2 puncte – se obtin din prezenta activa (raspunsuri corecte date la curs, rezolvări de probleme propuse in timpul cursului, diverse exercitii interactive, teme de curs)



BAZE DE DATE

CURS 1

Cuprinsul cursului

1. Generalități despre baze de date (structură, evoluție, caracteristici, funcționalități, perspective etc.)
2. Proiectarea (*design*) bazelor de date (diagrame *E/R*, diagrame conceptuale, *UML*, modelul relațional, proiectare, optimizare, normalizare, denormalizare, prelucrarea și optimizarea cererilor, regulile lui Codd etc.)
3. Introducere în bazele de date nerelaționale
4. Limbaje pentru gestionarea datelor. Neprocedural în baze de date (standardul *SQL*)

Bibliografie

- Connolly, T.M., Begg, C.E., Database Systems: A Practical Approach to Design, Implementation and Management, 6th edition, Pearson Education, 2014
- Popescu I., Alecu A., Velcescu L., Florea (Mihai) G., Programare avansată în Oracle9i, Editura Tehnică, București, 2004.
- Popescu, I., Velcescu, L., ***Proiectarea bazelor de date***, Editura Universității din București, 2008
- Popescu, I., Velcescu, L., ***Neprocedural în Oracle 10g***, Editura Universității din București, 2008

Concepte de baza

- Bază de date
- Sistem de gestiune a bazelor de date
- Dicționarul datelor

Concepte de baza – Baze de Date

- Ce este o bază de date (BD) ?
- Exemple?
- Cui aparțin datele?
- Cine le poate accesa?

Concepte de baza – Baze de Date

- **BAZA DE DATE** = ansamblu **structurat de date coerente**, fără **redundanță inutilă**, astfel încât acestea pot fi prelucrate eficient de mai mulți utilizatori într-un **mod concurent**
- Colecție de date **persistente**, care sunt folosite de către sistemele de aplicații ale unei anumite „întreprinderi“

Concepte de baza – Baze de Date

- „Întreprindere“ (eng. *Enterprise*):
 - **Reguli** proprii de funcționare
 - **Mulțime de date** referitoare la modul său de operare
- Datele din BD:
 - **Integrate**
 - **Partajate**

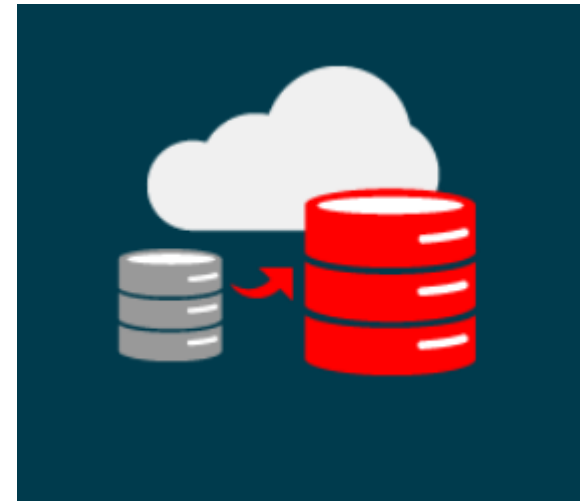
Concepte de baza - SGBD

- Ce este un Sistem de Gestiune a Bazelor de Date?
- Exemple?

Concepte de baza - SGBD

- **Sistem de Gestiune a Bazelor de Date (SGBD = DBMS – *Data Base Management System*)** este **un produs *software*** care asigură interacțiunea cu o bază de date
 - definirea
 - consultarea
 - actualizarea datelor din baza de date.
- Cererile de **acces la baza de date** sunt tratate și controlate de către **SGBD**.

Concepte de baza – BD si SGBD



<https://www.oracle.com/database/technologies/upgrades.html>

Concepte de baza - Dicționarul datelor

- **Dicționarul datelor** (catalog de sistem) = metabază de date
 - „date despre date“
 - structurat și administrat ca o bază de date
- **Ce informații regăsim în DD?**
- **Cum obținem aceste informații?**

Concepte de baza - Dicționarul datelor

- DD conține:
 - **descrierea** tuturor obiectelor unei baze de date
 - **starea** acestor obiecte
 - diversele **constrângeri** de securitate și de integritate
- DD poate fi interogat ca orice altă bază de date

Concepte de baza - DBA

- **Administratorul bazei de date** (DBA – *Data Base Administrator*) = persoană sau un grup de persoane ce răspund de ansamblul activităților legate de baza de date
 - analiză
 - proiectare
 - implementare
 - exploatare
 - întreținere etc.

Concepte de baza - DBA

- Categori de atribuții DBA:
- atribuții de proiectare
 - atribuții administrative
 - atribuții operative
 - atribuții de coordonare

Gestiunea Bazelor de Date

Arhitectura unui sistem de baze de date are urmatoarele componente:

1. baza de date propriu-zisă în care se memorează datele
2. sistemul de gestiune a bazei de date, care realizează gestionarea și prelucrarea complexă a datelor
3. un dicționar al bazei de date (metabaza de date), ce conține informații despre date, structura acestora, statistici, documentație
4. mijloace *hardware* (comune sau specializate)
5. reglementări administrative destinate bunei funcționări a sistemului
6. personalul implicat (utilizatori finali, administratorul datelor, administratorul bazei de date, proiectanți / programatori de aplicații) → **4 categorii de utilizatori**

Categorii de utilizatori

- **Administratorul de date (DA)** este un manager care:
 - decide ce date trebuie stocate în baza de date
 - stabilește regulile de întreținere și de tratare a acestor date după ce sunt stocate
- **Administratorul bazei de date (DBA)** este un profesionist în domeniul IT, care:
 - creează baza de date reală
 - implementează elementele tehnice de control
 - este responsabil cu asigurarea funcționării sistemului la performanțe adecvate, cu monitorizarea performanțelor
 - furnizează diverse servicii tehnice

Categorii de utilizatori

- **Proiectanții de baze de date** pot acoperi 2 aspecte ale concepției (*design*-ului):
 - Proiectare fizică
 - Proiectare logică
- **Utilizatorii finali**

GESTIUNEA BAZELOR DE DATE – Proiectarea logica

- **Proiectarea logică** presupune o cunoaștere completă și amănunțită:
 - a **modelului real** de proiectat
 - a **regulilor de funcționare** ale acestuia
- Proiectarea **conceptuală** a bazei de date
- Modelul creat este **independent** de programele de aplicații, de limbajele de programare
- Proiectarea logică a bazei de date, îndreptată spre un anumit **model de date** (relațional, orientat obiect, ierarhic etc.)

GESTIUNEA BAZELOR DE DATE – Proiectarea fizica

- **Proiectarea fizică:**
 - preia modelul logic de date
 - stabilește cum va fi realizat fizic
 - presupune cunoașterea funcționalităților SGBD-ului, avantajele și dezavantajele fiecărei alternative
- Transpunerea modelului logic într-un set de **tabele** supuse unor **constrângeri**. Se selectează **structuri de stocare** și **metode de acces** specifice, astfel încât să se asigure **performanțe**. Se iau măsuri privind **securitatea** datelor

Cerinte minimale BD

Cerintele minimale ale unei baze de date sunt:

- **redundanță minimă** în date
- furnizarea în **timp util** a informațiilor solicitate (timpul de răspuns la o interogare)
- asigurarea unor **costuri minime** în prelucrarea și întreținerea informației
- capacitatea de a satisface, cu aceleași date, **necesități informaționale** ale unui număr mare de utilizatori
- posibilitatea de adaptare la cerințe noi, răspunsuri la interogări neprevăzute inițial (**flexibilitate**)
- exploatarea simultană a datelor de către mai mulți utilizatori (**sincronizare**)

Cerinte minimale BD

(continuare)

- asigurarea **securității datelor** prin mecanisme de protecție împotriva accesului neautorizat (confidențialitate)
- înglobarea unor facilități destinate validării datelor și recuperării lor în cazul unor deteriorări accidentale, garantarea (atât cât este posibil) că datele din baza de date sunt corecte (**integritate**)
- posibilitatea de valorificare a eforturilor anterioare și anticiparea nevoilor viitoare (**compatibilitate** și **expandabilitate**)
- permisivitatea, prin ierarhizarea datelor după criteriul frecvenței acceselor, a unor reorganizări (eventual dinamice) care sporesc **performanțele** bazei

Niveluri de abstractizare și de percepție a datelor

Sunt 4 niveluri de abstractizare și de percepție a datelor:

- intern (fizic)
- conceptual
- logic
- extern

Datele există doar la nivel **fizic**, iar celelalte trei niveluri reprezintă **virtualizări** ale acestora.

Niveluri de abstractizare și de percepție a datelor

- **Nivelul fizic (intern)** → schema fizică a datelor (bit, octet, adresă);
- **Nivelul conceptual** → schema conceptuală a datelor și reprezintă **viziunea programatorilor de sistem** asupra datelor;
- **Nivelul logic** → una din schemele logice posibile ale datelor și reprezintă **viziunea programatorului de aplicație** asupra datelor;
- **Nivelul virtual (extern)** → reprezintă **viziunea utilizatorului final** asupra datelor;

Independenta datelor

Independența datelor → două aspecte fundamentale:

- o modificare a structurii fizice nu va afecta aplicația
- reciproc, modificări ale aplicației vor lăsa nealterată structura fizică de date

Exista 2 tipuri principale de independență a datelor:

- Independența fizică
- Independența logică

Acestora li se adaugă independența față de strategiile de acces

Independenta datelor

- **Independența fizică** → posibilitatea **modificării schemei fizice** a datelor fără ca aceasta să implice modificarea schemei conceptuale, a schemei logice și a programelor de aplicație
- **Independența logică** → posibilitatea **modificării schemei conceptuale** a datelor fără ca aceasta să implice modificarea schemei logice și a programelor de aplicație
 - Independența logică $\leftrightarrow$ fiecare utilizator are iluzia că este singurul beneficiar al unor date pe care, în realitate, le folosește în comun cu alți utilizatori
- **Independența față de strategiile de acces** → permite programului să **precizeze data pe care dorește să o acceseze**, dar nu modul cum accesează această dată

SGBD-ul va stabili drumul optim de acces la date

Limbaje pentru baze de date

Limbaje pentru baze de date

- **Limbaje pentru definirea datelor** (LDD – *Data Description Language*)
- **Limbaje pentru prelucrarea datelor** (LMD – *Data Manipulation Language*)
- **Limbaje pentru controlul datelor** (LCD – *Data Control Language*)

Limbaje pentru baze de date

Limbaje pentru definirea datelor (LDD – *Data Description Language*) permit:

- definirea **entităților** și a **atributelor** acestora
- precizarea **relațiilor** dintre date și strategiilor de acces la ele
- stabilirea de criterii diferențiate de **confidențialitate** și de validare automată a datelor utilizate

Limbaje pentru baze de date

Limbaje pentru prelucrarea datelor (LMD – *Data Manipulation Language*) – permit adaugarea, modificarea sau stergerea datelor

- o comandă are următoarea structură: operația, criterii de selecție, mod de acces (secvențial, indexat etc.)

Există limbaje LMD:

- **procedurale** – care specifică **cum** se obține rezultatul unei comenzi LMD
- **neprocedurale** – care descriu doar datele **ce** vor fi obținute și nu modalitatea de obținere a acestora

Limbaje pentru baze de date

Limbaje pentru controlul datelor (LCD – *Data Control Language*) – permit:

- asigurarea **confidențialității** și **integrității** datelor
- salvarea informației în cazul unor **defecțiuni**
- rezolvarea unor probleme de **concurență**

Obiectivele unui SGBD

Un **SGBD** (Sistem de Gestiune a Bazelor de Date) are următoarele **obiective fundamentale**:

- **Independența fizică** – independența structurilor de stocare în raport cu structurile de date din lumea reală
- **Independența logică** – fiecare grup de lucru poate să cunoască doar o parte a semanticii datelor, să vadă doar o submulțime a datelor și numai sub forma în care le dorește
- **Prelucrarea datelor de către neinformaticieni**
- **Administrarea centralizată a datelor**
- **Coerența datelor** – informația trebuie să satisfacă constrângeri statice sau dinamice, locale sau generale

Obiectivele unui SGBD

(continuare)

- **Neredundanța datelor**
 - Administrarea coerentă a datelor trebuie să asigure neduplicarea fizică a datelor
 - **Pentru a realiza performanțe referitoare la timpul de acces la date și răspuns la solicitările utilizatorilor, se acceptă o anumită redundanță a datelor**
- **Partajabilitatea datelor**
 - Aplicațiile pot să partajeze datele din baza de date în timp și simultan
 - O aplicație poate folosi date ca și cum ar fi singura care le utilizează, fără a ști că altă aplicație, concurent, le poate modifica

Obiectivele unui SGBD

Securitatea și confidențialitatea datelor

- Datele trebuie protejate de un acces neautorizat sau rău intenționat
 - există mecanisme care permit identificarea și **autentificarea** utilizatorilor
 - există proceduri de **acces autorizat** care depind de date și de utilizator
- Sistemul de gestiune trebuie să asigure securitatea fizică și logică a informației și să garanteze că **numai utilizatorii autorizați** pot efectua operații corecte asupra bazei de date

Dezavantajele SGBD-urilor

Un sistem de gestiune a bazelor de date are o **serie de limitări / dezavantaje** inerente:

- **complexitatea și dimensiunea sistemelor** pot să crească considerabil, din cauza necesității extinderii funcționalităților sistemului
- **costul**, care variază în funcție de mediu și funcționalitatea oferită, la care se adaugă cheltuieli periodice de întreținere
- **costuri adiționale** pentru elemente de *hardware*
- **costul conversiei** aplicațiilor existente, necesară pentru ca acestea să poată funcționa în noua configurație *hardware* și *software*
- **impactul unei defecțiuni** asupra aplicațiilor, bazei de date sau sistemului de gestiune

Structura unui SGBD

Structura unui Sistem de Gestiune a Bazelor de Date:

- are complexitate variabilă
- nivelul real de funcționalitate diferă de la produs la produs
- conține cel puțin 5 clase de module

Structura unui SGBD

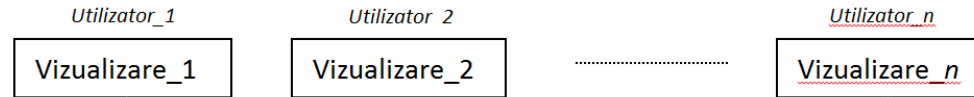
Module SGBD:

- **programe de gestiune a bazei de date (PGBD)** care realizează accesul fizic la date ca urmare a unei comenzi
- **module pentru tratarea LDD** ce permit traducerea unor informații în obiecte ce pot fi apoi exploatare în manieră procedurală sau neprocedurală
- **module pentru tratarea LMD** care permit utilizatorilor inserarea, ștergerea, reactualizarea sau consultarea informației dintr-o bază de date
- **module utilitare** care asigură întreținerea, prelucrarea, exploatarea corectă și ușoară a bazei de date
- **module de control** care permit controlul programelor de aplicație, asigurarea confidențialității și integrității datelor, rezolvarea unor probleme de concurență, recuperarea informației în cazul unor avarii sau defecțiuni *hardware* sau *software*

Independența logică și fizică

Nivelul extern

(modul în care
utilizatorii percep
datele)



Nivelul conceptual

(realizează transpunerea și
independența dintre
nivelul extern și cel intern)

Schema conceptuală

CE date sunt stocate și relațiile
dintre ele

Nivelul intern

(nivelul la care SGBD și SO percep
datele)

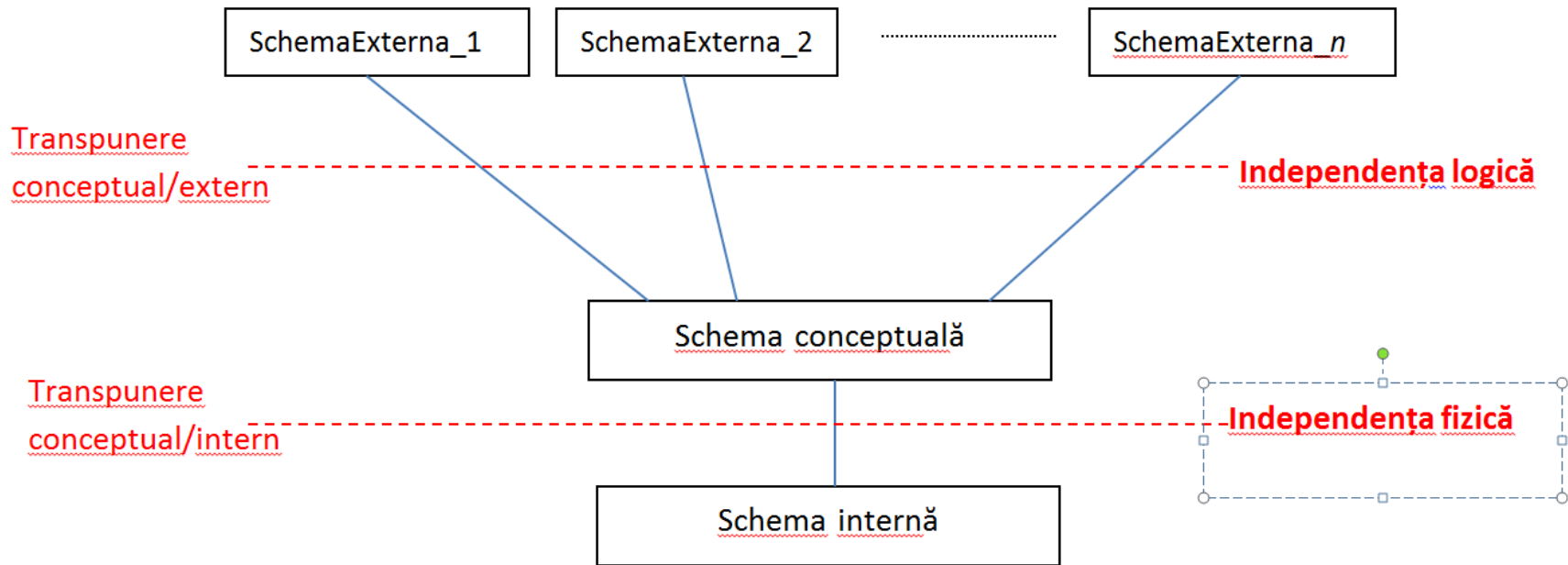
Schema internă

CUM sunt stocate datele

Organizarea fizică a datelor



Independența logică și fizică



Independența logică – imunitatea schemelor externe față de modificările efectuate în schema conceptuală

Independența fizică – imunitatea schemei conceptuale față de modificările efectuate în schema internă

Corespondența extern-extern – permite definirea unei vizualizări în funcție de altele, fără a necesita o definiție explicită a corespondenței cu nivelul conceptual

Arhitectura sistemelor de gestiune a bazelor de date

Niveluri ale arhitecturii sistemelor de gestiune a bazelor de date

Independența fizică și logică a datelor => adoptarea unei arhitecturi de baze de date organizată pe trei niveluri:

- **nivelul intern** (baza de date fizică);
- **nivelul conceptual** (modelul conceptual, schema conceptuală);
- **nivelul extern** (modelul extern, subschema, vizualizarea).

Nivelul conceptual al arhitecturii SGBD

Nivelul central -> nivelul conceptual

- structura canonică a datelor ce caracterizează procesul de modelat (structura semantică a datelor fără implementarea pe calculator)

Schema conceptuală permite:

- definirea tipurilor de date ce caracterizează proprietățile elementare ale entităților
- definirea tipurilor de date compuse care permit regruparea atributelor pentru a descrie entitățile modelului și legăturile între aceste entități
- definirea regulilor pe care trebuie să le respecte datele

Nivelul intern al arhitecturii SGBD

Nivelul intern = structura internă de stocare a datelor

Schema internă permite:

- descrierea datelor unei baze sub forma în care sunt stocate în memoria calculatorului

La **nivel conceptual sau intern**, schemele descriu o bază de date

La **nivel extern** schemele descriu doar o parte din date care prezintă interes pentru un utilizator sau un grup de utilizatori

Nivelul extern al arhitecturii SGBD

Nivel extern

- **Schema externă** reprezintă o descriere a unei părți a bazei de date ce corespunde viziunii unui program sau unui utilizator
- Modelul extern folosit este dependent de limbajul utilizat pentru prelucrarea bazei de date
- Schema externă permite **asigurarea unei securități** a datelor
 - Un grup de lucru va accesa doar datele descrise în schema sa externă, iar restul datelor sunt protejate împotriva accesului neautorizat sau rău intenționat

Pentru o bază de date particulară există o singură schemă internă, o singură schemă conceptuală, dar există mai multe scheme externe

Coreșpondențe între niveluri

- **coreșpondența conceptual-intern** → modul în care înregistrările și câmpurile conceptuale sunt reprezentate la nivel intern
- **coreșpondența extern-conceptual** → cheia independenței logice de date
- **coreșpondența extern-extern** → permite definirea unor vizualizări externe în funcție de altele, fără a necesita o definiție explicită a coreșpondenței cu nivelul conceptual

Arhitectura sistemelor de gestiune a bazelor de date

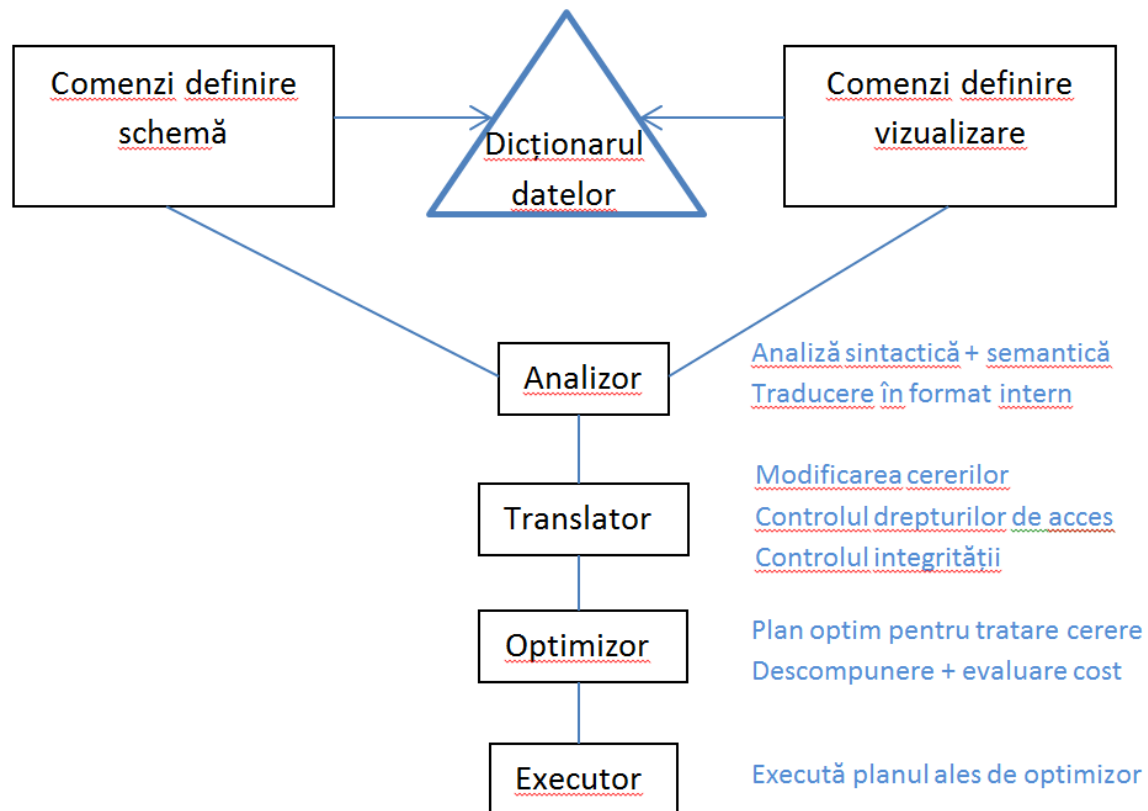
Arhitectura funcțională de referință propusă de grupul de lucru **ANSI/X3/SPARC** este axată pe dicționarul datelor și cuprinde două părți:

- prima, permite **descrierea datelor** (compoziția dicționarului datelor)
- a doua, permite **prelucrarea datelor** (interogarea și reactualizarea bazei de date)

Arhitectura sistemelor de gestiune a bazelor de date

- Gardarin a propus o arhitectură funcțională apropiată de arhitectura sistemelor de gestiune actuale.
- Arhitectura are la bază doar două niveluri:
 - **Schema** – care corespunde integrării schemelor interne și conceptuale
 - **Vizualizarea** – care este o schemă externă

Arhitectura sistemelor de gestiune a bazelor de date



Evolutia bazelor de date

Istoria BD și SGBD poate fi rezumată în **trei generații**:

- sisteme ierarhice și rețea
- sisteme relaționale
- sisteme avansate (orientate obiect, relaționale orientate obiect, deductive, multimedia, multibaze, active, temporale, distribuite, decizionale, magazine de date etc.)

Evolutia bazelor de date

Baze de date ierarhice și rețea

- datele sunt reprezentate la nivel de articol prin legături ierarhice (arbore) sau de tip graf
- slaba independență fizică a datelor complică administrarea și prelucrarea acestora
- limbajul de prelucrare a datelor impune programatorului să specifice drumurile de acces la date

Evolutia bazelor de date

Baze de date relaționale

- Modelul relațional (1970) tratează entitățile ca niște relații. Piața actuală de baze de date este acoperită în majoritate de sisteme **relaționale**.
- Bazele de date relaționale sunt caracterizate de:
 - structuri de date simple, intuitive
 - inexistența pointerilor vizibili pentru utilizator
 - constrângeri de integritate
 - operatori aplicați relațiilor care permit definirea, căutarea și reactualizarea datelor

Evolutia bazelor de date

- Bazele de date relaționale nu folosesc obiecte complexe și dinamice
- Nu realizează gestiunea datelor distribuite
- Nu realizează gestiunea cunoștințelor

A treia generație de SGBD-uri, **sistemele avansate**, încearcă să depășească aceste limite ale sistemului relațional

Evolutia bazelor de date

Baze de date orientate obiect

Ce nu realizează BD relaționale?

- Suportul obiectelor complexe și dinamice, prelucrarea acestora
- Sistemele relaționale nu modelează obiecte complexe ca grafuri, liste etc.
 - Un obiect complex poate să fie descompus în relații, dar apar dificultăți atât la descompunerea, cât și la refacerea acestuia prin compunere
- Un sistem relațional nu suportă obiecte dinamice care încorporează atât partea de date (informații) efective, cât și o parte relativă la tratarea acestora.

Îmbinarea tehnicii limbajelor orientate obiect cu a bazelor de date a permis realizarea bazelor de date orientate obiect

Evolutia bazelor de date

Baze de date orientate obiect (continuare)

Avantaje SGBDOO:

- realizează o modelare superioară a informației
- furnizează posibilități superioare de deducție (ierarhie de clase, moștenire)
- permit luarea în considerare a aspectelor dinamice și integrarea descrierii structurale și comportamentale

Dezavantaje:

- absența unui SGBDOO de referință
- gestiunea obiectelor complexe este mai dificilă
- utilizatorii au investit sume uriașe în sistemele relaționale și nu le pot abandona cu ușurință. Trecerea la tehnologia orientată obiect implică investiții mari și nu păstrează aproape nimic din vechile soluții

Evolutia bazelor de date

Baze de date relaționale orientate obiect

Simplitatea modelului relațional + puterea tehnologiei orientate obiect => baze de date relaționale orientate obiect

Construcția unui sistem de gestiune de baze de date relaționale orientate obiect (SGBDROO) trebuie să pornească de la cele existente. Aceasta se poate realiza în două moduri:

- dezvoltând un sistem relațional prin adăugarea caracteristicilor obiectuale necesare
- pornind de la un sistem orientat obiect și adăugând caracteristicile relaționale

Evolutia bazelor de date

Baze de date deductive

- O relație este o mulțime de înregistrări ce reprezintă **fapte**
- Cunoștințele sunt **asertiuni** generale și abstracte asupra faptelor
 - Cunoștințele permit să raționezi, ceea ce permite deducerea de noi fapte, plecând de la fapte cunoscute
- Un SGBD relațional suportă o formă limitată de cunoștințe, și anume constrângerile de integritate, iar restul trebuie integrate în programele de aplicație
- Prin programarea logică se gestionează cunoștințe relativ la baze de date care, în general, sunt relaționale
- Deducerea de noi informații, plecând de la informațiile stocate în baza de date

Evolutia bazelor de date

Baze de date deductive (continuare)

Un SGBD deductiv posedă:

- un **limbaj de definire a datelor** care permite definirea structurii predicatelor sub formă de relații și constrângeri de integritate asociate
- un **limbaj de prelucrare a datelor** care permite efectuarea reactualizărilor asupra datelor și formularea unor cereri
- un **limbaj de reguli de deducție** care permite ca, plecând cu predicatele definite anterior, să se specifice cum pot fi construite predicate derivate

Evolutia bazelor de date

Baze de date distribuite

- Sistem distribuit = ansamblu de mașini ce sunt interconectate printr-o rețea de comunicație și utilizate într-un scop global
- **Obiectiv BDD: Administrarea și prelucrarea datelor distribuite**, situate pe diferite calculatoare și exploatate de sisteme eterogene
- Bazele de date distribuite sunt sisteme de baze de date cooperante care rezidă pe mașini diferite, în locuri diferite
- Această mulțime de baze de date este **exploatată de utilizator ca și cum ar fi o singură bază de date**
- Programul de aplicație care exploatează o bază de date distribuită poate avea acces la date rezidente pe mai multe mașini, fără ca programatorul să cunoască localizarea datelor

Modelul **relațional** a rămas instrumentul principal prin care se realizează prelucrarea datelor distribuite.

Evolutia bazelor de date

Baze de date cu suport decizional

- Sistemele informatice, în particular bazele de date, au ajuns la maturitate
- Cantitate mare de informații ale companiilor, păstrată în tabele istorice
 - nefolositoare sistemelor operaționale ale companiei, care funcționează cu date curente
- Analiza date istorice → informații despre tendințe și evoluții care ar putea interesa compania
 - Sunt necesare tehnologii și instrumente speciale
 - Sunt analizate colecții de date provenind din sistemele operaționale ale companiei sau din surse externe
- Principalul scop al acestor baze de date a fost de a întâmpina nevoile sistemelor operaționale, a căror natură este inherent tranzacțională

Evolutia bazelor de date

Baze de date cu suport decizional (continuare)

- Sistemele tranzacționale sunt interesate, în primul rând, să controleze la un moment dat o singură tranzacție
- Un sistem operațional tipic operează cu evenimente predefinite și necesită acces rapid la date
- Nevoile sistemelor operaționale nu se schimbă mult în timp

Evolutia bazelor de date

Baze de date cu suport decizional (continuare)

- S-au pus la punct principii și tehnologii noi care să servească procesului de **analiză** și **administrare** a datelor
- O bază de date optimizată în acest scop definește o **Data Warehouse** (magazie de date)
- Principiul pe care îl urmează este cunoscut sub numele de procesare analitică (**OLAP – On Line Analytical Processing**)
- Principiul pe care se bazează sistemele tranzacționale a fost numit procesare tranzacțională (**OLTP – On Line Transactional Processing**)

Evolutia bazelor de date

Baze de date cu suport decizional (continuare)

- Aplicațiile unei *Data Warehouse* trebuie să ofere răspunsuri unor întrebări de tipul:
 - „Care zi din săptămână este cea mai aglomerată?”
 - „Ce clienți fideli nu au beneficiat de reduceri de prețuri?”
- Interogările pe BD analitice sunt *ad-hoc*, nu sunt predefinite → baza de date trebuie **optimizată** astfel încât să fie capabilă **să răspundă la orice fel de întrebare** care poate implica mai multe tabele

Evolutia bazelor de date

OLTP *versus* Data Warehouse

Sistemele OLTP	Data Warehouse
Păstrează date curente	Păstrează date istorice
Stochează date detaliate	Stochează date detaliate, agregate ușor sau puternic
Datele sunt dinamice	Datele sunt în mare măsură statice
Prelucrare repetitivă	Prelucrare ad-hoc, nestructurată și euristică
Nivel înalt de transfer al tranzacțiilor	Nivel mediu sau scăzut de transfer al tranzacțiilor
Tipar de utilizare previzibil	Tipar de utilizare imprevizibil
Conduse prin tranzacții	Conduse prin analiză
Susțin deciziile de zi cu zi	Susțin deciziile strategice
Deservesc un număr mare de utilizatori	Deservesc un număr relativ redus de utilizatori din administrație
Orientate spre aplicații	Orientate spre subiect

Evolutia bazelor de date

Baze de date cu suport decizional (continuare)

Marii producători de sisteme de gestiune a bazelor de date relaționale, precum *Oracle*, au introdus în produsele lor construcții care să faciliteze accesul la datele din sistemele fundamentale pentru luarea de decizii

- modalitate mai inteligentă de a realiza **operația de compunere** între două sau mai multe tabele
- metode de **indexare** noi, potrivite pentru marile cantități de date statice cu care operează sistemele *Data Warehouse*
- capacitatea de a detecta și **optimiza** interogări de un tip special
- posibilitatea de a folosi **mai multe procesoare** pentru a rezolva o interogare

Evolutia bazelor de date

Baze de date cu suport decizional (continuare)

- Un efort ce trebuie făcut pentru construirea unui sistem de suport pentru decizii (**DSS – *Decision Support System***) constă în procesul de descoperire a informațiilor utile din baza de date
- Acest proces, numit ***Data Mining*** sau ***Knowledge Discovery in Databases (KDD)***, procesează mari cantități de date, a căror corelare nu este neapărat evidentă, în vederea descoperirii de tendințe și tipare

Arhitectura *multitier* a sistemului *Oracle*

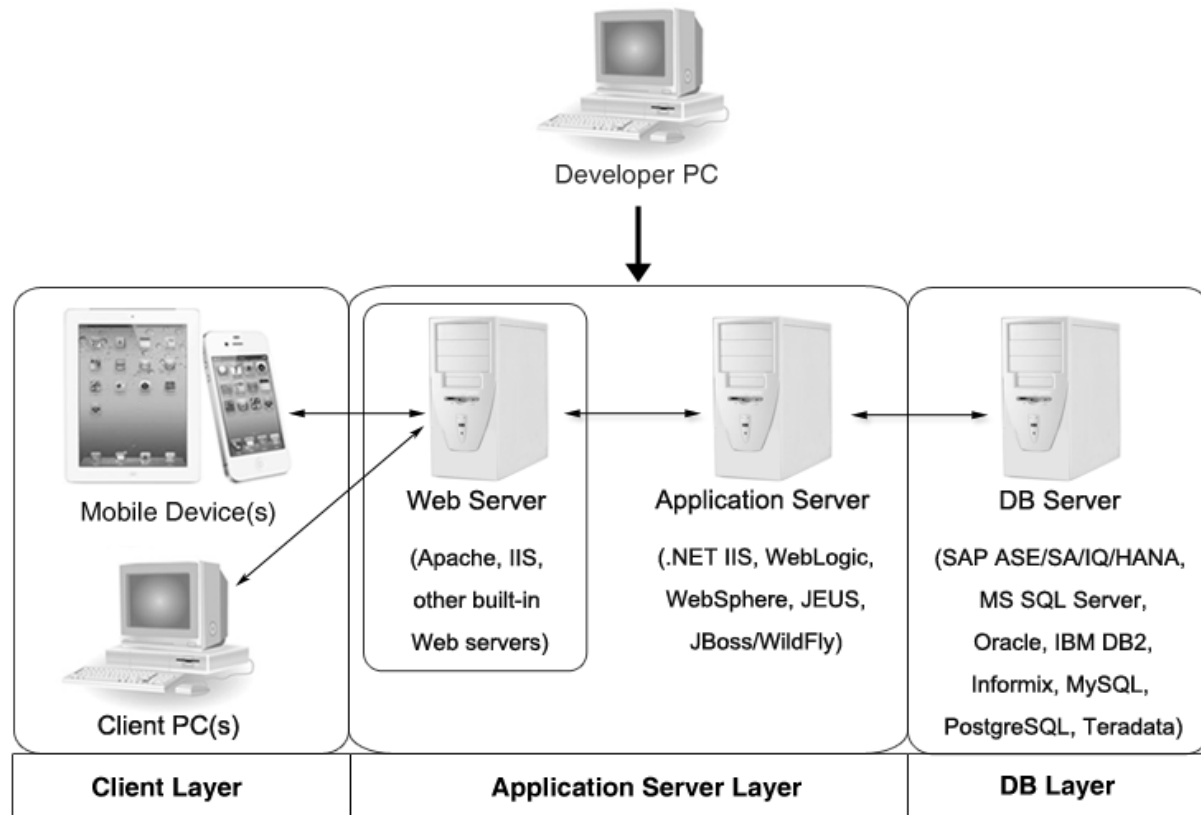
Arhitectura cu mai multe niveluri (*multitier*) conține următoarele elemente:

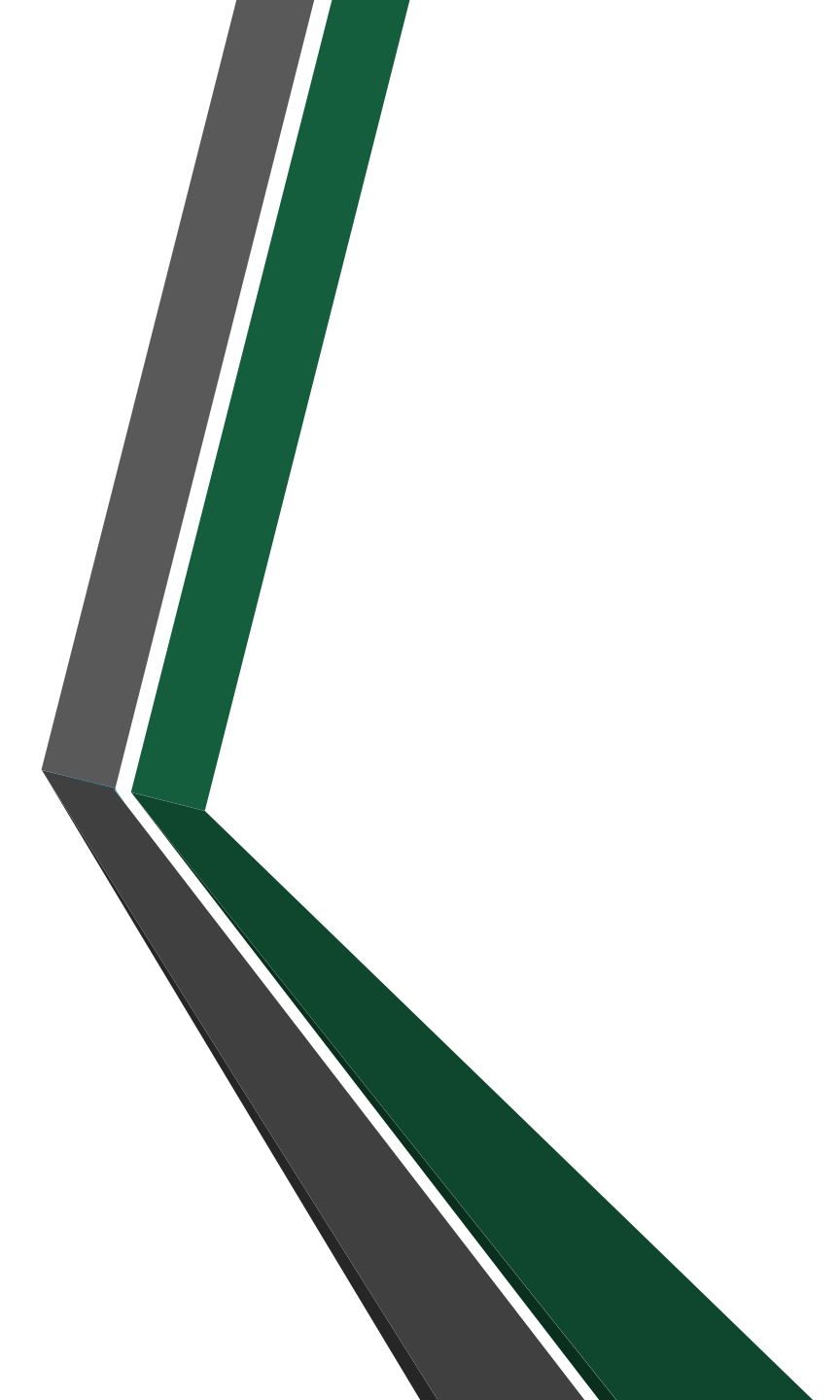
- unul sau mai mulți *clienti* care inițiază operații
- unul sau mai multe *servere* de aplicații care execută părți ale operațiilor
- un *server* de baze de date care stochează datele folosite de operații
- privilegiile *server*-ului de aplicații sunt limitate pentru a preveni execuția operațiilor nedorite sau inutile în timpul unei operații *client*

Arhitectura *multitier* a sistemului *Oracle*

- **Clientul** (un browser Web sau un proces user) → cerere pentru baza de date
 - Conectarea la server-ul bazei de date se face printr-unul sau mai multe servere de aplicații
- Server-ul de aplicații constituie interfața dintre clienți și serverul bazei de date, asigurând accesul la informații. De asemenea, el include un nivel adițional pentru securitate
- Server-ul de baze de date pune la dispoziția server-ului de aplicații informațiile necesare pentru soluționarea operațiilor lansate de către client

Arhitectura *multitier* a sistemului *Oracle*





BAZE DE DATE

CURS 2

Partea I

PROIECTAREA BAZELOR DE DATE RELAȚIONALE

- Modelarea entitate-relație (E/R)
- Diagrame entitate-relație
- Modelul relațional

MODELAREA ENTITATE-RELAȚIE

- Ce înțelegem prin model?
- Model vs. Implementare?
 - Ce trebuie să cunoască utilizatorii?

MODELAREA ENTITATE-RELAȚIE

- **Model** = reprezentare a **obiectelor** și **evenimentelor** lumii reale și a **asocierilor** dintre ele
 - abstractizare asupra aspectelor semnificative ale unei „întreprinderi“, ale unui sistem real
- **Model vs. Implementare?**
 - Caz particular al deosebirii uzuale dintre logic și fizic

MODELAREA ENTITATE-RELAȚIE

3 tipuri fundamentale de modele, care descriu aspecte:

- Statice
 - Dinamice
 - Funcționale
- } ale procesului de **modelat**

MODELAREA ENTITATE-RELAȚIE

Ce este un model de date?

MODELAREA ENTITATE-RELAȚIE

- **Model de date** = o colecție integrată de **concepte** necesare descrierii:
 - **datelor**
 - **relațiilor** dintre ele
 - **constrângerilor** existente asupra datelor sistemului real analizat

MODELAREA ENTITATE-RELAȚIE

Ce reprezintă modelarea unei baze de date?

MODELAREA ENTITATE-RELAȚIE

- **Modelarea unei baze de date** → trecerea de la **percepția** unor fapte din lumea reală la reprezentarea lor prin **date**
- **Modelul de date** trebuie:
 - să **reflecte** fidel fenomene ale lumii reale
 - să **urmărească evoluția** acestei lumi
 - să permită **comunicarea** dintre fenomenele lumii reale

MODELAREA ENTITATE-RELAȚIE

Modelul de date → 3 componente :

- o mulțime de **reguli** conform cărora sunt construite bazele de date (**partea structurală**);
- o mulțime de **operații** permise asupra datelor, care sunt utilizate pentru **reactualizarea** sau **regăsirea** datelor (**partea de prelucrare**);
- o mulțime de **reguli de integritate**, care asigură **coerența** datelor.

MODELAREA ENTITATE-RELAȚIE

Modelarea semantică a datelor → 4 etape:

1. Se identifică o **mulțime de concepte semantice** care sunt utile în descrierea lumii reale.

Se presupune că lumea reală (modelul real analizat) este formată din **entități** care au anumite **proprietăți**, că fiecare entitate are o **identitate**, că există **legături**, corelații între entități. Conceptul de corelație, ca și cel de entitate, este util, în mod intuitiv, la descrierea modelului.

2. Se caută o **mulțime de obiecte formale**, simbolice care sunt utilizate pentru reprezentarea conceptelor semantice anterioare.

MODELAREA ENTITATE-RELAȚIE

3. Se dau **reguli de integritate formale și generale** (constrângeri) care să reflecte restricțiile la care este supus modelul
4. Se definește o **mulțime de operatori formali** prin care pot fi prelucrate și analizate obiectele formale

MODELUL ENTITATE-RELAȚIE

- P. Chen, 1976
- **abordare a modelării semantice**
- model de date conceptual, pentru a ușura proiectarea bazelor de date
- de nivel înalt, independent de platforma *hardware* utilizată și de tipul SGBD-ului
- reprezentat grafic prin **Diagrame E/R**

MODELUL ENTITATE-RELAȚIE

Baza de date → mulțime de date ce **modelează** un sistem real format din:

- Obiecte
- Legături între ele

=> **Modelul E/R** împarte elementele unui sistem real în două categorii:

- **entități**
- **relații** (legături, asocieri) între aceste entități

Entitățile și legăturile → **caracteristici** (atribute)

MODELUL ENTITATE-RELAȚIE

- Conceptul de *relație, în sensul de asociere*, care intervine în definirea diagramei E/R

!=

- conceptul de relație care este specific *modelului relațional*

DIAGrame ENTITATE-RELATIE

- Diagrama E/R – model neformalizat pentru reprezentarea unui sistem din lumea reală
- **Entitate** = persoană, loc, concept, activitate, eveniment care este semnificativ pentru ceea ce modelăm

DIAGrame ENTITATE-RELATIE

- **Entitățile** devin **tabele** în modelele relaționale
- În general, entitățile se scriu cu litere mari
- Entitățile sunt **substantive**, dar nu orice substantiv este o entitate
- Pentru fiecare entitate este obligatoriu să se dea o descriere detaliată
- Nu pot exista, în aceeași diagramă, două entități cu același nume, sau aceeași entitate cu nume diferite

DIAGrame ENTITATE-RELATIE

- **Cheia primară** este un **identificator** unic în cadrul entității, făcând distincție între valori diferite ale acesteia.
- Cheia primară:
 - trebuie să fie **unică** și **cunoscută** la orice moment;
 - trebuie să fie controlată de administratorul bazei;
 - trebuie să nu conțină informații descriptive, să fie **simplă**, fără ambiguități;
 - să fie stabilă;
 - să fie familiară utilizatorului;

DIAGrame ENTITATE-RELATIE

Exemplu:

Se dau urmatoarele specificatii:

- In cadrul unui departament lucreaza mai multi angajati, dar un angajat poate sa lucreze intr-un singur departament
- Un angajat conduce un departament, iar departamentul poate avea un singur manager
- Un angajat poate sa lucreze (sa fie atasat) la mai multe proiecte in acelasi timp
- La dezvoltarea unui proiect pot participa mai multi angajati din companie
- Un proiect are alocate sarcini

Sa se identifice entitatile, plecand de la definitia entitatii existenta in cadrul slide-urilor anterioare.

DIAGrame ENTITATE-RELATIE

Exemplu:

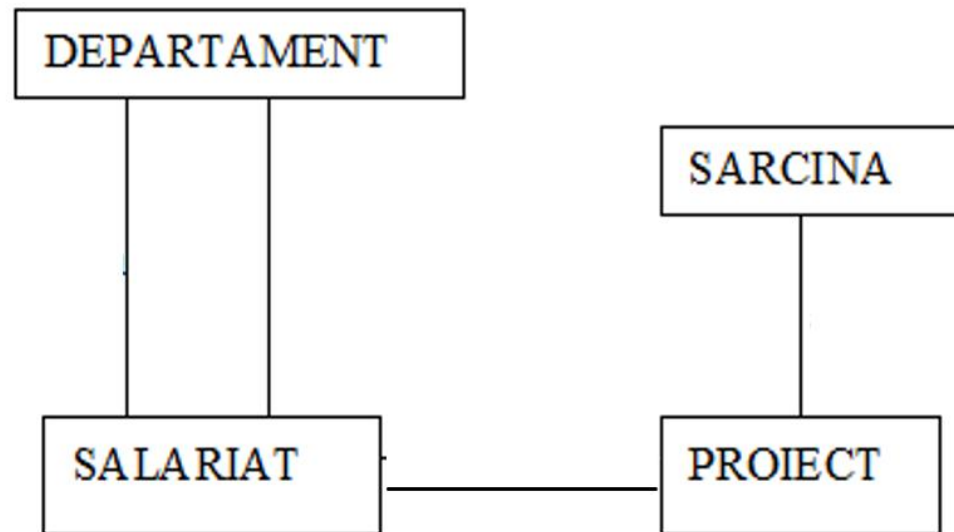
Se dau urmatoarele specificatii:

- In cadrul unui **departament** lucreaza mai multi **angajati**, dar un angajat poate sa lucreze intr-un singur departament
- Un angajat conduce un departament, iar departamentul poate avea un singur manager
- Un angajat poate sa lucreze (sa fie atasat) la mai multe **proiecte** in acelasi timp
- La dezvoltarea unui proiect pot participa mai multi angajati din companie
- Un proiect are alocate **sarcini**

Sa se identifice entitatile, plecand de la definitia entitatii existenta in cadrul slide-urilor anterioare.

Sa se identifice si legaturile dintre entitati (fara precizarea cardinalitatii).

DIAGrame ENTITATE-RELATIE



DIAGrame ENTITATE-RELAȚIE

- **Relație** (asociere): o **comunicare** între două sau mai multe entități. Existența unei relații este subordonată existenței entităților pe care le leagă.
 - În modelul relațional, **relațiile** devin **tabele speciale** sau **coloane speciale** care referă chei primare
 - Relațiile sunt **verbe**, dar nu orice verb este o relație.
 - Pentru fiecare relație este important să se dea o descriere detaliată.
 - În aceeași diagramă pot exista relații diferite cu același nume. În acest caz, le diferențiază entitățile care sunt asociate prin relația respectivă.
 - Pentru fiecare relație trebuie stabilită **cardinalitatea** (**maximă și minimă**) relației, adică numărul de tuple care aparțin relației.

DIAGrame ENTITATE-RELATIE

Exemplu (continuare):

Se dau urmatoarele specificatii:

- In cadrul unui departament lucreaza mai multi angajati, dar un angajat poate sa lucreze intr-un singur departament
- Un angajat conduce un departament, iar departamentul poate avea un singur manager
- Un angajat poate sa lucreze (sa fie atasat) la mai multe proiecte in acelasi timp
- La dezvoltarea unui proiect pot participa mai multi angajati din companie
- Un proiect are alocate sarcini (sarcina apartine proiectului)

Sa se identifice relatiile dintre entitati (verbele) (fara precizarea cardinalitatii).

DIAGrame ENTITATE-RELATIE

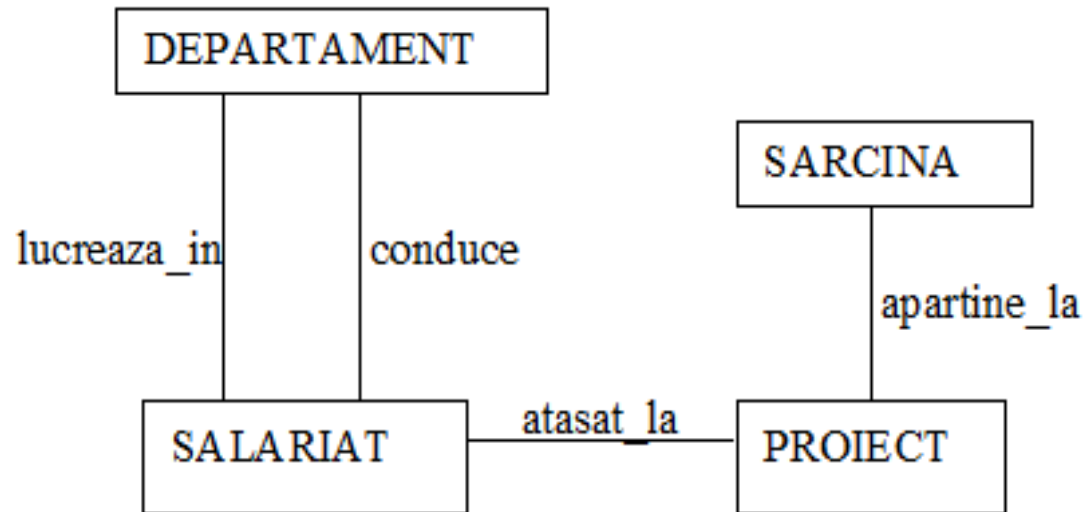
Exemplu (continuare):

Se dau urmatoarele specificatii:

- In cadrul unui departament **lucreaza** mai multi angajati, dar un angajat poate sa lucreze intr-un singur departament
- Un angajat **conduce** un departament, iar departamentul poate avea un singur manager
- Un angajat poate sa **lucreze** (sa fie atasat) la mai multe proiecte in acelasi timp
- La dezvoltarea unui proiect pot participa mai multi angajati din companie
- Un proiect **are** alocate sarcini (sau sarcina **apartine** proiectului)

Sa se identifice relatiile dintre entitati (verbele) (fara precizarea cardinalitatii).

DIAGrame ENTITATE-RELATIE



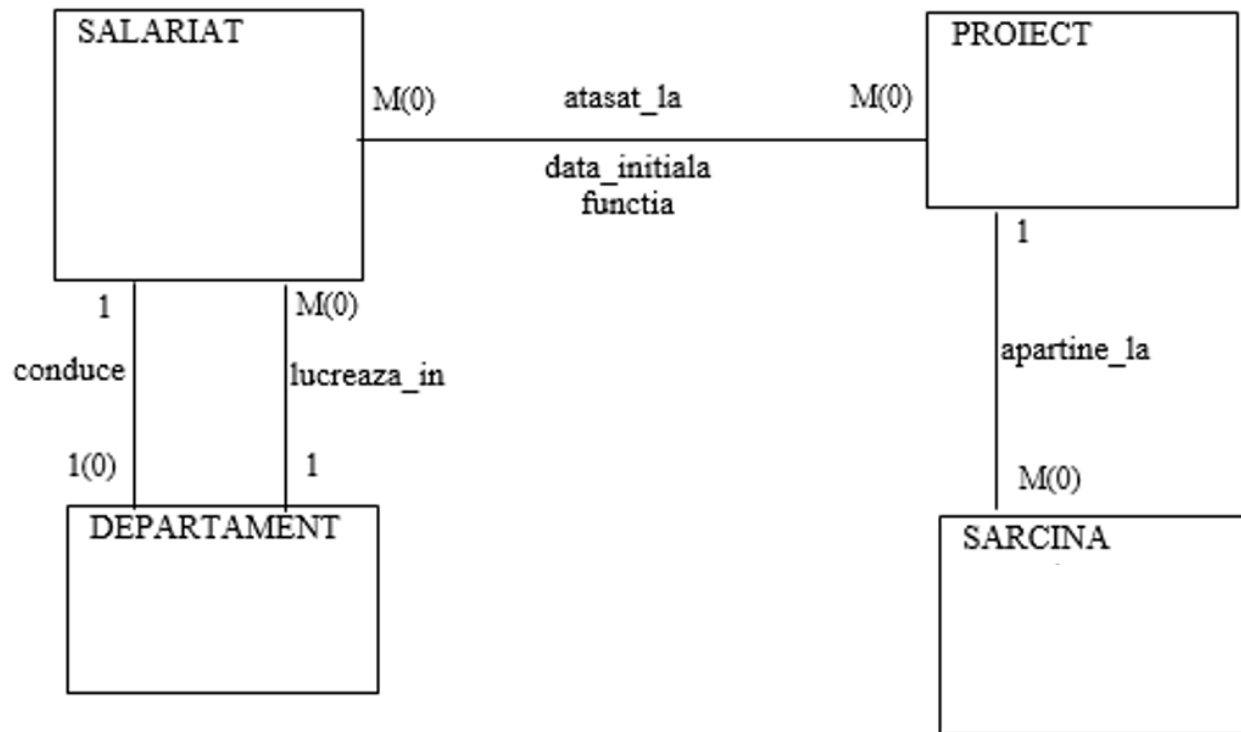
DIAGrame ENTITATE-RELAȚIE

poate (cardinalitate maximă) → **trebuie** (cardinalitate minimă)

- Relația **SALARIAT_lucraza_in_DEPARTAMENT** are cardinalitatea maximă **many-one** => (relatia one-to-many – 1:n)
 - Câți salariați **pot** lucra într-un departament? Mulți!
 - În câte departamente **poate** lucra un salariat? In cel mult unul!

- Relația **SALARIAT_conduce_DEPARTAMENT** are cardinalitatea minimă **one-zero** (1:0).
 - Câți salariați **trebuie** să conducă un departament? Cel puțin unul!
 - Câte departamente **trebuie** să conducă un salariat? Zero!

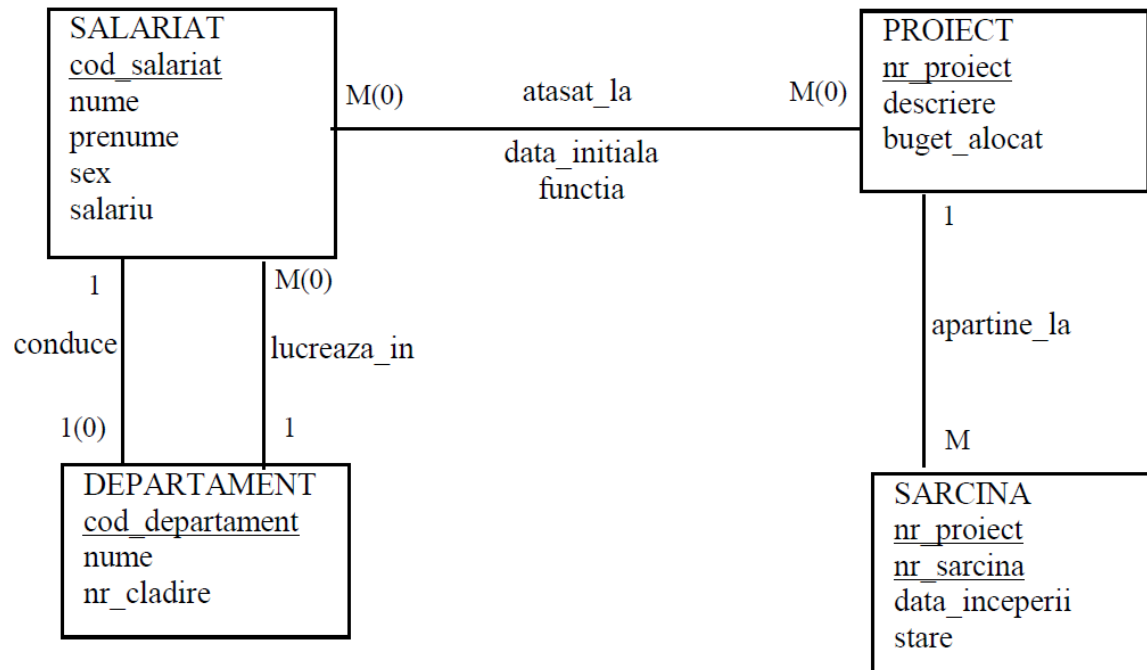
DIAGrame ENTITATE-RELATIE



DIAGrame ENTITATE-RELATIE

- **Atribut** = **proprietate** descriptivă a unei **entități** sau a unei **relații**
 - Trebuie făcută distincția între **atribut** (devine coloană în modelele relaționale) și **valoarea acestuia** (devine valoare în coloane)
 - Atributele sunt substantive, dar nu orice substantiv este atribut
 - Fiecărui atribut trebuie să i se dea o descriere completă (exemple, contraexemple, caracteristici)
 - Pentru fiecare atribut trebuie specificat **numele**, **tipul fizic** (*integer, float, char* etc.), **valori posibile**, **valori implicite**, **reguli de validare**

DIAGrame ENTITATE-RELATIE



DIAGrame ENTITATE-RELAȚIE

Reguli (neunice) pentru proiectarea unei diagrame E/R:

- **entitățile** sunt reprezentate prin **dreptunghiuri**
- **relațiile** dintre entități sunt reprezentate prin **arce neorientate**
- atributele care reprezintă **chei primare** trebuie **subliniate** sau marcate prin simbolul „#”, plasat la sfârșitul numelui acestor atribute
- **cardinalitatea minimă** este indicată în paranteze, iar cardinalitatea **maximă** se scrie fără paranteze
- nu este necesar să fie specificate, în cadrul diagramei, toate atributele

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

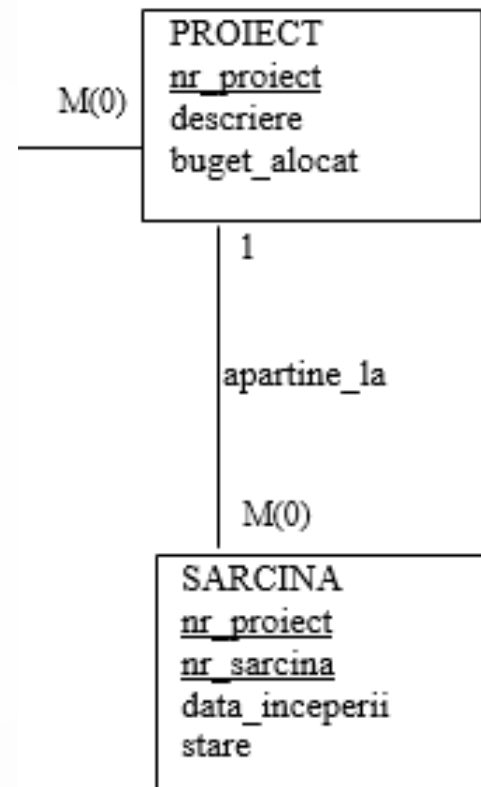
- Dependența (**entitatea dependentă**)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Specializare, generalizare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Atribute optionale

DIAGrame ENTITATE-RELATIE

Entitate dependentă – nu poate exista în mod independent

(SARCINA depinde de PROIECT)

- Cheia primară a unei entități dependente include cheia primară a sursei (*nr_proiect*) și cel puțin un atribut caracteristic entității (*nr_sarcina*).
- Entitatea dependentă se desenează prin dreptunghiuri cu linii mai subțiri.



DIAGrame ENTITATE-RELATIE

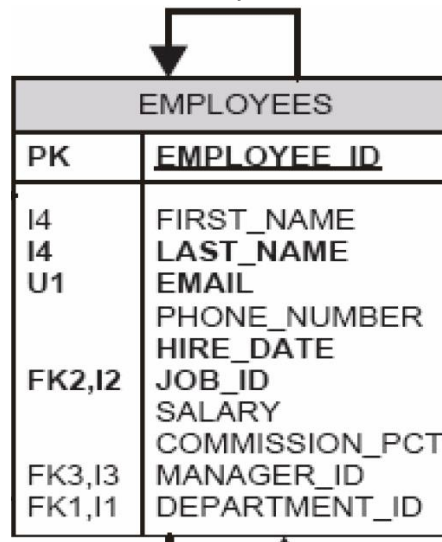
Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Specializare, generalizare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Atribute optionale

DIAGrame ENTITATE-RELAȚIE

Într-o diagramă E/R se pot defini relații recursive

O **relație recursivă** este o relație între o entitate și ea însăși.



- **Relația recursivă este definită de următoarea regulă de proiectare:**
Un angajat poate fi managerul unui alt angajat din cadrul companiei.
Există și angajați care nu au manageri. **Comentați această regulă**
(dpdv cardinalitate și implementare în cadrul tabelului Employees)

DIAGrame ENTITATE-RELATIE

Algoritmul pentru proiectarea diagramei E/R

1. identificarea **entităților** din cadrul sistemului analizat;
2. identificarea **relațiilor** (asocierilor) dintre entități și stabilirea **cardinalității**;
3. identificarea **atributelor** aferente entităților și asocierilor dintre entități;
4. stabilirea atributelor de identificare a entităților, adică stabilirea **cheilor primare**;

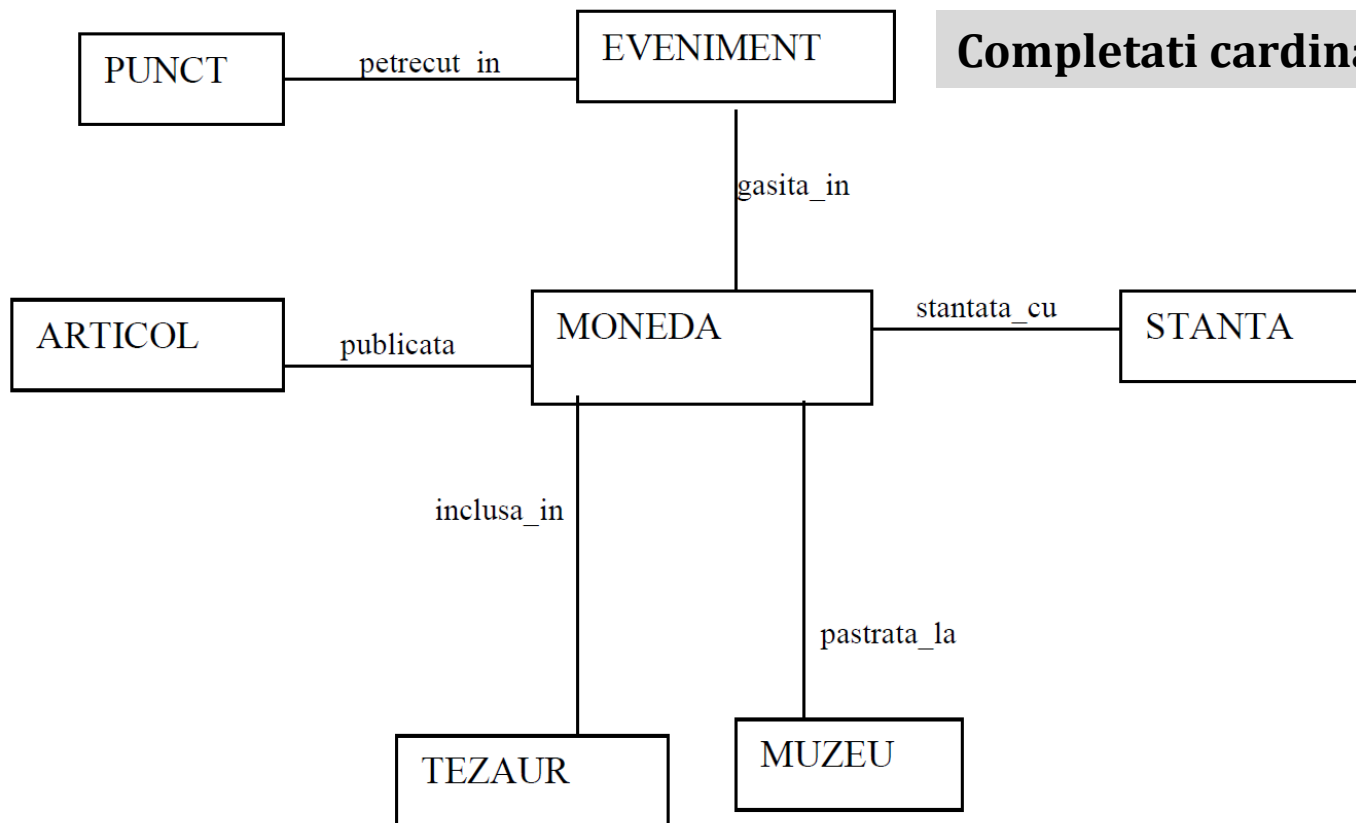
Exercitiul 1: Campionatele de fotbal ale diferitelor tari

Regulile de functionare:

- O echipa poate fi sponsorizata de mai multi sponsori. Suma sponsorizarii depinde de fiecare sponsor in parte si de echipa pe care acesta alege sa o sponsorizeze;
- Un campionat este format din mai multe etape, iar in cadrul unei etape se joaca mai multe meciuri de catre echipele care iau parte la campionat;
- O echipa joaca mai multe meciuri, iar un meci este jucat de doua echipe;

Urmand algoritmul pentru proiectarea diagramei E/R (**VEZI SLIDE 35**) sa se identifice **entitatile**, **relatiile** si **cardinalitatile**, **atributele** si **cheile primare**, dupa care sa se proiecteze **Diagrama Entitate/Relatie**.

Exercitiul 2: Descoperiri de monede antice din România



Completati cardinalitatea!

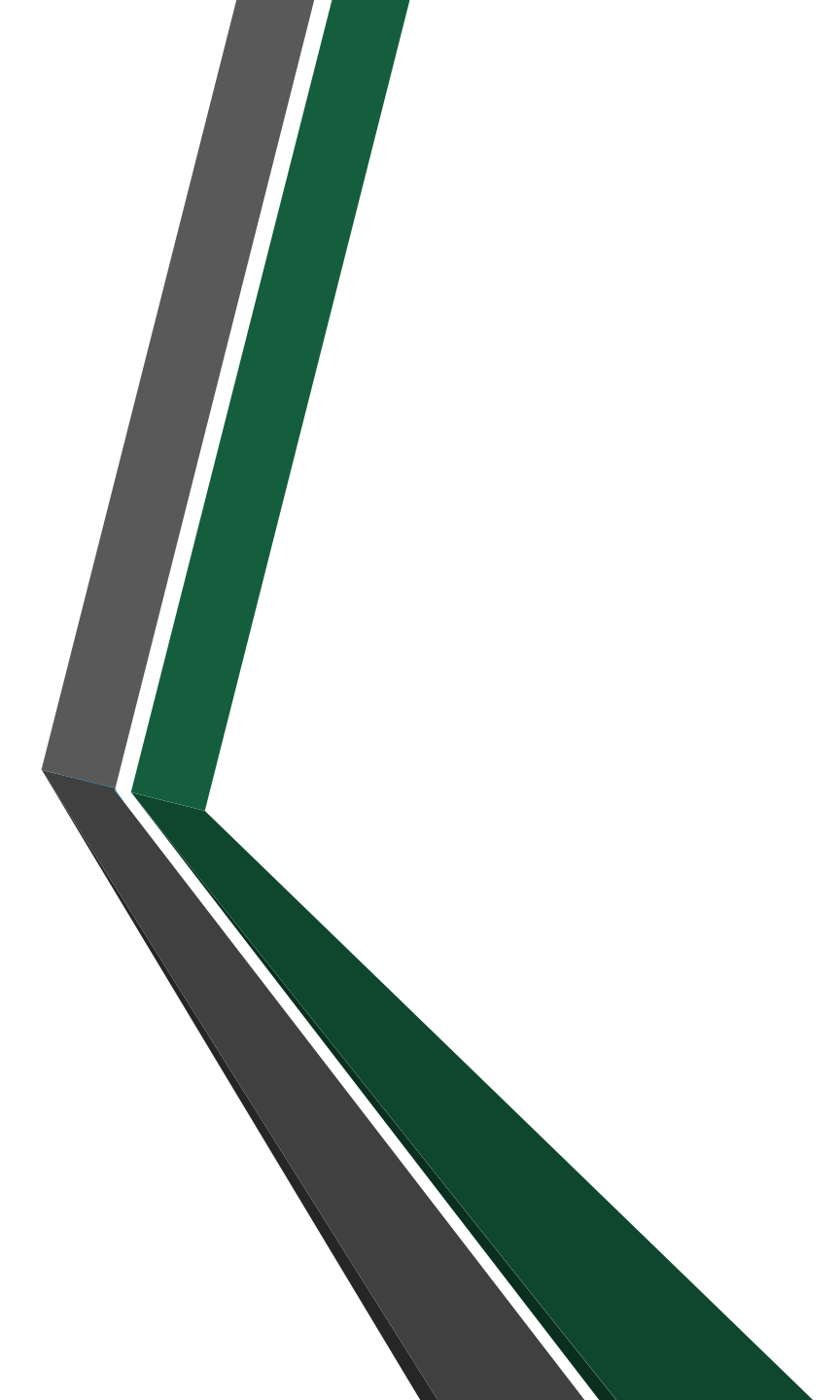
TEZÁUR, *tezaure*, s. n. **1.** Cantitate mare de monede, bijuterii, pietre scumpe sau alte obiecte de preț, strânse și păstrate în loc sigur => cardinalitatea **M(0)** de la TEZAU la MONEDA pentru ca un tezaur nu trebuie sa contina neaparat monede.

ȘTANȚĂ – nr_stanță, împărat emitent, valoare nominală, an emitere, monetăria, legenda de pe avers, legenda de pe revers.

PUNCT – un moment in care s-a petrecut evenimentul.

TEMA

- Studiați notiunile învățate în acest curs!
- Analizați diagramele studiate.
- În cadrul cursului următor să aveți coli și pix.



BAZE DE DATE

CURS 2

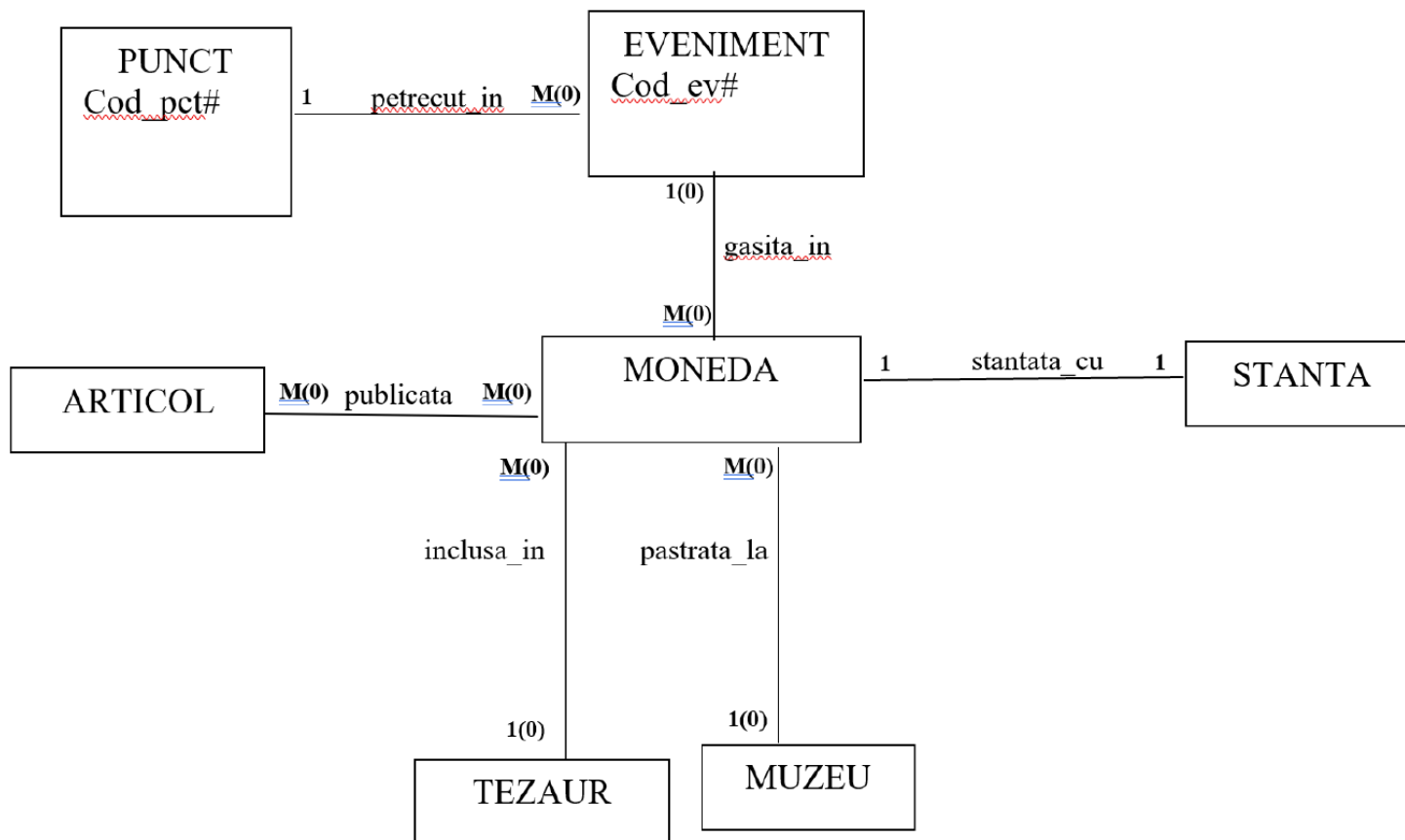
Partea II

DIAGrame ENTITATE-RELATIE

Notiuni recapitulative – Curs 2 Partea I:

- Model de date (date, relatii, constrangeri)
- Entitate
- Cheia primara
- Relatie / Cardinalitate
- Atribut
- Diagrame Entitate-Relatie

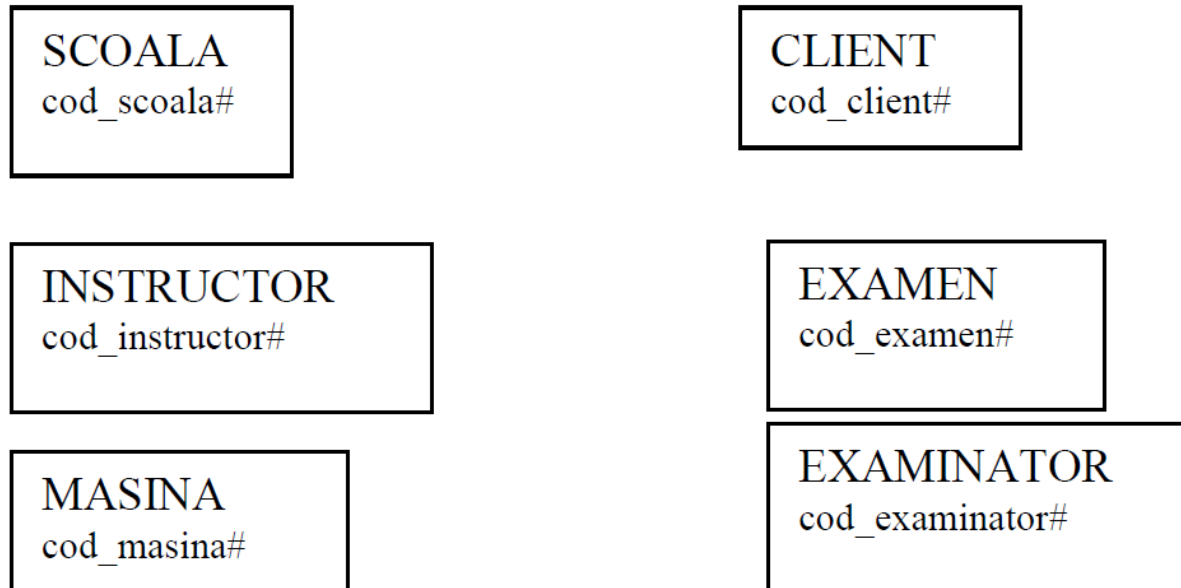
Discutie din cursul anterior: Descoperiri de monede antice din România



EXERCITIU RECAPITULATIV

Exercitiul 1 – Evidenta scolilor de soferi din Romania – 0.25p

Completați relațiile (*lucreaza_la, conduce, sustine, asista, instruieste*) dintre entități și specificați cardinalitatea

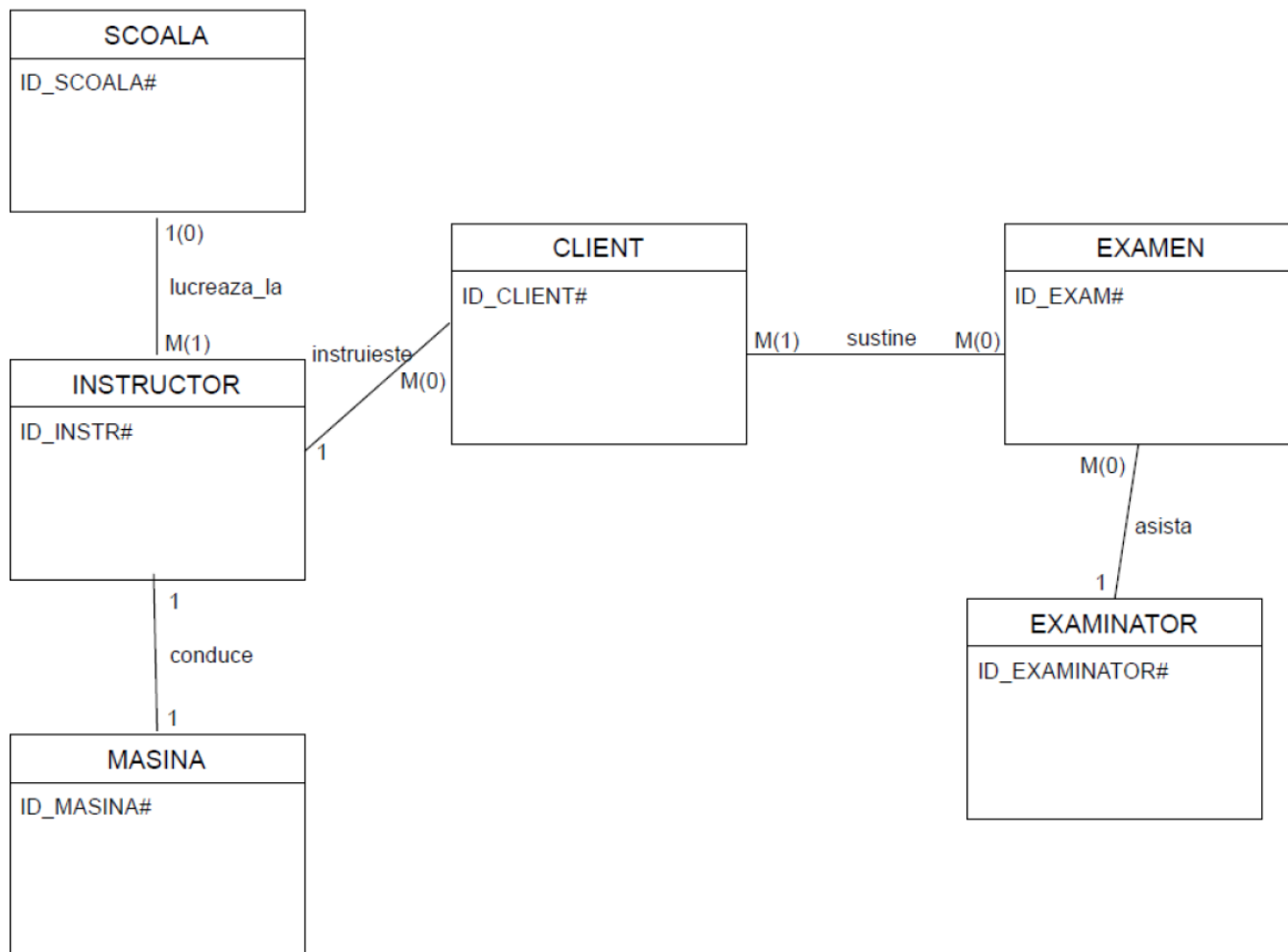


EXERCITIU RECAPITULATIV

Regulile de functionare:

- Baza de date modeleaza activitatea din cadrul unei scoli de soferi;
- Scoala de soferi angajeaza instructori care se ocupa de pregatirea clientilor. In cadrul scolii pot lucra mai multi instructori, dar un instructor nu poate lucra pentru mai multe scoli simultan. Instructorul poate face parte din cadrul unei singure scoli de soferi sau poate lucra fara a face parte, in mod obligatoriu, dintr-o scoala;
- Fiecare instructor detine doar o singura masina si poate sa instruiasca mai multi clienti. Pot exista situatii in care instructorul nu are inca alocati cursati. De asemenea, un cursant trebuie sa fie pregatit de acelasi instructor pe toata durata cursului;
- Un cursant poate fi inscris in cadrul scolii de soferi doar pentru ore suplimentare de condus. Cursantul poate sustine de mai multe ori examenul final. In cadrul examenului este prezent intotdeauna un singur examinator;

EXERCITIU RECAPITULATIV



DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (**subentitate** / **superentitate**)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Atribute optionale

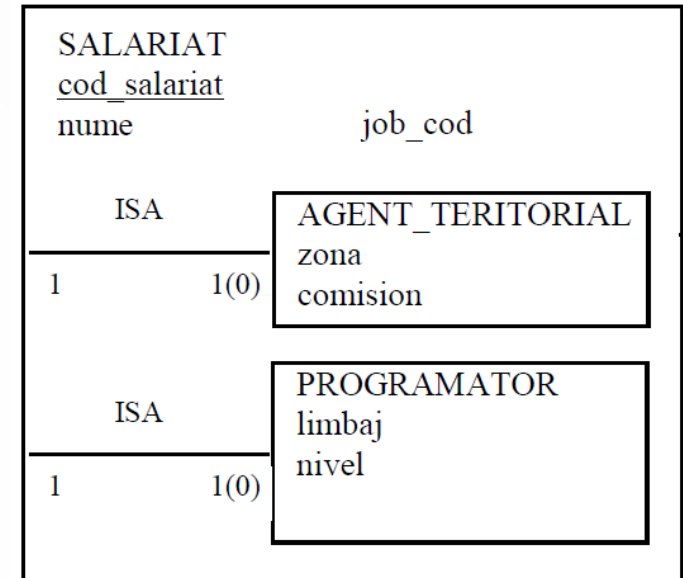
DIAGrame ENTITATE-RELAȚIE

Moștenirea atributelor

- **Subentitate (subclasă)** – submulțime a unei alte entități, numită **superentitate (superclasă)**

SALARIAT < — > PROGRAMATOR

- Subentitatea se desenează prin dreptunghiuri incluse în superentitate.
- Există o relație între o subentitate și o superentitate, numită **ISA**, care are cardinalitatea maximă 1:1 și minimă 1:0.
- Cheile primare, attributele și relațiile unei superentități sunt valabile pentru orice subentitate. Afirmția reciprocă este falsă.



DIAGrame ENTITATE-RELATIE

Superclasa (superentitate) / Subclasa (subentitate):

- **Superclasa** (superentitatea) este o entitate care include **subclase** (subentități) distincte, ce trebuie reprezentate în modelul de date
- **Subclasa** are un rol distinct și, evident, este membră a unei superclase. O subclasă, fiind o entitate, poate să posede propriile subclase

O entitate împreună cu subclasele ei, subclasele acestora și așa mai departe definește o **ierarhie de tip** (ierarhie de specializare)

De exemplu: ANGAJAT reprezintă o superclasă pentru entitatea PROGRAMATOR

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- **Generalizare, specializare**
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Atribute optionale

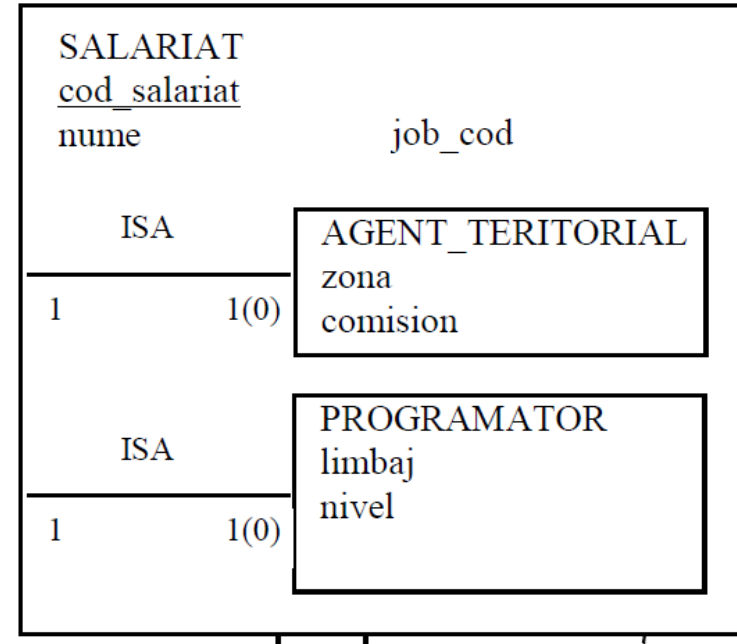
DIAGrame ENTITATE-RELATIE

Generalizare

- Din entități similare care au mai multe **atribute comune** se pot crea **superentități**.
- Aceste superentități conțin attributele comune, iar attributele speciale sunt asignate la subentități. Pentru noile superentități se introduc chei primare artificiale.

Specializare

- După valorile unor attribute clasificatoare se pot determina **clase**.
- Un grup de subentități reciproc exclusive definește o clasă.
- Clasele se aliniază în desen vertical.



DIAGrame ENTITATE-RELATIE

Generalizarea este procesul de **minimizare a diferențelor** dintre entități, prin identificarea caracteristicilor comune ale acestora. Generalizarea are ca rezultat identificarea unei **superclase** generalizate din subclasele inițiale.

Specializarea este procesul de **maximizare a diferențelor** dintre membrii unei entități, prin identificarea caracteristicilor distinctive ale acestora. (**subclasa**).

- Dacă subclasele unei clase sunt disjuncte (nu au elemente comune), atunci o instanța (valoare/inregistrare/intrare) a entității poate fi membră doar a unei subclase a acesteia (**constrângere de disjuncție**) (un angajat poate fi ori programator ori agent teritorial, nu ambele în același timp).
- O specializare cu participare totală specifică faptul că fiecare instanță (intrare) din superclasă trebuie să fie membră a unei subclase din specializare (**constrângere de participare**) (un angajat trebuie obligatoriu să fie programator sau agent teritorial).
- O specializare cu participare parțială specifică faptul că nu este necesar ca o instanță/valoare din entitate să aparțină vreunei subclase a acesteia. **De exemplu**, există salariați în **SALARIAȚ** care nu aparțin niciunei subentități.

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- **Relații de tip 2, 3**
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Atribute optionale

DIAGrame ENTITATE-RELAȚIE

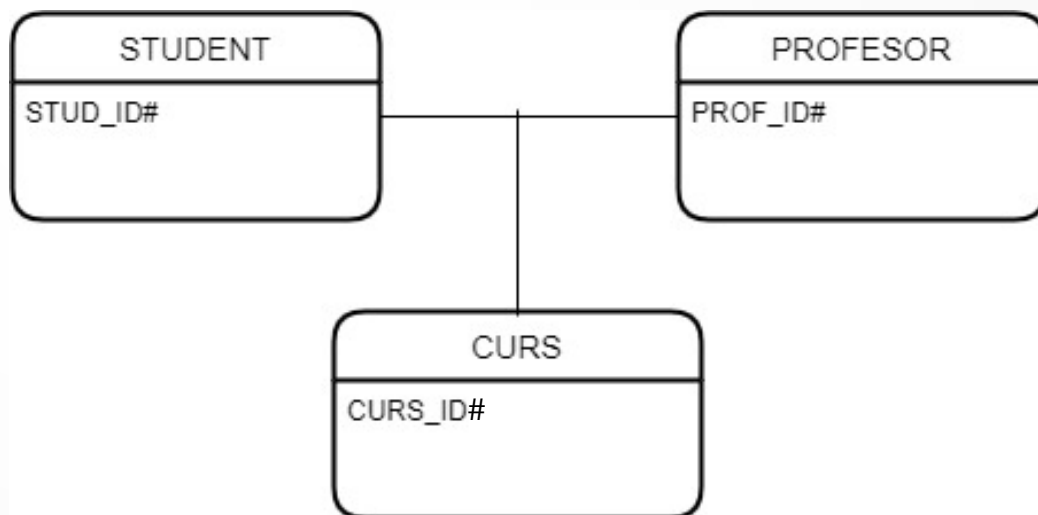
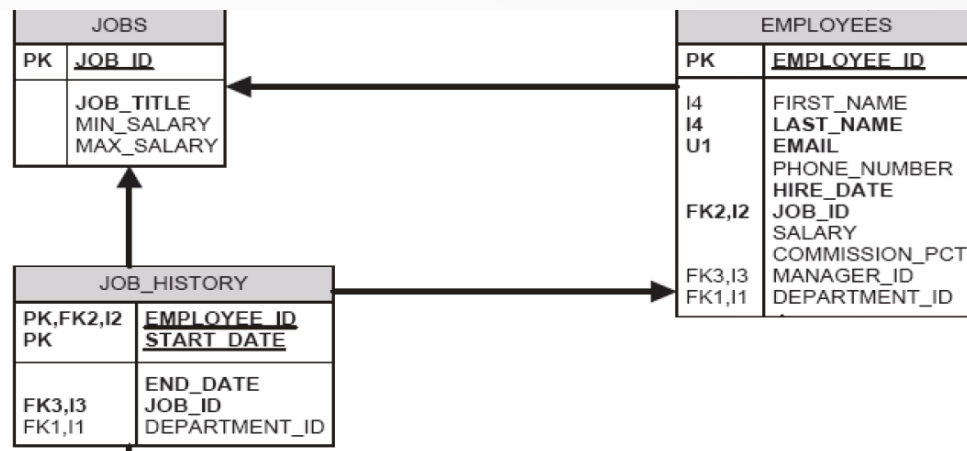
Unele relații sunt relative la două entități și le numim de **tip 2**, iar dacă relațiile implică mai mult de două entități, le vom numi de **tip 3**.

- Trei relații de tip 2 sunt diferite de o relație de tip 3
- Rupând o relație de tip 3 în trei relații de tip 2, pot apărea informații incorecte

Trei relații de tip 2 sunt diferite de o relație de tip 3.

În relația de tip 3 – un student participă la mai multe cursuri, predate de mai mulți profesori

- relație many-to-many între toate tabelele implicate



DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- Atribute optionale

DIAGrame ENTITATE-RELAȚIE

- Trebuie **exclude** din model **relațiile indirecte** (ex: un angajat este coleg cu un alt angajat) deoarece ele pot conduce la redundanță în baza de date
- **Atributele derivabile** trebuie eliminate și introduse expresii prin care aceste atribute pot fi calculate
- **EXAMPLE?**

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- **Relație sau atribut?**
- **Entitate sau relație?**
- Reassignarea atributelor indirecte
- Atribute optionale

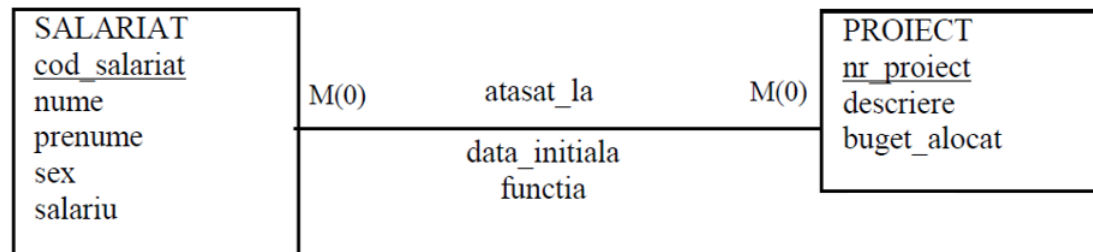
DIAGrame ENTITATE-RELAȚIE

➤ Relație sau atribut?

- Dacă un atribut al unei entități reprezintă cheia primară a unei alte entități, atunci el referă o relație (*cod_departament* în tabelul SALARIAT).

➤ Entitate sau relație?

- Se cercetează cheia primară. Dacă aceasta combină cheile primare a două entități, atunci este vorba de o relație.
- **EX:** cheia primară a relației *atasat_la* combină *cod_salariat* cu *nr_proiect*, prin urmare, *SALARIAT_atasat_la_PROIECT* va defini o relație și nu o entitate.



DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- **Reassignarea atributelor indirecte**
- Atribute optionale

DIAGrame ENTITATE-RELATIE

- **Un atribut indirect** nu descrie real relația sau entitatea
- Un atribut indirect este un caz special de relație indirectă care trebuie eliminată
- Prin urmare, attributele indirecte trebuie reasignate deoarece introduc redundanță în date

EX: numărul clădirii în care lucrează un salariat este un atribut al entității DEPARTAMENT și nu este o caracteristică a entității SALARIAT

DIAGrame ENTITATE-RELATIE

Cazuri speciale de entități, relații, atribute și modul lor de reprezentare în cadrul diagramei entitate-relație:

- Dependența (entitatea dependentă)
- Relații recursive
- Moștenirea atributelor (subentitate / superentitate)
- Generalizare, specializare
- Relații de tip 2, 3
- Excluderea relațiilor indirecte
- Eliminarea atributelor derivabile
- Relație sau atribut?
- Entitate sau relație?
- Reassignarea atributelor indirecte
- **Atribute optionale**

DIAGrame ENTITATE-RELATIE

- Există **attribute opționale**, a căror valoare este uneori necunoscută, alteori neaplicabilă
- Aceste attribute trebuie introduse la subentități

EX: comisionul pentru deplasare și zona de lucru sunt attribute specifice unui agent teritorial și trebuie introduse in cadrul subentitatii AGENT_TERITORIAL

DIAGrame ENTITATE-RELATIE

Algoritmul pentru proiectarea diagramei E/R

1. identificarea **entităților** din cadrul sistemului analizat;
2. identificarea **relațiilor** (asocierilor) dintre entități și stabilirea **cardinalității**;
3. identificarea **atributelor** aferente entităților și asocierilor dintre entități;
4. stabilirea atributelor de identificare a entităților, adică stabilirea **cheilor primare**;

DIAGrame ENTITATE-RELATIE

Aceași realitate poate fi percepută diferit de către diverși analiști pentru un același sistem => **modele structurale distincte.**

- Construirea **diagramei conceptuale**
- Obținerea **schemelor relaționale**
- **Normalizarea** acestora



=> un **model relațional** care va elimina anumite clase de anomalii ce pot să apară în proiectarea modelului de date

MODELUL E/R EXTINS

- Conceptele de bază ale modelării E/R nu sunt suficiente pentru a reprezenta cerințe complexe
- Modelul E/R susținut cu concepte semantice adiționale definește **modelul E/R extins (EER)**.
 - include toate **conceptele modelului original**
 - + conceptele adiționale de **subclasă, superclasă, moștenire, specializare, generalizare**.

Modelul EER (modelul E/R extins) = Diagrama E/R + concepte adiționale (subclasă, superclasă, moștenire, specializare, generalizare).

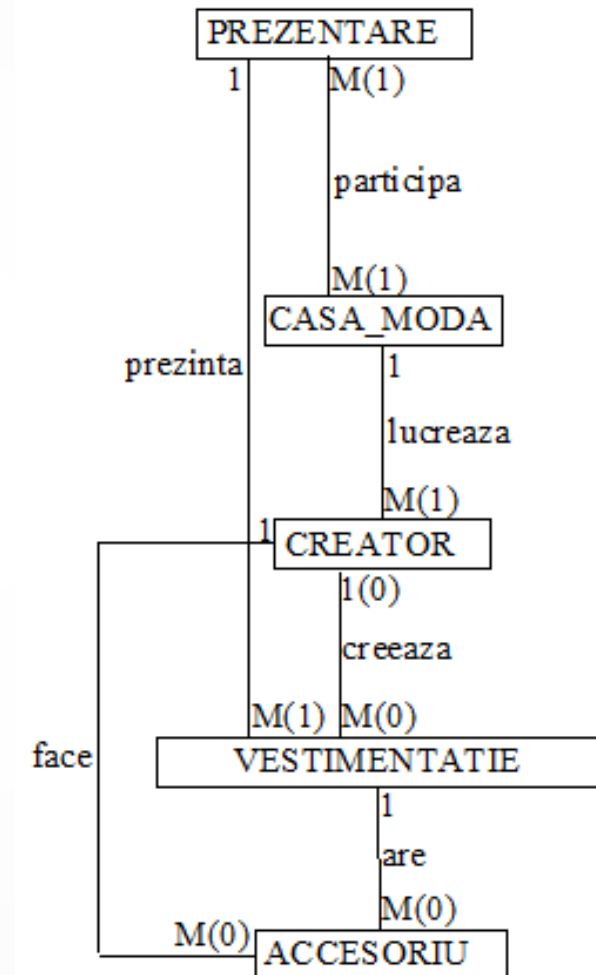
DEFICIENȚE ALE MODELULUI E/R

- Cauza: interpretare eronată a sensului unei relații => **capcane de conectare**
- Posibil să necesite restructurarea modelului
- 2 clase de capcane de conectare:
 - **de întrerupere**
 - **în avantaj**

DEFICIENȚE ALE MODELULUI E/R

1. Capcane de intrerupere

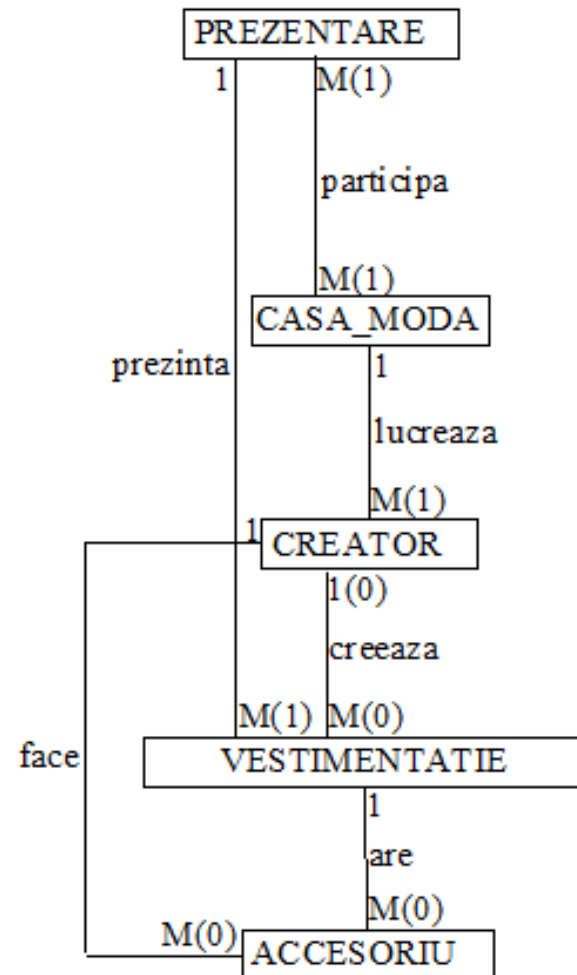
- Modelul sugerează existența unei relații între entități, dar nu există o cale între anumite apariții ale entităților
- Această capcană poate să apară acolo unde există o relație cu **participare parțială** (0 la cardinalitatea minimă) care face parte din calea dintre entitățile ce sunt legate



DEFICIENȚE ALE MODELULUI E/R

- Aceste capcane generează situațiile în care, așa cum a fost proiectat modelul de date, **el nu poate să răspundă la anumite interogări.**

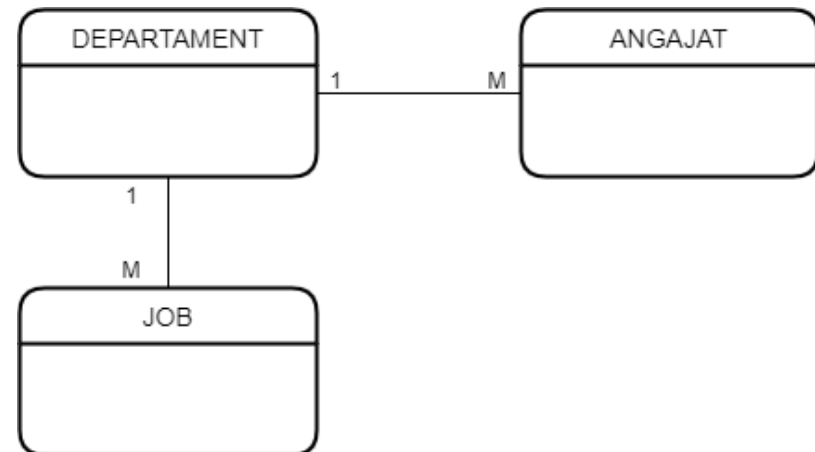
De exemplu: pentru a afla pentru ce prezentare de modă a fost creată o anumită vestimentație, a fost necesară introducerea unei legături între entitățile PREZENTARE și VESTIMENTATIE, care însă a generat redundanță în modelul de date



DEFICIENȚE ALE MODELULUI E/R

2. Capcane în evantai

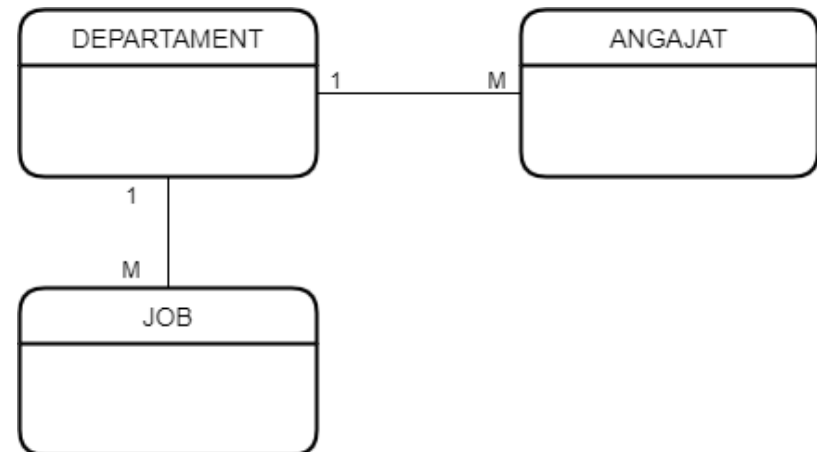
- Modelul ia în considerare o relație între entități, dar calea dintre anumite apariții ale entităților este ambiguă
- Aceste capcane apar când două sau mai multe relații *one_to_many* provin din aceeași entitate



DEFICIENȚE ALE MODELULUI E/R

2. Capcane în evantai

- Modelul ia în considerare o relație între entități, dar calea dintre anumite apariții ale entităților este ambiguă
- Aceste capcane apar când două sau mai multe relații *one_to_many* provin din aceeași entitate

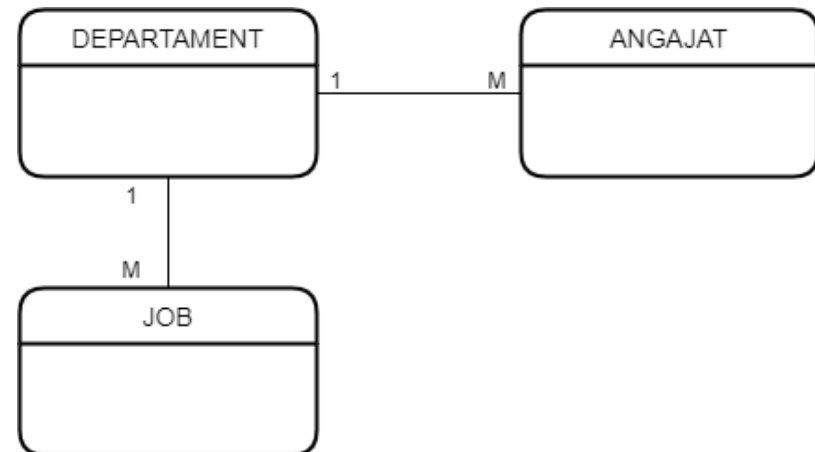


**La ce întrebare nu
răspunde modelul?**

DEFICIENȚE ALE MODELULUI E/R

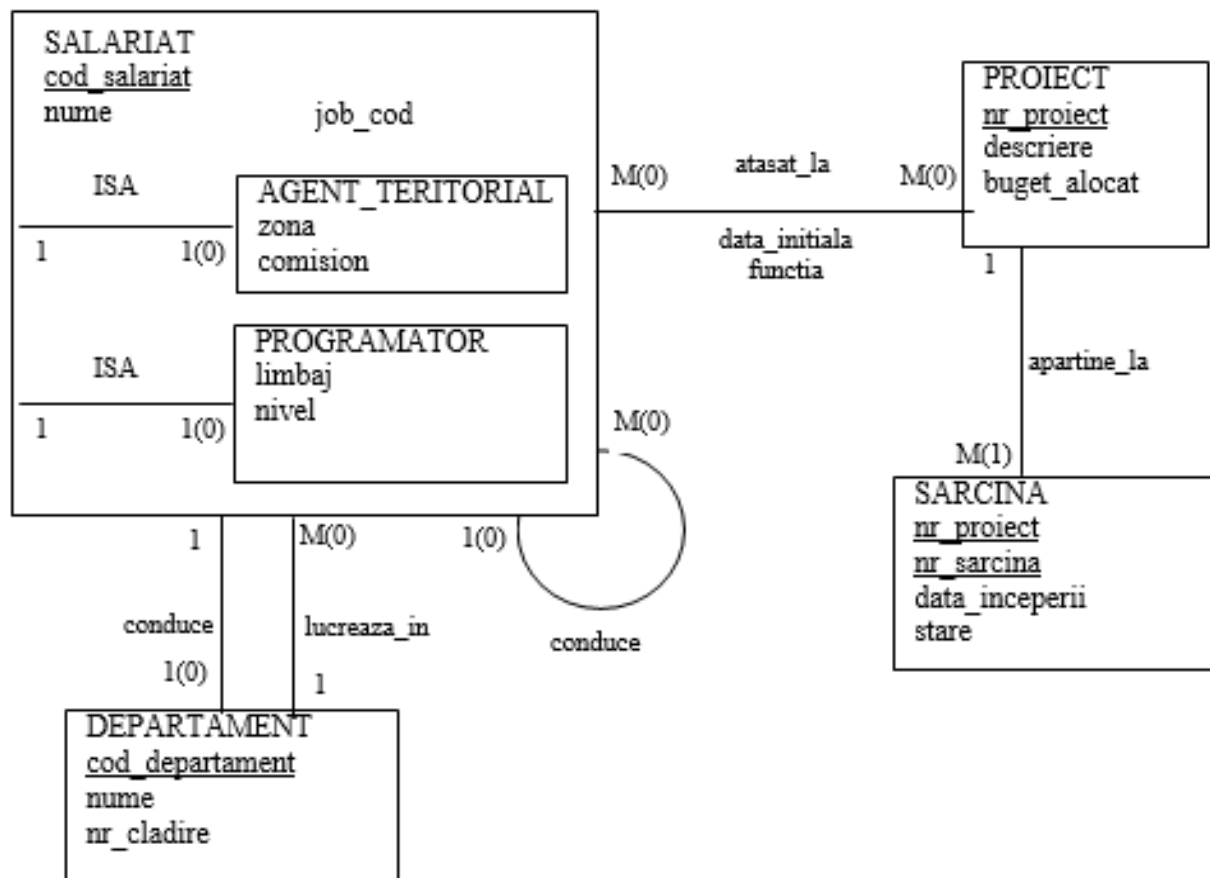
2. Capcane în evantai

- Modelul ia în considerare o relație între entități, dar calea dintre anumite apariții ale entităților este ambiguă
- Aceste capcane apar când două sau mai multe relații *one_to_many* provin din aceeași entitate

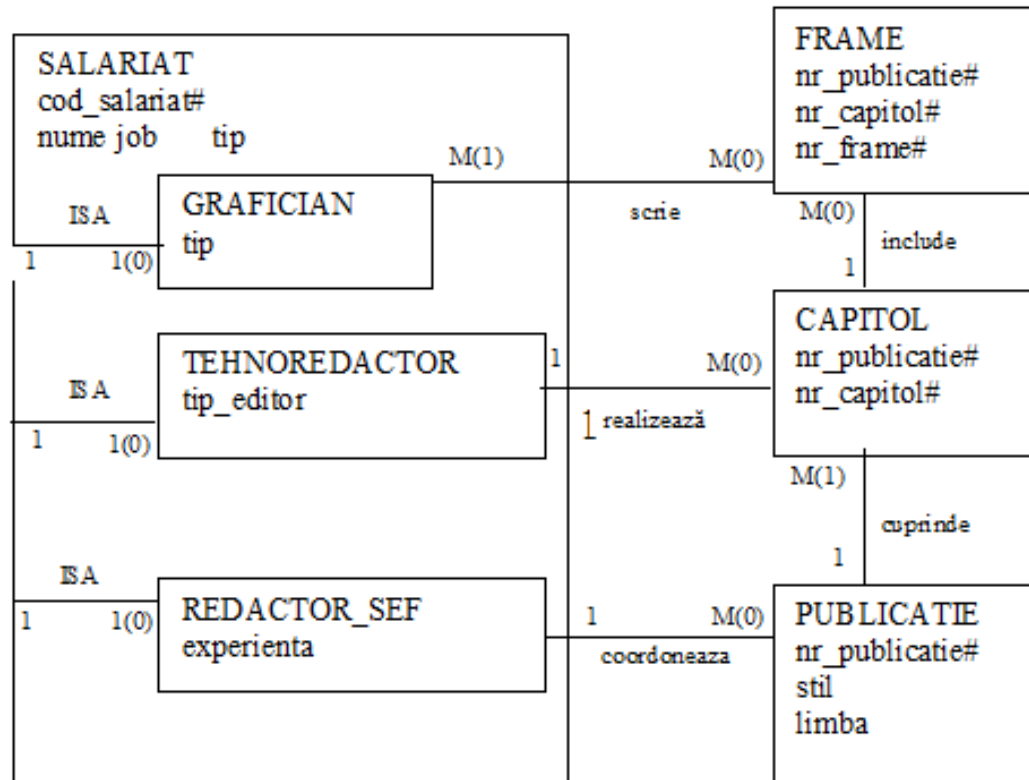


Nu se poate raspunde la
intrebarea – **Ce job are un
angajat?**

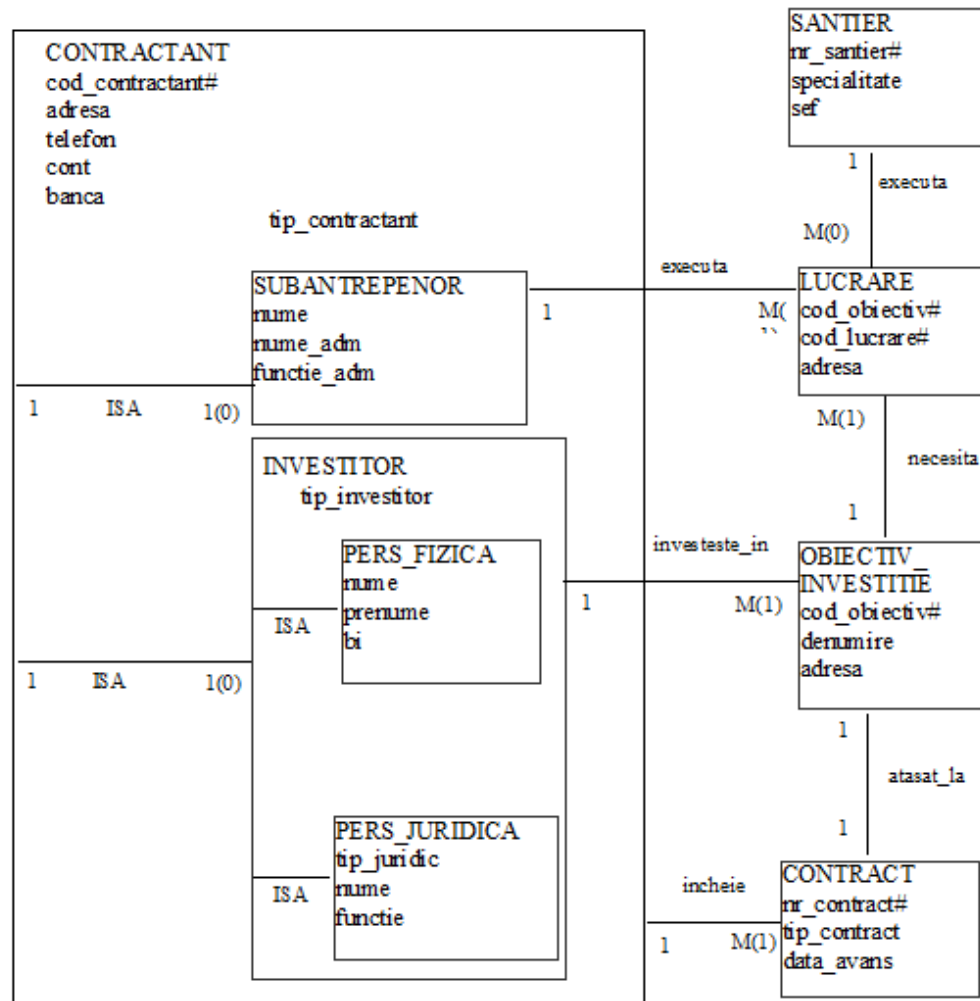
EXEMPLUL 1: Gestiunea activităților din cadrul unei companii



EXEMPLUL 2: Gestiunea activităților de editare dintr-o editură



EXEMPLUL 3: Gestiunea unei firme de constructii



Discutie erori!

EXEMPLUL 4: Gestiunea activitatilor de împrumut dintr-o biblioteca

Entitățile și relațiile care intervin în acest model sunt următoarele:

- CARTE** (entitate independentă) – orice carte care se găsește în inventarul bibliotecii. Cheia primară este atributul *cod_carte*;
- CITITOR** (entitate independentă) – orice cititor care poate împrumuta cărți. Cheia primară este atributul *cod_cititor*;
- DOMENIU** (entitate independentă) – domeniul cărui îi aparține o carte. Cheia primară este atributul *cod_domeniu*;
- IMPRUMUTA** – relație care leagă entitățile CITITOR și CARTE;
- APARTINE** – relație care leagă attributele CARTE și DOMENIU;

Obs: S-a presupus (restrictiv) că într-o zi un cititor nu poate împrumuta, de mai multe ori, aceeași carte -> **regulă a modelului.**

- **Ce cardinalități au cele două relații?**
- **Reprezentați diagrama E/R a acestui model.**

EXEMPLUL 5: Gestiunea activitatilor dintr-o agentie de turism

Sa se proiecteze Diagrama Entitate-Relatie pentru **Gestiunea activitatilor dintr-o agentie de turism**.

Descrierea modelului si a regulilor de functionare:

Modelul de date va gestiona informatii legate de rezervarea si organizarea activitatilor din cadrul unei agentii de turism. Intr-o agentie de turism lucreaza ghizi, soferi si agenti de vanzari. Agentia se ocupa de clientii care doresc sa achizitioneze sejururi sau excursii. Clientii sunt preluati de agentii de vanzari, dupa care, in urma ofertelor, ei aleg un sejur sau o excursie din oferta agentiei. Excursiile pot avea ghizi si soferi.

EXEMPLUL 5: Gestiunea activitatilor dintr-o agentie de turism

Modelul de date prezinta anumite restrictii (constrangeri) de functionare – acestea se implementeaza la nivel de cardinalitate:

- Din oferta agentiei fac parte doar sejururi sau excursii.
- Ghizii conduc excursii, la care pot fi atasati mai multi soferi, iar o excursie poate avea un singur ghid. Pot exista ghizi care sa nu fie alocati excursiilor. Turistii pot opta pentru excursii fara ghid.
- O excursie poate avea mai multi soferi sau pot exista si oferte fara soferi. In cazul acestor oferte, clientii aleg sa calatoreasca pe cont propriu. De asemenea, pot exista si soferi care sa nu fie alocati momentan excursiilor existente.
- Clientii agentiei achizitioneaza sejururi sau excursii.
- Clientii pot fi deserviti de catre mai multi agenti de vanzari ai agentiei si pot alege mai multe oferte.
- Un sejur se desfasoara intr-o anumita locatie, iar intr-o locatie au loc mai multe sejururi. Pot exista si locatii in care sa nu aiba loc sejururi.

EXEMPLUL 5: Gestiunea activitatilor dintr-o agentie de turism

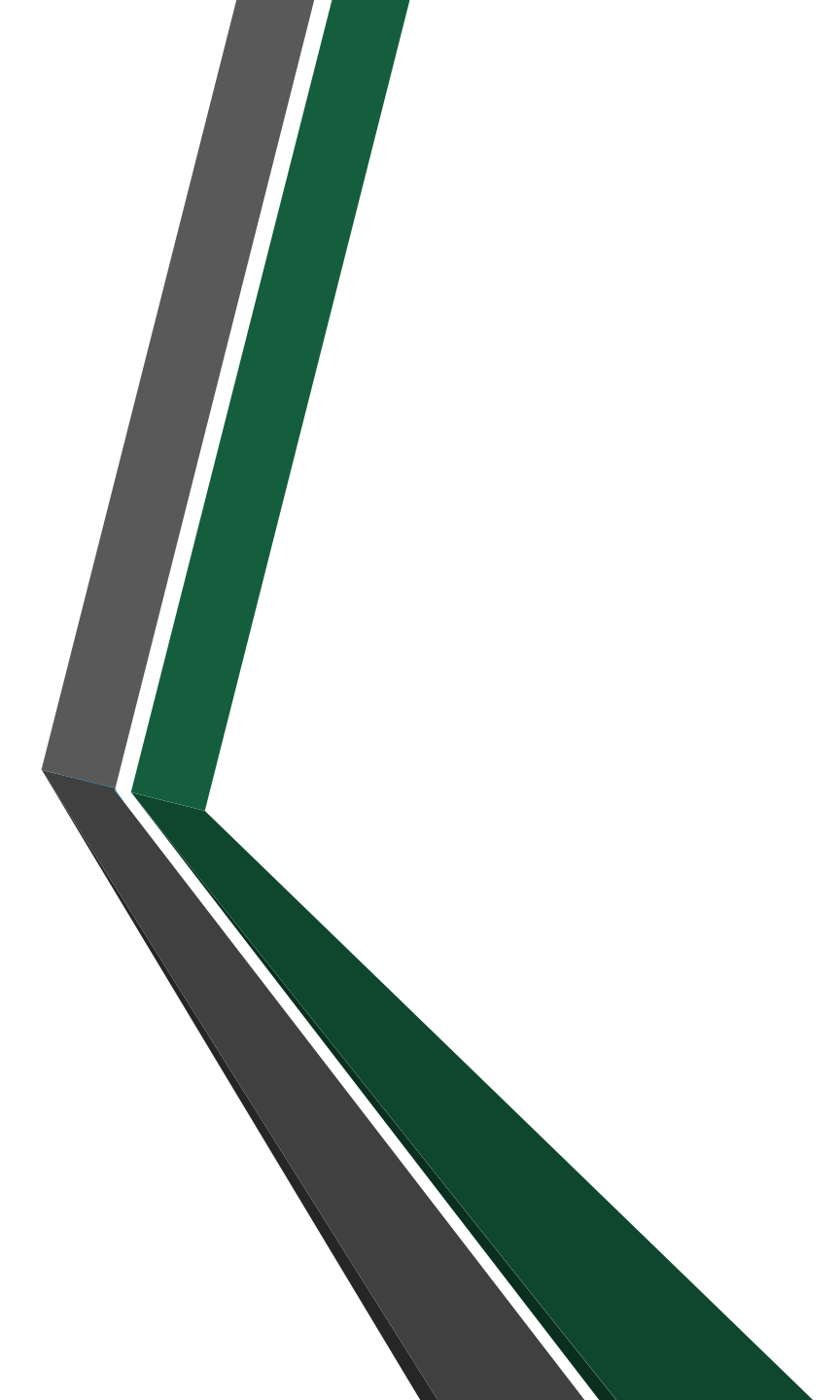
In continuare vom parcurge urmatoorii pasi:

1. Descrierea modelului real, a utilitatii acestuia si a regulilor de functionare;
2. Prezentarea constrangerilor (restrictii, reguli) impuse asupra modelului;
3. Descrierea entitatilor, incluzand precizarea cheii primare;
4. Descrierea relatiilor, incluzand precizarea cardinalitatii acestora;
5. Descrierea atributelor, incluzand tipul de date si eventualele constrangeri, valori implicite, valori posibile ale atributelor;
6. Realizarea diagramei entitate-relatie;

Documentul cu explicatiile in intregime si Diagrama E/R se afla pe site, fiind intitulat: “Agentie_Turism”

TEMA 1 – Pregatire pentru proiect

Conform exercitiului anterior – Gestiunea activitatilor dintr-o agentie de turism – realizati tema 1 aflata pe site, intitulata “**Tema1_Curs**”

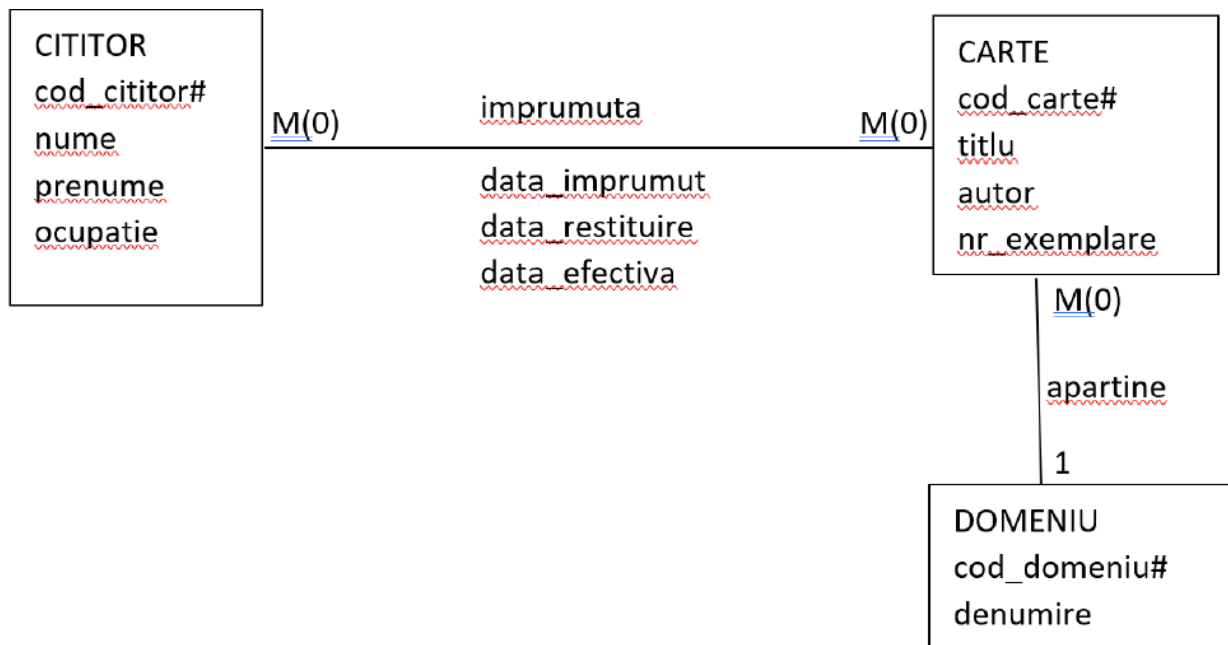


BAZE DE DATE

CURS 4

EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca

- Sa se proiecteze Diagrama Conceptuala, transformand Diagrama Entitate/Relatie:



EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca

CITITOR

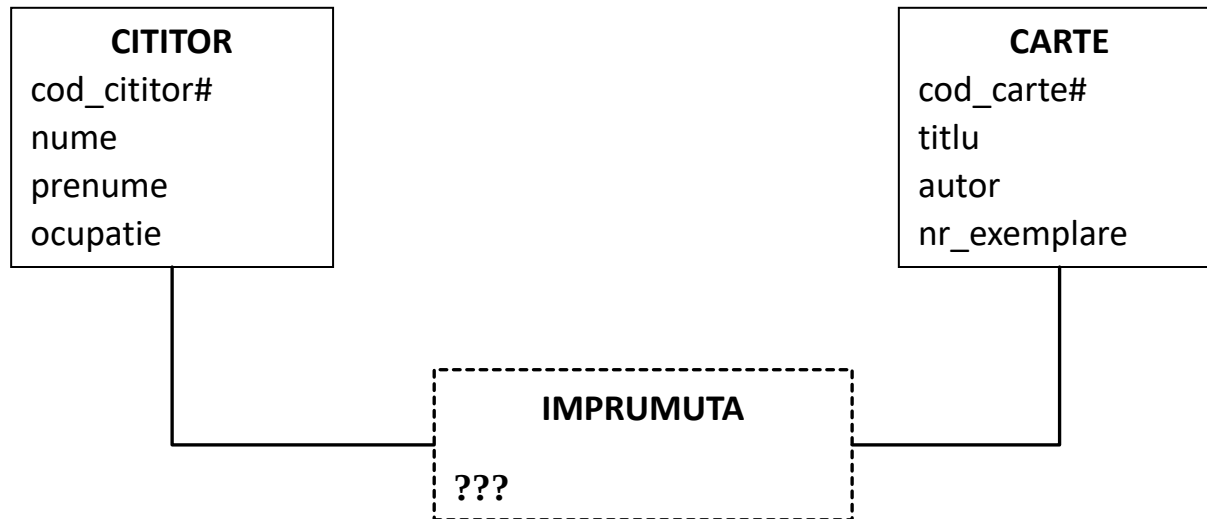
cod_cititor#
nume
prenume
ocupatie

CARTE

cod_carte#
titlu
autor
nr_exemplare

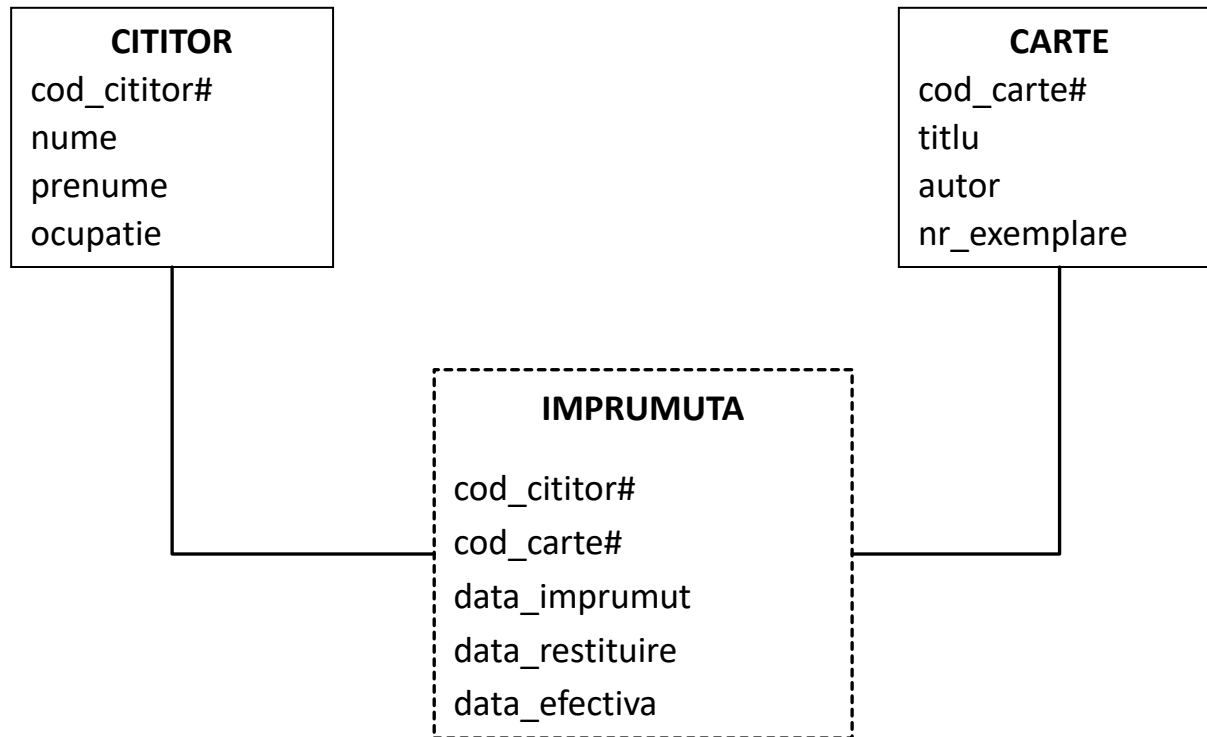
Cum se transforma relatia **IMPRUMUTA**?

EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca



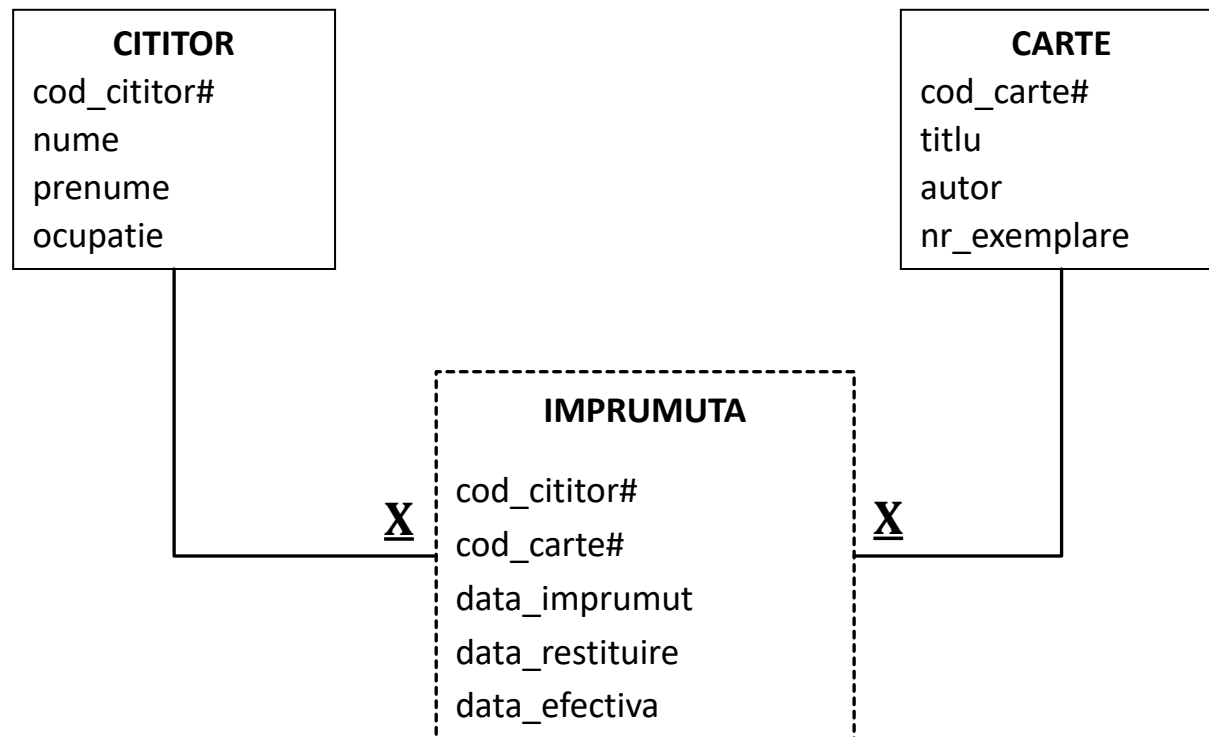
Ce attribute contine tabelul asociativ **IMPRUMUTA**?

EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca

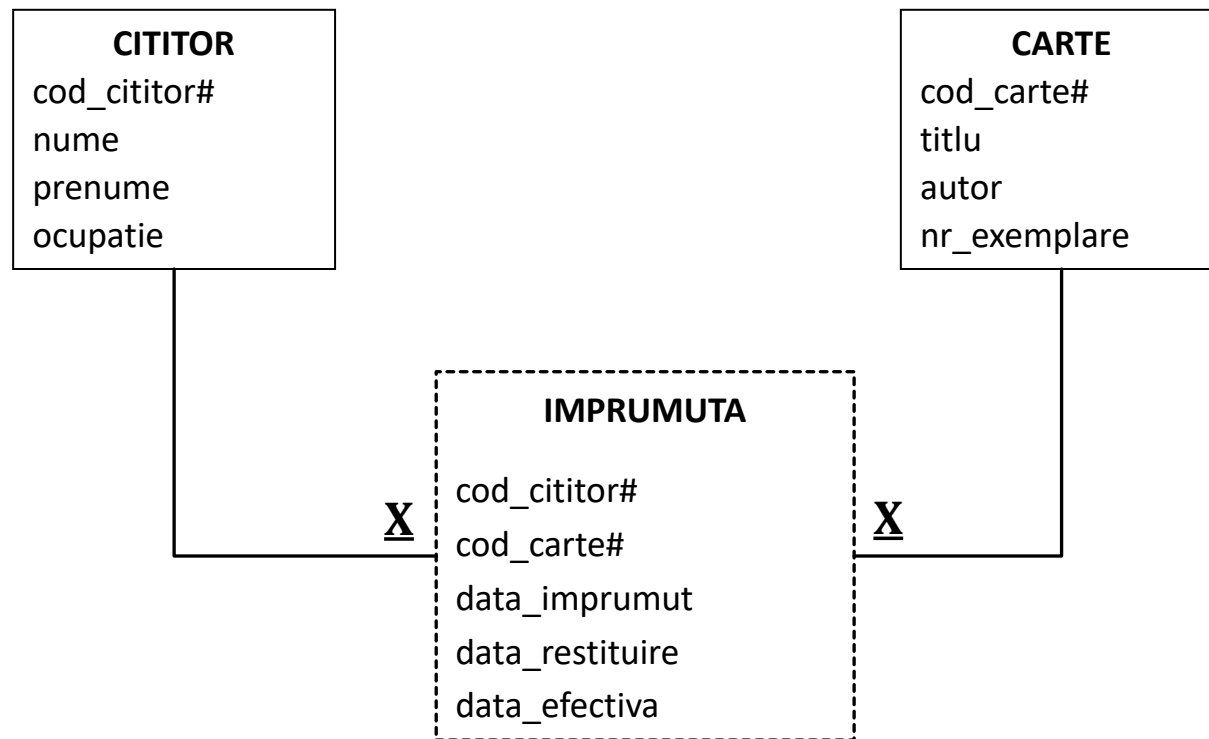


Ce simbol se foloseste pentru marcarea cheii primare? Unde este plasat acesta?

EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca

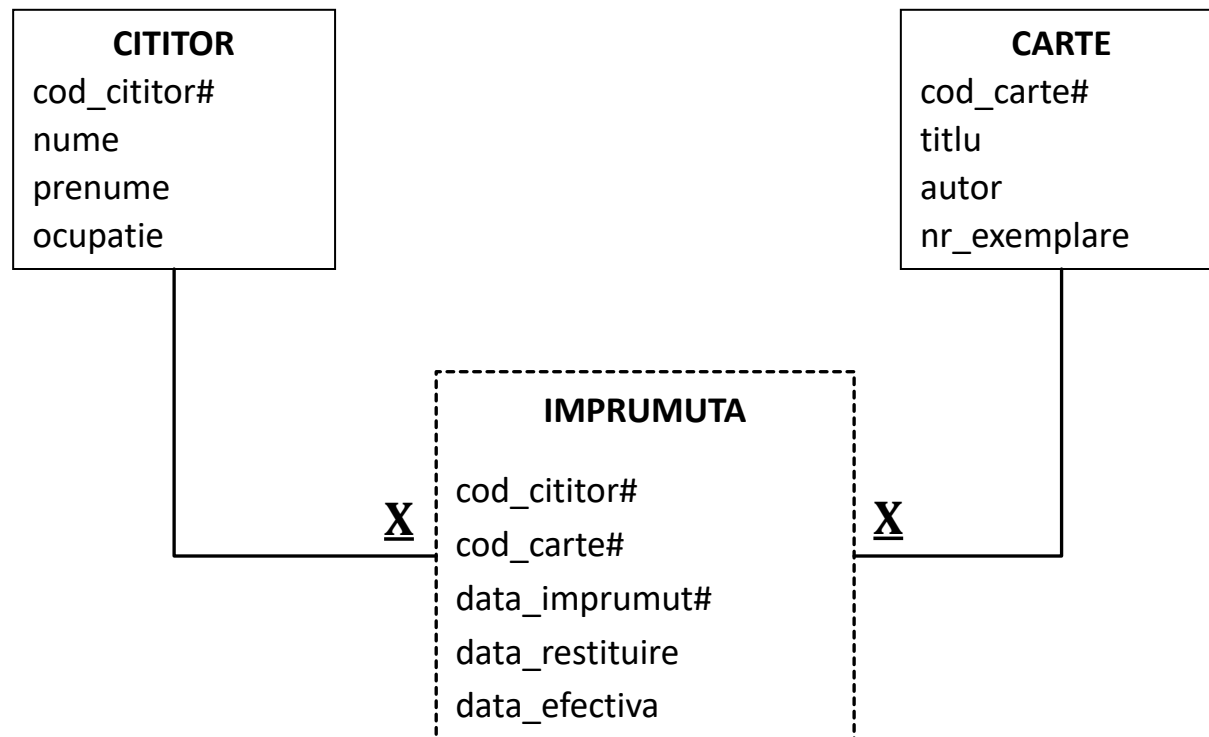


EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca



Cum se poate proiecta baza de date stiind ca un cititor poate sa imprumute aceeasi carte de mai multe ori?

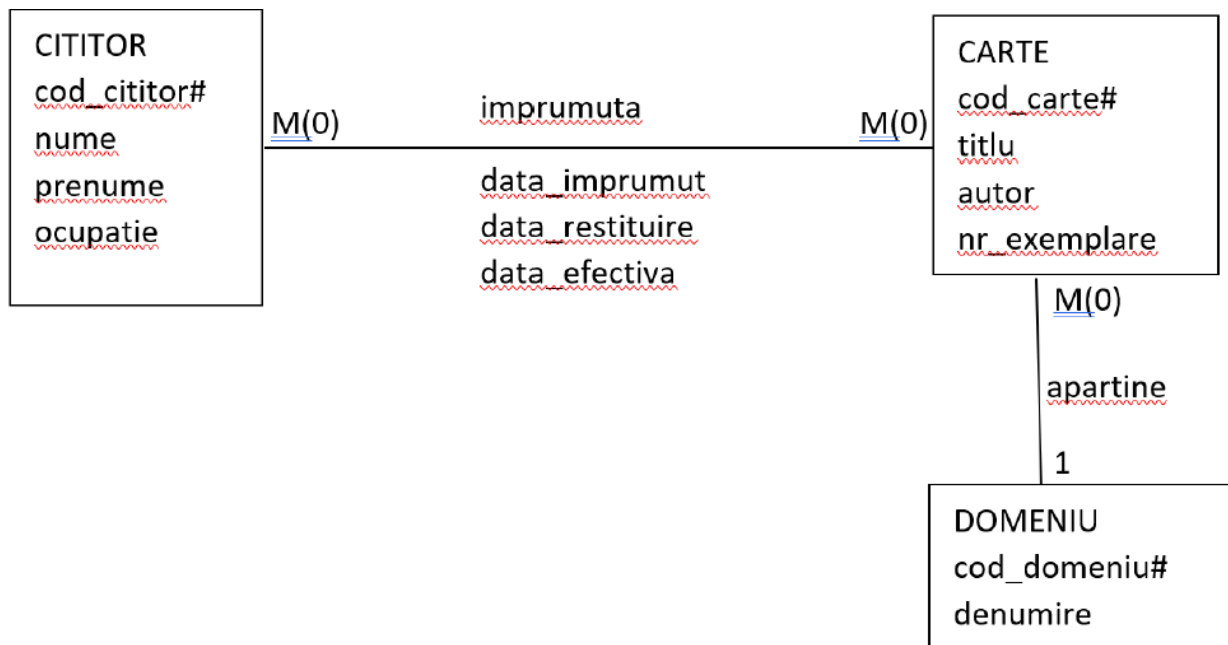
EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca



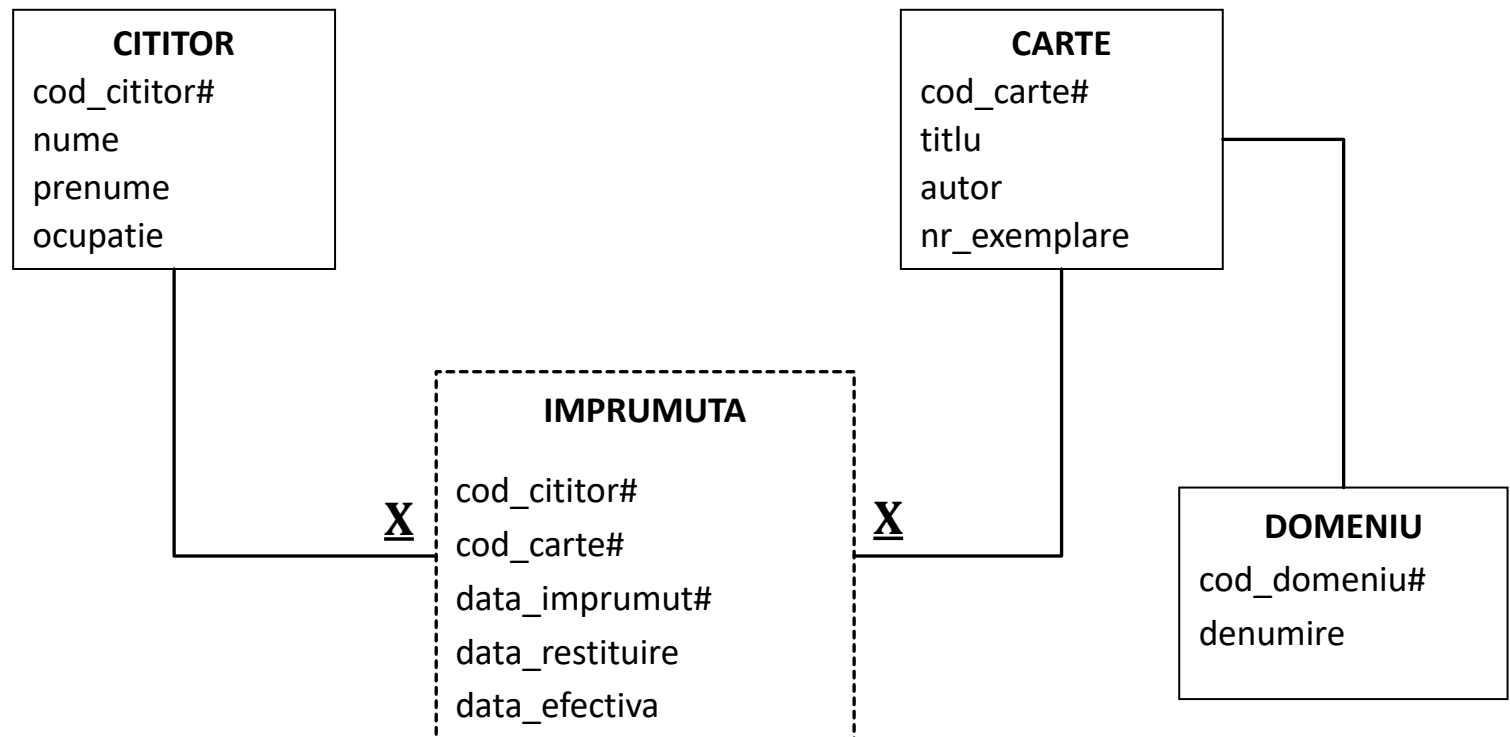
Raspuns: Tabelul asociativ **IMPRUMUTA** o sa aiba o **cheie primara compusa** (`cod_cititor`, `cod_carte`, `data_imprumut`)

EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca

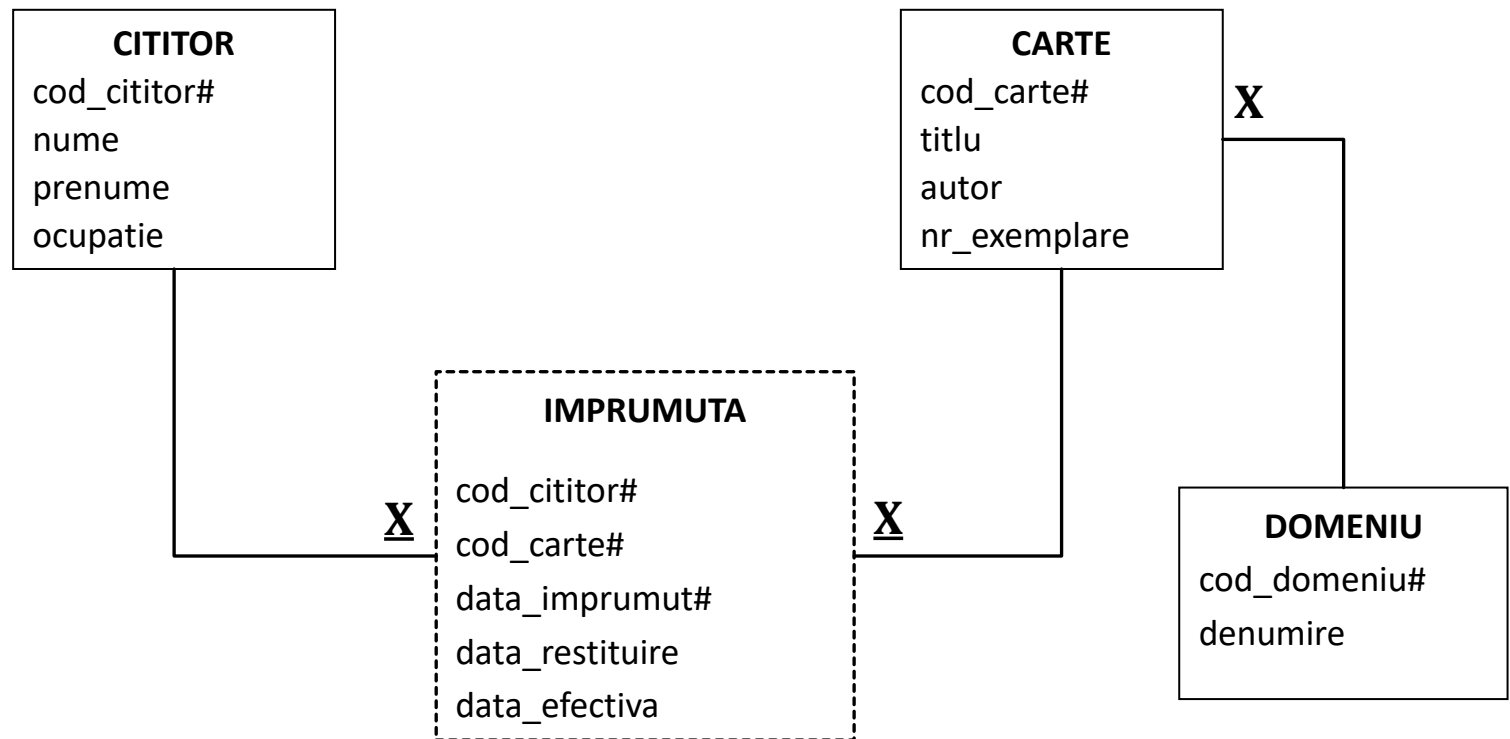
Cum se transforma tabelul **DOMENIU** impreuna cu relatia **APARTINE**?



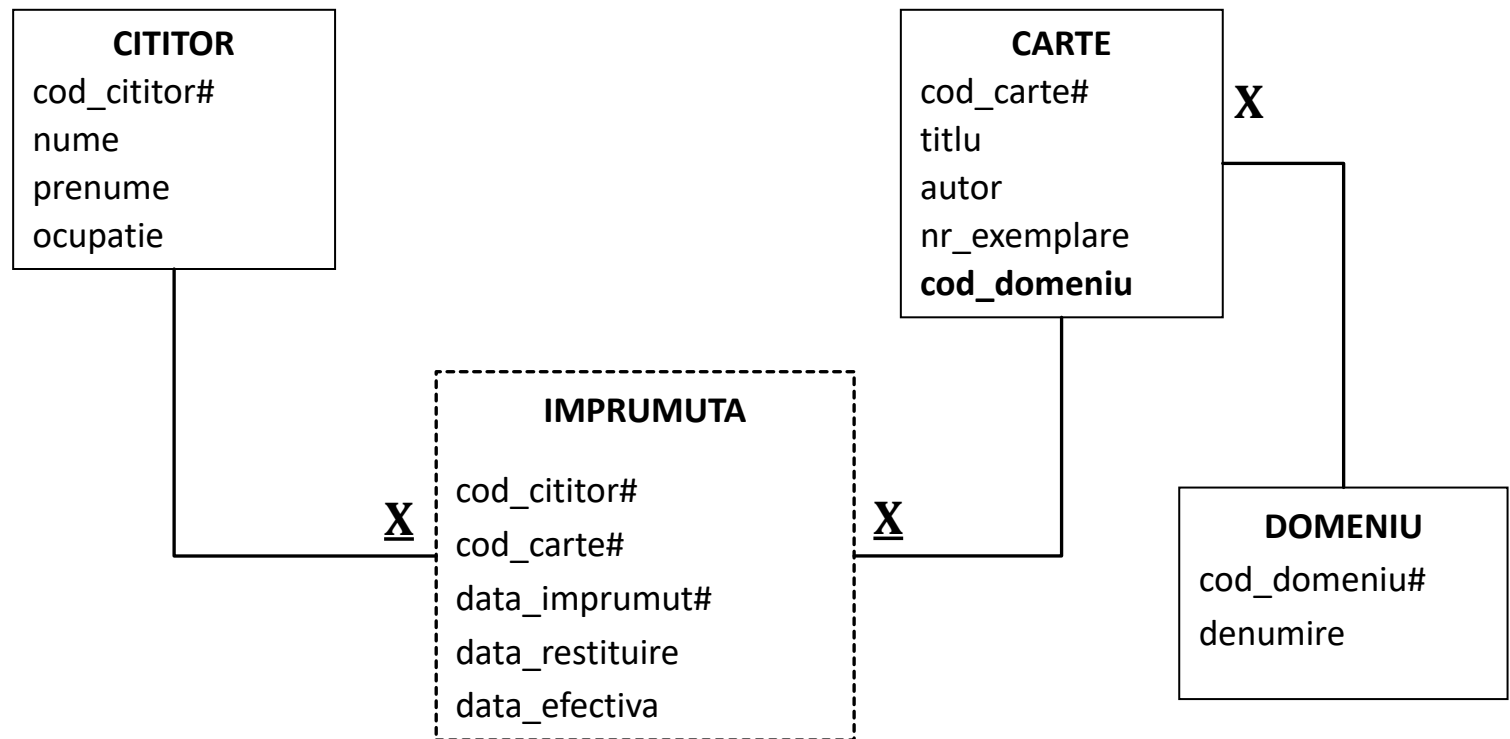
EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca



EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca

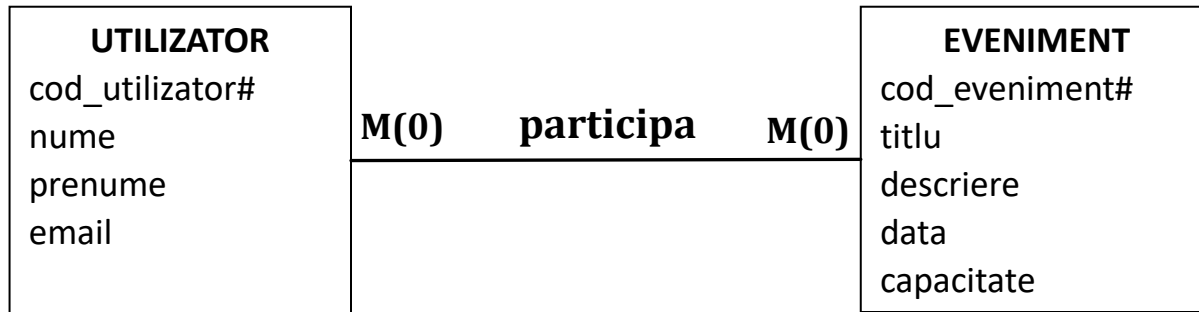


EXEMPLUL 1: Gestiunea activitatilor de imprumut dintr-o biblioteca



Se pot trece in tabel si cheile externe. De exemplu, cheia externa **cod_domeniu** din CARTE.

EXEMPLUL 2: Aplicatie de organizare de evenimente – Diagrama E/R



Ce semnifica acea cardinalitate M(0) din perspectiva implementarii?

- Cum se proiecteaza Diagrama Conceptuala?
- Cum se implementeaza la nivel de backend?
- Cum se implementeaza la nivel de frontend?

CUM SE PROIECTEAZA DIAGRAMA CONCEPTUALA?

UTILIZATOR

cod_utilizator#
nume
prenume
email

EVENIMENT

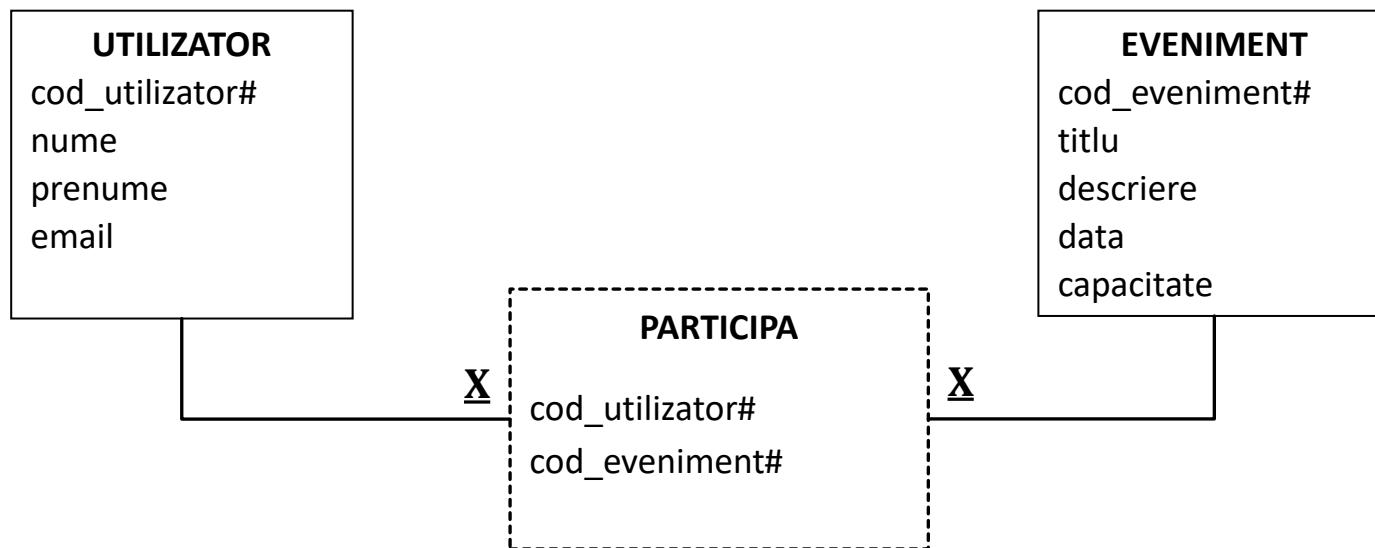
cod_eveniment#
titlu
descriere
data
capacitate

Ce fel de tabel proiectam?

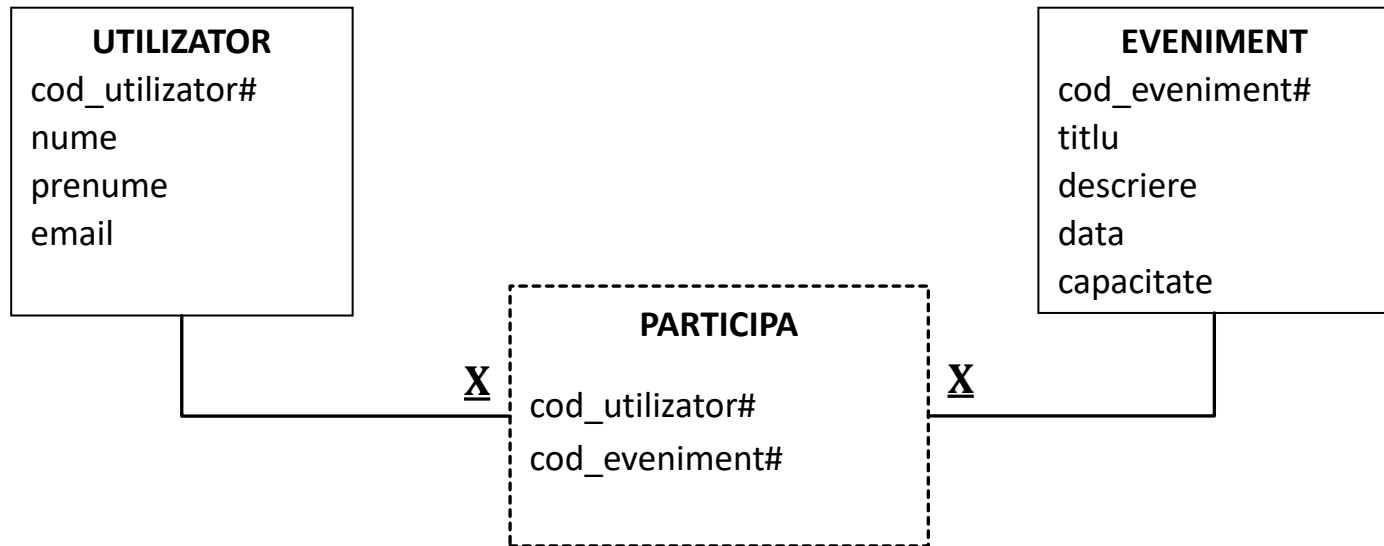
Care este cheia primara a tabelului?

Ce simbol folosim si unde se plaseaza acesta?

CUM SE PROIECTEAZA DIAGRAMA CONCEPTUALA?

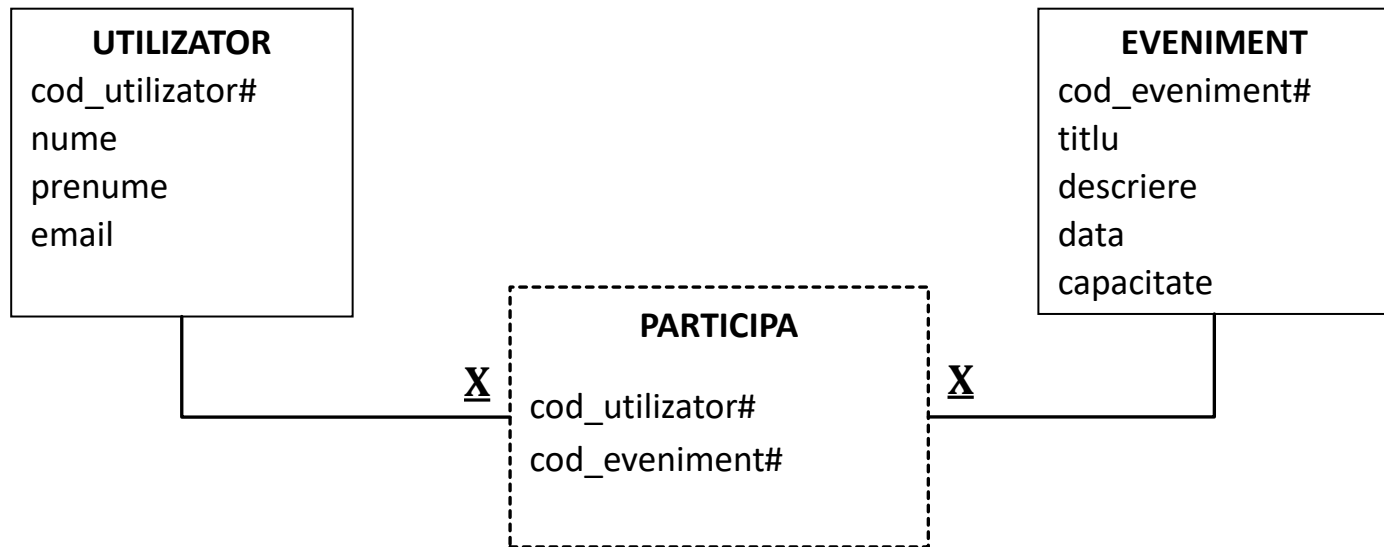


CUM SE IMPLEMENTEAZA LA NIVEL DE BACKEND – la nivel logic?



Ce reprezinta M(0)?

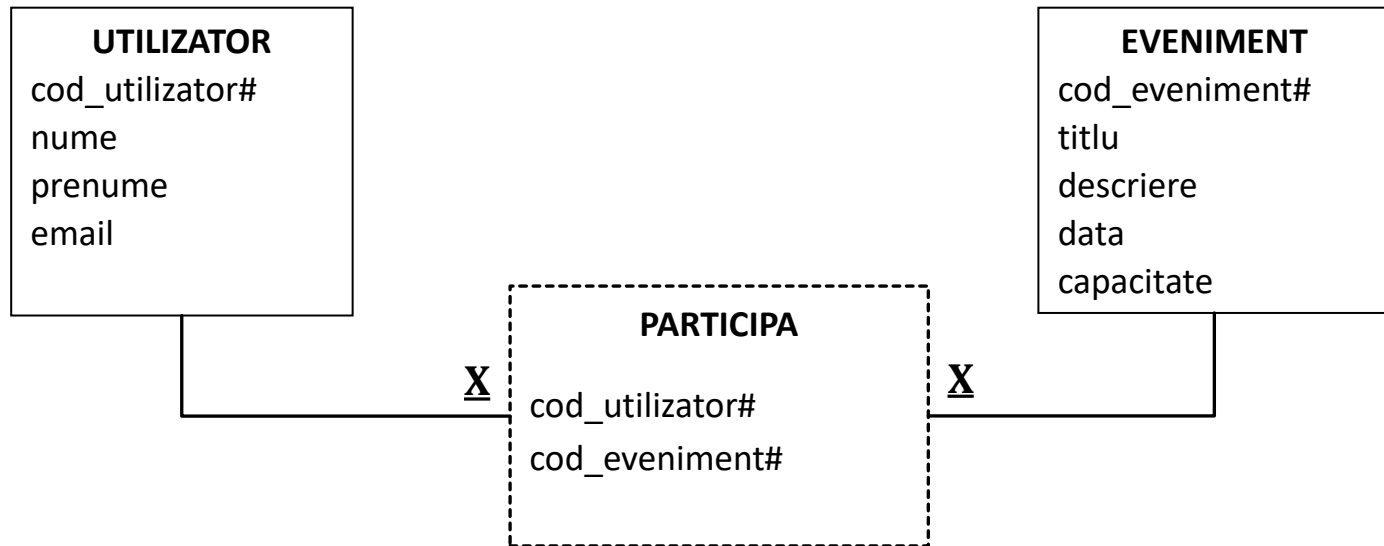
CUM SE IMPLEMENTEAZA LA NIVEL DE BACKEND – la nivel logic?



La nivel de implementare/aplicatie – nu este obligatoriu ca atunci cand un user este adaugat in tabelul **UTILIZATOR** (se inregistreaza in platforma) sa se insereze si in tabelul asociativ **PARTICIPA**.

La fel se intampla si in cazul inserarii unui nou eveniment in baza de date.

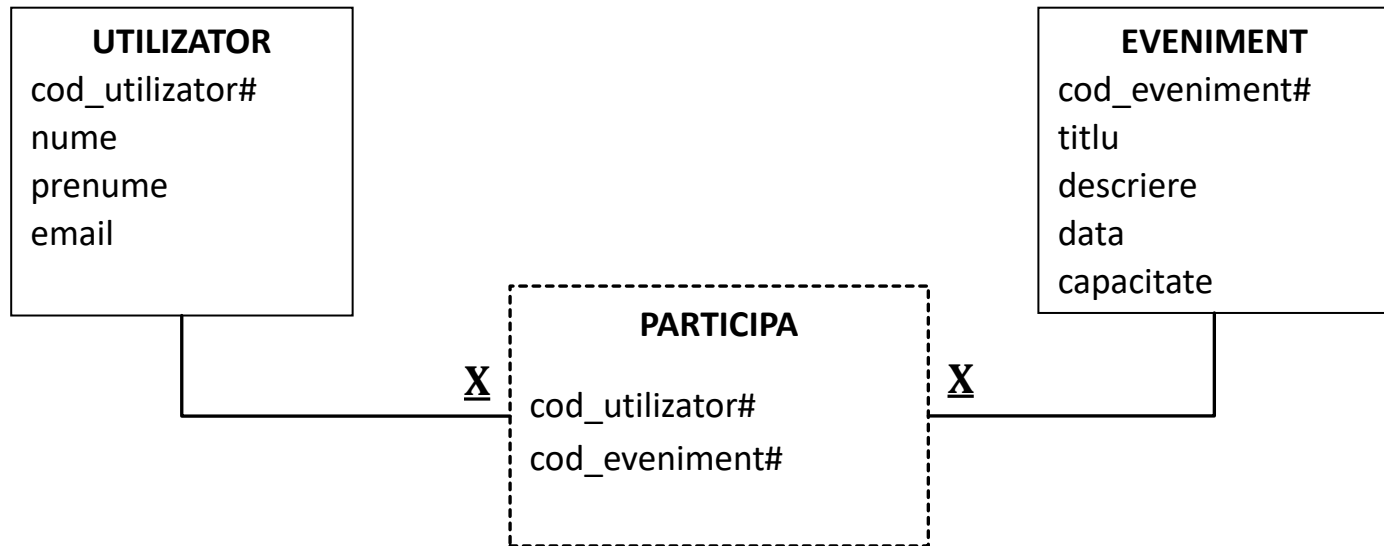
CUM SE IMPLEMENTEAZA LA NIVEL DE BACKEND – la nivel logic?



Daca relatiile erau M(1), atunci era obligatoriu ca in momentul inserarii unui nou user in baza de date, sa se insereze si in tabelul asociativ.

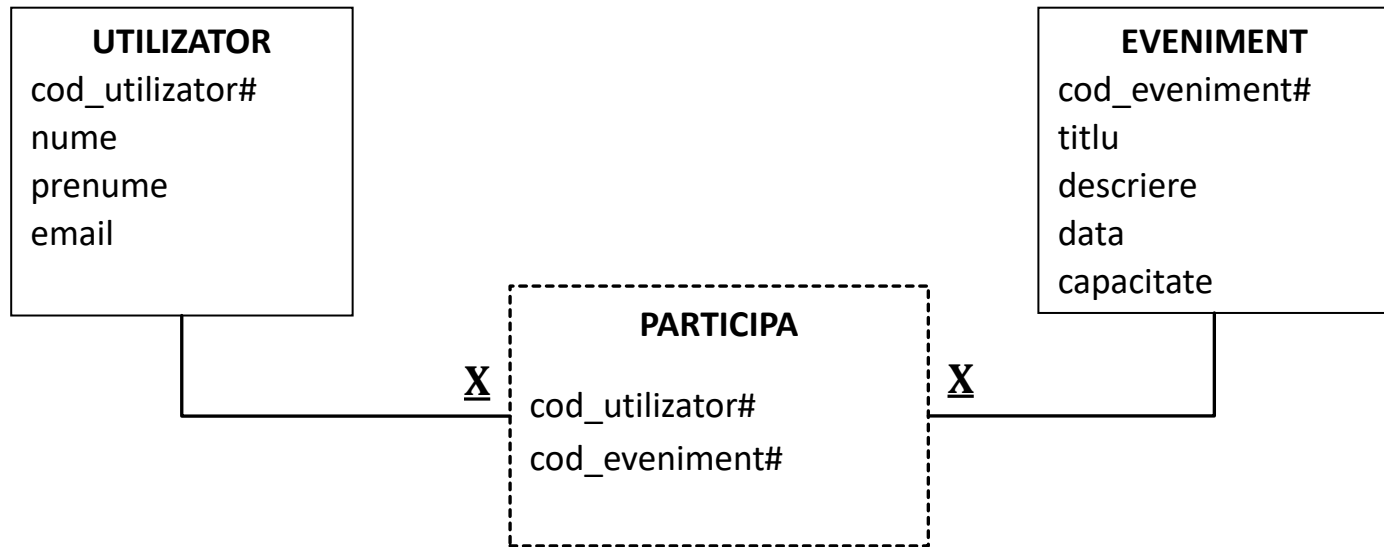
In cazul evenimentului, relatia corecta este intotdeauna M(0) deoarece nu se doreste ca in momentul adaugarii evenimentului, sa se adauge si utilizatorii care vor participa la eveniment.

CUM SE IMPLEMENTEAZA LA NIVEL DE FRONTEND



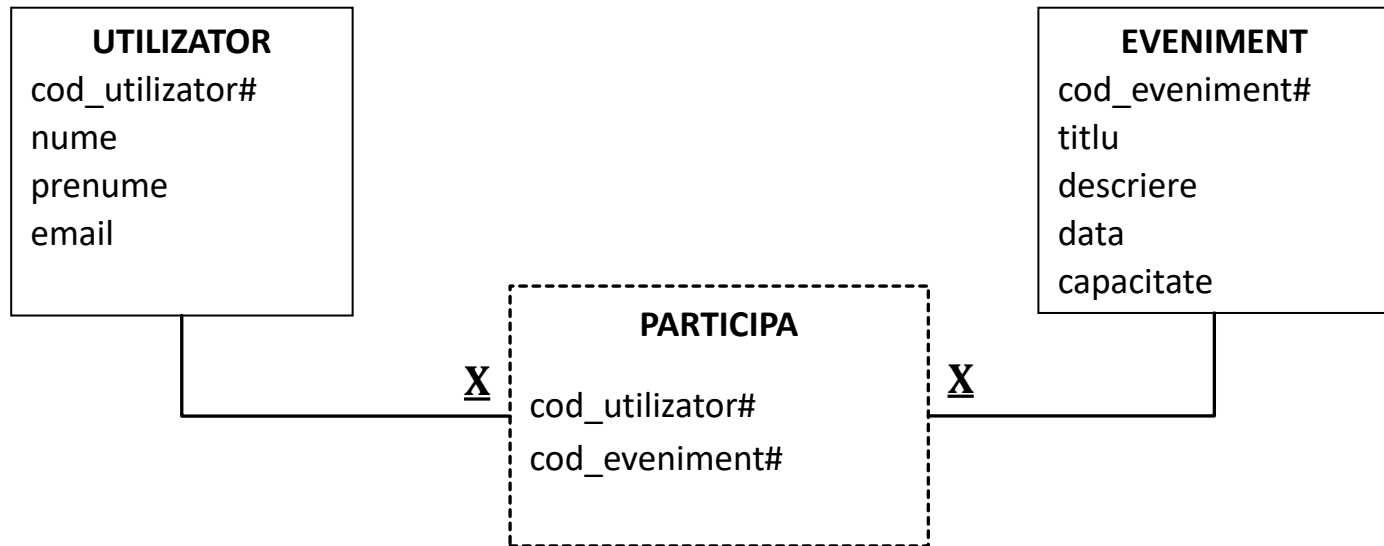
Pentru utilizator –
Pentru eveniment –

CUM SE IMPLEMENTEAZA LA NIVEL DE FRONTEND



Pentru utilizator – in momentul in care un utilizator isi completeaza datele in platforma (insert in UTILIZATOR) – pentru relatia M(0) acesta nu este obligat sa aleaga si un eveniment la care sa participe, deci nu se realizeaza inserarea si in PARTICIPA; pentru relatia M(1) utilizatorul o sa fie obligat sa selecteze si un eveniment, deci se insereaza si asocierea dintre utilizator si eveniment in tabelul asociativ PARTICIPA

CUM SE IMPLEMENTEAZA LA NIVEL DE FRONTEND



Pentru eveniment – relatia corecta este M(0) deci se pot insera evenimente independent de utilizatori. Inserarea in PARTICIPA se realizeaza ulterior.



SCHEMELE RELATIONALE - Salariat

Schemele relaționale corespunzătoare acestei diagrame conceptuale sunt următoarele:

- **SALARIAT**(cod_salariat#, salariu, nume, job_cod, prenume, cod_sef, nr_depart);
- **DEPARTAMENT**(cod_departament#, nume, numar_cladire, cod_sef);
- **ATASAT_LA**(cod_salariat#, nr_proiect#, functie);
- **PROIECT**(nr_proiect#, descriere, buget_alocat);
- **SARCINA**(nr_proiect#, nr_sarcina#, data_inceperii, stare);
- **AGENT_TERITORIAL**(cod_salariat#, zona, comision);
- **PROGRAMATOR**(cod_salariat#, limbaj, nivel);
- **TELEFON**(cod_salariat#, nr_telefon#);

EXEMPLUL 4: DIAGRAMA E/R - Editura

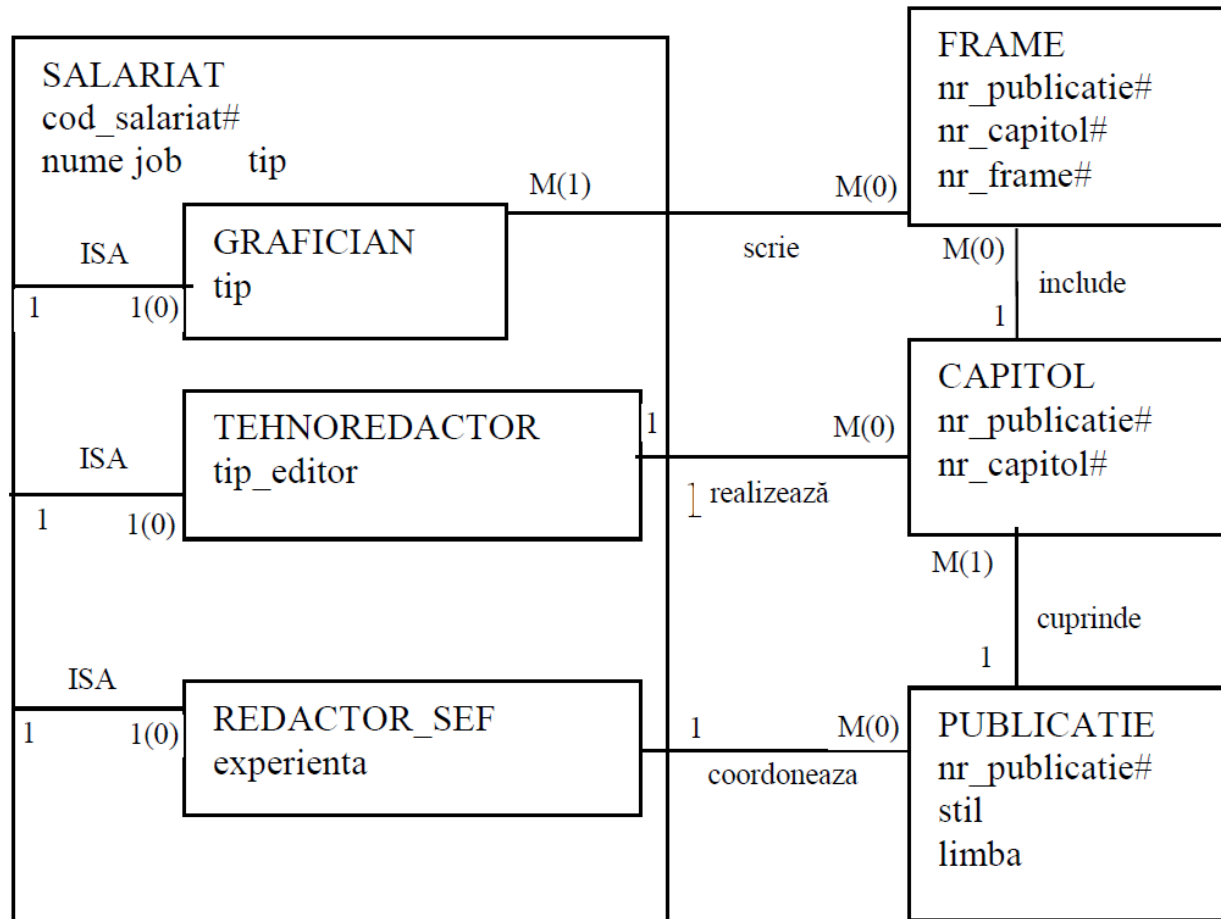
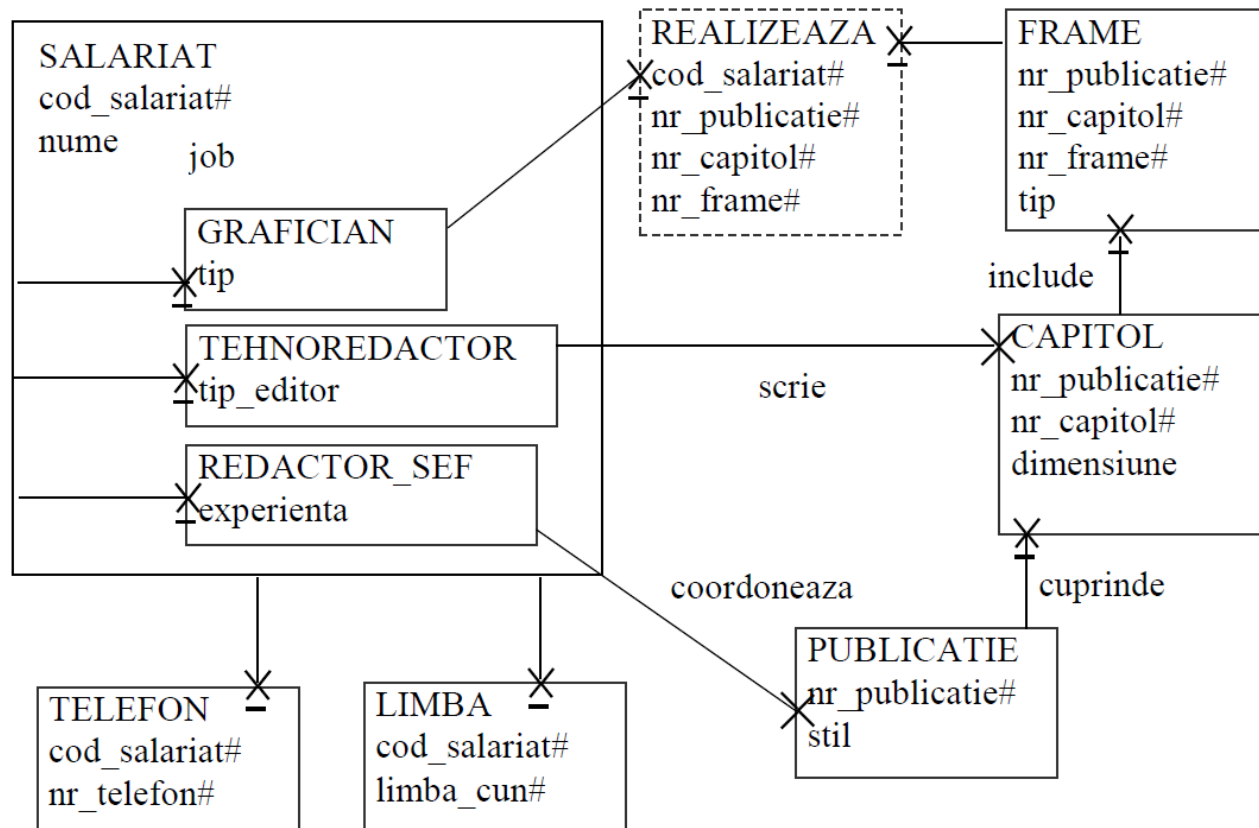


DIAGRAMA CONCEPTUALA - Editura



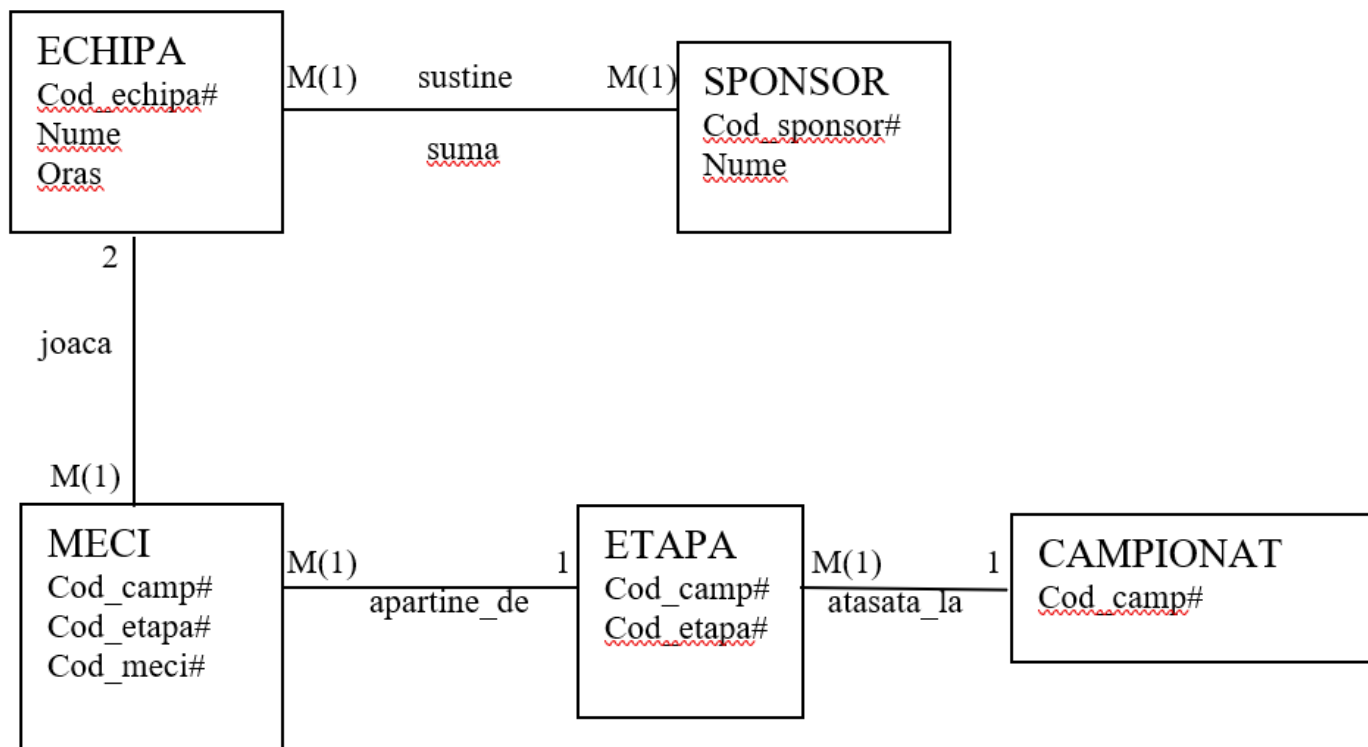
SCHEMELE RELATIONALE - Editura

Schemele relaționale:

- SALARIAT(cod_salariat#, nume, prenume, data_angajare, salariu, job);
- GRAFICIAN(cod_salariat#, tip);
- TEHNOREDACTOR(cod_salariat#, tip_editor);
- REDACTOR_SEF(cod_salariat#, experienta);
- LIMBA(cod_salariat#, limba_cunoscuta#);
- TELEFON(cod_salariat#, nr_telefon#);
- REALIZEAZA(cod_salariat#, nr_frame#, nr_publicatie#, nr_capitol#, data_inceput, data_limita);
- FRAME(nr_frame#, nr_publicatie#, nr_capitol#, tip, format);
- CAPITOL(nr_publicatie#, nr_capitol#, dimensiune, cod_salariat);
- PUBLICATIE(nr_publicatie#, stil, cod_salariat, autor, cost, titlu);

EXEMPLUL 5: Campionate Fotbal

Transformați următoarea Diagramă E/R în Diagramă Conceptuală și enumerați Schemele Relaționale:



EXEMPLUL 5: Campionate Fotbal

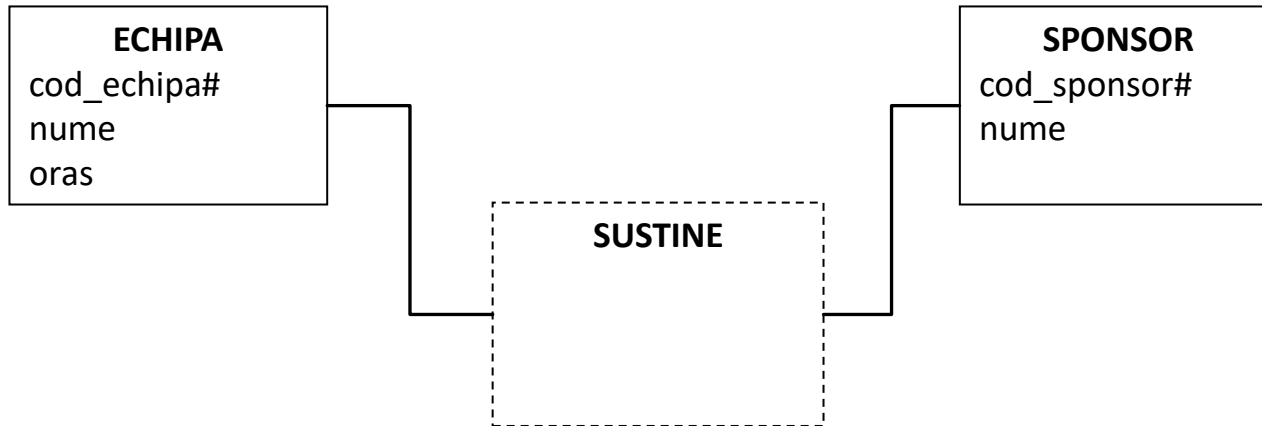
ECHIPA

cod_echipa#
nume
oras

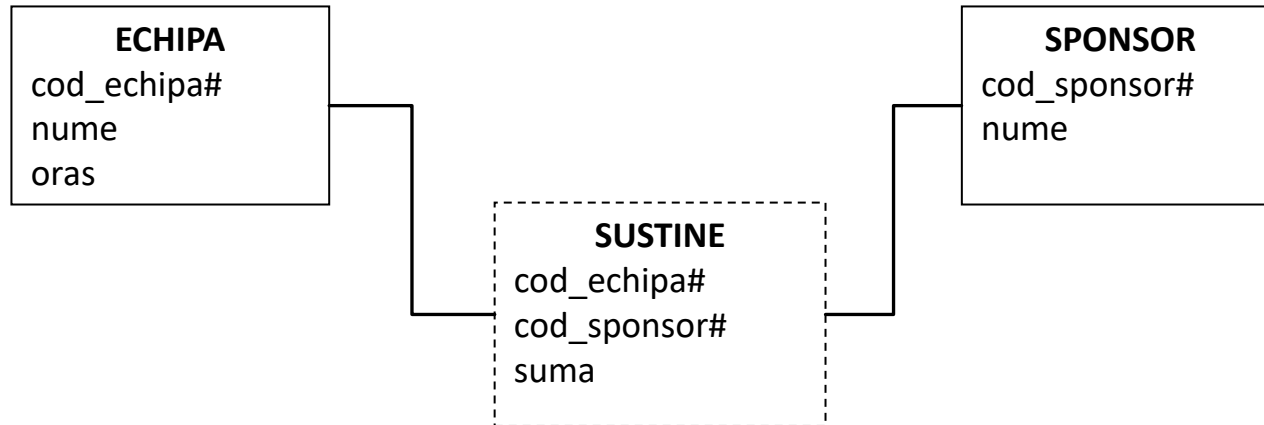
SPONSOR

cod_sponsor#
nume

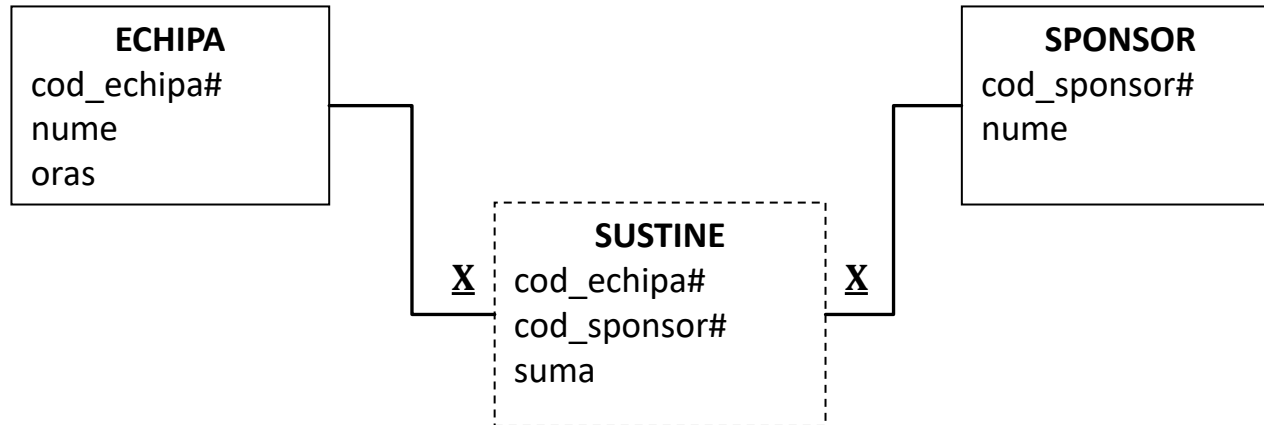
EXEMPLUL 5: Campionate Fotbal



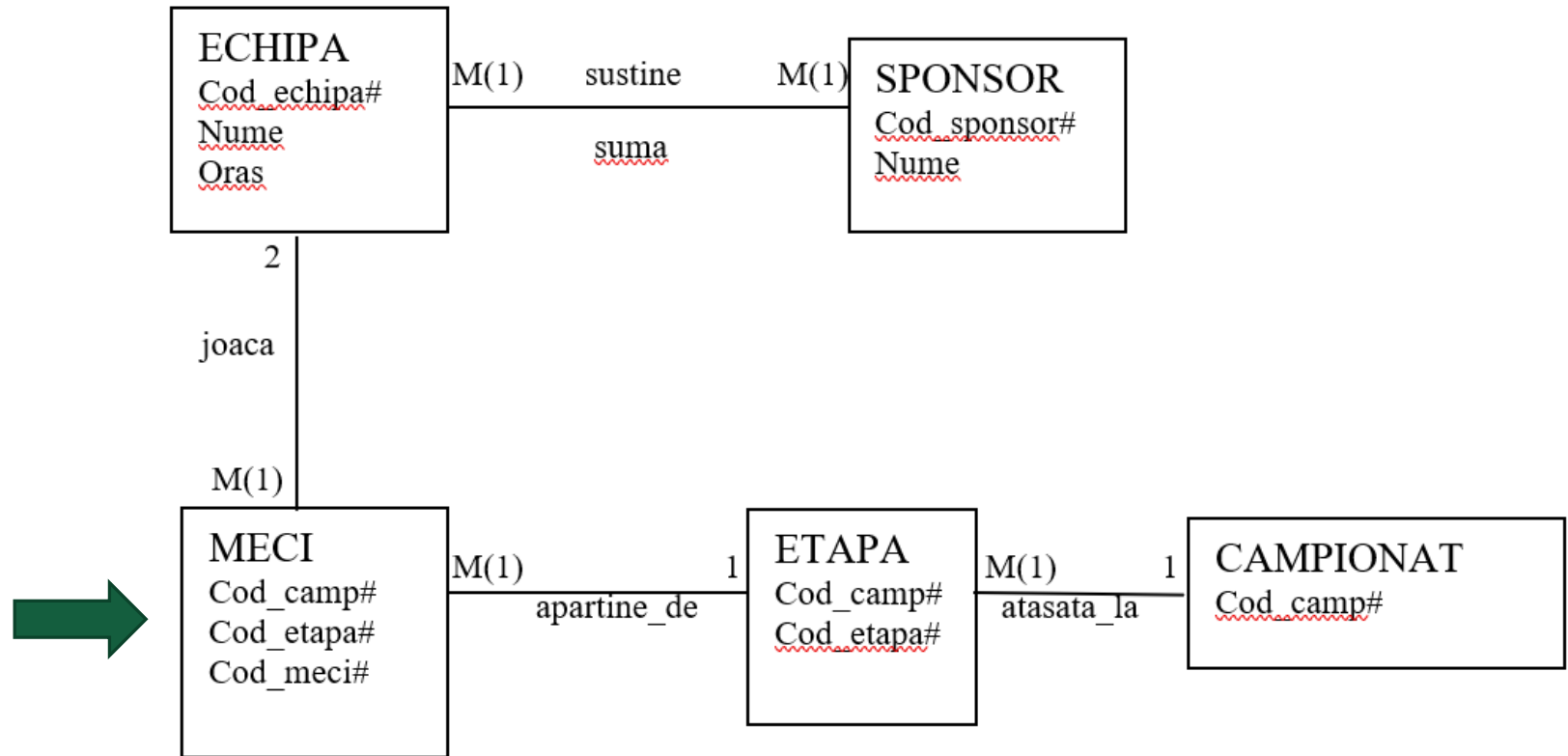
EXEMPLUL 5: Campionate Fotbal



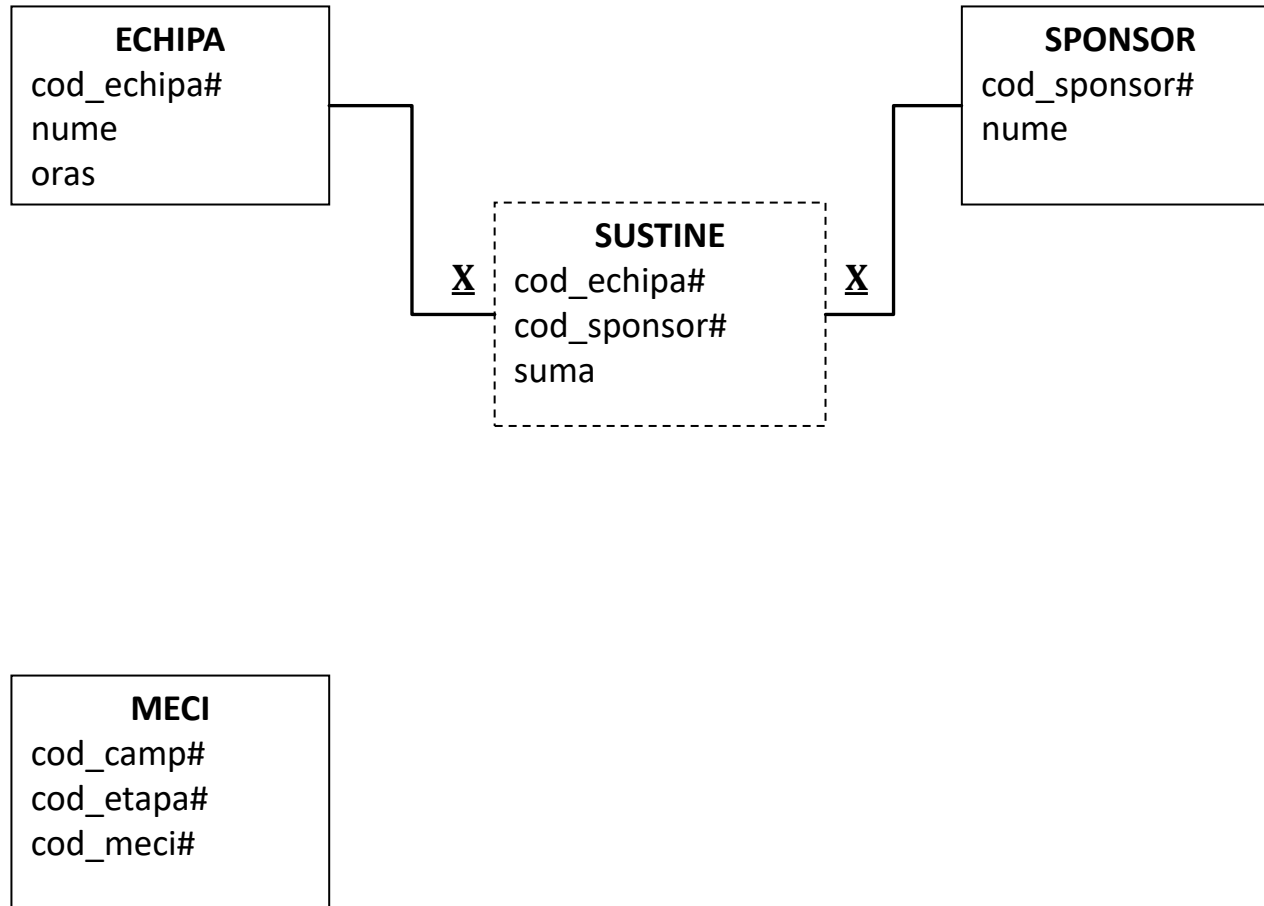
EXEMPLUL 5: Campionate Fotbal



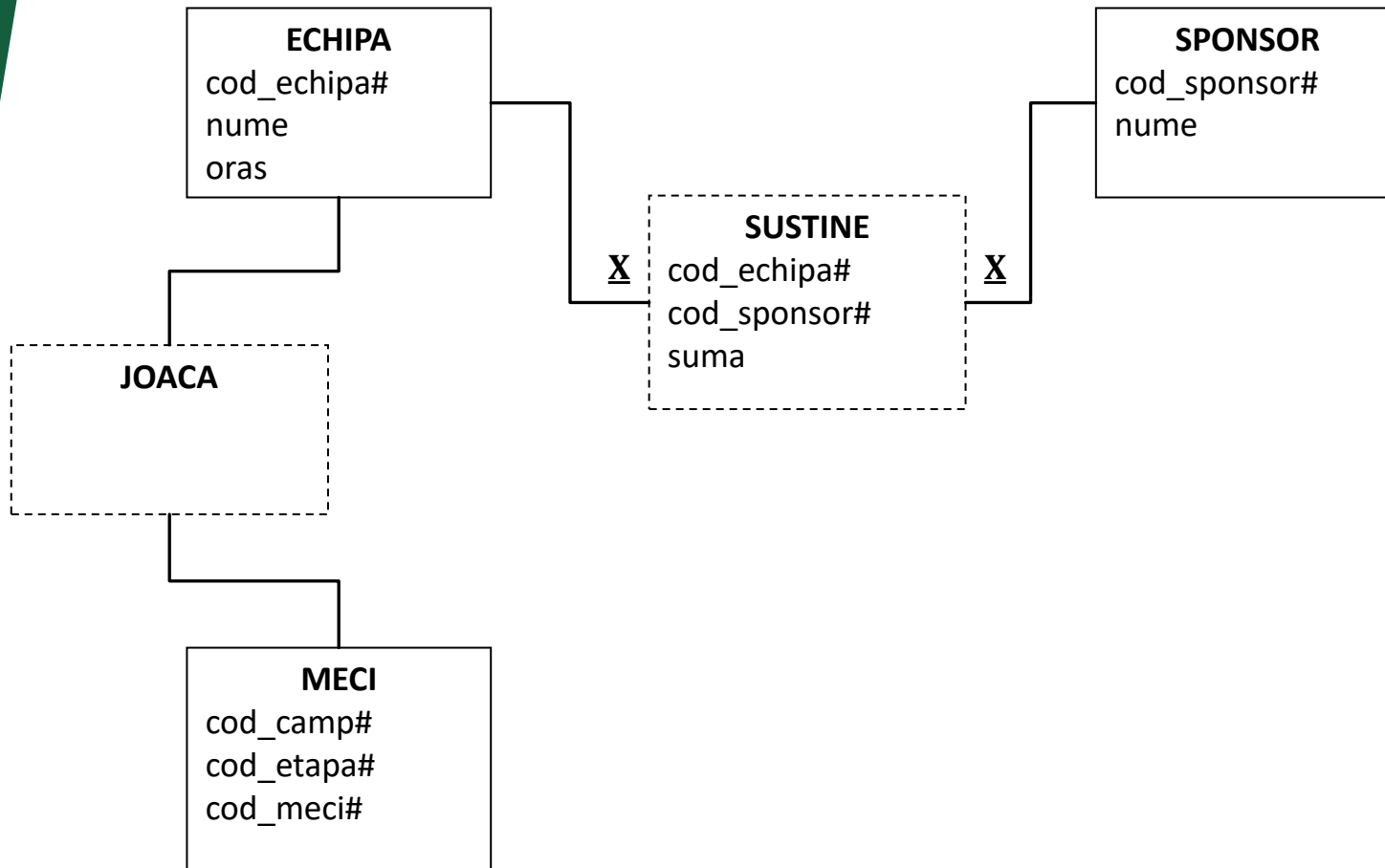
EXEMPLUL 5: Campionate Fotbal



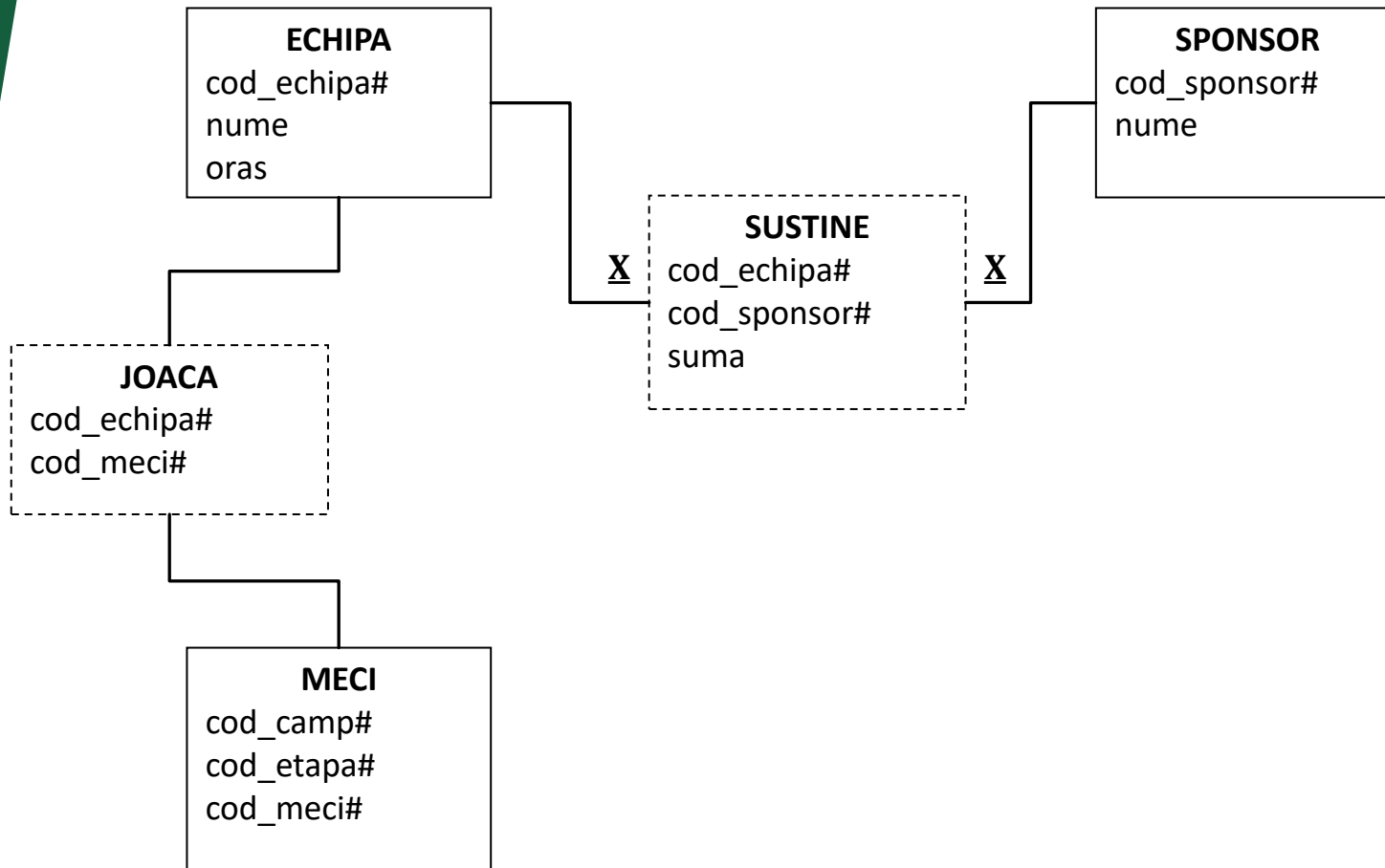
EXEMPLUL 5: Campionate Fotbal



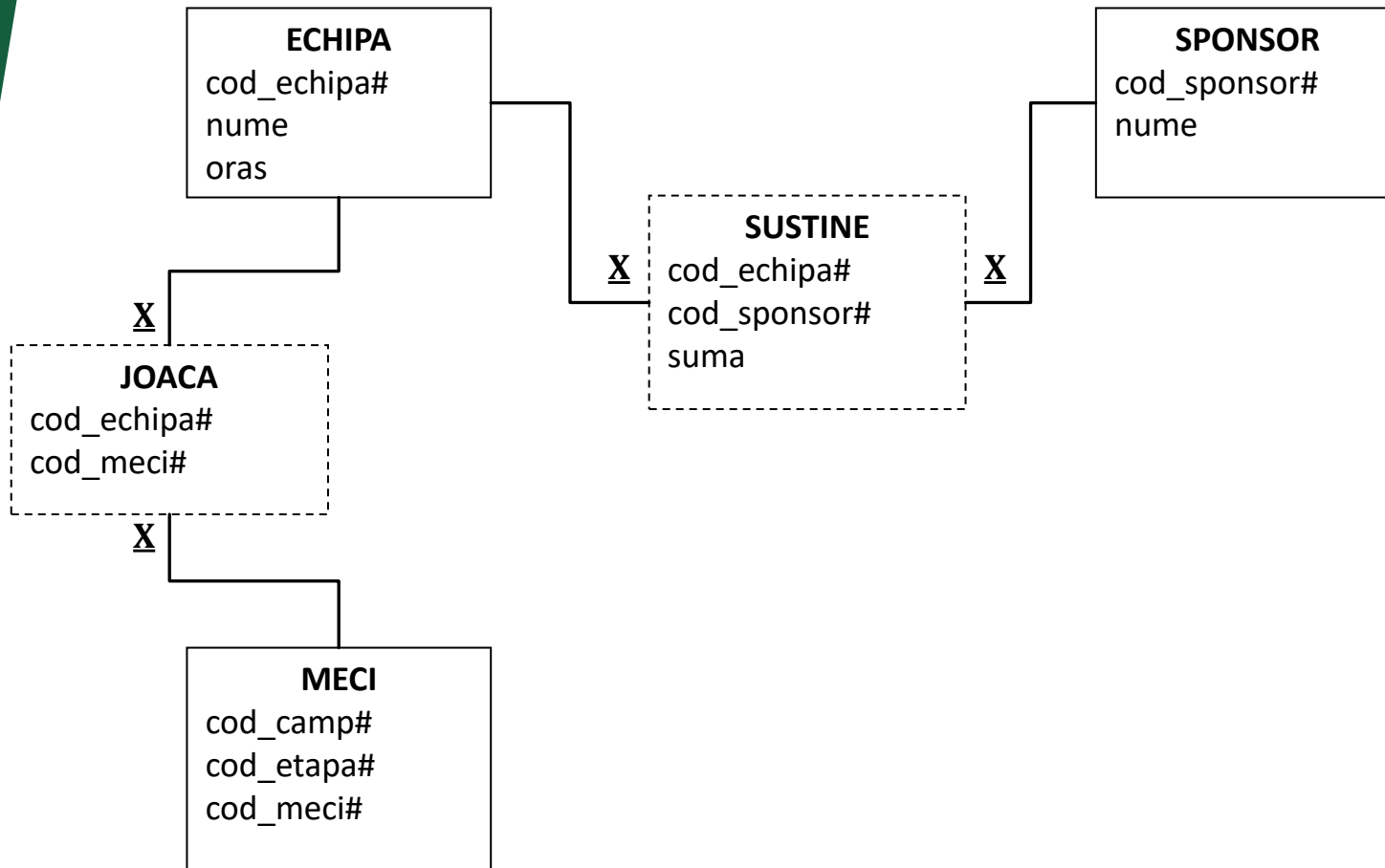
EXEMPLUL 5: Campionate Fotbal



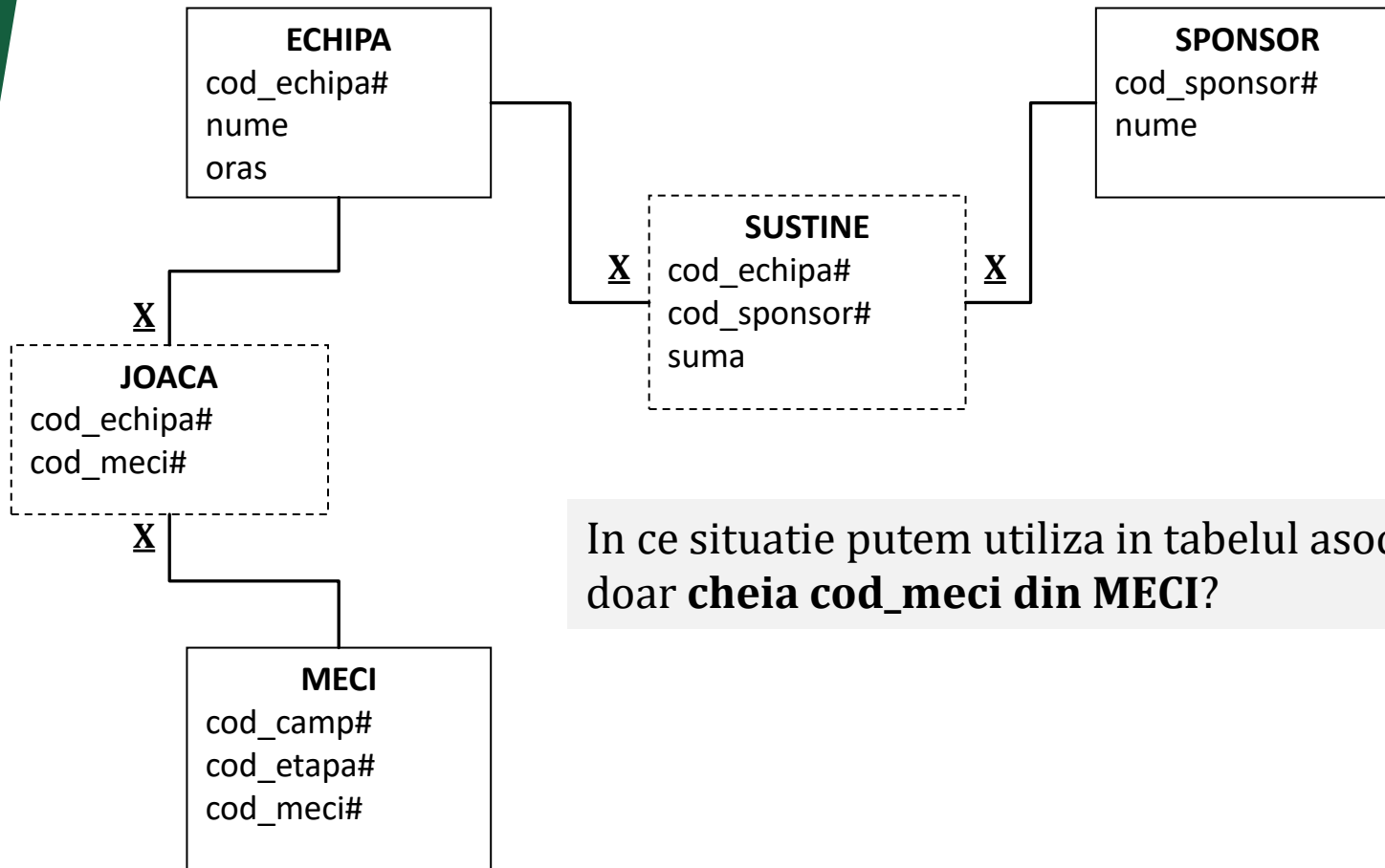
EXEMPLUL 5: Campionate Fotbal



EXEMPLUL 5: Campionate Fotbal

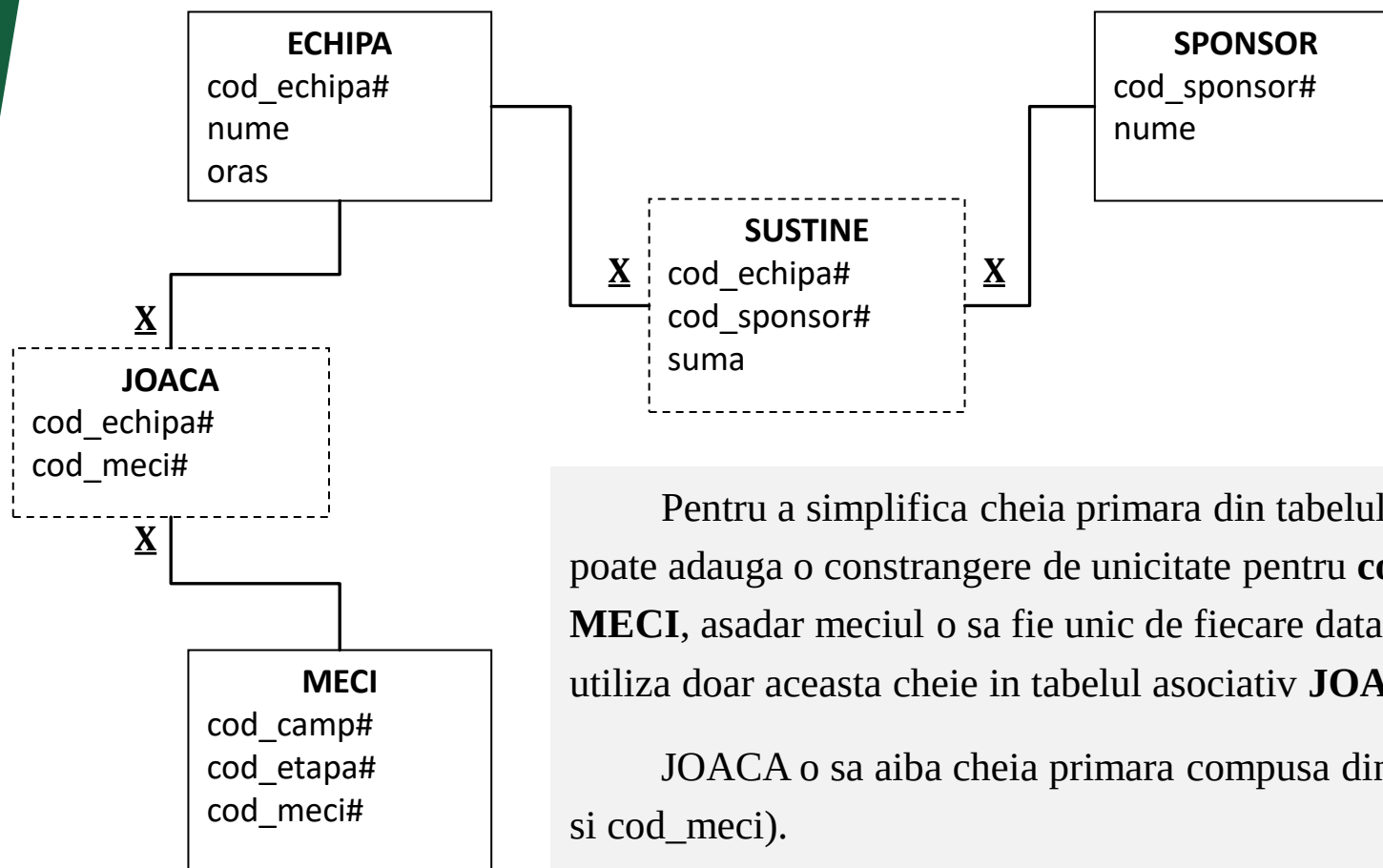


EXEMPLUL 5: Campionate Fotbal



In ce situatie putem utiliza in tabelul asociativ doar **cheia cod_meci din MECI**?

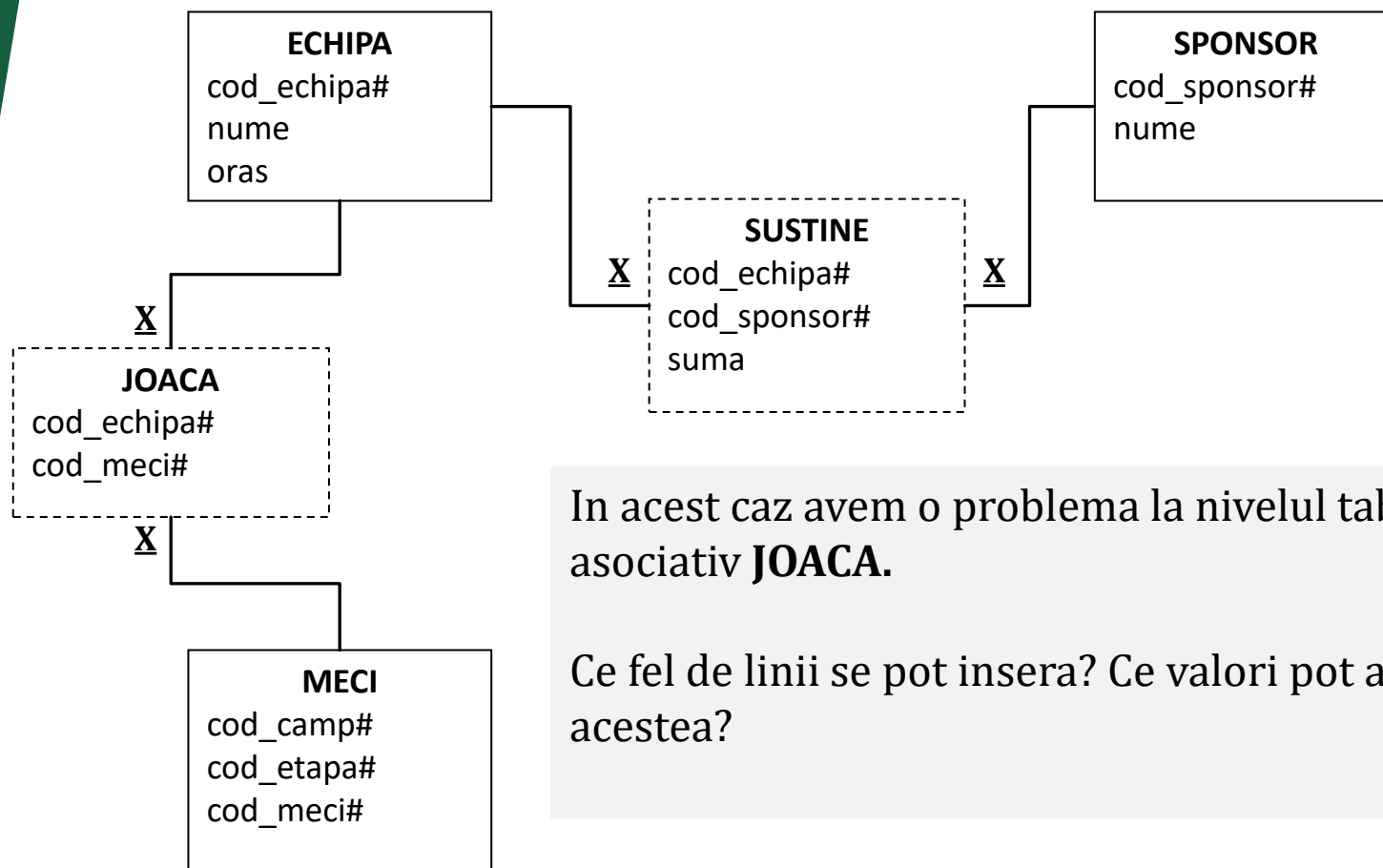
EXEMPLUL 5: Campionate Fotbal



Pentru a simplifica cheia primara din tabelul asociativ se poate adauga o constrangere de unicitate pentru **cod_meci din MECI**, asadar meciul o sa fie unic de fiecare data si se poate utiliza doar aceasta cheie in tabelul asociativ **JOACA**.

JOACA o sa aiba cheia primara compusa din (`cod echipa` si `cod_meci`).

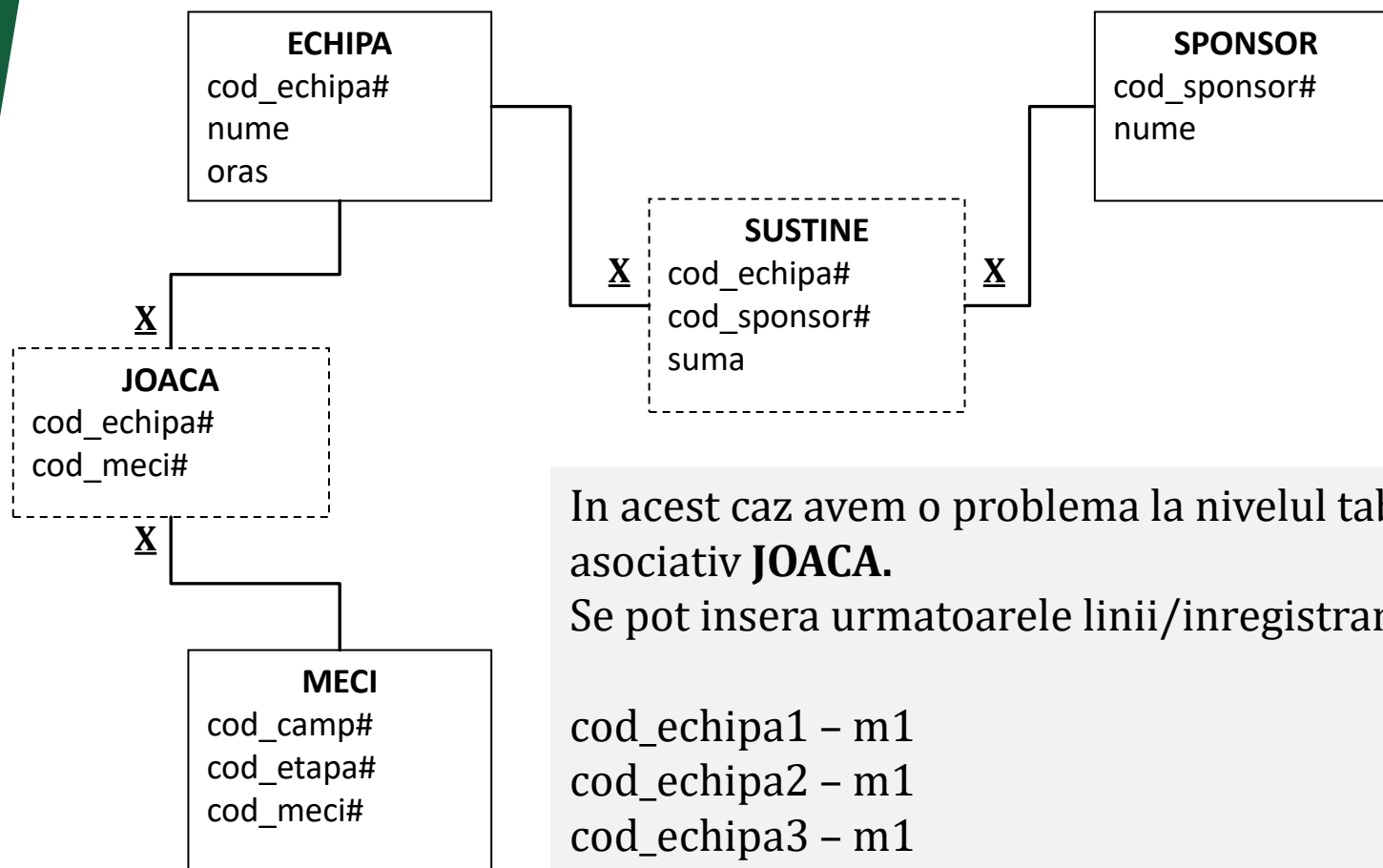
EXEMPLUL 5: Campionate Fotbal



In acest caz avem o problema la nivelul tabelului asociativ **JOACA**.

Ce fel de linii se pot insera? Ce valori pot avea acestea?

EXEMPLUL 5: Campionate Fotbal



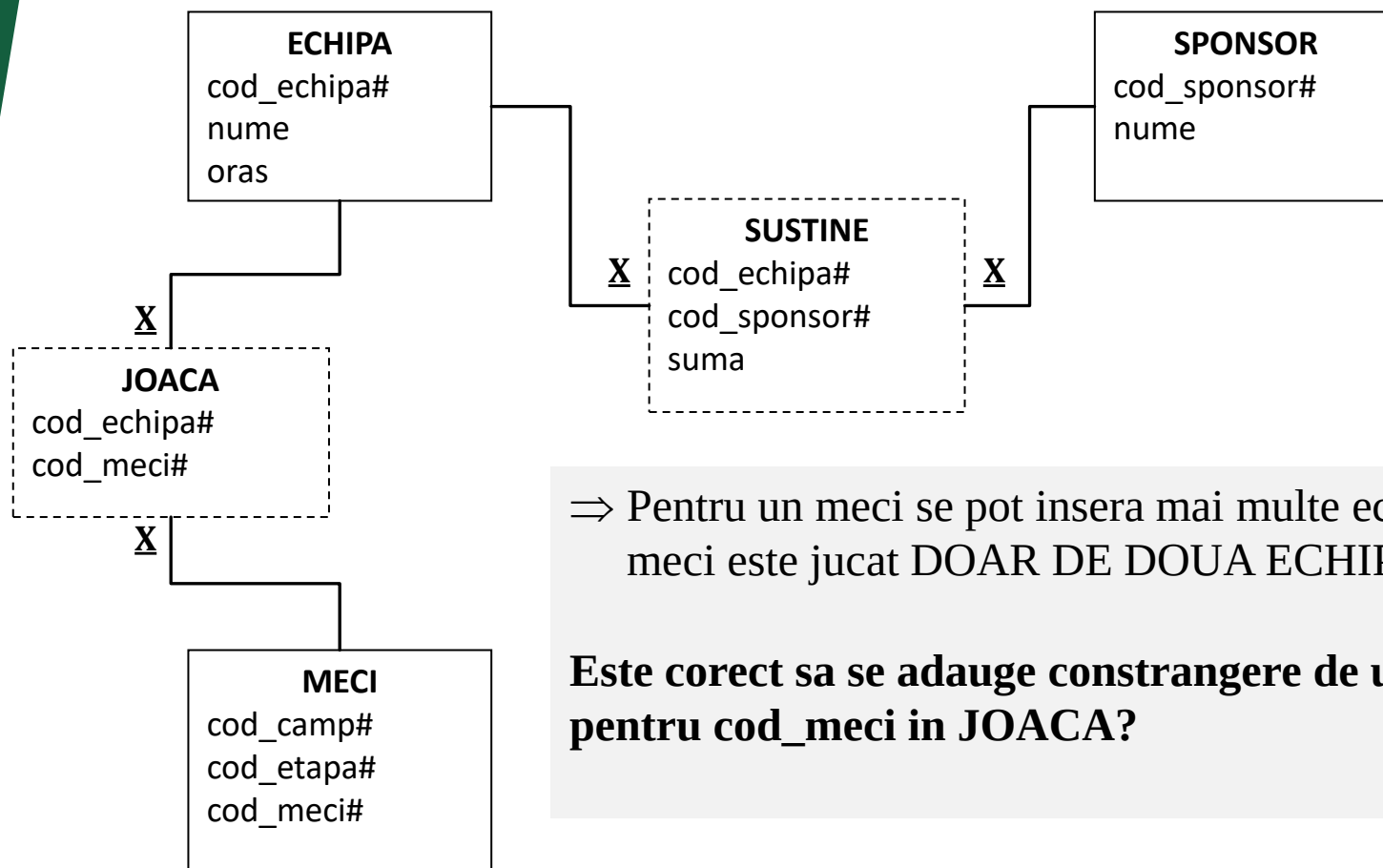
In acest caz avem o problema la nivelul tabelului asociativ **JOACA**.

Se pot insera urmatoarele linii/inregistrari:

cod_echipa1 – m1
cod_echipa2 – m1
cod_echipa3 – m1

Ce se observa?

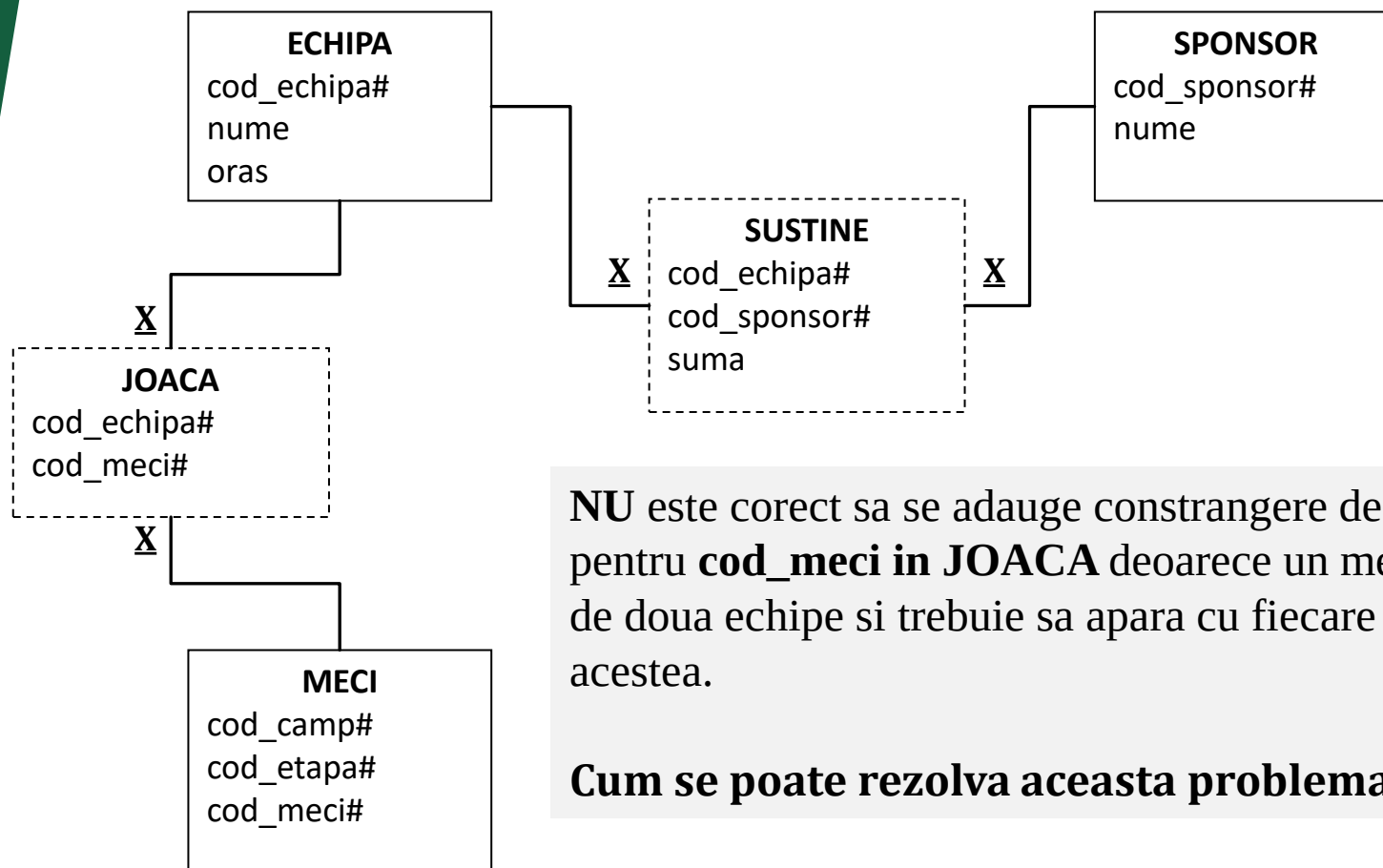
EXEMPLUL 5: Campionate Fotbal



⇒ Pentru un meci se pot insera mai multe echipe, dar un meci este jucat DOAR DE DOUA ECHIPE.

Este corect sa se adauge constrangere de unicitate pentru cod_meci in JOACA?

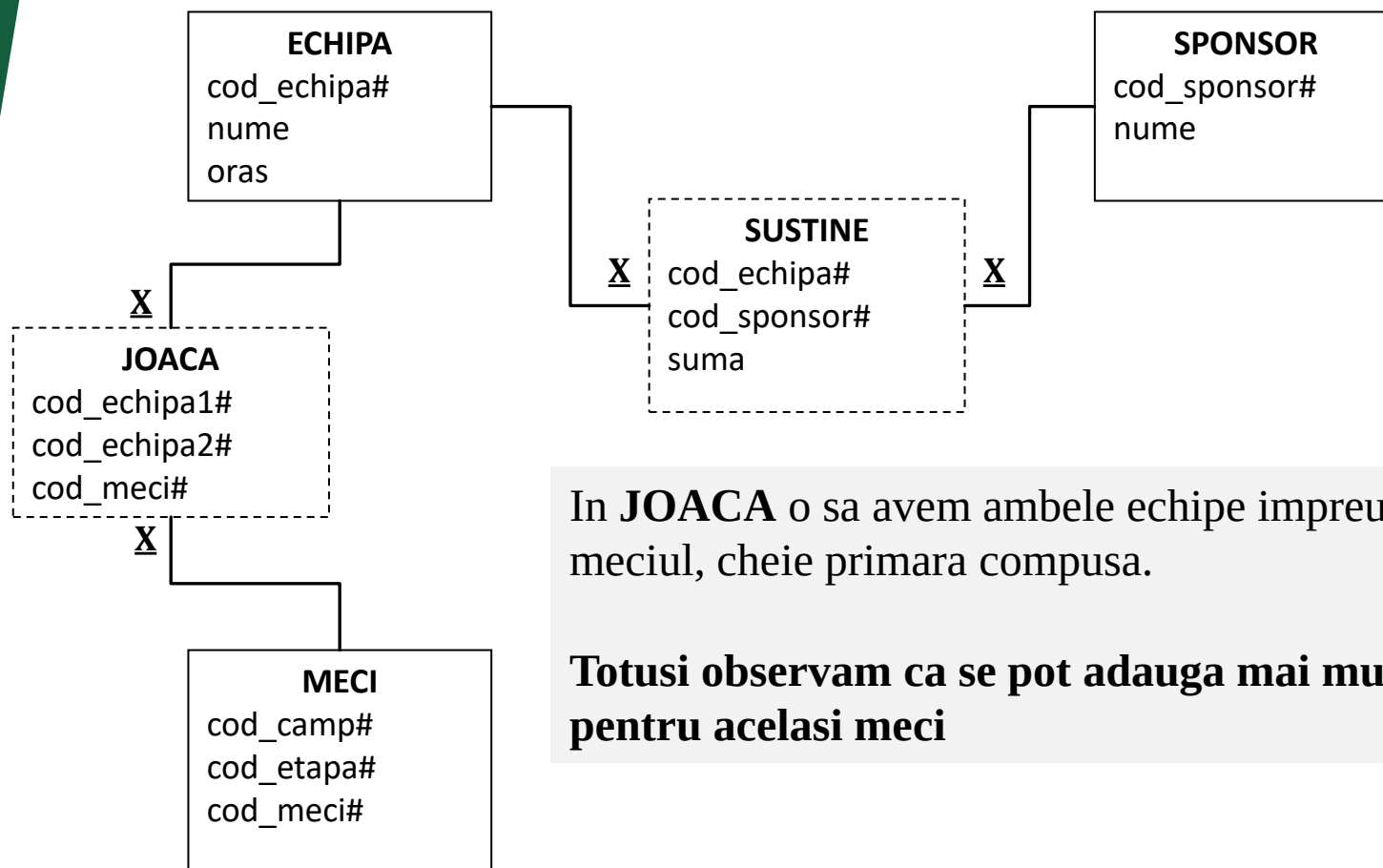
EXEMPLUL 5: Campionate Fotbal



NU este corect sa se adauge constrangere de unicitate pentru **cod_meci** in **JOACA** deoarece un meci este jucat de doua echipe si trebuie sa apara cu fiecare dintre acestea.

Cum se poate rezolva aceasta problema?

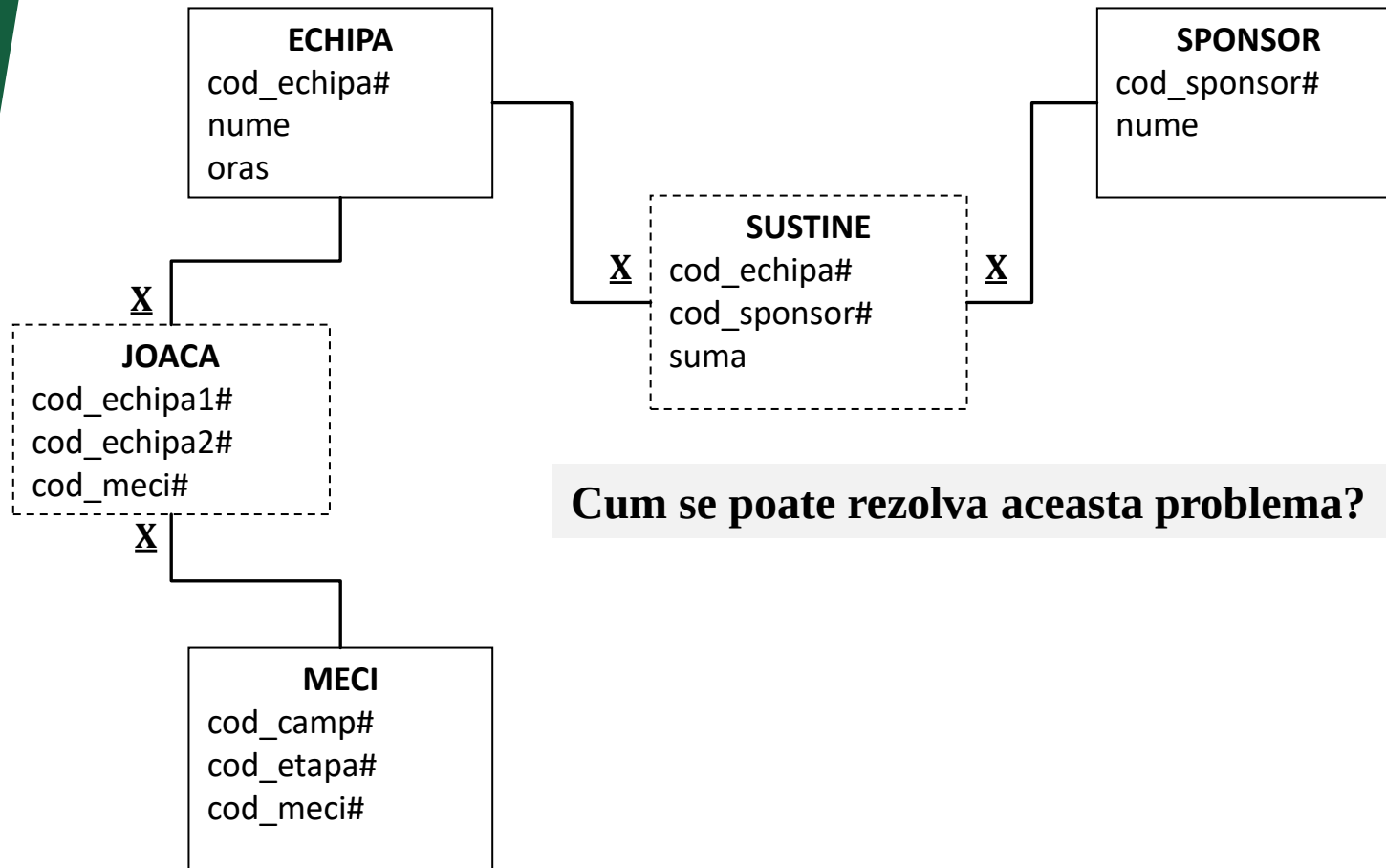
EXEMPLUL 5: Campionate Fotbal



In **JOACA** o sa avem ambele echipe impreuna cu meciul, cheie primara compusa.

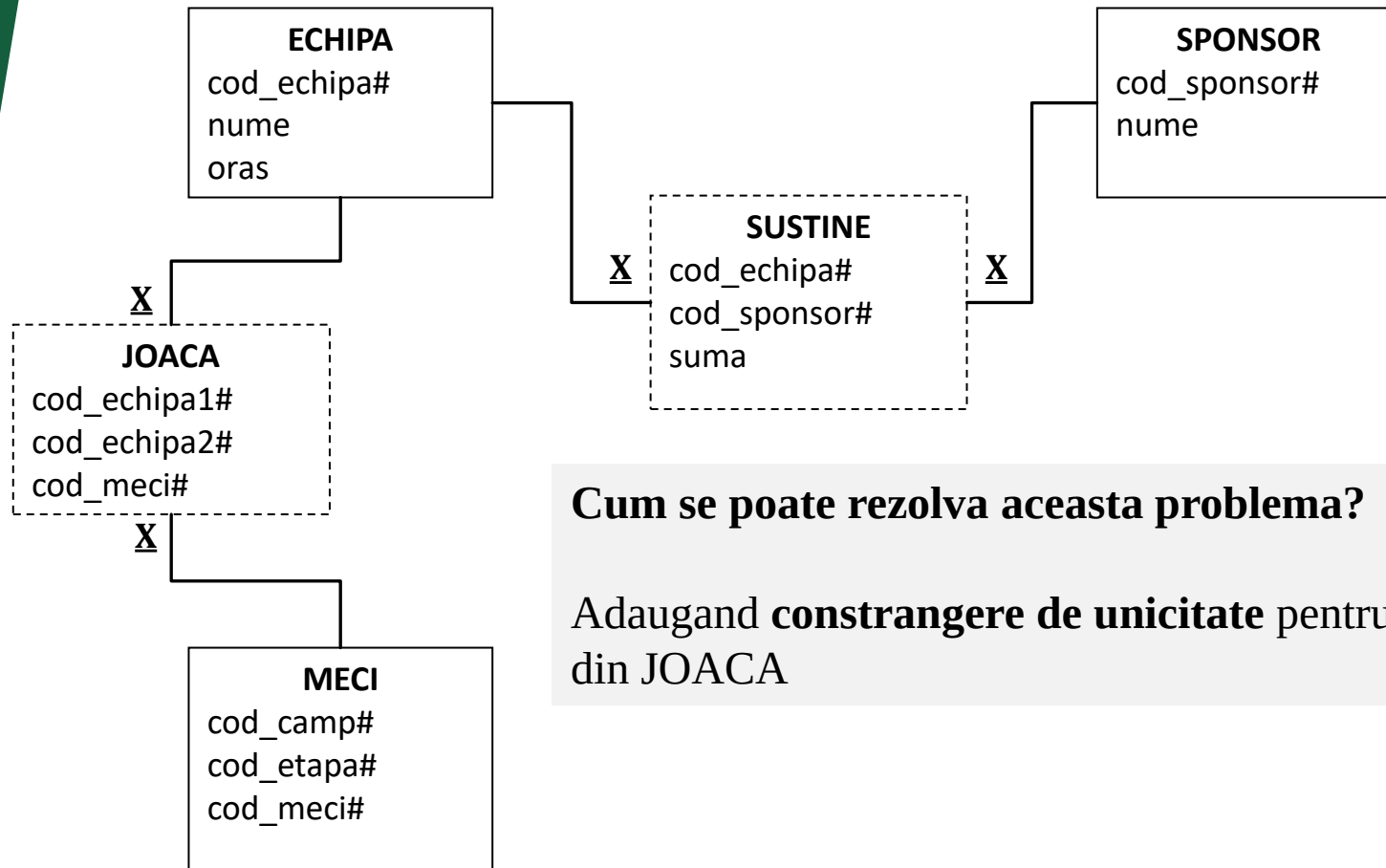
Totusi observam ca se pot adauga mai multe echipe pentru acelasi meci

EXEMPLUL 5: Campionate Fotbal



Cum se poate rezolva aceasta problema?

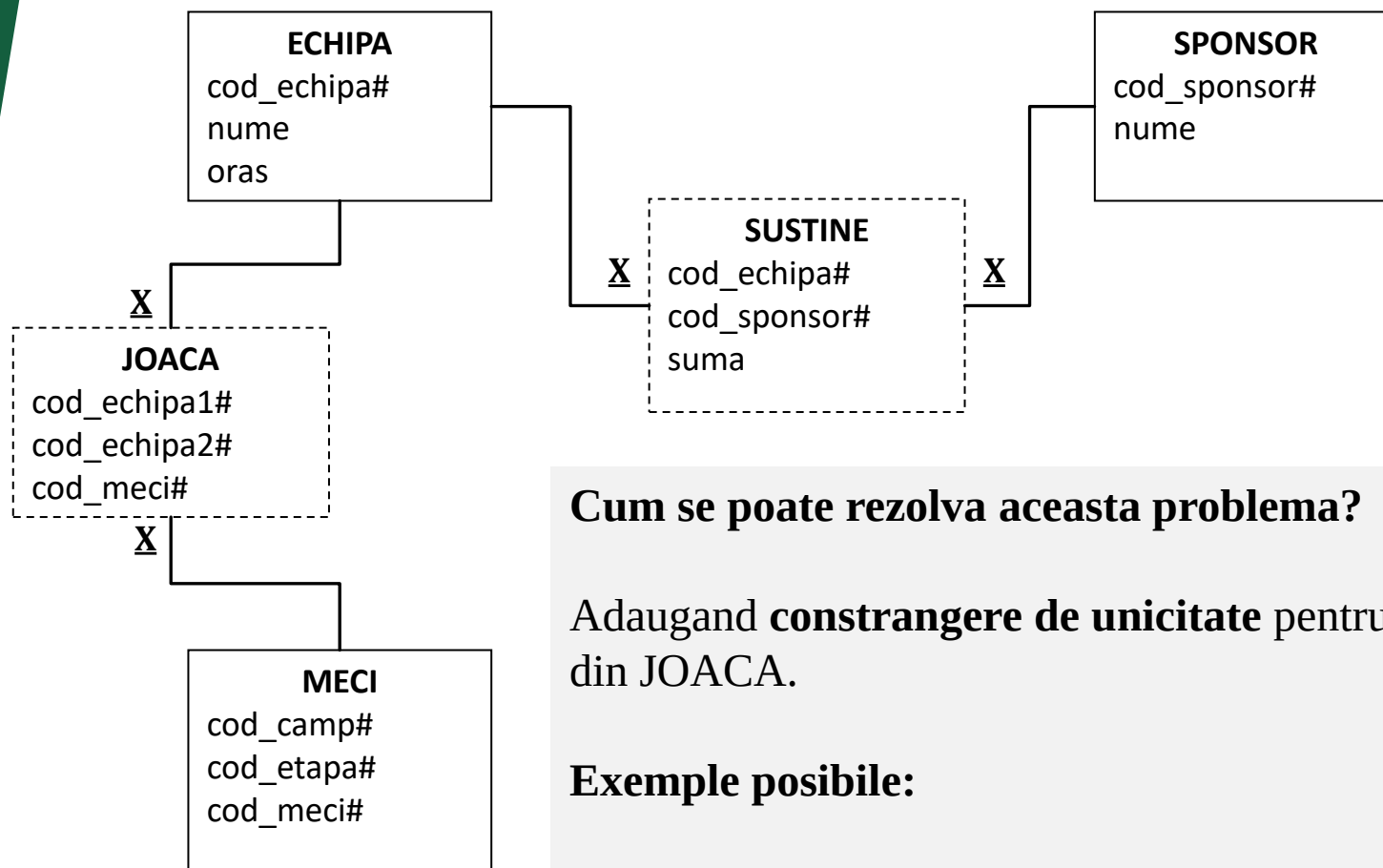
EXEMPLUL 5: Campionate Fotbal



Cum se poate rezolva aceasta problema?

Adaugand **constrangere de unicitate** pentru `cod_meci` din JOACA

EXEMPLUL 5: Campionate Fotbal



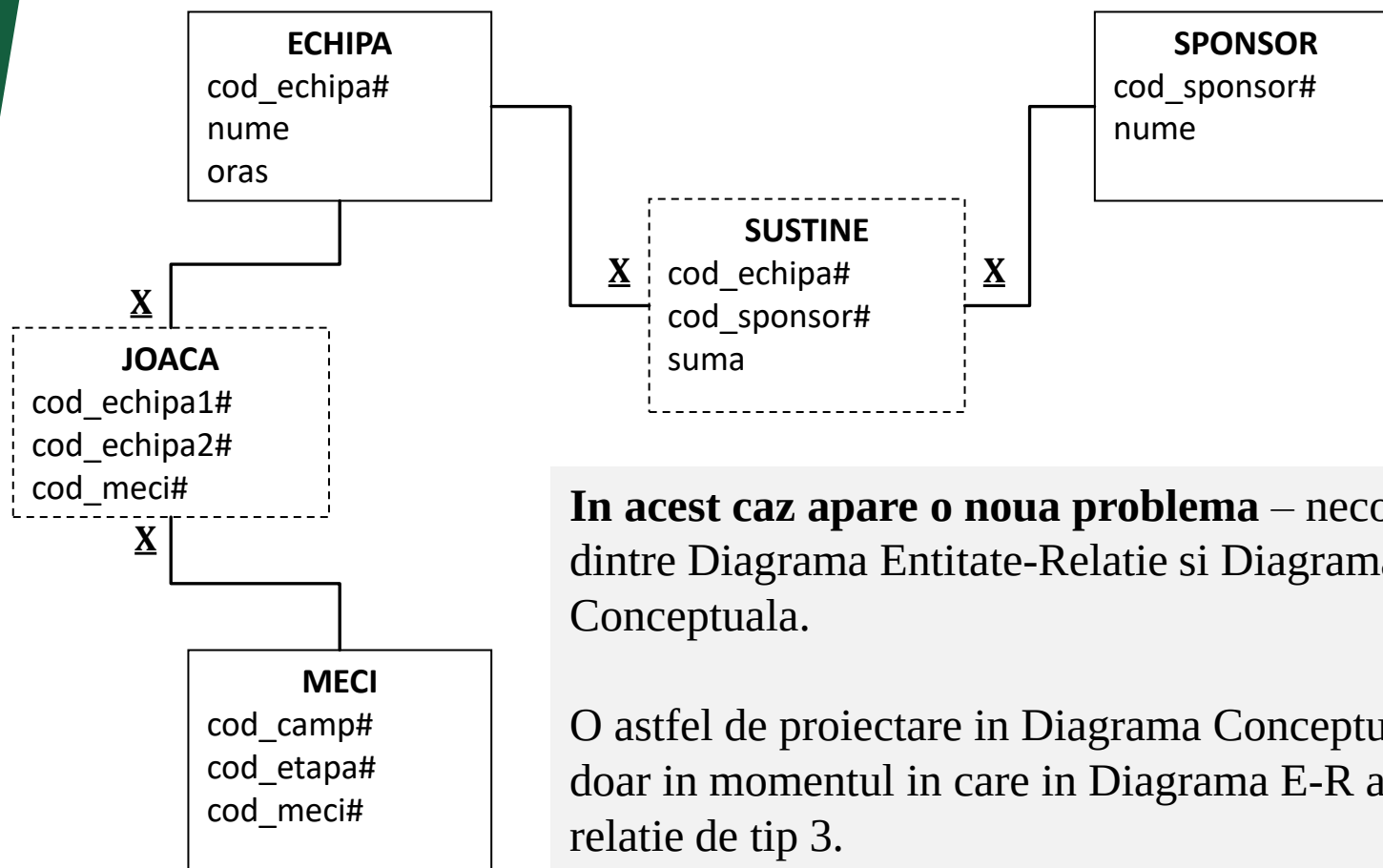
Cum se poate rezolva aceasta problema?

Adaugand **constrangere de unicitate** pentru cod_meci din JOACA.

Exemple posibile:

ech1, ech2, m1;
ech1, ech3, m2;
ech2, ech3, m3, etc

EXEMPLUL 5: Campionate Fotbal



In acest caz apare o noua problema – neconcordanța dintre Diagrama Entitate-Relație și Diagrama Conceptuală.

O astfel de proiectare în Diagrama Conceptuală apare doar în momentul în care în Diagrama E-R avem o relație de tip 3.

EXEMPLUL 5: Campionate Fotbal

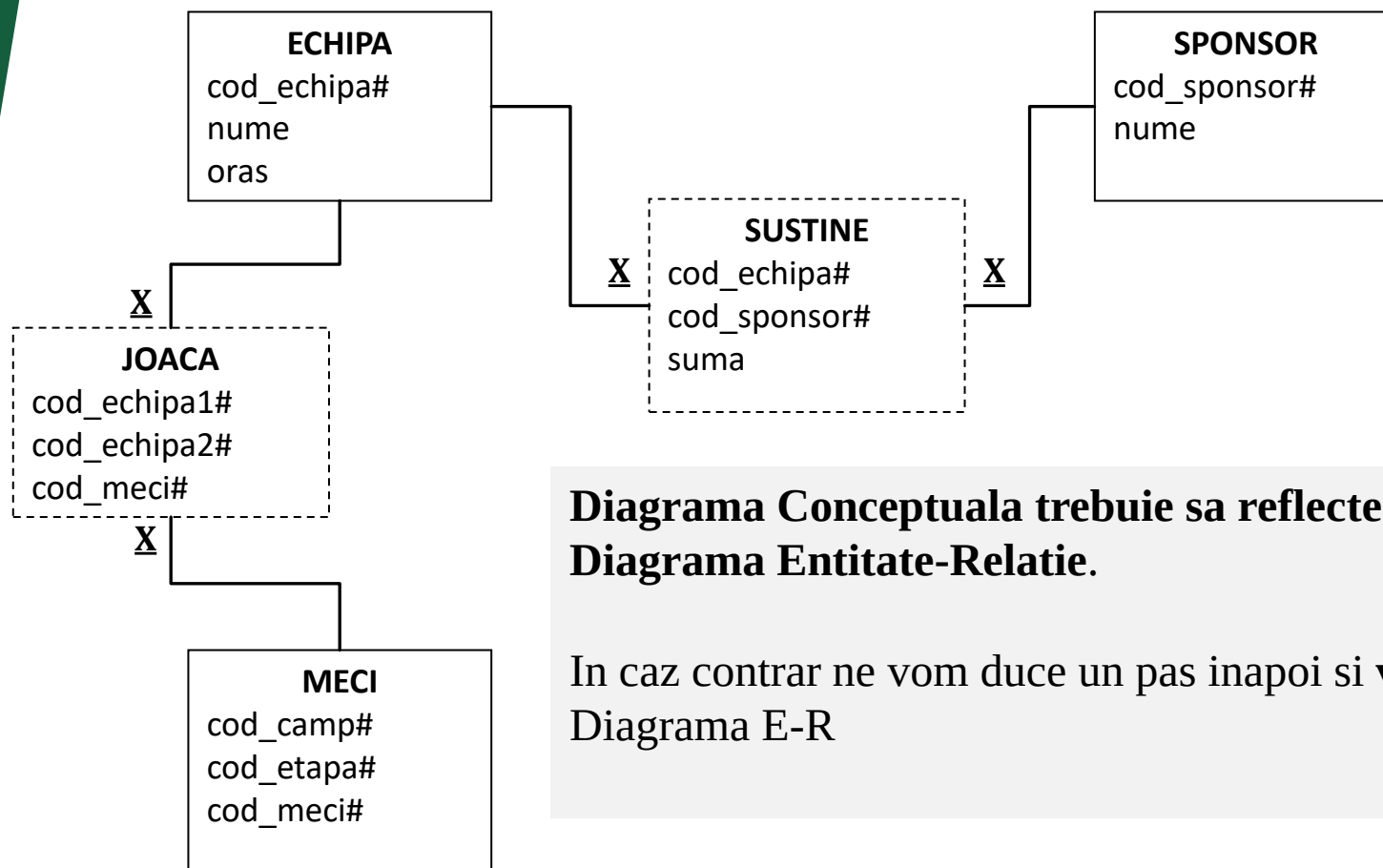
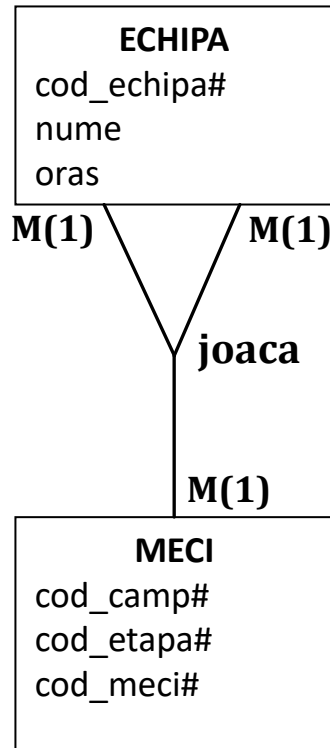


Diagrama Conceptuala trebuie sa reflecte 100% Diagrama Entitate-Relatie.

In caz contrar ne vom duce un pas inapoi si **vom reface** Diagrama E-R

EXEMPLUL 5: Campionate Fotbal

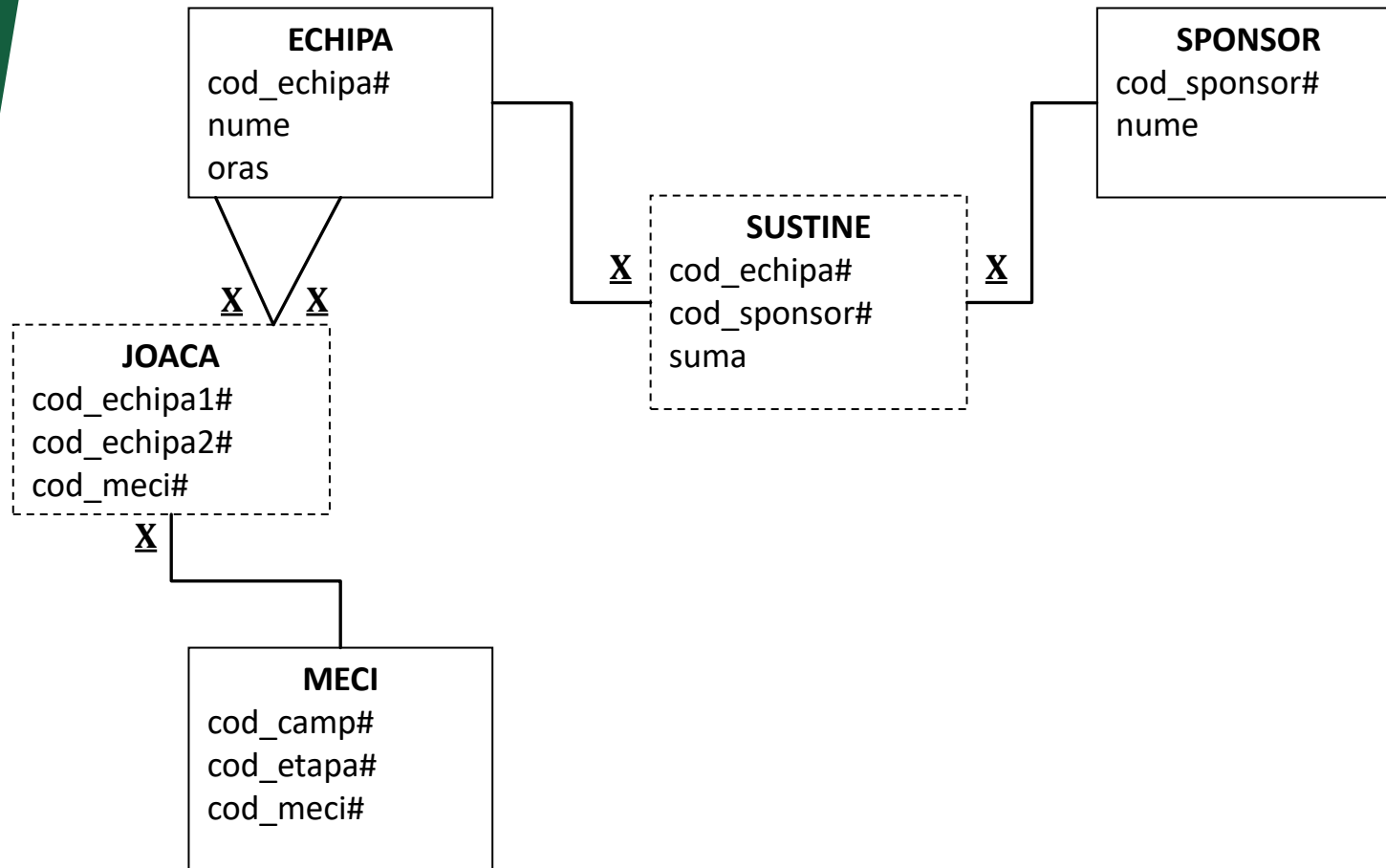


Ne intoarcem la Diagrama Entitate-Relatie si reproiectam relatia ca fiind relatie de **tip 3** intre **ECHIPA** si **MECI**.

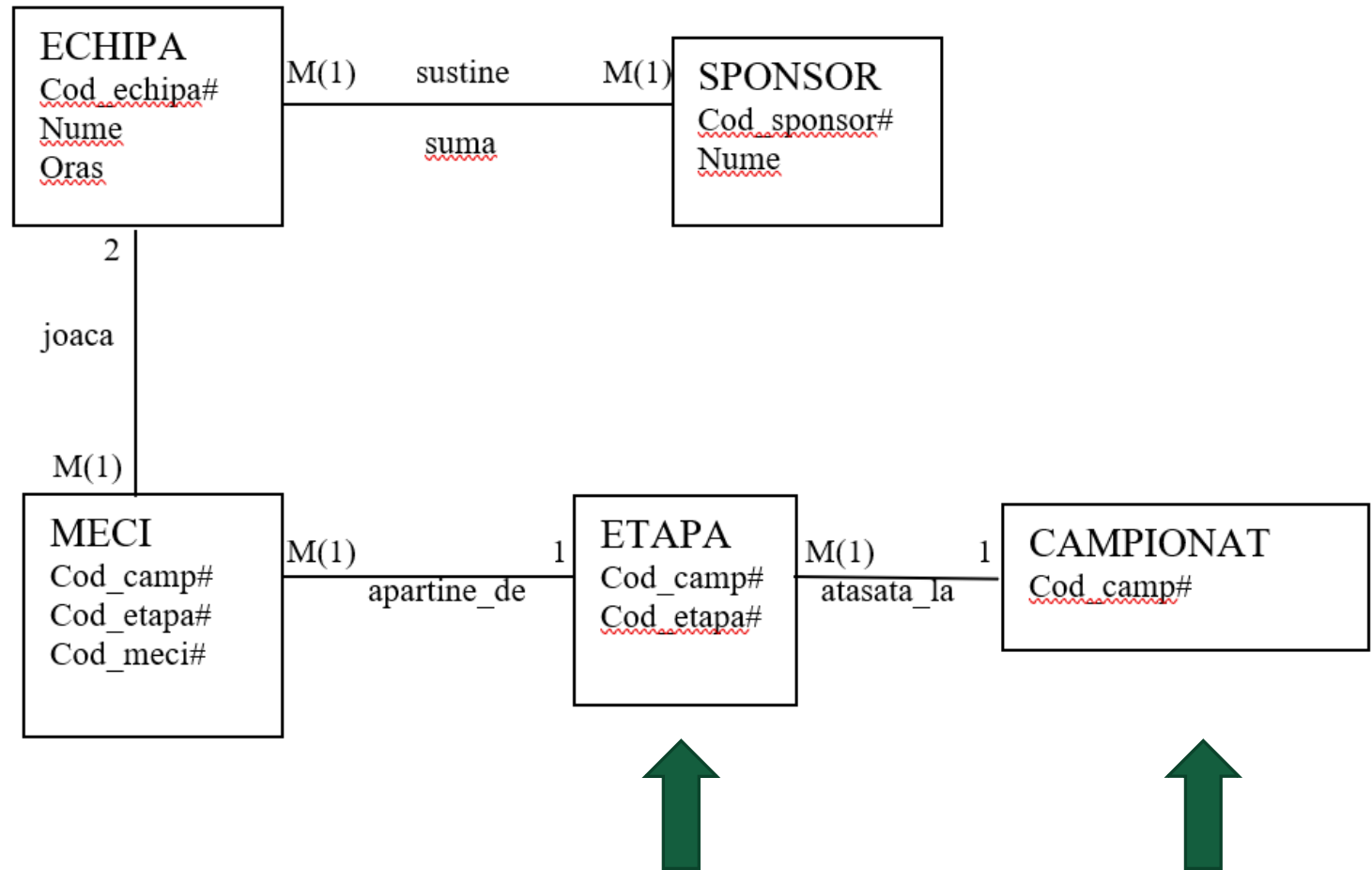
Relatii – o echipa joaca mai multe meciuri cu mai multe echipe ; un meci este jucat de mai multe echipe

In acest moment transformarea discutata anterior coincide cu Diagrama E-R, cu observatia de a proiecta corect legaturile dintre tabele.

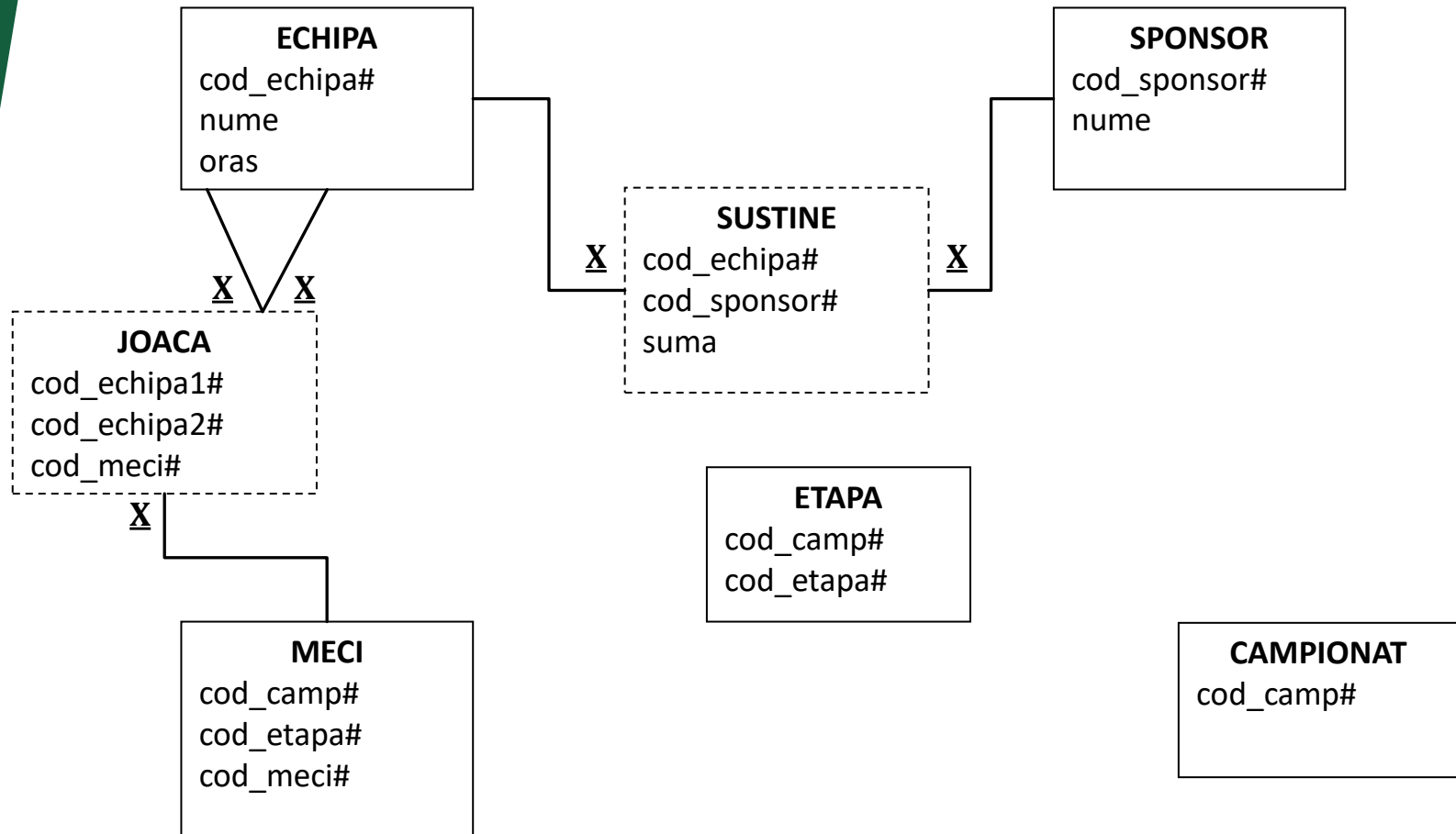
EXEMPLUL 5: Campionate Fotbal



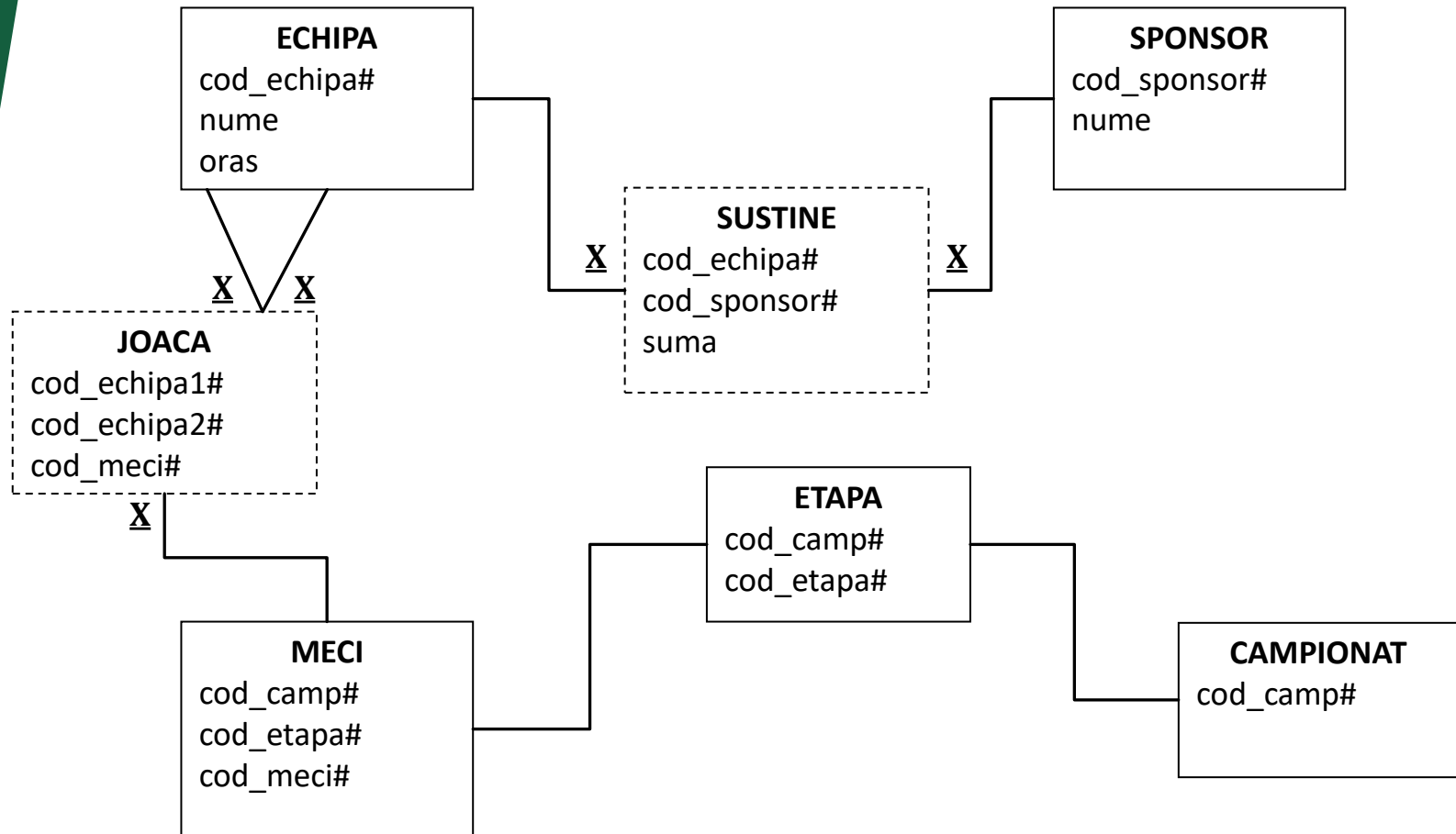
EXEMPLUL 5: Campionate Fotbal



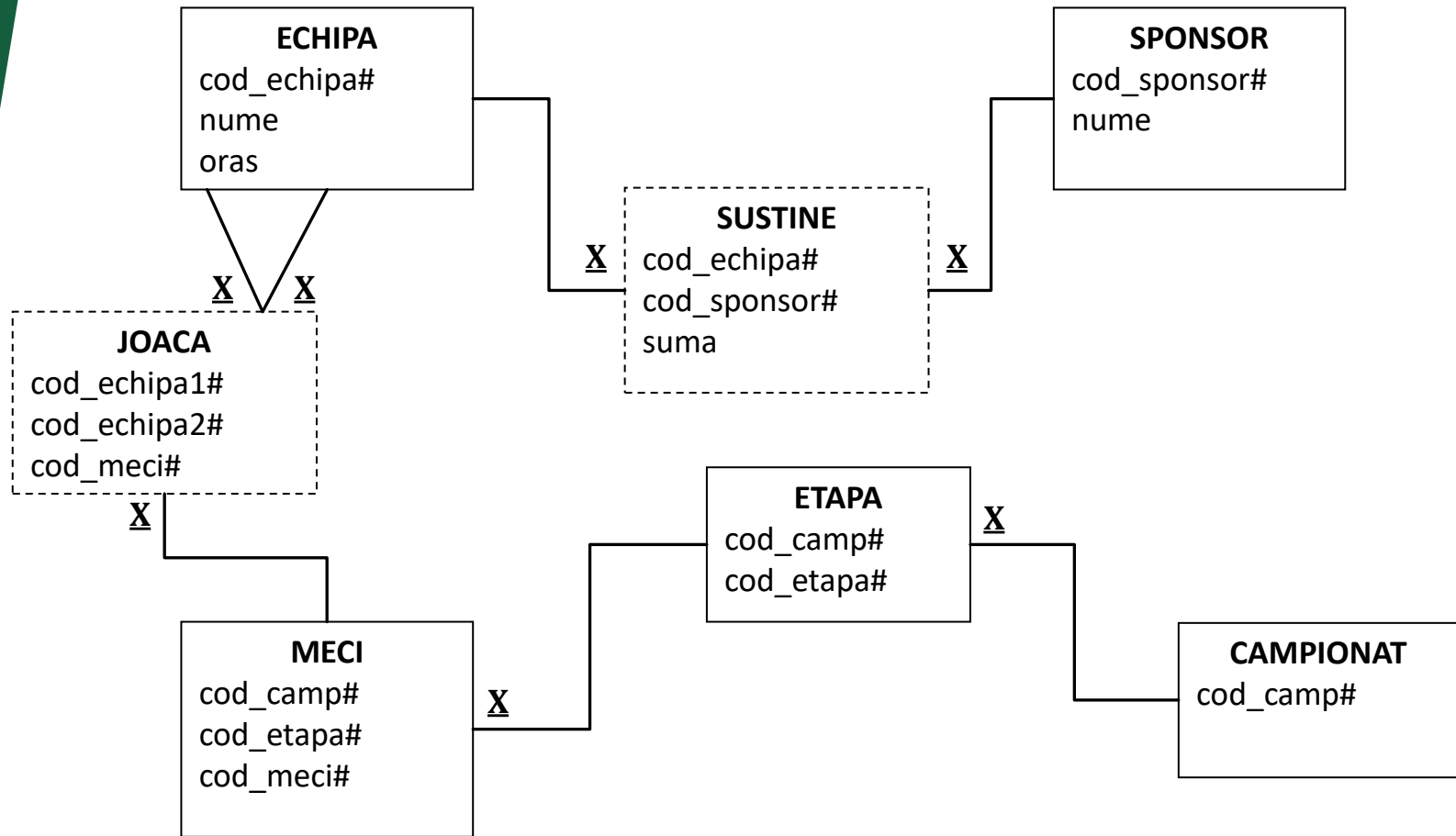
EXEMPLUL 5: Campionate Fotbal



EXEMPLUL 5: Campionate Fotbal

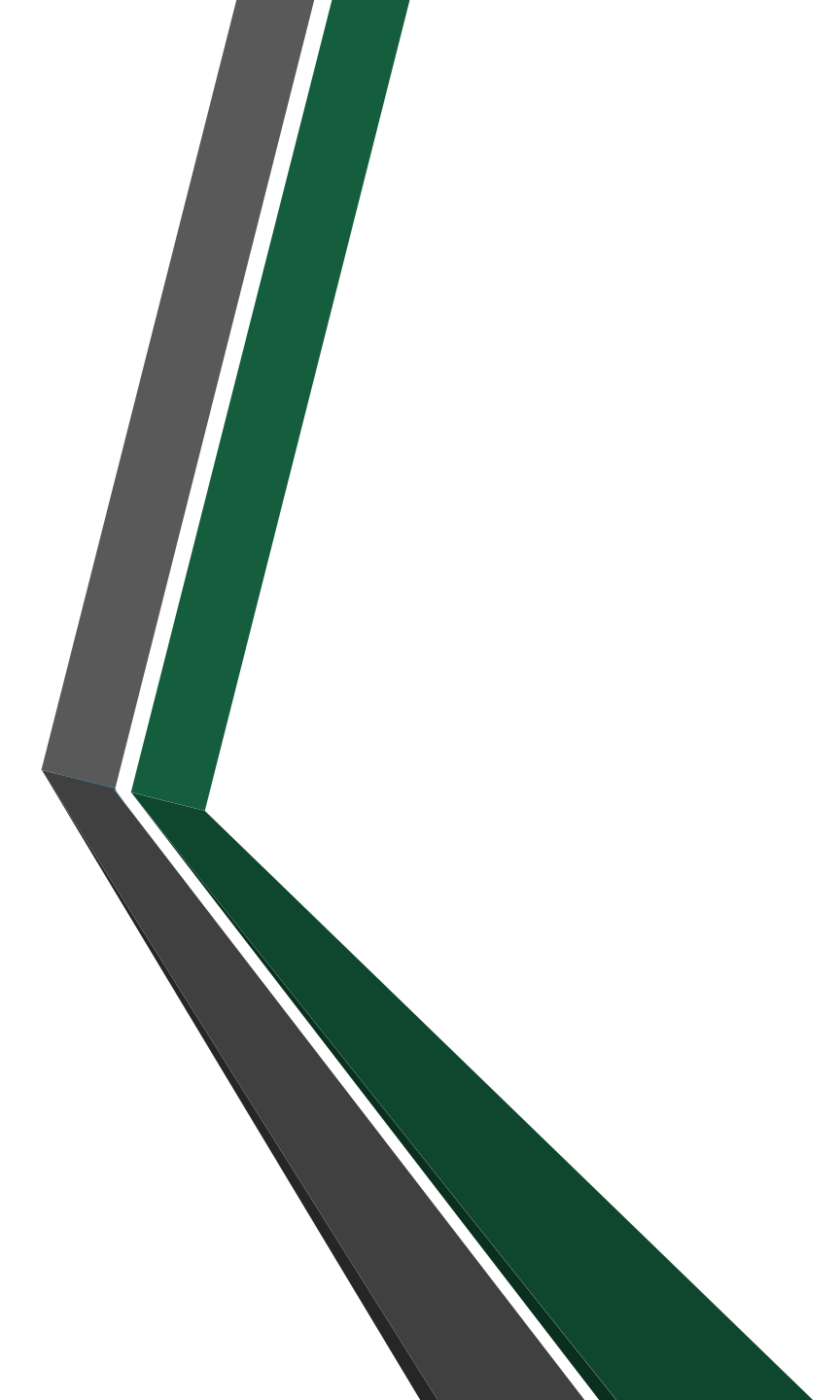


Campionate Fotbal – Diagrama Conceptuala



TEMA 2

Studiati notiunile invatate in **Cursurile 2 si 3** impreuna cu exemplele din **Cursul 4** si realizati tema 2 aflata pe site, intitulata “**Tema2_Curs**”



BAZE DE DATE

CURS 3

MODELUL RELAȚIONAL

- Conceput și dezvoltat de **E.F. (Edgar Frank) Codd**
- **MODEL FORMAL** de organizare conceptuală a datelor, destinat reprezentării legăturilor dintre date, bazat pe **teoria matematică a relațiilor**
- Modelul relațional este alcătuit numai din relații și prin urmare, orice interogare asupra bazei de date este tot o relație

MODELUL RELAȚIONAL

➤ Calități:

- este **simplu**;
- **riguros** din punct de vedere matematic;
- nu este orientat spre sistemul de calcul;

➤ Modalități pentru **definirea unui SGBD relațional**:

- prezentarea datelor în **tabele** supuse anumitor operații de tip proiecție, selecție, reuniune, compunere, intersecție etc.
- un sistem de baze de date ce suportă un limbaj de tip **SQL** – *Structured Query Language*;
- un sistem de baze de date care respectă **principiile modelului relațional** introdus de E.F. Codd.

MODELUL RELAȚIONAL

- **Caracteristicile** unui model relațional:
 - **structura** relațională a datelor (structurile de date sunt definite de un limbaj de definire a datelor);
 - **operatorii** modelului relațional (manipularea datelor – relațiile pot fi manipulate utilizând un limbaj de manipulare a datelor; în modelul relațional se folosesc operatori specifici algebrei relationale);
 - **regulile de integritate** care guvernează folosirea cheilor în model (constrângeri interne);
- Aceste trei elemente corespund celor trei componente ale ingineriei *software*:
 - **informație**
 - **proces**
 - **integritate**

MODELUL RELAȚIONAL

Structura datelor

- Definirea noțiunilor de **domeniu**, **relație**, **schemă relațională**, **valoare null** și **tabel vizualizare** (*view*).
- Conceptele utilizate pentru a descrie formal, uzual sau fizic elementele de bază ale organizării datelor:

FORMAL	UZUAL	FIZIC
Relație Tuplu Atribut Domeniu	Tablou / Tabel Linie Coloană Tip de date	Fișier Înregistrare Câmp Tip de date

MODELUL RELAȚIONAL

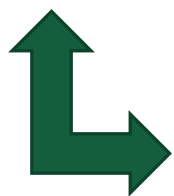
- **DOMENIU** – mulțime de valori care poate fi definită fie enumerând elementele componente, fie definind o proprietate distinctivă a domeniului valorilor.
- Fie $D_1, D_2, \dots, D_n$ domenii finite, nu neapărat disjuncte. **Produsul cartezian** $D_1 \times D_2 \times \dots \times D_n$ al domeniilor $D_1, D_2, \dots, D_n$ este definit de mulțimea tuplurilor $(V_1, V_2, \dots, V_n)$, unde $V_1 \in D_1, V_2 \in D_2, \dots, V_n \in D_n$. Numărul n definește **aritatea tuplului**.
- Cu alte cuvinte, **domeniul (tipul de date)** este o mulțime finită în care un **atribut (coloana)** poate lua valori. Atributele din exemplul de mai jos sunt: ID, NUME, PRENUME și EMAIL

NUMBER		VARCHAR	
ID	NUME	PRENUME	EMAIL
100	KING	STEVEN	SKING
200	De Haan	Lex	LDEHAAN
300	Hunold	Alexander	AHUNOLD

MODELUL RELAȚIONAL

- O **RELAȚIE** R pe mulțimile $D_1, D_2, \dots, D_n$ este o submulțime a produsului cartezian $D_1 \times D_2 \times \dots \times D_n$, deci este o mulțime de tuple. Asadar, o **relatie** este o multime finita de tuple.
- **Caracteristicile unei relații:**
 - nu exista tuple identice (intrari/linii);
 - ordinea liniilor si a coloanelor este arbitrara;
 - elementele care fac parte dintr-un domeniu (adica un singur tip de date) sunt de acelasi tip;
 - fiecare coloana apartine unui singur domeniu si nu se poate repeta coloana in cadrul aceleiasi relatii;

**Relatia
ANGAJAT**



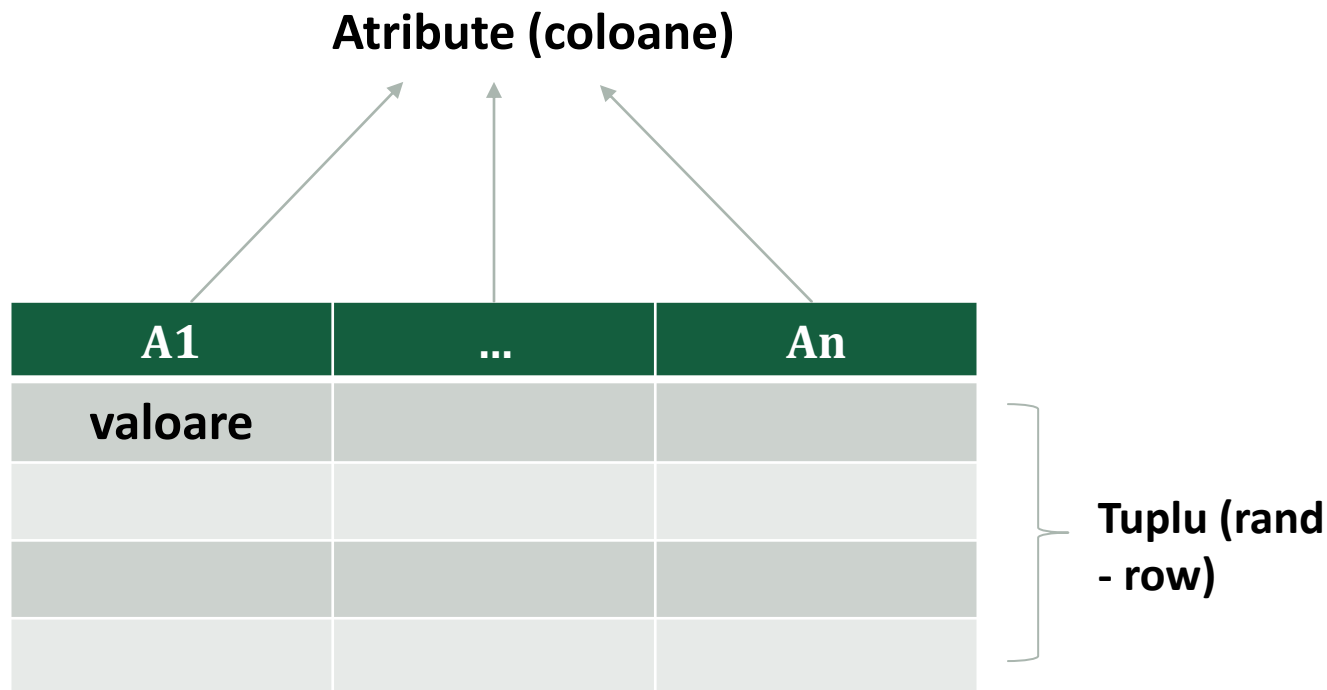
ID	NUME	PRENUME	EMAIL
100	KING	STEVEN	SKING
200	De Haan	Lex	LDEHAAN
300	Hunold	Alexander	AHUNOLD

MODELUL RELAȚIONAL

- Definirea unei relații se referă la mulțimi care variază în timp
- Este necesar un element invariant în timp: **structura relației** (**schema relațională**)
- Mulțimea numelor atributelor corespunzătoare unei relații R definește **schema relațională** a relației respective. Vom nota schema relațională prin $R(A_1, A_2, \dots, A_n)$ – **exemplu**: Angajat(ID, NUME, PRENUME, EMAIL)
- Putem reprezenta o relație printr-un **tabel bidimensional**
 - O coloană corespunde de fapt unui atribut;
 - Numărul atributelor unei relații definește **gradul** relației (**exemplu**: relația ANGAJAT este de grad 4 sau are aritate 4), iar numărul de tupluri din relație definește **cardinalitatea** relației.

ID	NUME	PRENUME	EMAIL
100	KING	STEVEN	SKING
200	De Haan	Lex	LDEHAAN
300	Hunold	Alexander	AHUNOLD

MODELUL RELAȚIONAL



Domeniu = multimea de valori in care poate lua valori un atribut

Relatie = Tabel

Numele relatiei = Denumirea tabelului

Tuplu = o succesiune de valori de diferite tipuri

MODELUL RELAȚIONAL

Exemplu (crearea unui tabel în *SQL*):

```
CREATE TABLE salariat
```

```
(
```

```
    cod_salariat    NUMBER(4) PRIMARY KEY,
```

```
    nume            VARCHAR2(25),
```

```
    prenume         VARCHAR2(20),
```

```
    salariu         NUMBER(8, 2),
```

```
    sef             NUMBER(4),
```

```
    job_cod         VARCHAR2(6),
```

```
    cod_departament NUMBER(3)
```

```
);
```

MODELUL RELAȚIONAL

Valoarea *NULL*

- Când se inserează tupluri într-o relație, de multe ori un atribut este **necunoscut** sau **neaplicabil**.
- Pentru a reprezenta acest atribut a fost introdusă o valoare convențională în relație, și anume valoarea ***null***.
- Este necesară o aritmetică și o logică nouă care să cuprindă acest element.
- Rezultatul operatorilor aritmetici este *null* când unul din argumente este *null*.
- Prin urmare, „*null* = *null*” are valoarea *null*, iar $\neg \text{null}$ este *null*.

MODELUL RELAȚIONAL

- Tabelele de adevăr pentru operatorii AND și OR:

AND	T	F	Null		OR	T	F	Null
T	T	F	Null		T	T	T	T
F	F	F	F		F	T	F	Null
Null	Null	F	Null		Null	T	Null	Null

MODELUL RELAȚIONAL

Tabelul vizualizare

- **view**, filtru, relație virtuală, vedere
- constituie un filtru relativ la unul sau mai multe tabele, care conține numai informația necesară unei anumite abordări sau aplicații.
Securitate, reactualizări
- Vizualizarea este **virtuală** deoarece datele pe care le conține nu sunt în realitate memorate într-o bază de date. Este memorată numai **definiția** vizualizării
- Vizualizarea nu este definită explicit, ca relațiile de bază, prin mulțimea tuplurilor componente, ci implicit, pe baza altor relații prin intermediul unor expresii relaționale
- Stabilirea efectivă a tuplurilor care compun vizualizarea se realizează prin evaluarea expresiei atunci când utilizatorul se referă la acest tabel

MODELUL RELAȚIONAL

Exemplu (crearea unei vizualizări în *SQL*):

```
CREATE VIEW PROGRAMATOR (nume, departament)  
  AS SELECT nume, cod_departament  
    FROM SALARIAT  
   WHERE job_cod = 'IT_PROG';
```


MODELUL RELAȚIONAL

Reguli de integritate

- **asertiuni** pe care datele conținute în baza de date trebuie să le satisfacă
- trebuie făcută distincția între:
 - **regulile structurale** inerente modelării datelor;
 - regulile de funcționare specifice unei aplicații particular;

Există trei tipuri de constrângeri structurale:

- de cheie
- de referință
- de entitate

constituie **mulțimea minimală de reguli de integritate** pe care **trebuie** să le respecte un SGBD relațional

Restricțiile de integritate minimale sunt definite în raport cu noțiunea de cheie a unei relații.

MODELUL RELAȚIONAL

Modelul relațional respectă **trei reguli de integritate structurală**:

- **Regula 1 – unicitatea cheii**. Cheia primară trebuie să fie **unică** și **minimală**.
- **Regula 2 – integritatea entității**. Atributele cheii primare trebuie să fie **diferite de valoarea NULL**.
- **Regula 3 – integritatea referirii**. O cheie externă trebuie să fie ori **NULL în întregime**, ori să corespundă unei **valori a cheii primare asociate**.

MODELUL RELAȚIONAL

O mulțime minimală de attribute ale căror valori identifică unic un tuplu (adica un rand/inregistrare din baza de date) într-o relație reprezintă o **cheie** pentru relația respectivă.

- Fiecare relație are cel puțin o cheie;
- Una dintre **cheile candidat** va fi aleasă pentru a identifica efectiv tupluri și ea va primi numele de **cheie primară**;
- Cheia primară nu poate fi reactualizată;
- Attributele care reprezintă cheia primară sunt fie subliniate, fie **urmate de simbolul #**;

MODELUL RELAȚIONAL

- O **cheie** identifică linii și este diferită de un **index** care localizează liniile
- O **cheie secundară** este folosită ca index pentru a accesa tupluri
- Un grup de attribute din cadrul unei relații care conține o cheie a relației poartă numele de **supercheie**.
- Fie schemele relaționale $R1(P1, S1)$ și $R2(S1, S2)$, unde $P1$ este cheie primară pentru $R1$, $S1$ este cheie secundară pentru $R1$, iar $S1$ este cheie primară pentru $R2$. În acest caz, vom spune că $S1$ este **cheie externă** (cheie străină) pentru $R1$.

PROIECTAREA MODELULUI RELAȚIONAL

Transformarea entităților

- Entitățile independente devin **tabele independente**
 - Cheia primară nu conține chei externe
- Entitățile dependente devin **tabele dependente**
 - Cheia primară a entităților dependente conține cheia primară a entității de care depinde (cheie externă) plus unul sau mai multe attribute adiționale
- Subentitățile devin **subtabele**
 - Cheia externă referă cheia primară din supertabel, iar cheia primară este această cheie externă (cheia primară a subentității PROGRAMATOR este cod_salariat care este o cheie externă).

PROIECTAREA MODELULUI RELAȚIONAL

Transformarea relațiilor

- Relațiile 1:1 și 1:n devin **chei externe**
 - Relația *conduce* devine coloană în tabelul DEPARTAMENT, iar relația *lucreaza_in* devine coloană în tabelul SALARIAT;
 - Simbolul „×” indică plasamentul cheii externe, iar simbolul „⊗” exprimă faptul că această cheie externă este conținută în cheia primară. Relația 1:1 plasează cheia externă în tabelul cu mai puține linii;
- Relația $m:n$ devine un tabel special, numit **tabel asociativ**, care are două chei externe pentru cele două tabele asociate.
 - Cheia primară este compunerea acestor două chei externe plus eventuale coloane adiționale;
 - Tabelul se desenează punctat;
- Relațiile de tip trei devin **tabele asociative**
 - Cheia primară este compunerea a trei chei externe plus eventuale coloane adiționale.

PROIECTAREA MODELULUI RELAȚIONAL

Transformarea atributelor

- Un atribut singular devine o **coloană**
- Atributele multiple devin **tabele dependente** ce conțin cheia primară a entității și atributul multiplu
 - Cheia primară este o cheie externă, plus una sau mai multe coloane adiționale;
- Ce devin atributele relațiilor?
 - Pentru relații 1:1 și 1: n , atributele relațiilor vor aparține tabelului care conține cheia externă;
 - Pentru relații $m:n$ și de tipul trei, atributele vor fi plasate în tabelele asociative;

PROIECTAREA MODELULUI RELAȚIONAL

- Cele patru tipuri de tabele (independente, dependente, subtabele și asociative) se deosebesc prin structura cheii primare.

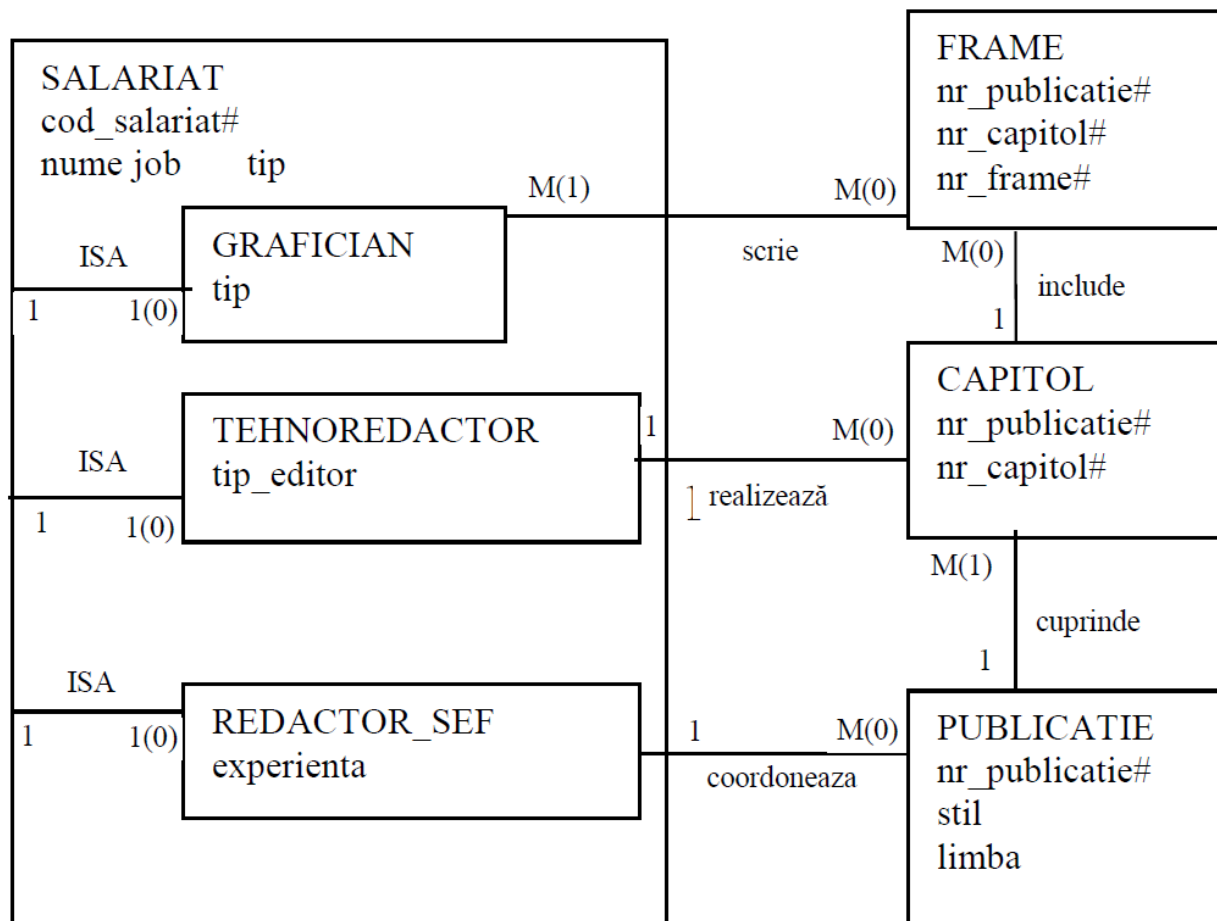
Tabel	Reprezintă	Cheie primară
Independent	entitate independentă	nu conține chei externe
Subtabel	Subentitate	o cheie externă
Dependent	entitate dependentă	o cheie externă și una sau mai
	atribute multiple	multe coloane adiționale
Asociativ	relație m:n	două sau mai multe chei
	relații de tip 3	externe și (opțional) coloane adiționale

- Diagramele conceptuale pentru proiectarea modelelor relaționale comentate vor fi construite din diagramele E/R prin adăugarea tabelelor asociative și prin marcarea cheilor externe.

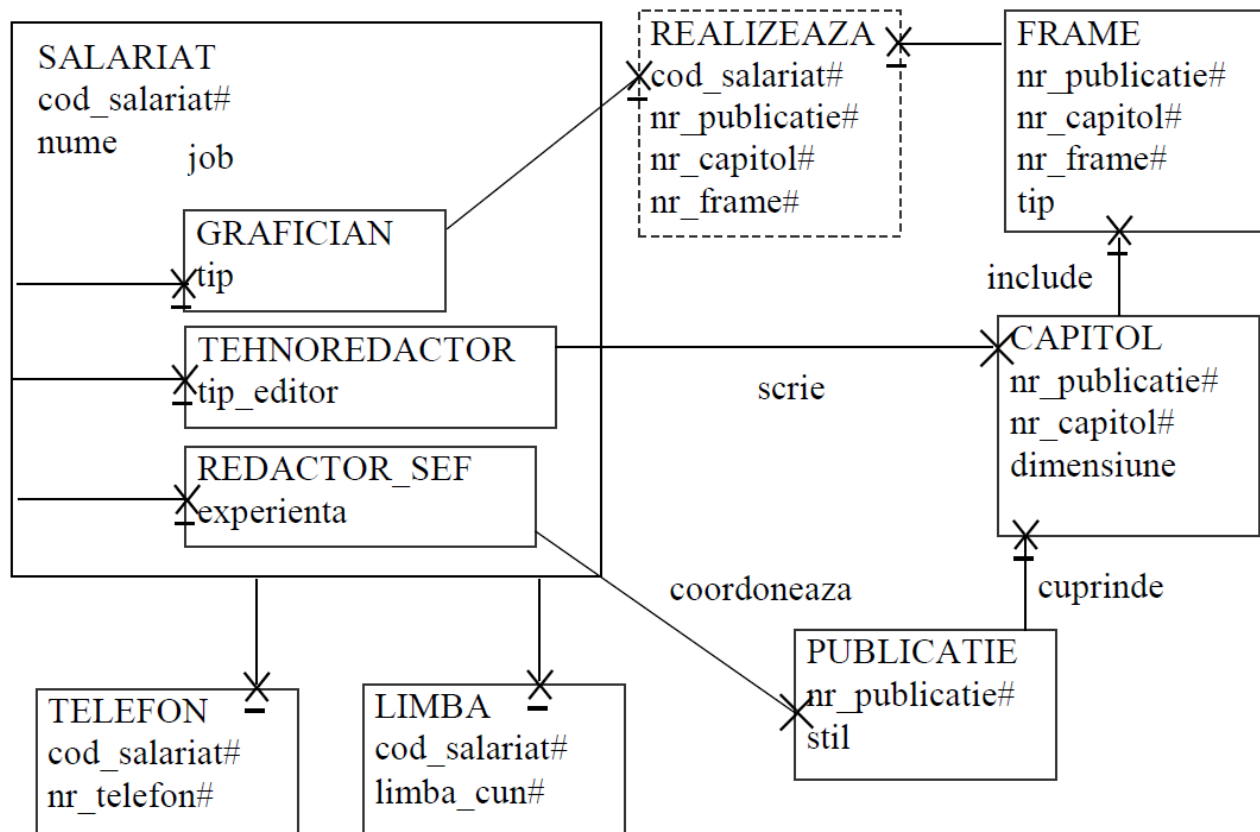
TEMA

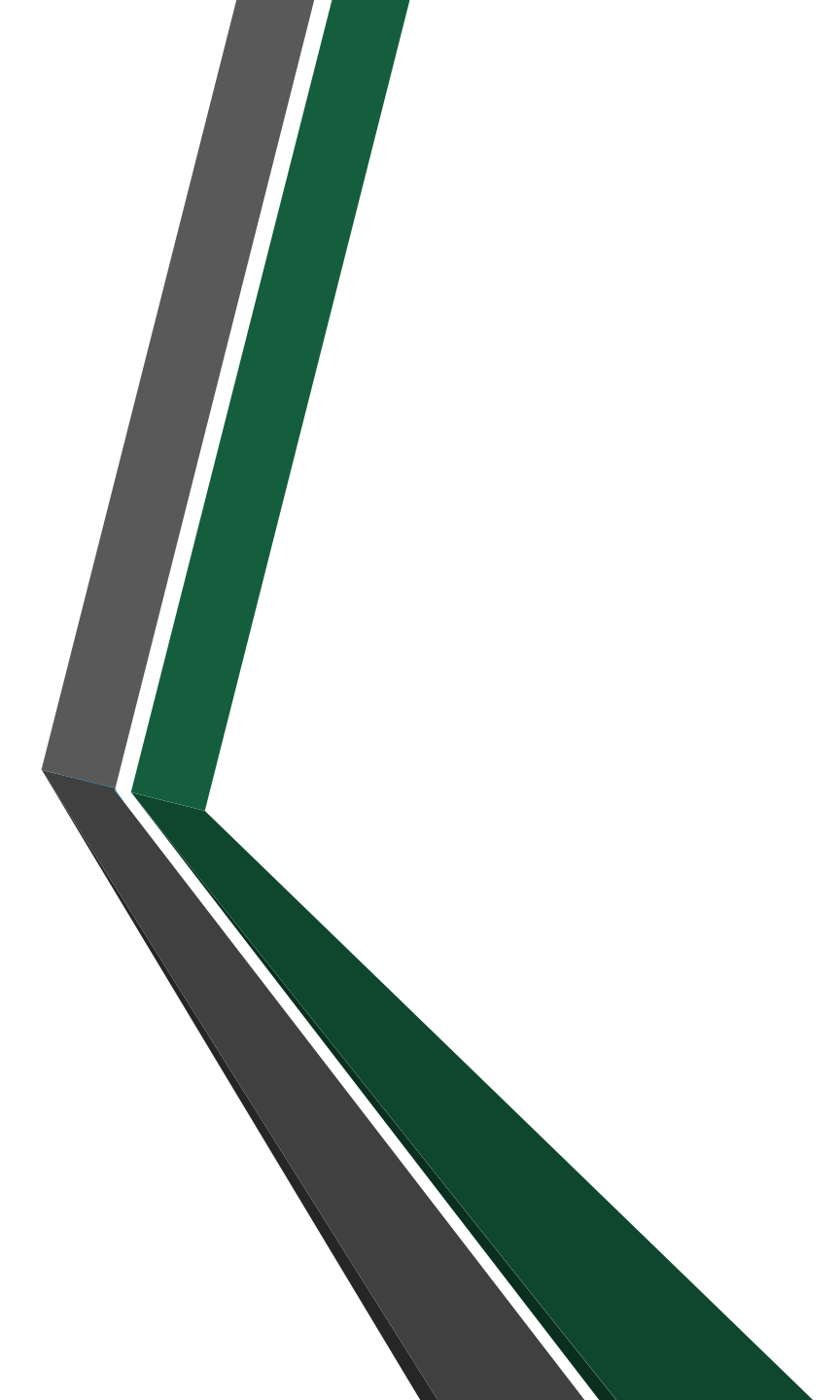
- Sa se studieze notiunile parcurse in cadrul cursului si sa se analizeze Diagrama Conceptuala proiectata pentru **Gestiunea activitatilor de editare dintr-o editura**, plecand de la Diagrama Entitate/Relatie.
- Studiati regulile de transformare si modul in care acestea au fost aplicate.

TEMA – Diagrama E/R



TEMA – Diagrama Conceptuala





BAZE DE DATE

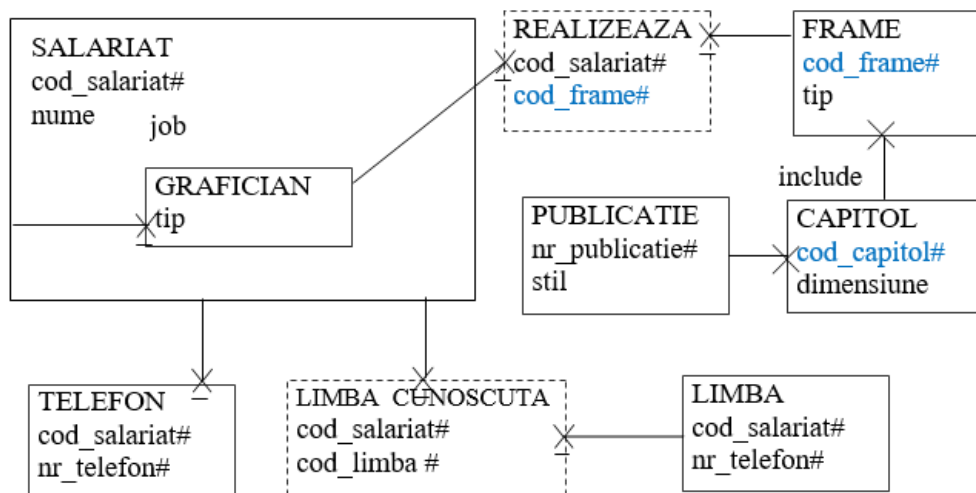
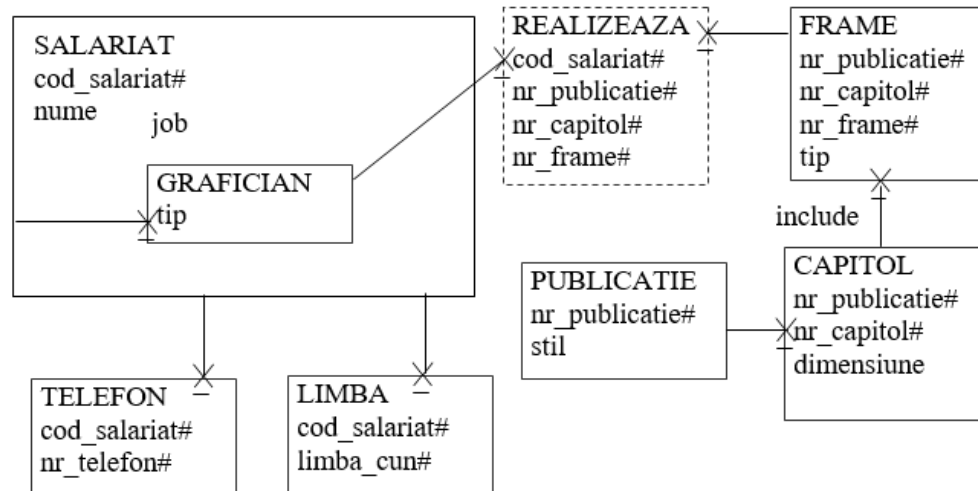
CURS 5

Completări design BD – explicatii teme

Observatiile referitoare la primele doua teme din cadrul cursului (Tema1_Curs si Tema2_Curs) se afla in documentul **Observatii_Teme_1_2_Curs**.

Pe site se afla si **diagramele asociate** fiecarei teme (documentele Tema1_Curs_Rezolvare si Tema2_Curs_Rezolvare).

Completări design BD



Completări design BD

- Introducem o **cheie primara artificială** dacă avem o cheie primară compusă din multe attribute (în practică dacă sunt mai mult de doua-trei attribute)
- Introducerea unei CP artificiale conduce la pierderea dependentei
- Se poate pune cheie artificială și pe tabelul asociativ, caz în care tabelul rămâne punctat în diagrama concepuală, însă fără dependențe (“x” simplu, nu subliniat). În acest caz cheia primară a tabelului dependent nu va fi referită de nicio cheie externă
- În cazul atributelor multiple alegem modelarea ca tabel dependent sau relație M:M (tabel asociativ)?
 - Dacă aceeași valoare poate aparține mai multor entități, atunci poate fi o opțiune de design relația M:M.

Agentie_Turism – Diagrama Conceptuala

Pe site se afla documentul in care exista atat Diagrama E-R (studiata intr-un curs anterior), cat si Diagrama Conceptuala asociata.

Fisierul se numeste: **Agentie_Turism_ER_Conceptuala.**

Exemplu Proiect

Exemplul pentru proiect se afla pe site, numit **Exemplu_Proiect**

Progres Proiect

- Studiați notiunile studiate din cadrul cursurilor 2 și 3 (regulile de proiectare, notiunile teoretice, definiții – nu se învăța pe de rost, ci se înțeleg logic – așa cum le-am studiat împreună la curs);
- Parcurgeți exemplele studiate până în acest moment;
- Analizați exemplele pentru proiect – **Agentie de Turism și Casa de Moda** (exemplul discutat în cursul curent);
- Demarați proiectul (cerințele se afla pe site – Săptămăna 1 – **Cerinte_Proiect**);
- Citiți și sistemul de notare aflat tot în cadrul săptămânii 1;

Modelare orientată pe obiecte cu *UML*

- Un **limbaj de modelare** reprezintă o modalitate de a **comunica** despre un sistem și de a **exprima** diversele modele produse în cadrul procesului de analiză și dezvoltare a sistemului.
 - Limbajul furnizează o **notație** care permite reprezentarea unui design.
- Ca orice limbaj, un limbaj de modelare are o **sintaxă** și o **semantică**.
 - Pentru semantica modelului trebuie aleasă **sintaxa cea mai expresivă**.
 - De cele mai multe ori, un limbaj de modelare este un **limbaj grafic** (diagramatic).

Modelare orientată pe obiecte cu *UML*

- *The Unified Modelling Language* (**UML**) este un limbaj **vizual** de **modelare** și de **comunicare** despre sistem.
- UML **nu este un limbaj de programare**.
- UML este un limbaj pentru modelarea orientată pe obiecte, cu suport pentru modelare vizuală, funcționând ca o modalitate de a comunica și de a exprima cunoștințe.

Modelare orientată pe obiecte cu *UML*

- **UML** este un **limbaj grafic de modelare** pentru **specificarea, vizualizarea, construcția** și **documentarea** componentelor:
 - unui sistem *software* (pentru întreprinderi, telecomunicații, sisteme bancare și financiare, transporturi etc.)
 - sau pentru modelarea organizațională a unor sisteme *non-software* (din justiție, medicină, afaceri etc.).
- **UML** permite realizarea tuturor **documentelor** necesare înțelegerii modelului și diagramelor utilizate pe tot parcursul ciclului de viață al unui proces de realizare a unui sistem.
 - specificarea **cerințelor**, **arhitecturii** și **proiectării** sistemului; elaborarea **codului** sursă, planuri de dezvoltare și de **management** al proiectului.

Modelare orientată pe obiecte cu *UML*

- UML **nu** este un limbaj de programare, dar modelele exprimate în UML pot fi implementate ușor în limbaje de programare orientate pe obiecte (C++, Java, C#) sau în **baze de date relaționale**.
 - Este posibilă nu numai generarea codului dintr-un model UML, dar și **ingineria inversă**, constând în construirea dintr-un cod dat a unui model UML.

Modelare orientată pe obiecte cu *UML*

- Sintaxa limbajului UML implică diagrame, în timp ce semantica lui se bazează pe paradigma orientării pe obiecte.
 - Din punct de vedere al modelării orientate pe obiecte, entitățile (conceptele) se numesc **clase**, iar relațiile dintre ele se numesc **asocieri**.
- UML conține **mai multe tipuri de diagrame**, fiecare reflectând unul sau mai multe dintre tipurile de vizualizări posibile asupra unui sistem *software*.

Modelare orientată pe obiecte cu *UML*

- **Diagrama claselor** descrie structura unui sistem în general. În componența ei intră **clase**, **stereotipuri** și **relațiile** dintre acestea.
- **Diagrama obiectelor** descrie structura unui sistem la un anumit moment. Acest tip de diagramă este o variantă a diagramei claselor care, în locul unei clase, prezintă mai multe instanțe ale ei, fiind formată din obiecte și legături dintre ele.

Modelare orientată pe obiecte cu *UML*

- **Diagrama cazurilor de utilizare** descrie **funcționalitatea** unui sistem, prezentând actorii externi, cazurile de utilizare identificate numai din punct de vedere al actorilor (comportamentul sistemului, așa cum este perceput de utilizatorii lui), precum și relațiile dintre actori și cazurile de utilizare.
 - Un actor poate fi orice sau oricine interacționează cu sistemul (trimite sau recepționează mesaje de la sistem sau schimbă informații cu acesta). Actorul are un rol în cadrul unui sistem, nu este un utilizator individual al acestuia și, din acest motiv, el este o entitate (o clasă), nu o instanță. Un caz de utilizare este inițiat mereu de un actor și furnizează o valoare actorului.

Modelare orientată pe obiecte cu *UML*

- **Diagrama componentelor** (diagrama de implementare) descrie structura fizică a codului în termenii componentelor de cod și relațiilor dintre acestea.
- **Diagrama de desfășurare** (de exploatare) indică arhitectura fizică pe care este implementat sistemul, calculatoarele, nodurile sistemului și conexiunile dintre ele.
- **Diagrama secvențelor** este o diagramă de interacțiune, care prezintă colaborarea dinamică dintre un număr de obiecte, punând accentul pe secvențele de mesaje trimise între acestea pe măsura trecerii timpului.
- **Diagrama de colaborare** este tot o diagramă de interacțiune, dar care, pe lângă interacțiunea dintre obiecte (schimbul de mesaje), prezintă obiectele și legăturile dintre ele.

Modelare orientată pe obiecte cu *UML*

- **Diagrama de stare** descrie ciclul de viață al unui element (al obiectelor, subsistemelor și sistemelor), prin specificarea stărilor în care se găsește elementul și a evenimentelor care îi modifică starea.
- **Diagrama de activitate** prezintă activitățile și responsabilitățile elementelor sistemului, fiind utilizată pentru modelarea funcțiilor sistemului. Ea are ca elemente constitutive stări de acțiune și mesaje care vor fi trimise sau recepționate ca parte a acțiunii realizate.

Modelare orientată pe obiecte cu *UML*

În UML, diagramele fac parte din două categorii.

- **Diagrame dinamice** sau comportamentale – descriu comportamentul și interacțiunile dintre diverse entități ale sistemului informatic.
 - diagramele de secvență, colaborare, stare și activitate.
- **Diagrame statice** sau structurale - descriu structura, responsabilitățile sistemului informatic, componentele executabile ale sistemului, locațiile fizice de execuție și nodurile de stocare a datelor.
 - diagramele claselor, obiectelor, cazurilor de utilizare, componentelor și diagramele de exploatare.

Modelare orientată pe obiecte cu *UML*

- Diagramele UML pot fi desenate și administrate utilizând un utilitar **CASE** (*Computer Aided Software Engineering*).
 - Aceste utilitare sunt deosebit de utile în cazul unor diagrame complexe.
 - Totuși, dacă diagrama este prea complicată, atunci este necesară partiționarea ei în mai multe diagrame sau reprezentarea la un nivel superior de abstractizare.
- Exemple de utilitare CASE care permit realizarea diagramelor UML sunt reprezentate de: *Microsoft Office Visio*, *IBM Rational Rose Professional Data Modeler*, *Lucidchart*, *Altova UModel*, *Borland Together*, *Visual Paradigm for UML*, *ArgoUML* etc.

Modelare orientată pe obiecte cu *UML*

- Interesul pentru suportul **modelării bazelor de date cu ajutorul UML** a condus la crearea unor **profile specifice**.
 - Un profil constituie o **propunere a unei comunități** și regroupează o mulțime de elemente UML care se aplică unui context particular și care conservă metamodelul UML.
 - *IBM Rational Rose* a inclus un astfel de profil adaptat bazelor de date.
- Există o **diferență între un model** (de exemplu, modelul conceptual de date) și un **formalism** (în care este descris un model).
- Astfel, putem vorbi despre **modelarea conceptuală a datelor urmând formalismul entitate-relație sau formalismul UML**. Notația exprimă doar aspectul referitor la reprezentare.

Modelare orientată pe obiecte cu *UML*

- Pe lângă **formalismul entitate-relație**, considerat **standardul *de facto* pentru modelarea datelor**, o opțiune alternativă este oferită de către UML.
- Acesta include **primitive pentru modelarea datelor**, inițial concepute pentru reprezentarea structurii claselor unei aplicații orientate obiect, dar care pot fi folosite pentru **specificarea modelului de date al domeniului unei aplicații**.
 - În particular, diagramele de clase UML pot fi utilizate ca alternativă la diagramele entitate-relație.

Modelare orientată pe obiecte cu *UML*

- **Diferența majoră** dintre o diagramă de clase UML și o diagramă entitate-relație este reprezentată de **diferența dintre o clasă și o entitate**: o clasă este o generalizare a noțiunii de entitate, care permite proiectantului să specifice nu numai attribute, ci și funcții (numite metode) aplicabile instanțelor clasei.
- **UML este mai general decât modelul entitate-relație**
 - iar proiectantul poate exploata diagramele de clase UML pentru a realiza **aceeași specificație** pe care o poate obține cu ajutorul modelului entitate-relație.

Modelare orientată pe obiecte cu *UML*

- Tabelul următor stabilește echivalențele dintre formalismele modelului entitate-relație și al notației UML:

Entitate-relație	UML
Tip entitate	Clasă
Asociere (relație)	Asociere (relație)
Entitate	Obiect
Cardinalitate	Multiplicitate
Model conceptual de date	Diagramă de clase

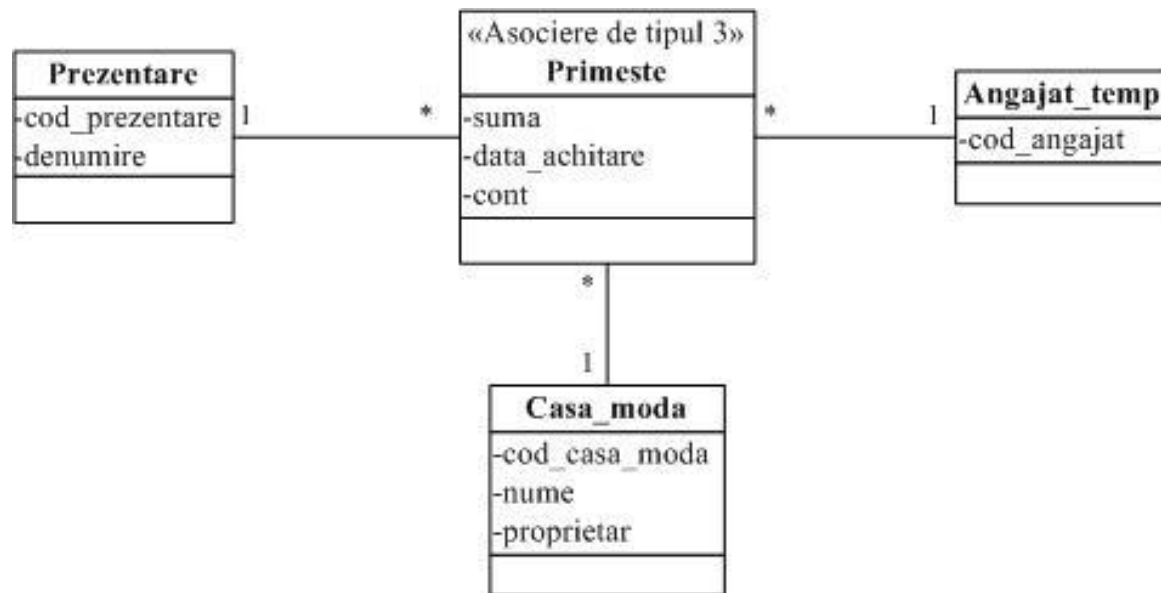
Modelare orientată pe obiecte cu *UML*

- Există **două forme de agregare**: compusă și partajată.
- **Compunerea** exprimă o relație de apartenență mai puternică, apărând atunci când relația este de tipul „compune” sau „**face parte din**”. Reprezentarea compunerii se face prin intermediul unui romb plin, poziționat lângă clasa care reprezintă întregul. Multiplicitatea extremității agregat nu poate depăși 1.
- **Agregarea** partajată presupune că un obiect **poate face parte din mai multe agregate**, iar întregul se poate modifica în timp. O astfel de relație se reprezintă prin intermediul unui romb gol, poziționat lângă clasa care reprezintă întregul.

Modelare orientată pe obiecte cu *UML*

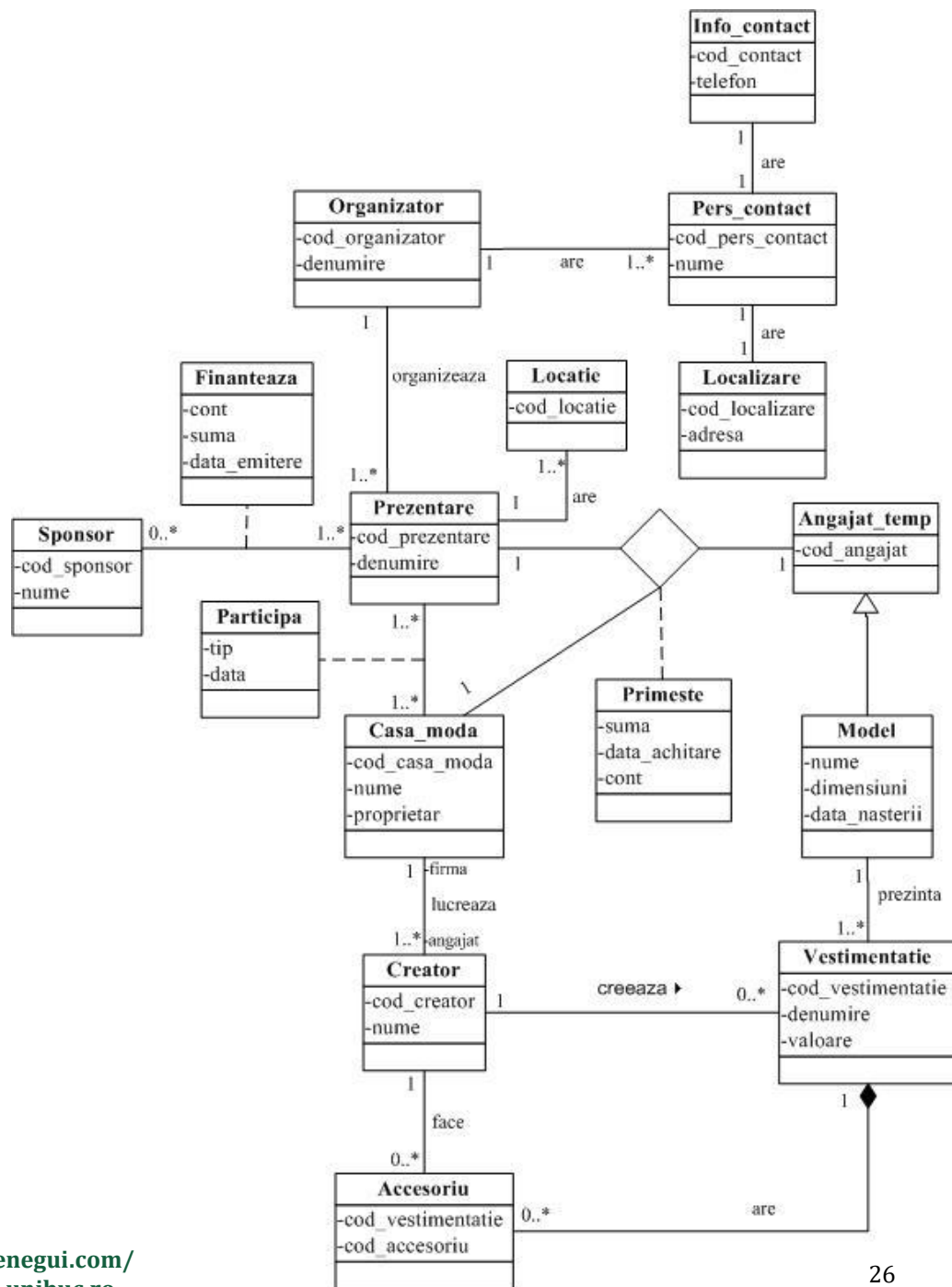
- Relațiile de grade strict mai mari decât 2 sunt reprezentate în UML cu ajutorul unui romb sau prin intermediul unei clase cu **stereotip**.
- Stereotipurile constituie un **mecanism de extensibilitate al UML**.
 - Ele permit extinderea vocabularului UML astfel încât să poată fi definite **noi elemente ale modelului**, derivate din cele existente dar cu proprietăți specifice domeniului problemei.
 - Reprezentarea grafică a unui stereotip se face prin plasarea numelui său, încadrat între caracterele „<<” și „>>” deasupra numelui unui alt element.

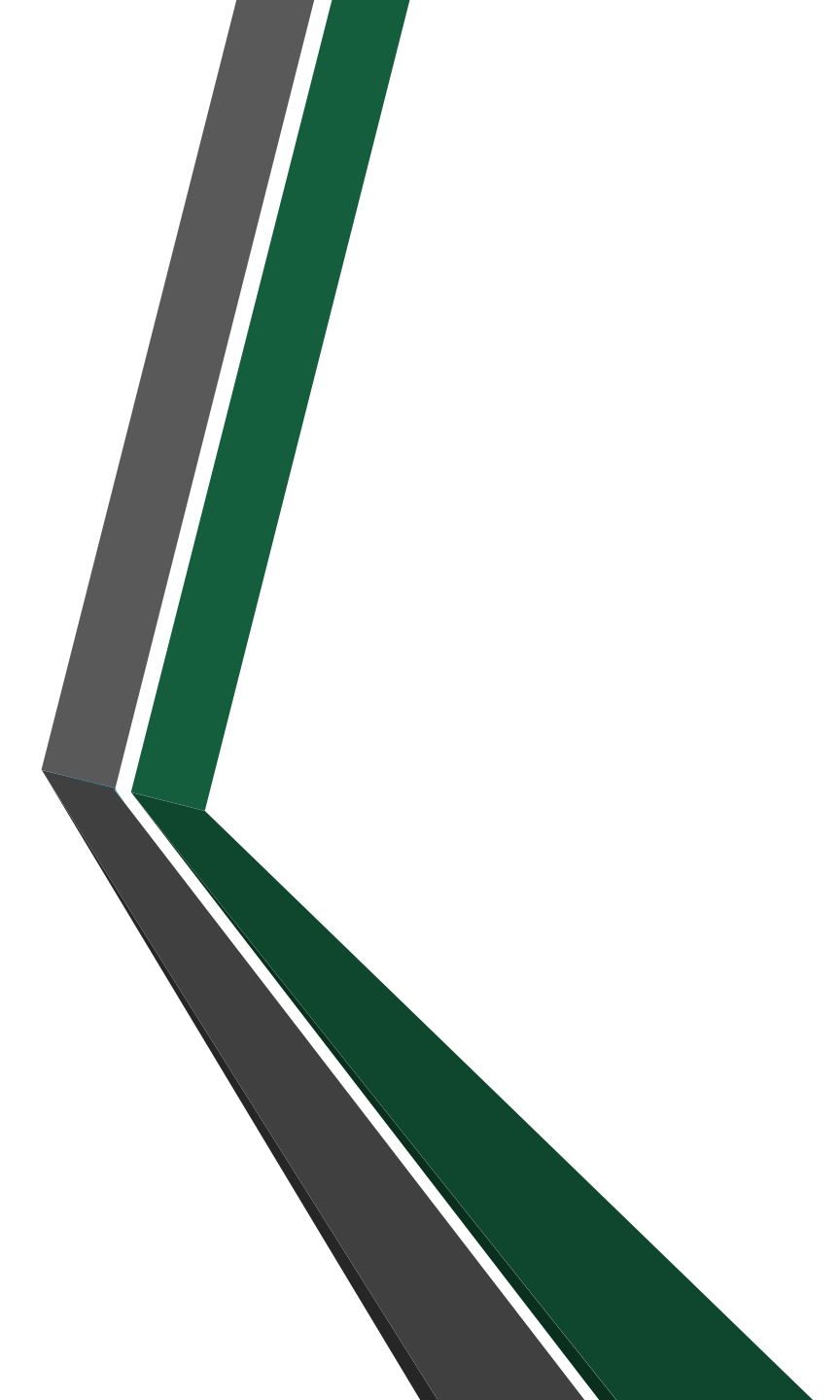
Modelare orientată pe obiecte cu *UML*



Asociere UML de tipul 3 cu stereotip.

Diagrama de clase corespunzătoare unei restricții a modelului.





BAZE DE DATE

CURS 6

ALGEBRA RELAȚIONALĂ

- **LDD** – precizează **entitățile**, relațiile dintre ele, **atributele**, structura atributelor, **cheile**, **constrângerile**, prin urmare definește **structura obiectelor bazei de date** (**schema** bazei de date);
- **LMD** – cuprinde aspecte referitoare la introducerea, modificarea, eliminarea și căutarea datelor;

ALGEBRA RELAȚIONALĂ

- Modelul relațional oferă **două mulțimi de operatori pe relații**:
 - **algebra relațională** (filtrele se obțin aplicând operatori specializați asupra uneia sau mai multor relații din cadrul bazei relaționale);
 - **calculul relațional** (filtrele se obțin cu ajutorul unor formule logice pe care tuplurile rezultatului trebuie să le satisfacă);
- Echivalența dintre algebra relațională și calculul relațional a fost demonstrată de Jeffrey David Ullman. Această echivalență arată că orice relație posibil de definit în algebra relațională poate fi definită și în cadrul calcului relațional, și reciproc.

ALGEBRA RELAȚIONALĂ

- **Algebra relationala** a fost introdusa de *Edgar Frank Codd* si are la baza o **multime de operatii** care se aplica asupra unor relatii, rezultatul fiind tot o relatie;
- Este un limbaj abstract cu ajutorul caruia se pot scrie cererile (se poate face o paralela cu pseudocodul care se transforma in cod interpretat de masina);
- Pentru a scrie cereri folosind ***algebra relationala*** avem nevoie de simboluri specifice, operatii si implicit operatori;
- Cu alte cuvinte, in algebra relationala folosim anumiti operatori specifici, reprezentati cu ajutorul unor simboluri si notatii, pe care ii aplicam unor relatii, rezultatul final fiind tot o relatie;

ALGEBRA RELAȚIONALĂ

De exemplu:

Fie R și S doua **relatii** asupra carora se aplica operatorul **UNION** (reuniune). Rezultatul reuniunii celor doua relatii este tot o relatie și se noteaza:

$$\text{Rezultat} = \text{UNION}(R, S)$$

Prin algebra relationala se pot construi cererile pas cu pas pana la obtinerea unui rezultat final, prin scrierea ***expresiilor relationale***.

ALGEBRA RELAȚIONALĂ

Scopul fundamental al algebrei relaționale este de a permite scrierea **expresiilor relaționale**

- acestea sunt o **reprezentare** de nivel superior, simbolică, a intențiilor utilizatorului și pot fi supuse unei diversități de **reguli de transformare (optimizare)**.

Relațiile sunt închise față de algebra relațională

- operanzii și rezultatele sunt relații → ieșirea unei operații poate deveni intrare pentru alta → posibilitatea imbricării expresiilor în algebra relațională.

ALGEBRA RELAȚIONALĂ

Operatorii algebrei relaționale:

- operatori clasici pe mulțimi (**UNION, INTERSECT, PRODUCT, DIFFERENCE**);
- operatori relaționali speciali (**PROJECT, SELECT, JOIN, DIVISION**).

OPERATORUL PROJECT

- Operatorul **PROJECT** – se utilizeaza atunci cand se doreste preluarea anumitor valori din baza de date (atributele/coloanele $A_1, A_2, \dots, A_n$), fara a fi necesara o conditie.
- Proiecția este o operație unară care elimină anumite atribute ale unei relații producând o submulțime „pe verticală” a acesteia.
 - Suprimarea unor atribute poate avea ca efect apariția unor tupluri duplicate, care trebuie eliminate.
- **Notatii:**
 - **PROJECT** ($R, A_1, \dots, A_m$)
 - $\Pi_{A_1, \dots, A_m} (R)$
 - $R[A_1, \dots, A_m]$

unde $A_1, A_2, \dots, A_m$ sunt parametrii proiecției relativ la relația R .

OPERATORUL PROJECT

Notatie: *PROJECT(R, A1, A2, ... , An)*

Exemplu : Sa se obtina o lista care cuprinde numele si job-ul angajatilor.

Cerere SQL:

```
SELECT last_name, job_id  
FROM employees;
```

Algebra relationala (expresia algebrica):

Rezultat = PROJECT(EMPLOYEES, nume, job);

OPERATORUL PROJECT

1. Proiecție cu dubluri în SQL:

```
SELECT department_id  
FROM employees;
```

2. Proiecție fără dubluri în SQL:

```
SELECT DISTINCT department_id  
FROM employees;
```

OPERATORUL SELECT

- Operatorul **SELECT** – se utilizeaza atunci cand preluam date pe baza unei conditii.
- Selecția (restricția) este o operație unară care produce o submulțime pe „orizontală” a unei relații R .
 - Această submulțime se obține prin extragerea tuplurilor din R care **satisfac o condiție specificată**.
- **Notatii:**
 - **SELECT(R , condiție)**
 - $\sigma_{\text{condiție}}(R)$
 - $R[\text{condiție}]$
 - **RESTRICT(R , condiție).**

OPERATORUL SELECT

Notatie: *SELECT(R, conditie)*

Exemplu : Sa se obtina toate informatiile despre angajatii care au jobul 'IT_PROG'.

Cerere SQL:

```
SELECT *  
FROM employees  
WHERE job_id = 'IT_PROG';
```

Algebra relationala (expresia algebrica):

Rezultat = SELECT(EMPLOYEES, job_id = 'IT_PROG')

OPERATORUL UNION

- Operatorul **UNION** – reuniunea a doua relatii R si S (elementele comune si necomune afisate o singura data).
- **Reuniunea** a două relații R și S este mulțimea tuplurilor aparținând fie lui R , fie lui S , fie ambelor relații.
- **Notatii:**
 - **UNION(R, S)**
 - $R \cup S$
 - $OR(R, S)$
 - $APPEND(R, S)$

OPERATORUL UNION

Notatie: *UNION(R, S)*

Exemplu: Se cer codurile departamentelor al caror nume contine sirul “re” sau in care lucreaza angajati avand codul job-ului “SA_REP”.

Cerere SQL:

```
SELECT department_id  
FROM departments  
WHERE lower(department_name) like '%re%'
```

UNION

```
SELECT department_id  
FROM employees  
WHERE upper(job_id) like '%SA_REP%';
```

OPERATORUL UNION

```
SELECT department_id
FROM departments
WHERE lower(department_name) like '%re%'

UNION

SELECT department_id
FROM employees
WHERE upper(job_id) like '%SA_REP%';
```



Algebra relationala (expresia algebrica):

R1 = SELECT(DEPARTMENTS, department_id LIKE '%re%')

R2 = PROJECT(R1, department_id)

R3 = SELECT(EMPLOYEES, job_id LIKE '%SA_REP%')

R4 = PROJECT(R3, department_id)

Rezultat = *UNION(R2, R4)*

OPERATORUL UNION

Algebra relationala (expresia algebrica):

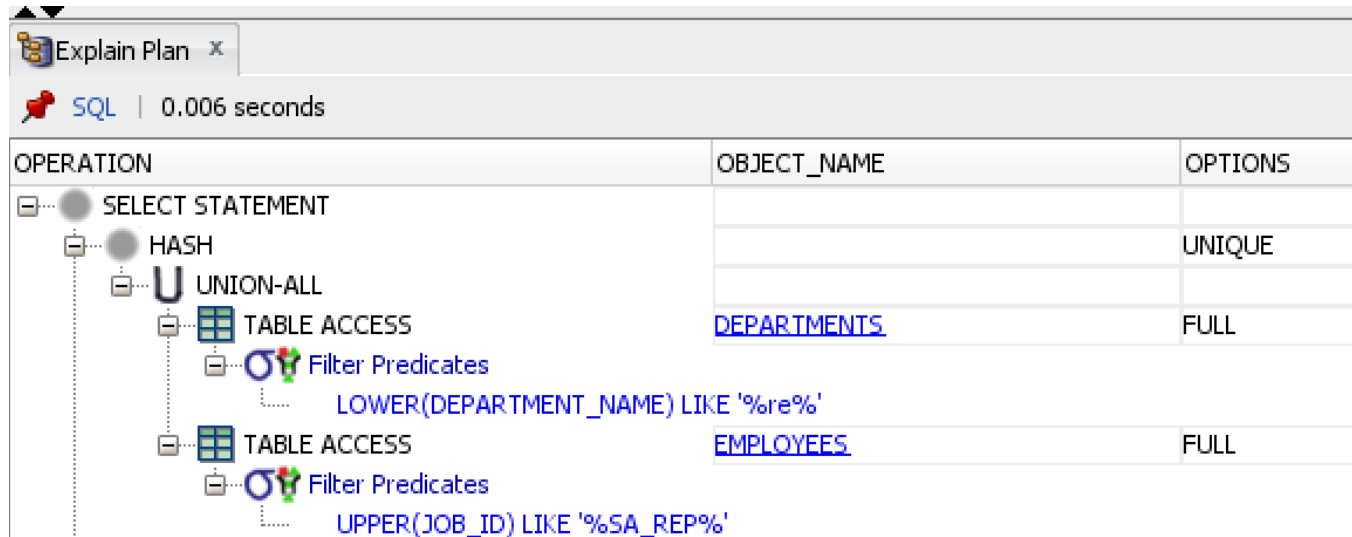
R1 = SELECT(DEPARTMENTS, department_id LIKE '%re%')

R2 = PROJECT(R1, department_id)

R3 = SELECT(EMPLOYEES, job_id LIKE '%SA_REP%')

R4 = PROJECT(R3, department_id)

Rezultat = *UNION(R2, R4)*



OPERATORUL DIFFERENCE

- Operatorul **DIFFERENCE** – diferența a două relații R și S (elementele care sunt în R și nu sunt în S) – este mulțimea tuplurilor care aparțin lui R , dar nu aparțin lui S .
- Diferența este o operație binară necomutativă care permite obținerea tuplurilor ce apar numai într-o relație.
- **Notatii:**
 - **DIFFERENCE(R, S)**
 - $R - S$
 - **REMOVE(R, S)**
 - **MINUS(R, S)**

OPERATORUL DIFFERENCE

Notatie: *DIFFERENCE(R, S)*

Exemplu: Sa se obtina codurile departamentelor in care nu lucreaza nimeni.

Cerere SQL:

```
SELECT department_id
FROM departments -- department_id este cheie primara, deci avem o lista
                  -- unica de departamente
```

MINUS -- din lista tuturor departamentelor dorim sa eliminam
-- departamentele in care lucreaza angajati

```
SELECT department_id
FROM employees; -- department_id este cheie externa, deci sunt
                 -- departamente in care lucreaza angajati
```

Algebra relationala (expresia algebrica):

R1 = PROJECT(DEPARTMENTS, department_id)

R2 = PROJECT(EMPLOYEES, department_id)

Rezultat = DIFFERENCE(R1, R2)

OPERATORUL INTERSECT

- Operatorul **INTERSECT** – intersectia a doua relatii R si S (elementele comune celor doua relatii afisate o singura data).
- Intersecția a două relații R și S este mulțimea tuplurilor care aparțin atât lui R , cât și lui S . Operatorul INTERSECT este un operator binar, comutativ, derivat:
 - $R \cap S = R - (R - S)$
 - $R \cap S = S - (S - R)$.
- **Notatii:**
 - **INTERSECT(R, S)**
 - $R \cap S$
 - **AND(R, S)**
- Operatorii **INTERSECT** și **DIFFERENCE** pot fi simulați în SQL (în cadrul comenzii SELECT) cu ajutorul opțiunilor EXISTS, NOT EXISTS, IN, NOT IN (!= ANY) – o sa implementam in laborator toate exemplele

OPERATORUL INTERSECT

Notatie: *INTERSECT(R, S)*

Exemplu: Sa se afiseze angajatii (codul lor) care sunt manageri in cadrul departamentelor si in acelasi timp sunt si managerii altor angajati.

Cerere SQL:

```
SELECT manager_id  
FROM departments -- managerii de departament
```

INTERSECT

```
SELECT manager_id  
FROM employees; -- managerii angajatilor
```

Algebra relationala (expresia algebrica):

$R1 = \text{PROJECT}(\text{DEPARTMENTS}, \text{manager_id})$

$R2 = \text{PROJECT}(\text{EMPLOYEES}, \text{manager_id})$

Rezultat = $\text{INTERSECT}(R1, R2)$

OPERATORUL INTERSECT

Intersectia se poate simula utilizand operatorul **IN** sau operatorul **EXISTS** (il vom studia in laborator).

Cum procedam atunci cand utilizam operatorul **IN**?

Cerere SQL (simularea intersectiei folosind operatorul **IN**):

```
SELECT manager_id  
FROM departments  
WHERE manager_id IN (SELECT manager_id  
                      FROM employees );
```

OPERATORUL PRODUCT

- Operatorul **PRODUCT** – produsul cartezian dintre relatia R si relatia S .
- Fie R și S relații de aritate m , respectiv n . Produsul cartezian al lui R cu S este mulțimea tuplurilor de aritate $m + n$ unde primele m componente formează un tuplu în R , iar ultimele n componente formează un tuplu în S .
- **Notatii:**
 - **PRODUCT(R, S)**
 - $R \times S$
 - **TIMES(R, S)**

OPERATORUL PRODUCT

Notatie: *PRODUCT(R, S)*

Exemplu: Sa se afiseze toti angajatii (codurile si numele angajatilor) impreuna cu toate departamentele (numele si codurile departamentelor).

Cerere SQL:

```
SELECT employee_id, last_name, d.department_id, department_name  
FROM employees e, departments d;
```

Algebra relationala (expresia algebrica):

$R1 = \text{PROJECT}(\text{EMPLOYEES}, \text{employee_id}, \text{last_name})$

$R2 = \text{PROJECT}(\text{DEPARTMENTS}, \text{department_id}, \text{department_name})$

Rezultat = $\text{PRODUCT}(R1, R2)$

Cerere SQL (utilizand CROSS JOIN):

```
SELECT employee_id, last_name, d.department_id, department_name  
FROM employees e CROSS JOIN departments d;
```

OPERATORUL JOIN

- Operatorul **JOIN** – permite regasirea informatiei din mai multe relatii corelate. Operatorul combina **produsul cartezian** cu **selectia** si **proiectia**.
- 4 tipuri de **JOIN**:
 - **NATURAL JOIN**
 - **θ-JOIN**
 - **SEMI-JOIN**
 - **OUTER JOIN**

NATURAL JOIN

- **NATURAL JOIN** – operatorul de compunere naturală combină tupluri din două relații R și S , cu condiția ca **atributele comune să aibă valori identice**.
- Cu alte cuvinte, acest tip de join realizează join automat bazat pe coloanele comune existente în cele două tabele între care se realizează operația de join. Coloanele comune sunt cele care au același nume în ambele tabele.

NATURAL JOIN

Algoritmul care realizează compunerea naturală este următorul:

1. se calculează produsul cartezian $R \times S$;
2. pentru fiecare atribut comun A care definește o coloană în R și o coloană în S , se selectează din $R \times S$ tuplurile ale căror valori coincid în coloanele $R.A$ și $S.A$ (atributul $R.A$ reprezintă numele coloanei din $R \times S$ corespunzătoare coloanei A din R);
3. pentru fiecare astfel de atribut A se elimină coloana $S.A$, iar coloana $R.A$ se va numi A .

NATURAL JOIN

- Operatorul NATURAL JOIN poate fi exprimat formal astfel:

$$\text{JOIN}(R, S) = \Pi_{i_1, \dots, i_m} \sigma_{(R.A_1 = S.A_1) \wedge \dots \wedge (R.A_k = S.A_k)}(R \times S),$$

unde $A_1, \dots, A_k$ sunt attributele comune lui R și S , iar $i_1, \dots, i_m$ reprezintă lista componentelor din $R \times S$ (păstrând ordinea inițială) din care au fost eliminate componentele $S.A_1, \dots, S.A_k$.

NATURAL JOIN

Notatie: *JOIN(R, S)*

Exemplu: Sa se afiseze informatii despre angajatii care lucreaza in departamente si care sunt in acelasi timp atat manageri de departament, cat si managerii altor angajati.

Algoritmul compunerii naturale:

1. Se calculeaza produsul cartezian dintre relatiile R si S (R reprezinta relatia employees, iar S relatia departments);

```
SELECT employee_id, last_name, d.department_id,  
department_name, d.manager_id  
FROM employees e, departments d;
```

NATURAL JOIN

2. Se selecteaza tuplurile pentru care exista valori comune atat in relatia R, cat si in relatia S (valorile comune sunt – department_id si manager_id – acestea facand parte din ambele relatii)

*WHERE e.department_id = d.department_id AND
e.manager_id = d.manager_id;*

3. Pentru fiecare atribut existent in ambele relatii, in afisare se pastreaza doar o singura data coloana comuna (in cazul nostru o sa avem o singura data coloana department_id si o singura data coloana manager_id).

NATURAL JOIN

Algoritmul anterior se aplica automat atunci cand utilizam **NATURAL JOIN**, nefiind nevoie de aliasuri.

```
SELECT employee_id, last_name, department_id, department_name,  
manager_id  
FROM employees NATURAL JOIN departments;
```

Algebra relationala (expresia algebrica):

Rezultat = JOIN(EMPLOYEES, DEPARTMENTS)

θ -JOIN

- **θ -JOIN** – combina tupluri (coloane/atribute) din doua relatii R si S pe baza unei conditii specificata in cadrul operatiei de JOIN. Aceasta conditie poate fi bazata pe egalitatea unor coloane (de obicei cheia primara si cheia externa) – acesta fiind un caz particular de **θ -JOIN**, si anume **EQUIJOIN (sau inner join)**, dar sunt utilizate in egala masura si conditii care implica operatorii <, <=, >, >=, != (acest tip de join fiind un **NONEQUIJOIN**)
- Operatorul **θ -JOIN** este un operator derivat, fiind o combinație de produs scalar și selecție:

$$\text{JOIN}(R, S, \text{condiție}) = \sigma_{\text{condiție}} (R \times S)$$

θ -JOIN

Notatie: *JOIN(R, S, conditie)*

Exemplu: Sa se afiseze numele angajatilor impreuna cu denumirea jobului in cadrul caruia lucreaza.

Cerere SQL:

```
SELECT last_name, job_title  
FROM employees e, jobs j  
WHERE e.job_id = j.job_id;
```

Algebra relationala (expresia algebrica):

$R1 = \text{PROJECT}(\text{EMPLOYEES}, \text{last_name})$

$R2 = \text{PROJECT}(\text{JOBS}, \text{job_title})$

Rezultat = $\text{JOIN}(R1, R2, \text{EMPLOYEES.job_id} = \text{JOBS.job_id})$

SEMI-JOIN

- **SEMI-JOIN** – se utilizeaza atunci cand din cele doua relatii participante *R* si *S* se utilizeaza doar attributele relatiei *R* sau doar cele ale relatiei *S*.
- Operatorul SEMI-JOIN conservă **attributele unei singure relații participante** la compunere și este utilizat când nu sunt necesare toate attributele compunerii. Operatorul este asimetric.
 - Tupluri ale relației *R* care participă în compunerea (naturală sau θ -JOIN) dintre relațiile *R* și *S*.
- SEMI-JOIN este un operator derivat, fiind o combinație de proiecție și compunere naturală sau proiecție și θ -JOIN:

Notatie: $SEMIJOIN(R, S) = PROJECT(JOIN(R, S))$
 $SEMIJOIN(R, S, conditie) = PROJECT(JOIN(R, S, conditie))$

SEMI-JOIN

Exemplu: Sa se afiseze departamentele (codul si denumirea) care au cel putin un angajat.

Se afiseaza in final coloane care fac parte doar dintr-un tabel – departments => operatorul SEMI-JOIN conservă **atributele unei singure relații participante**.

```
SELECT d.department_id, department_name  
FROM employees e, departments d  
WHERE e.department_id = d.department_id;
```

Algebra relationala (expresia algebrica):

```
R1 = JOIN(EMPLOYEES, DEPARTMENTS, EMPLOYEES.department_id =  
DEPARTMENTS.department_id)  
Rezultat = PROJECT(R1, department_id, department_name)
```

OUTER JOIN

- **OUTER JOIN** – pentru doua relatii R si S , acest operator returneaza valorile atributelor care indeplinesc conditia de corelare (adica intersectia celor doua relatii pe baza unei valori comune – join clasic), impreuna cu valori NULL in functie de tipul acestui operator.
- Cu alte cuvinte, operația de **compunere externă** combină tupluri din două relații, tupluri pentru care nu sunt satisfăcute condițiile de corelare.
- În cazul aplicării operatorului JOIN se pot pierde tupluri, atunci când **există un tuplu în una din relații pentru care nu există nici un tuplu în cealaltă relație**, astfel încât să fie satisfăcută relația de corelare.
- Operatorul elimină acest inconvenient prin **atribuirea valorii null** valorilor atributelor care există într-un tuplu din una dintre relațiile de intrare, dar care nu există și în cea de-a doua relație.
- Practic, se realizează compunerea a două relații R și S la care se adaugă tupluri din R și S , care nu sunt conținute în compunere, completate cu valori *null* pentru attributele care lipsesc.

OUTER JOIN

➤ Compunerea externă poate fi: LEFT, RIGHT, FULL.

➤ De exemplu:

R LEFT OUTER JOIN S – reprezintă compunerea în care tuplurile din *R*, care nu au valori similare în coloanele comune cu relația *S*, sunt de asemenea incluse în relația rezultat.

➤ Asadar,

R LEFT OUTER JOIN S – se vor afisa valorile din *R* si *S* care respecta conditia de JOIN (intersectia) impreuna cu valorile care sunt in *R* (LEFT) si nu sunt in *S*. In acest caz valorile care lipsesc vor fi automat completate cu NULL.

OUTER JOIN

Exemplu:

Consideram tabelul **EMPLOYEES** ca fiind relatia **R** si tabelul **DEPARTMENTS** ca fiind relatia **S**

Sa se afiseze angajatii (cod si nume) impreuna cu departamentele in care lucreaza (cod si denumire). Rezultatul o sa includa toti angajatii chiar daca nu au departament.

```
SELECT employee_id, d.department_id, last_name, department_name  
FROM employees e LEFT JOIN departments d ON (e.department_id =  
d.department_id);
```

OUTER JOIN

- In output se observa angajatii care lucreaza intr-un departament, adica valorile din **EMPLOYEES** si **DEPARTMENTS** (din *R* si *S*) care indeplinesc conditia de corelare (**e.department_id = d.department_id**), impreuna cu valorile care sunt in **EMPLOYEES** (*R*) si nu sunt in **DEPARTMENTS** (*S*), adica angajatii care nu au departament.
- Valorile care lipsesc vor fi completate automat cu **NULL** – in exemplul nostru valoarea care lipseste este cea specifica departamentului deoarece sunt afisati si angajatii **care nu au departament**, ceea ce inseamna ca angajatul exista, dar departamentul nu. In acest caz o sa existe un rezultat ca acesta :

178 NULL Grant NULL – coloanele reprezentative departamentului (department_id si department_name sunt automat completate cu valoarea NULL)

OUTER JOIN

- In mod asemanator se vor comporta si variantele urmatoare de **JOIN** (*RIGHT OUTER JOIN* si *FULL OUTER JOIN*)
- **R RIGHT OUTER JOIN S** – se vor afisa valorile din R si S care respecta conditia de JOIN (intersectia) impreuna cu valorile care sunt in S (RIGHT) si nu sunt in R. In acest caz valorile care lipsesc vor fi automat completate cu NULL.
- **R FULL OUTER JOIN S** – se vor afisa valorile din R si S care respecta conditia de JOIN (intersectia) impreuna cu valorile care sunt in R si nu sunt in S si valorile care sunt in S si nu sunt in R. In acest caz valorile care lipsesc vor fi automat completate cu NULL.

OPERATORUL DIVISION

- **Diviziunea** este o operație binară care definește o relație ce conține valorile atributelor dintr-o relație care apar **în toate** valorile atributelor din cealaltă relație.
- **Notatii:**
 - **DIVISION(R, S)**
 - **DIVIDE(R, S)**
 - **$R \div S$**
- Diviziunea conține acele tupluri de dimensiune $n - m$ la care, adăugând orice tuplu din S , se obține un tuplu din R .
- Operatorul diviziune poate fi exprimat formal astfel:
 - **$R^{(n)} \div S^{(m)} = \{t^{(n-m)} \mid \forall s \in S, (t, s) \in R\}$** , unde $n > m$ și $S \neq \emptyset$.

ALGEBRA RELAȚIONALĂ

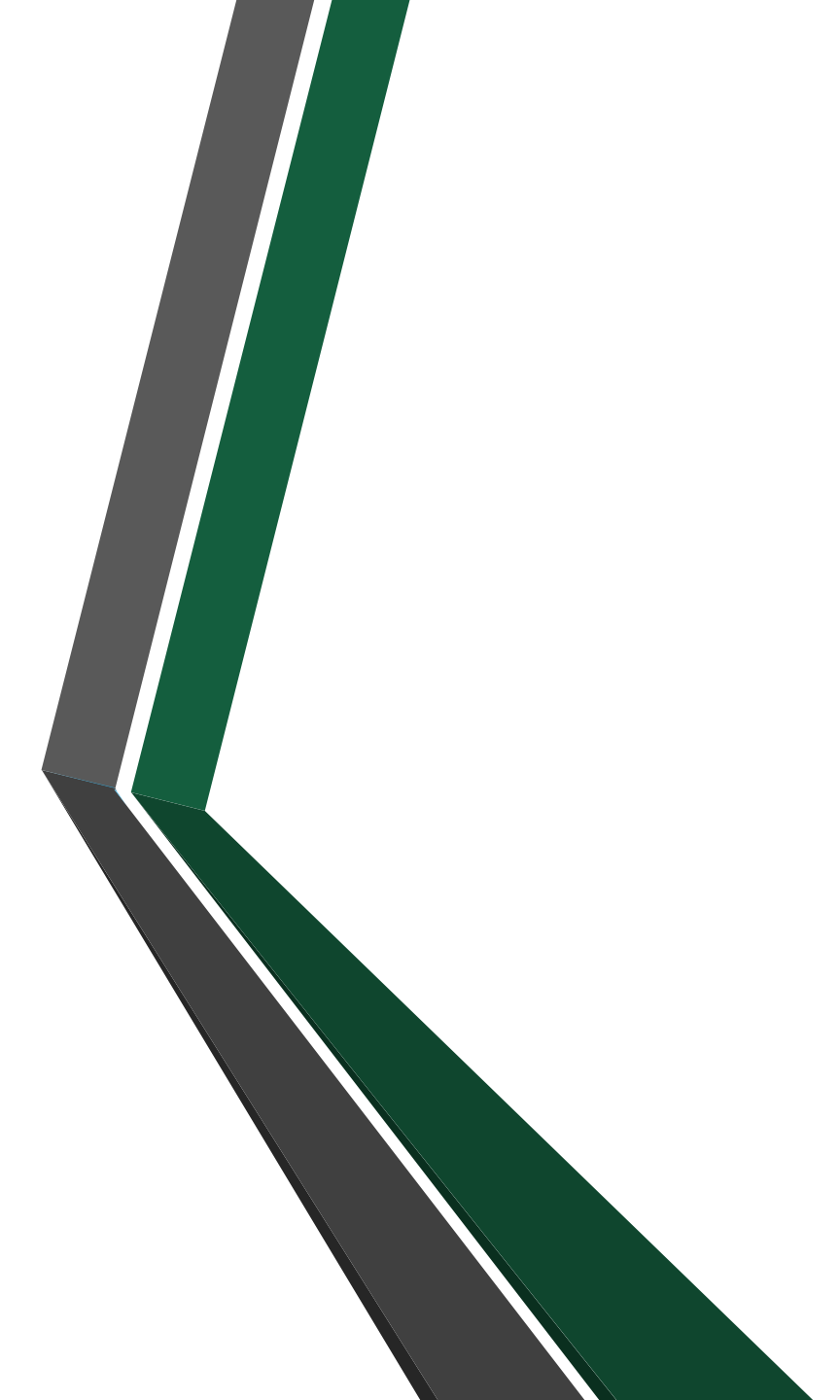
- Operatorul **DIVISION** este legat de cuantificatorul universal ($\forall$) care nu există în SQL.

- Cuantificatorul universal poate fi însă simulat cu ajutorul cuantificatorului existențial ($\exists$) utilizând relația:

$$\forall x P(x) \equiv \neg \exists x \neg P(x).$$

- Prin urmare, operatorul **DIVISION** poate fi exprimat în SQL prin succesiunea a doi operatori NOT EXISTS.

- **Acest operator o sa fie studiat in cadrul laboratorului**



BAZE DE DATE

CURS 7

Evaluarea și optimizarea interogărilor

Procesarea interogărilor

- O **expresie a algebrei relaționale** este constituită din **relații** legate prin **operații** din algebra relațională;
- O expresie se poate reprezenta **grafic** cu ajutorul unui **arbore**, numit **arbore algebric**, în care nodurile corespund operatorilor din cadrul expresiei respective;
- Evaluarea unei expresii presupune efectuarea prelucrărilor indicate de operatorii din expresie în ordinea aparițiilor sau în ordinea fixată prin paranteze;
- **Rezultatul** evaluării unei expresii este o **relație derivată** din relațiile menționate ca operanzi în cadrul expresiei;
- Două expresii sunt echivalente, dacă în urma evaluării lor se obține ca rezultat aceeași relație;

Procesarea interogărilor

- În majoritatea sistemelor de gestiune, în special în cele relaționale, interfața cu utilizatorul este de tip **neprocedural**;
 - Utilizatorul definește datele pe care dorește să le vizualizeze fără a da algoritmi de acces (**CE**, nu **CUM**)
 - Sistemul trebuie să convertească cererea utilizatorului:
 - într-o cerere optimală;
 - în proceduri de acces optimal la datele fizice;
- Garantarea absolută a performanțelor optime pentru procesorul limbajului relațional este imposibilă;
- Corectă ar fi utilizarea cuvântului „**ameliorare**” în locul cuvântului „**optimizare**”;

Procesarea interogărilor

În momentul în care se implementează o cerere (interogare) în SQL, sistemul de gestiune (Oracle Database) evaluează cererea.

Evaluarea unei interogări se efectuează în trei etape:

- **Analiza cererii** – presupune **studierea semantică și sintactică** a cererii pentru a verifica corectitudinea sa și a simplifica criteriul de căutare.

Semantic – se verifică dacă programul rulează și compilează fără erori. **Sintactic** – se verifică dacă programul rulează și compilează fără erori, dar și dacă este corectă sintaxa utilizată (pentru a verifica corectitudinea cererii). În acest moment se verifică și accesul la datele implicate în cererea SQL.

Procesarea interogărilor

- **Ordonanțarea/Ordonarea** – presupune **descompunerea cererii într-o mulțime de operații elementare** și determinarea unei ordini optime a acestor operații.

Operațiile sunt, în general, cele ale **algebrei relaționale** (SELECT, PROJECT, JOIN, DIVISION, UNION, INTERSECT, PRODUCT, DIFFERENCE), după care se stabilește o ordine de execuție optimă a acestor operații.

La sfârșitul etapei se obține un **plan de execuție al cererii**.

Procesarea interogărilor

- **Execuția** – presupune **efectuarea** (paralelă și/sau secvențială) operațiilor elementare furnizate de planul de execuție pentru a obține **rezultatul cererii**.

Procesarea interogărilor

- Presupunem că utilizatorul transmite sistemului de gestiune o **cerere** exprimată prin **comenzi SQL**
- Pentru a răspunde cererii, SGBD-ul trebuie să înțeleagă cererea utilizatorului
- Cererea trebuie să:
 - fie corectă sintactic;
 - aibă datele **disponibile** utilizatorului;
 - localizeze datele analizând diferite **drumuri de acces** la ele;

Procesarea interogărilor

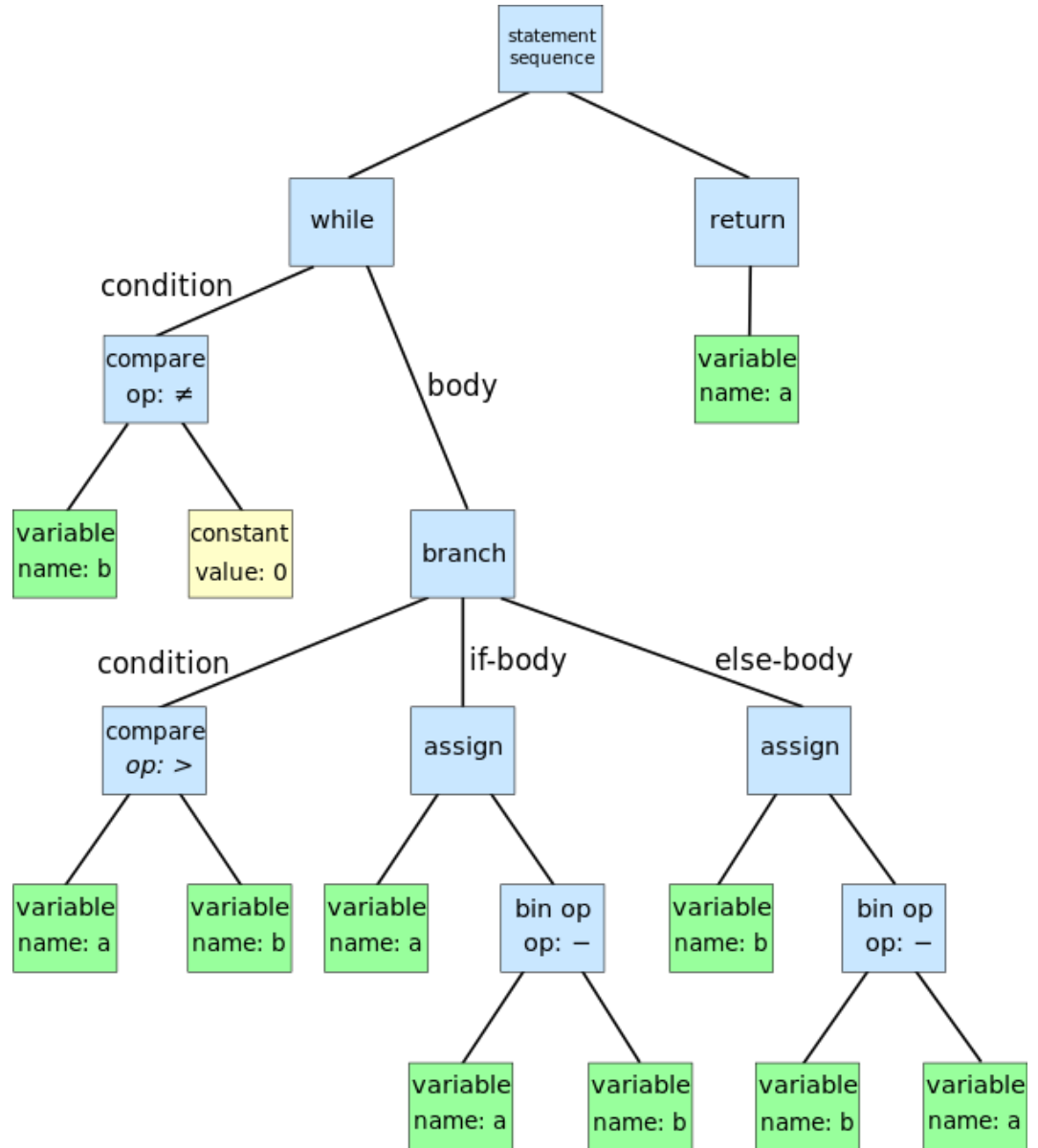
- Aceste funcții sunt realizate de SGBD cu ajutorul a **două module funcționale** care comunică permanent:
 - **analizorul cererilor**, care asigură:
 - verificarea sintactică și semantică a cererii;
 - localizarea datelor implicate în cerere (găsirea adresei blocurilor ce conțin datele);
 - furnizarea planului de execuție;
 - **administratorul datelor** (executorul), care execută efectiv cererea (primește planurile de execuție furnizate de modulul de optimizare și le execută)
 - Execuția presupune căutarea blocurilor ce conțin datele și transferul blocurilor în memoria *cache*;

Procesarea interogărilor

- In general, compilatorul efectueaza trei tipuri de analiza: **semantica, sintactica, lexicala**
- Se verifica daca programul ruleaza si compileaza fara erori. Erorile semantice sunt erorile de compilare care nu sunt erori de sintaxa (ex: nu avem tipuri de date compatibile, nu se realizeaza o conversie automata, nu are acces la o proprietate, etc).
- In final, pentru realizarea tuturor verificarilor, compilatorul realizeaza un arbore, dupa cum se observa in exemplul urmator:

Procesarea interogărilor

```
while b ≠ 0
  if a > b
    a = a - b
  else
    b = b - a
return a
```



Procesarea interogărilor

- In acelasi mod executa o cerere si compilatorul SQL, convertind cererea utilizatorului intr-o cerere optima cu ajutorul unui arbore algebric.

Ideea generală:

cerere → arbore algebric (nu este unic) → plan de execuție → optimizare

Procesarea interogărilor

- Cu ajutorul **planului de execuție** se realizează o evaluare a cererii în vederea realizării unor optimizări;
- **Arborele algebric** nefiind unic, pot exista planuri de execuție diferite, dar care în final să producă același rezultat;
- **Scopul optimizării** este acela de **a identifica planul de execuție optim** (de cost minim);
- Un plan de execuție implică:
 - **o secvență de pași pentru evaluarea cererii** - în mod obișnuit, fiecare pas din planul de execuție corespunde unei operații relaționale;
 - **metoda care va fi folosită pentru evaluarea operației** - de obicei, pentru o operație relațională dată, există mai multe metode ce pot fi folosite pentru evaluarea acesteia;

Procesarea interogărilor

- Două planuri de execuție diferite care au întotdeauna același rezultat se numesc **echivalente**. Planuri de execuție echivalente pot avea diferite costuri;
- Scopul optimizării cererilor este de a găsi, printre diversele planuri de execuție echivalente, pe acela de cost minim;
- Într-un sistem centralizat, costul evaluării unei cereri este suma a două componente:
 - **costul I/O** (transferuri de date) + **costul CPU** (verificare de condiții, operații *join*, etc.)

Ordinea de execuție a operațiilor

- O interogare constă dintr-un număr de operații
 - ordinea în care se efectuează operațiile are un rol important în evaluarea costului necesar realizării interogării;
- Există **două modalități** de abordare pentru a determina ordinea de execuție a operațiilor:
 - algebric;
 - bazat pe estimarea costului;
- Ambele folosesc o mulțime de **reguli** care permit **transformarea unui plan de execuție** (reprezentat ca o expresie scrisă în termenii algebrei relaționale) în altul, echivalent

Ordinea de execuție a operațiilor

- Astfel, o **cerere SQL** se poate scrie sub forma unei **expresii a algebrei relationale**, formata din relatii si operatii specifice algebrei relationale, dupa care se poate reprezenta grafic sub forma de arbore, numit **arbore algebric**, in care nodurile corespund operatorilor algebrei relationale utilizati in cadrul cererii, urmand etapa de **evaluare** si **optimizare** a cererii;
- Optimizarea cererilor utilizand algebra relationala se realizeaza pornind de la scrierea cererii folosind expresii algebrice, dupa care se aplica transformari echivalente, dar mult mai optime;
- Pentru optimizare se utilizeaza si **proprietatile operatorilor algebrei relationale**;

Proprietățile operatorilor algebrei relaționale

Proprietățile sunt explicate detaliat, fiind însoțite de exemple, în documentul
Curs_7_Proprietati_Operatorii_Exemple

Proprietățile operatorilor algebrei relaționale

Proprietatea 1. Comutativitatea operațiilor de *join* și produs cartezian:

$$\text{JOIN}(R1, R2) = \text{JOIN}(R2, R1)$$

$$R1 \times R2 = R2 \times R1$$

Proprietatea 2. Asociativitatea operațiilor de *join* și produs cartezian:

$$\text{JOIN}(\text{JOIN}(R1, R2), R3) = \text{JOIN}(R1, \text{JOIN}(R2, R3))$$

$$(R1 \times R2) \times R3 = R1 \times (R2 \times R3)$$

Proprietatea 3. Compunerea proiecțiilor:

$$\Pi_{A1, \dots, Am} (\Pi_{B1, \dots, Bn} (R)) = \Pi_{A1, \dots, Am} (R),$$

unde $\{A_1, A_2, \dots, A_m\} \subseteq \{B_1, B_2, \dots, B_n\}$.

Proprietatea 4. Compunerea selecțiilor:

$$\sigma_{cond1} (\sigma_{cond2} (R)) = \sigma_{cond1 \wedge cond2} (R) = \sigma_{cond2} (\sigma_{cond1} (R)),$$

unde am notat prin *cond* condiția după care se face selecția.

Proprietățile operatorilor algebrei relaționale

Proprietatea 5. Comutarea selecției cu proiecția:

$$\Pi_{A_1, \dots, A_m} (\sigma_{cond} (R)) = \sigma_{cond} (\Pi_{A_1, \dots, A_m} (R)),$$

unde condiția după care se face selecția implică numai attributele $A_1, \dots, A_m$.

Dacă condiția implică și attributele $B_1, \dots, B_n$, care nu aparțin mulțimii $\{A_1, \dots, A_m\}$, atunci:

$$\Pi_{A_1, \dots, A_m} (\sigma_{cond} (R)) = \Pi_{A_1, \dots, A_m} (\sigma_{cond} (\Pi_{A_1, \dots, A_m, B_1, \dots, B_n} (R)))$$

Proprietățile operatorilor algebrei relaționale

Proprietatea 6. Comutarea selecției cu produsul cartezian:

Dacă toate atributele menționate în condiția după care se face selecția sunt atribute ale relației $R1$, atunci:

$$\sigma_{cond} (R1 \times R2) = \sigma_{cond} (R1) \times R2$$

Dacă condiția este de forma $cond1 \wedge cond2$ și dacă $cond1$ implică numai atribute din $R1$, iar $cond2$ implică numai atribute din $R2$, atunci

$$\sigma_{cond} (R1 \times R2) = \sigma_{cond1} (R1) \times \sigma_{cond2} (R2)$$

Dacă $cond1$ implică numai atribute din $R1$, iar $cond2$ implică atribute atât din $R1$ cât și din $R2$, atunci:

$$\sigma_{cond} (R1 \times R2) = \sigma_{cond2} (\sigma_{cond1} (R1) \times R2)$$

Proprietățile operatorilor algebrei relaționale

Proprietatea 7. Comutarea selecției cu reuniunea:

$$\sigma_{cond} (R1 \cup R2) = \sigma_{cond} (R1) \cup \sigma_{cond} (R2)$$

Proprietatea 8. Comutarea selecției cu diferența:

$$\sigma_{cond} (R1 - R2) = \sigma_{cond} (R1) - \sigma_{cond} (R2)$$

Proprietatea 9. Comutarea proiecției cu produsul cartezian:

Dacă $A_1, \dots, A_m$ este o listă de atribute ce apar în schemele relaționale $R1$ și $R2$ și dacă lista este formată din atribute aparținând lui $R1$ (notate prin $B_1, \dots, B_n$) și din atribute aparținând lui $R2$ (notate prin $C_1, \dots, C_k$) atunci:

$$\Pi_{A_1, \dots, A_m} (R1 \times R2) = \Pi_{B_1, \dots, B_n} (R1) \times \Pi_{C_1, \dots, C_k} (R2)$$

Proprietatea 10. Comutarea proiecției cu reuniunea:

$$\Pi_{A_1, \dots, A_m} (R1 \cup R2) = \Pi_{A_1, \dots, A_m} (R1) \cup \Pi_{A_1, \dots, A_m} (R2)$$

Proprietățile operatorilor algebrei relaționale

Proprietatea 11. Compunerea proiecției cu operația *join*:

Dacă $A_1, \dots, A_m$ este o listă de attribute ce apar în schemele relaționale $R1$ și $R2$ și dacă lista este formată din attribute aparținând lui $R1$ (notate prin $B_1, \dots, B_n$) și din attribute aparținând lui $R2$ (notate prin $C_1, \dots, C_k$) atunci:

$$\Pi_{A_1, \dots, A_m} (\text{JOIN}(R1, R2, D)) = \Pi_{A_1, \dots, A_m} (\text{JOIN}(\Pi_{D, B_1, \dots, B_n}(R1), \Pi_{D, C_1, \dots, C_k}(R2), D),$$

unde am notat prin $\text{JOIN}(R1, R2, D)$ operația de compunere naturală între $R1$ și $R2$ după atributul comun D .

Proprietățile operatorilor algebrei relaționale

Proprietatea 12. Compunerea selecției cu operația *join*:

$$\sigma_{\text{cond}} (\text{JOIN} (R1, R2, D)) = \sigma_{\text{cond}} (\text{JOIN} (\Pi_{D,A} (R1), \Pi_{D,A} (R2), D)),$$

unde A reprezintă attributele care apar în condiția după care se face selecția.

Reguli de optimizare frecvent folosite

Regula de optimizare 1:

Selecțiile se execută cât mai devreme posibil. Motivația acestei reguli este că selecțiile reduc substanțial dimensiunea relațiilor. Regula de transformare 4 poate fi folosită pentru a separa două sau mai multe selecții în selecții individuale care pot fi distribuite *join*-ului sau produsului cartezian folosind comutarea selecției cu *join*-ul.

Reguli de optimizare frecvent folosite

Regula de optimizare 2:

Produsele carteziane se înlocuiesc cu *join*-uri, ori de câte ori este posibil. Un produs cartezian între două relații este de obicei mult mai scump (ca și cost) decât un *join* între cele două relații, deoarece primul generează concatenarea tuplurilor în mod exhaustiv și poate genera un rezultat foarte mare. Această transformare se poate realiza folosind legătura dintre produs cartezian, *join* și selecție.

Reguli de optimizare frecvent folosite

Regula de optimizare 3:

Dacă sunt mai multe *join*-uri atunci cel care se execută primul este cel mai restrictiv

- Un *join* este mai restrictiv decât altul dacă produce o relație mai mică;
- Se poate determina care *join* este mai restrictiv pe baza selectivității sau cu ajutorul informațiilor statistice;
- Algebric, acest lucru se poate realiza folosind regula de transformare 2;

Reguli de optimizare frecvent folosite

Regula de optimizare 4:

Proiecțiile se execută la început pentru a îndepărta attributele nefolositoare.

- Dacă un atribut al unei relații nu este folosit în operațiile ulterioare atunci trebuie îndepărtat. În felul acesta se va folosi o relație mai mică în operațiile ulterioare;
- Aceasta se poate realiza folosind comutarea proiecției cu *join*-ul;

Arbori algebrici. Simbolurile operatorilor algebrei relationale.

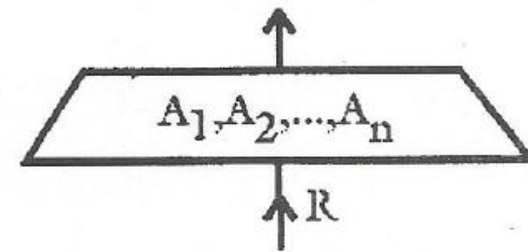
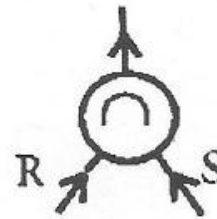
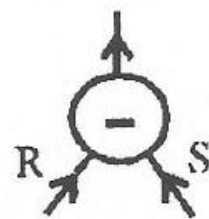
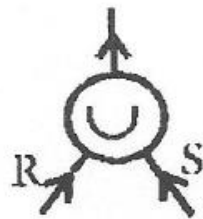
- In continuare vom studia simbolurile operatorilor algebrei relationale;
- Cu ajutorul acestora, fiecare operator al algebrei relationale se poate reprezenta grafic;
- Reprezentarea grafica o sa formeze in final arborele algebric asociat expresiei algebrice

UNION

DIFFERENCE

INTERSECT

PROJECT

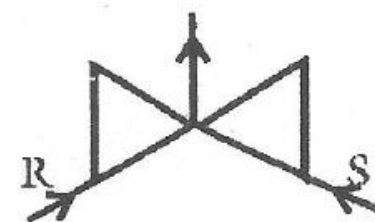
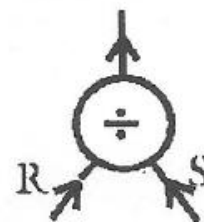
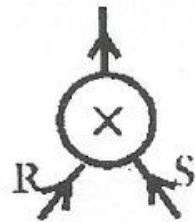
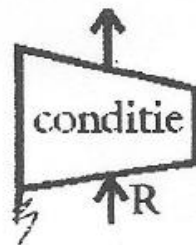


SELECT

PRODUCT

DIVISION

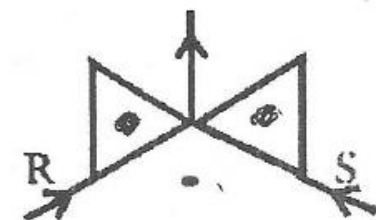
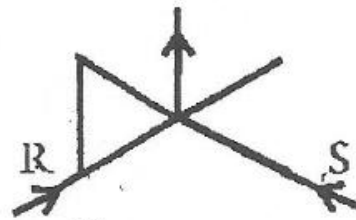
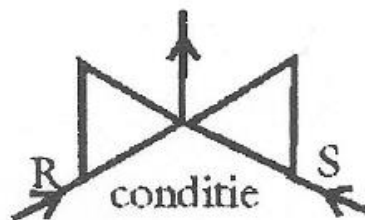
NATURAL JOIN



θ -JOIN

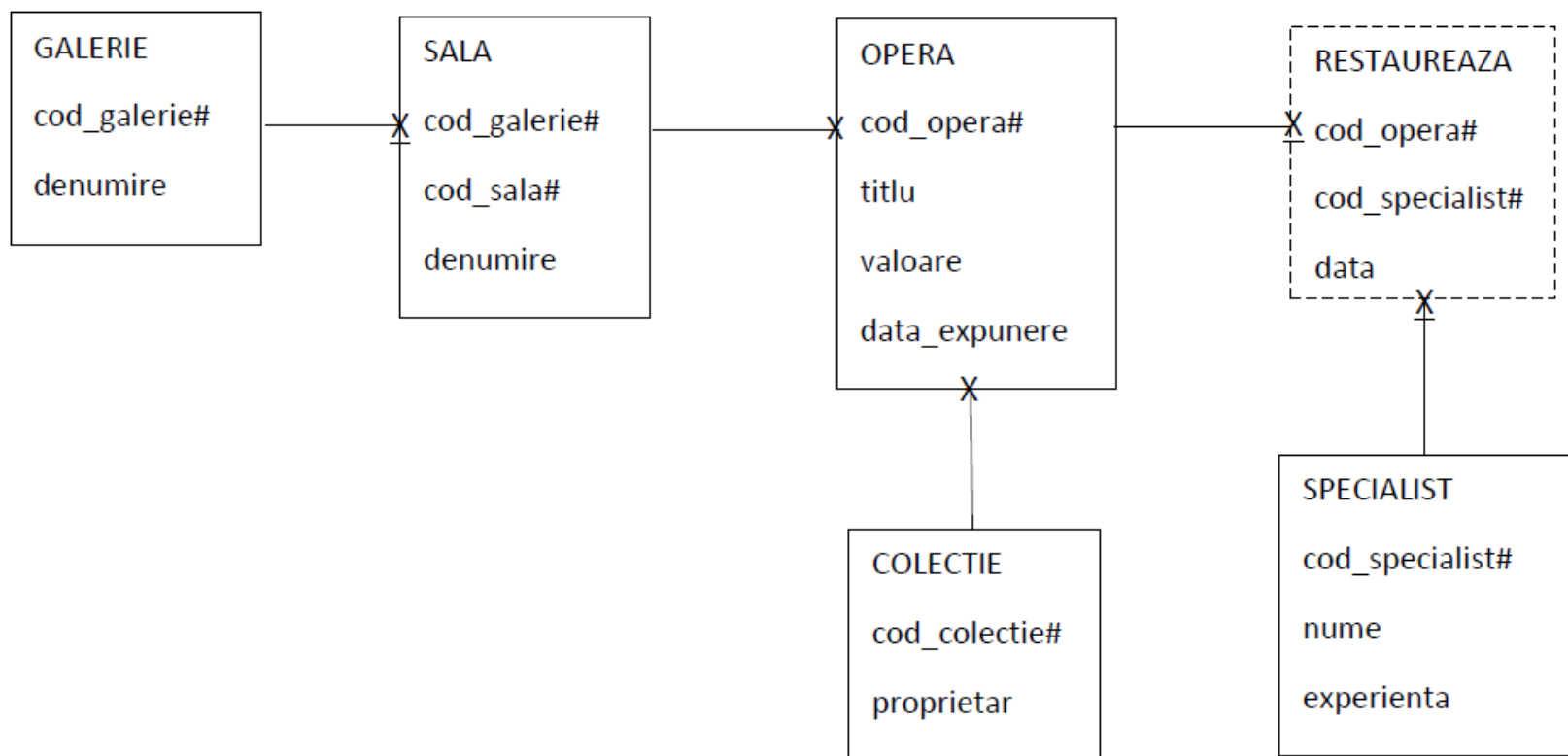
SEMI-JOIN

OUTER-JOIN



Exemplu

- Sa se obtina **titlul** si **valoarea** operelor de arta, expuse in galeria avand codul G1, care fac parte din colectiile ce apartin proprietarului cu numele King si care au fost restaurate dupa data 15/06/2000.



Exemplu

Cerere SQL :

```
SELECT titlu, valoare  
FROM opera o JOIN colectie c ON (o.cod_colectie =  
c.cod_colectie)  
JOIN restaureaza r ON (o.cod_opera = r.cod_opera)  
WHERE upper(o.cod_galerie) = 'G1'  
AND initcap(c.proprietar) = 'King'  
AND r.data > to_date('05/06/2000', 'dd/mm/yyyy');
```

Exemplu

Expresie algebrica:

R1 = SELECT(OPERA, cod_galerie = 'G1')

R2 = PROJECT(R1, cod_opera, titlu, valoare, cod_colectie)

R3 = SELECT(COLECTIE, proprietar = 'King')

R4 = PROJECT(R3, cod_colectie)

R5 = SEMIJOIN(R2, R4, cod_colectie)

R6 = SELECT(RESTAUREAZA, data > 05/06/2000)

R7 = PROJECT(R6, cod_opera)

R8 = SEMIJOIN(R5, R7, cod_opera)

Rezultat = R9 = PROJECT(R8, titlu, valoare)

Exemplu

Arbore algebric:

- Arborele algebric se afla pe slide-ul urmator, dar si in documentul numit: **Arbore_Algebric_Curs7**

R1 = SELECT(OPERA, cod_galerie = 'G1')

R2 = PROJECT(R1, cod_opera, titlu, valoare, cod_colectie)

R3 = SELECT(COLECTIE, proprietar = 'King')

R4 = PROJECT(R3, cod_colectie)

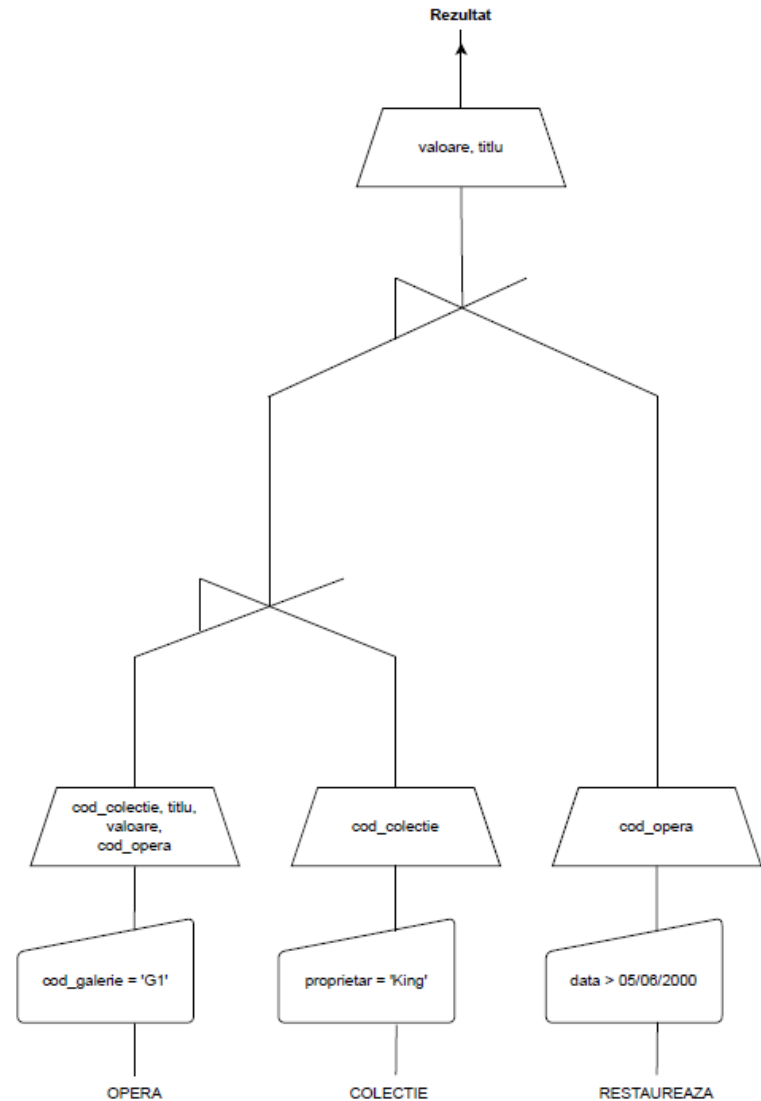
R5 = SEMIJOIN(R2, R4, cod_colectie)

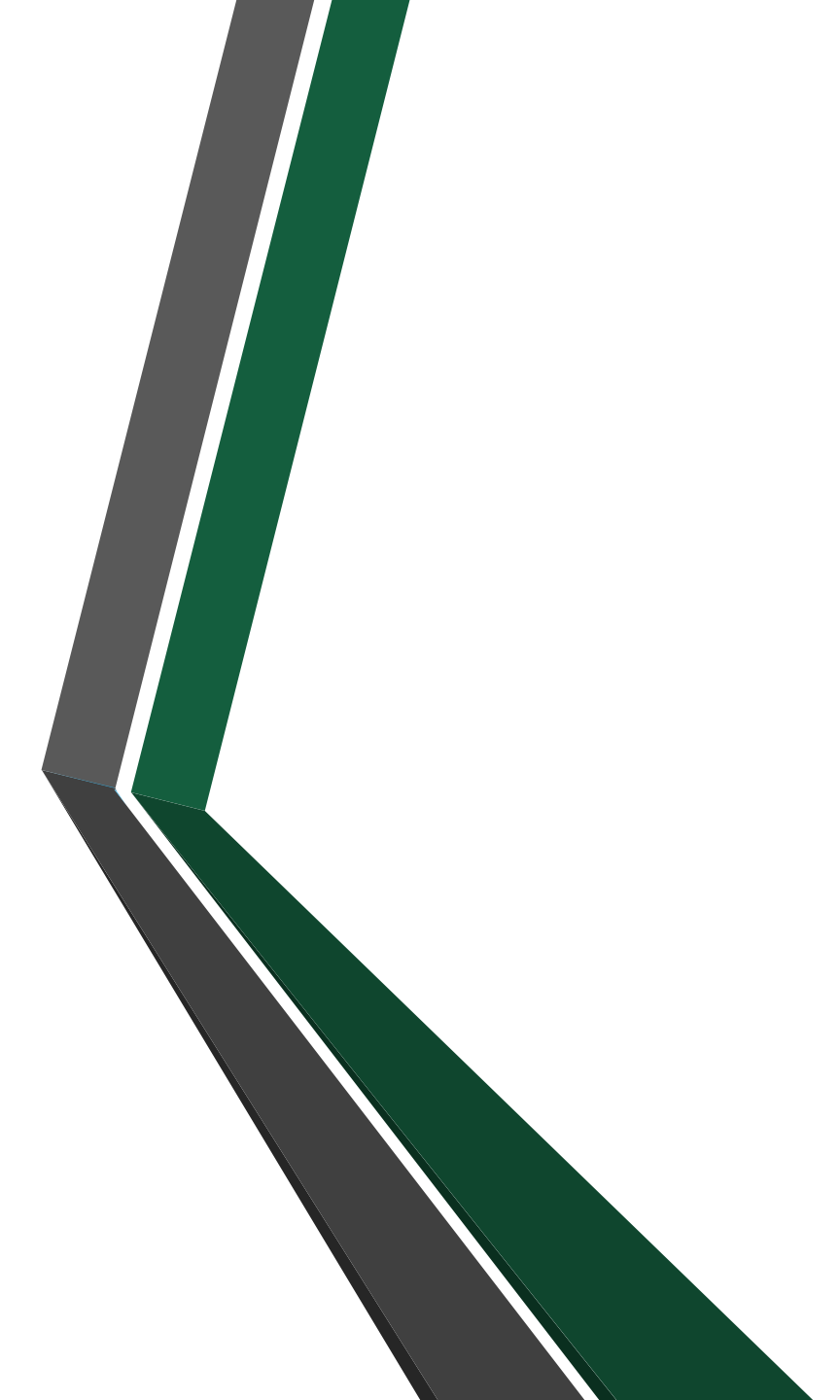
R6 = SELECT(RESTAUREAZA, data > 05/06/2000)

R7 = PROJECT(R6, cod_opera)

R8 = SEMIJOIN(R5, R7, cod_opera)

Rezultat = R9 = PROJECT(R8, titlu, valoare)





BAZE DE DATE

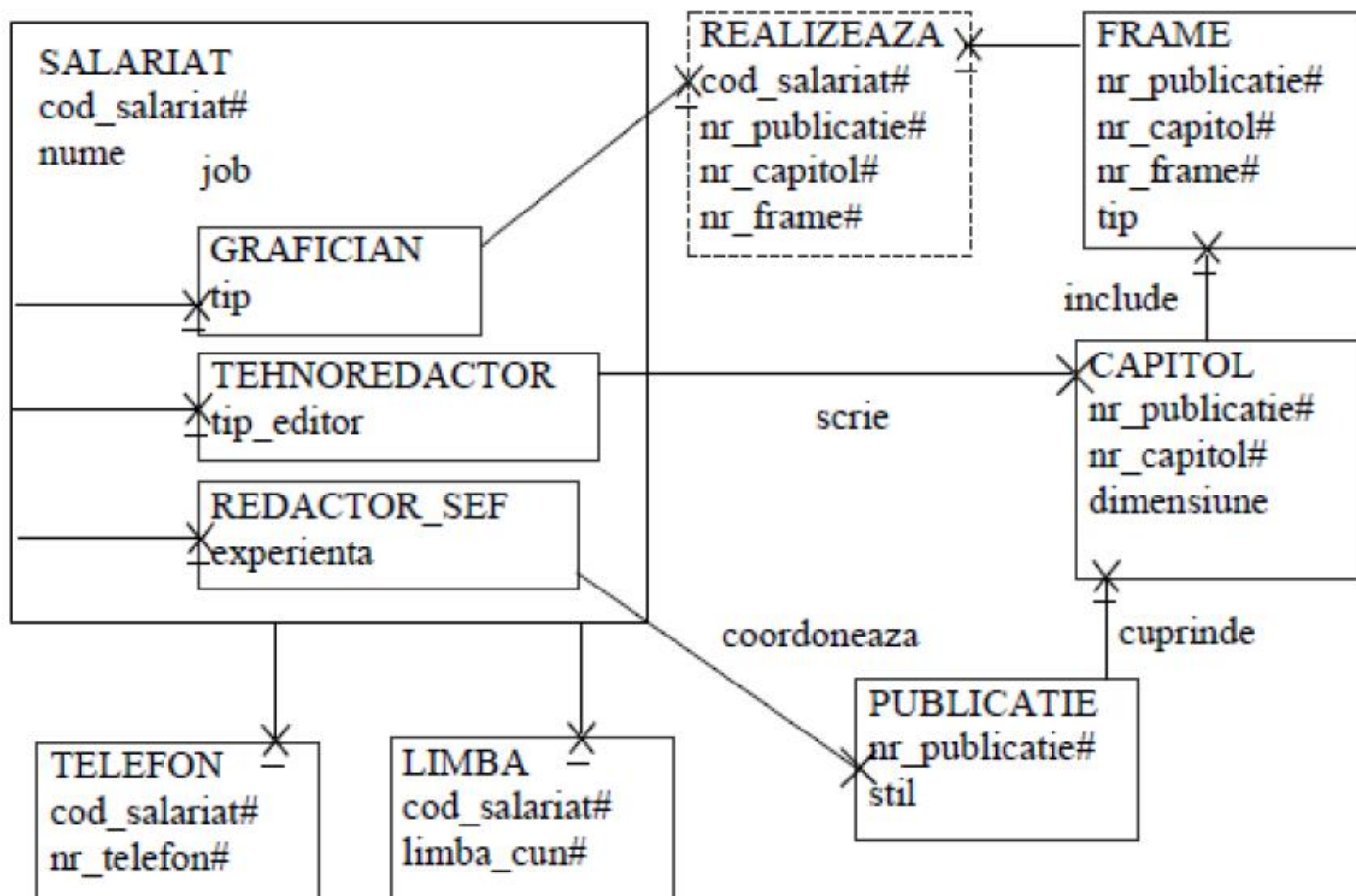
CURS 8

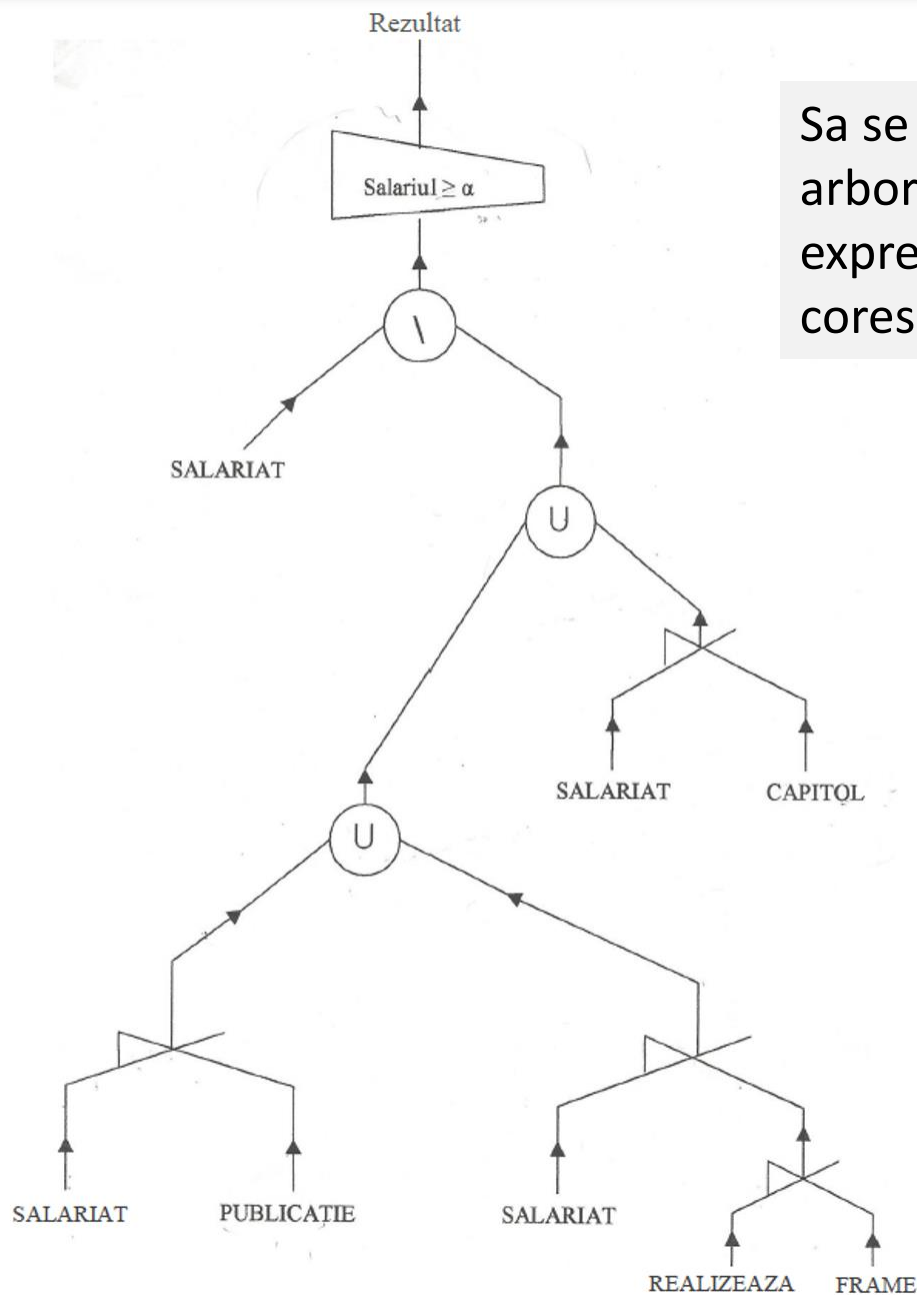
Exercitii Arbori

Rezolvarile exercitiilor din cadrul acestui curs se afla si pe site in documentul numit “**Exercitii_Abori**”

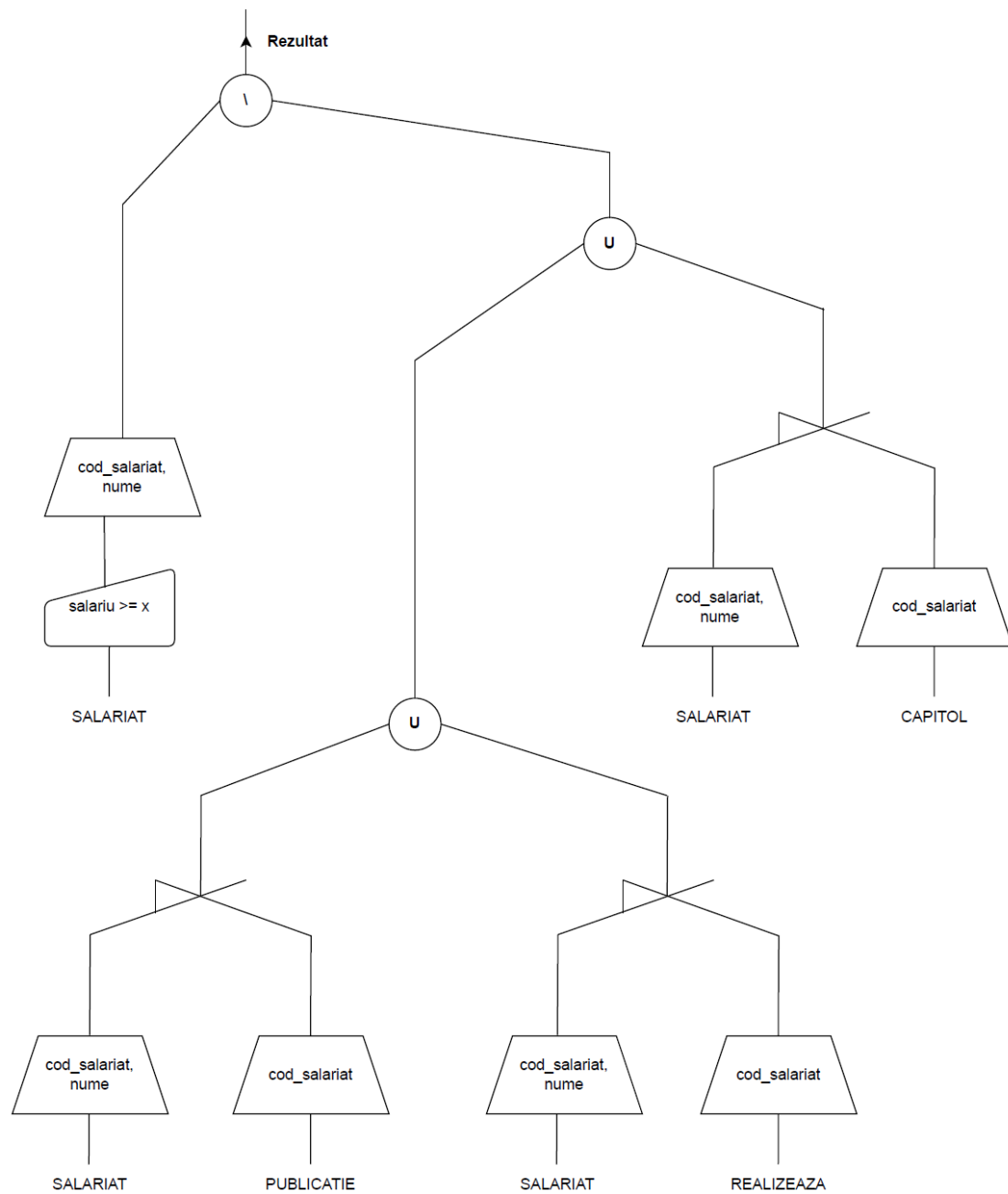
Exercitiul 1

Sa se afiseze informatii despre salariatii care nu contribuie la machetarea niciunei publicatii, dar au salariul mai mare decat o valoare data – **cerere in limbaj natural**





Sa se **optimizeze** urmatorul
arbore algebric si sa se scrie
expresia algebrica
corespunzatoare



Expresia algebrica:

R1 = PROJECT(REALIZEAZA, cod_salariat)

R2 = PROJECT(SALARIAT, cod_salariat, nume)

R3 = SEMIJOIN(R1, R2, cod_salariat)

R4 = PROJECT(PUBLICATIE, cod_salariat)

R5 = SEMIJOIN(R4, R2, cod_salariat)

R6 = UNION(R3, R5)

R7 = PROJECT(CAPITOL, cod_salariat)

R8 = SEMIJOIN(R7, R2, cod_salariat)

R9 = UNION(R8, R6)

R10 = SELECT(SALARIAT, salariu >= x)

R11 = PROJECT(R10, cod_salariat, nume)

R12 = MINUS(R11, R9)

Rezultat = PROJECT(R12, cod_salariat, nume)

Exercitiul 2

Codurile si numerele de telefon ale investitorilor, impreuna cu valoarea investitiei si durata de executie a lucrarilor pentru care valoarea investitiei este intre doua limite **I1** si **I2**, iar tipul contractantului este '**investitie**'.



Scriti **cererea SQL**, urmata de **expresia algebrica** si **arborele algebric**.

Codurile si numerele de telefon ale investitorilor, impreuna cu valoarea investitiei si durata de executie a lucrarilor pentru care valoarea investitiei este intre doua limite **I1** si **I2**, iar tipul contractantului este '**investitie**'.



Cerere SQL?

Codurile si numerele de telefon ale investitorilor, impreuna cu valoarea investitiei si durata de executie a lucrarilor pentru care valoarea investitiei este intre doua limite **I1** si **I2**, iar tipul contractantului este '**investitie**'.



```
SELECT cod_contractant, telefon, val_investitie, durata  
FROM contractant co JOIN contract ct ON(co.cod_contractant =  
ct.cod_contractant)  
WHERE val_investitie > I1 AND val_investitie < I2 AND  
tip_contractant = 'investitie';
```



```
SELECT cod_contractant, telefon, val_investitie, durata  
FROM contractant co JOIN contract ct ON(co.cod_contractant =  
ct.cod_contractant)  
WHERE val_investitie > l1 AND val_investitie < l2 AND  
tip_contractant = 'investitie';
```

Expresia algebrica - neoptima:

R1 = PROJECT(CONTRACT, cod_contractant, val_investitie, durata)

R2 = SELECT(R1, val_investitie > l1)

R3 = SELECT(R2, val_investitie < l2)

R4 = PROJECT(CONTRACTANT, cod_contractant, telefon,
tip_contractant)

R5 = SELECT(R4, tip_contractant = 'investitie')

R6 = PROJECT(R5, cod_contractant, telefon)

Rezultat = JOIN(R6, R3, cod_contractant)

Expresia algebrica:

R1 = PROJECT(CONTRACT, cod_contractant, val_investitie, durata)

R2 = SELECT(R1, val_investitie > l1)

R3 = SELECT(R2, val_investitie < l2)

R4 = PROJECT(CONTRACTANT, cod_contractant, telefon,
tip_contractant)

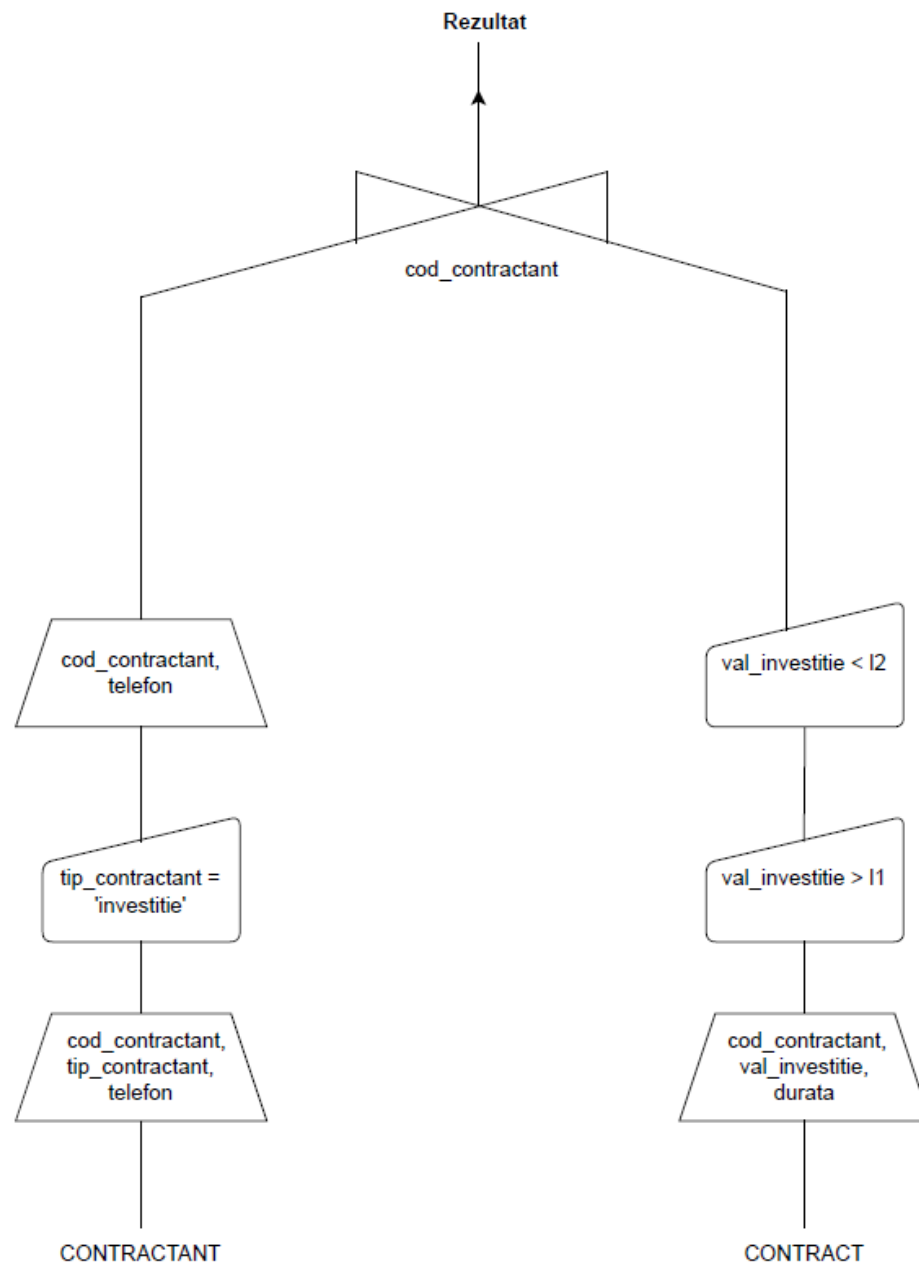
R5 = SELECT(R4, tip_contractant = 'investitie')

R6 = PROJECT(R5, cod_contractant, telefon)

Rezultat = JOIN(R6, R3, cod_contractant)

Join-ul din relatia **Rezultat** returneaza toate coloanele pe care le au relatiile primite ca argument.

Arborele algebric - neoptimizat:



Expresia algebrica - neoptimizata:

R1 = PROJECT(CONTRACT, cod_contractant, val_investitie, durata)

R2 = SELECT(R1, val_investitie > l1)

R3 = SELECT(R2, val_investitie < l2)

R4 = PROJECT(CONTRACTANT, cod_contractant, telefon, tip_contractant)

R5 = SELECT(R4, tip_contractant = 'investitie')

R6 = PROJECT(R5, cod_contractant, telefon)

Rezultat = JOIN(R6, R3, cod_contractant)

Expresia algebrica - optimizata:

R1 = SELECT(CONTRACT, val_investitie > l1 AND val_investitie < l2)

R2 = PROJECT(R1, cod_contractant, val_investitie, durata)

R3 = SELECT(CONTRACTANT, tip_contractant = 'investitie')

R4 = PROJECT(R3, cod_contractant, telefon)

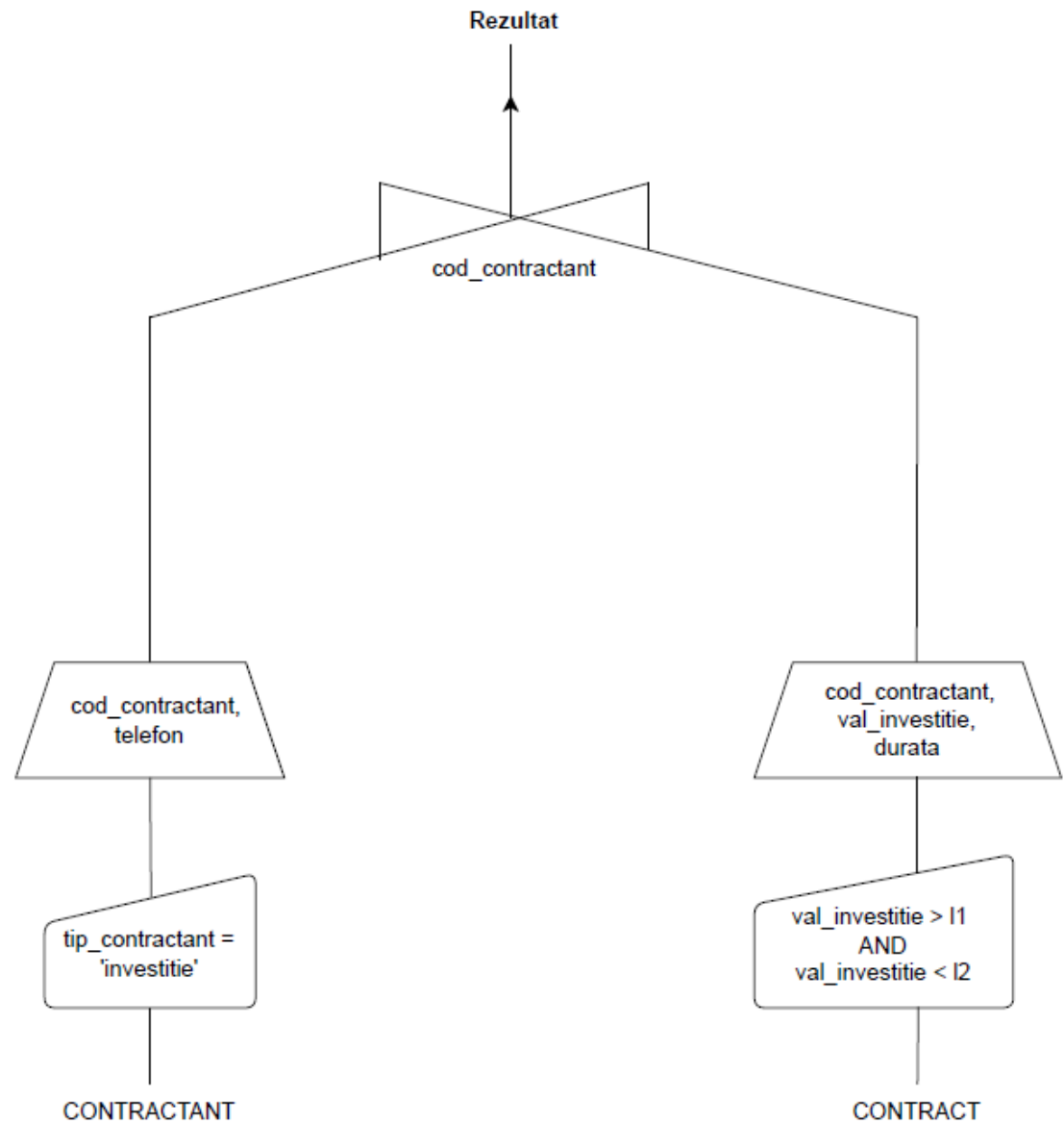
Rezultat = JOIN(R4, R2, cod_contractant) -> rezultat optimizat



Explicatiile detaliate si pasii optimizarii se afla in documentul **Exercitii_Arbori** de pe site.

In acelasi mod se procedeaza si in cadrul proiectului final.

Arborele algebric – optimizat:



Regulile lui Codd

Avantajele modelului relațional:

- fundamentare matematică riguroasă;
- independență fizică a datelor;
- posibilitatea filtrărilor;
- existența unor structuri de date simple;
- realizarea unei redundanțe minime;
- suplețe în comunicarea cu utilizatorul neinformatician;

Regulile lui Codd

Limite ale modelului relațional:

- rămâne totuși redundanță;
- ocupă spațiu;
- apar fenomene de inconsistență;
- nu există mecanisme pentru tratarea optimă a cererilor recursive;
- nu lucrează cu obiecte complexe;
- nu există mijloace perfecționate pentru exprimarea constrângerilor de integritate;
- nu realizează gestiunea totală a datelor distribuite;
- nu realizează gestiunea cunoștințelor;

Regulile lui Codd

- În anul 1985, E.F. Codd a publicat un set de **12 reguli** în raport cu care un sistem de gestiune a bazelor de date poate fi apreciat ca relațional
 - Niciun sistem de gestiune a bazelor de date nu respectă absolut toate regulile definite de Codd, dar acest lucru nu împiedică etichetarea acestor sisteme drept relaționale
- Nu trebuie apreciat un SGBD ca fiind relațional sau nu, ci **măsura în care acesta este relațional**, deci numărul regulilor lui Codd pe care le respectă

Regulile lui Codd

- **Regula 1 – regula reprezentării informației.** *Într-o bază de date relațională, informația este reprezentată la nivel logic sub forma unor **tabele ce poartă numele de relații**.*
- **Regula 2 – regula accesului garantat la date.** *Fiecare valoare dintr-o bază de date relațională trebuie să poată fi accesată în mod logic printr-o **combinație formată din numele relației, valoarea cheii primare și numele atributului**.*
- **Regula 3 – regula reprezentării informației necunoscute.** *Un sistem relațional trebuie să permită utilizatorului definirea unui tip de date numit „**null**” pentru reprezentarea unei informații necunoscute la momentul respectiv.*

Regulile lui Codd

- **Regula 4 – regula dicționarilor de date.** *Asupra descrierii bazelor de date (informații relative la relații, vizualizări, indecși etc.) trebuie să se poată aplica **aceleași operații** ca și asupra datelor din baza de date.*
- **Regula 5 – regula limbajului de interogare.** *Trebuie să existe cel puțin un **limbaj pentru prelucrarea bazei de date**.*
- **Regula 6 – regula de actualizare a vizualizării.** *Un SGBD trebuie să poată determina dacă o vizualizare poate fi actualizată și să stocheze rezultatul interogării într-un dicționar de tipul unui catalog de sistem.*
- **Regula 7 – regula limbajului de nivel înalt.** *Regulile de prelucrare asupra unei relații luate ca întreg sunt valabile atât pentru operațiile de regăsire a datelor, cât și asupra operațiilor de inserare, actualizare și ștergere a datelor.*

Regulile lui Codd

- **Regula 8 – regula independenței fizice a datelor:** *Programele de aplicație și activitățile utilizatorilor nu depind de modul de depunere a datelor sau de modul de acces la date.*
- **Regula 9 – regula independenței logice a datelor.** *Programele de aplicație trebuie să fie transparente la modificările de orice tip efectuate asupra datelor.*
- **Regula 10 – regula independenței datelor privind integritatea acestora.** *Regulile de integritate trebuie să fie definite într-un sublimbaj relațional, nu în programul de aplicație.*

Regulile lui Codd

- **Regula 11 – regula independenței datelor din punct de vedere al distribuirii.** *Distribuirea datelor pe mai multe calculatoare dintr-o rețea de comunicații de date nu trebuie să afecteze programele de aplicație.*
- **Regula 12 – regula versiunii procedurale a unui SGBD.** *Orice componentă procedurală a unui SGBD trebuie să respecte aceleași restricții de integritate ca și componenta relațională.*

Regulile lui Codd

Deoarece regulile lui Codd sunt prea severe pentru a fi respectate de un SGBD operațional, s-au formulat **criterii minimale de definire a unui sistem de gestiune relational**:

- Un SGBD este **minimal relațional** dacă:
 - toate datele din cadrul bazei sunt reprezentate prin **valori în tabele**;
 - nu există pointeri observabili de către utilizator;
 - sistemul suportă operatorii relaționali de **proiecție, selecție și compunere naturală**, fără limitări impuse din considerente interne;
- Un SGBD este **complet relațional** dacă este minimal relațional și satisface în plus condițiile:
 - sistemul suportă **restricțiile de integritate de bază** (unicitatea cheii primare, constrângerile referențiale, integritatea entității);
 - sistemul suportă **toate operațiile de bază ale algebrei relaționale**;

NORMALIZAREA RELAȚIILOR

- În procesul modelării unei baze de date relaționale, o etapă importantă o reprezintă **normalizarea relațiilor conceptuale**, adică obținerea de relații „moleculare” fără a pierde nimic din informație, pentru a elimina:
- **redundanța;**
 - **anomaliile reactualizării informațiilor;**

NORMALIZAREA RELAȚIILOR

- Tehnica normalizării permite:
 - obținerea unei **scheme conceptuale rafinate** printr-un proces de **ameliorare progresivă** a unei scheme conceptuale inițiale a bazei de date relaționale;
- După fiecare etapă de ameliorare, relațiile bazei de date ating un anumit **grad de perfecțiune**, deci se află într-o anumită **formă normală**
 - Trecerea unei relații dintr-o formă normală în alta, presupune **eliminarea unui anumit tip de dependență** nedorită, care este transformată în dependență admisibilă, adică dependență care nu provoacă anomalii;

NORMALIZAREA RELAȚIILOR

Procesul de ameliorare a schemei conceptuale trebuie:

- să garanteze **conservarea datelor**, adică în schema conceptuală finală trebuie să figureze toate datele din cadrul schemei inițiale;
- să garanteze **conservarea dependențelor** dintre date, adică în schema finală fiecare dependență trebuie să aibă determinantul și determinatul în schema aceleiași relații;
- să reprezinte o **descompunere minimală** a relațiilor inițiale, adică niciuna dintre relațiile care compun schema finală nu trebuie să fie conținută într-o altă relație din această schema;

NORMALIZAREA RELAȚIILOR

Există două metode pentru a modela **baze de date relaționale fără anomalii sau pierderi de informație**:

- **Schema descompunerii** pleacă de la o schemă relațională universală ce conține toate attributele BD.
 - Schema se descompune prin **proiecții succesive** în subrelații;
 - Descompunerea se oprește când continuarea ei ar duce la pierderi de informație;
 - Algoritmii de descompunere se bazează, în general, pe descrierea formală a dependenței dintre attribute;

NORMALIZAREA RELAȚIILOR

- **Schema sintezei** pleacă de la o mulțime de attribute independente.
 - Utilizând proprietăți de semantică și legături între attribute se pot compune noi relații, astfel încât, acestea să nu sufere de anumite anomalii;
 - Algoritmii se bazează, în general, pe teoria grafurilor pentru a reprezenta legăturile dintre attribute;

Dependențe funcționale

- O **relație universală** este o relație ce grupează toate atributele care modelează sistemul real cercetat
- Fie E , **mulțimea dependențelor** considerate de proiectantul bazei de date pentru o schemă relațională sau pentru o relație universală.
 - Plecând de la o mulțime de proprietăți formale ale dependențelor, proprietăți considerate drept reguli de deducție (**axiome**), poate fi obținută **mulțimea maximală de dependențe asociate lui E** . Această mulțime definește închiderea lui E .

Dependențe funcționale

- Fie E mulțimea dependențelor unei relații și $p_1, p_2, \dots, p_r, r \geq 1$, proprietăți formale ale acestor dependențe.
 - Dacă există o mulțime E' , astfel încât orice dependență a mulțimii E este derivabilă din E' prin aplicarea proprietăților $p_1, p_2, \dots, p_r$, atunci mulțimea E' definește **acoperirea** lui E pentru proprietățile $p_1, p_2, \dots, p_r$.
- E' este **o acoperire minimală** pentru E , dacă nu există nici o submulțime proprie, nevidă a lui E' care să fie o acoperire pentru E . **Evident, E și E' au închideri identice, deci dispun de același potențial informațional!**

Dependențe funcționale

- Fie $R(A_1, A_2, \dots, A_n)$ o schemă relațională și fie X, Y submulțimi de attribute ale lui R .
 - X **determină funcțional** Y sau Y depinde funcțional (FD) de X , dacă pentru orice relație r (valoare curentă a lui R) **nu există două tupluri care să aibă aceleași valori pentru attributele lui X și să aibă valori diferite pentru cel puțin un atribut din Y** . Cu alte cuvinte, o valoare a lui X , determină unic o valoare a lui Y .
- Notăție: $X \rightarrow Y$; X este numit **determinant**, iar Y este numit **determinat** (sau dependent).
 - Dependența funcțională $X \rightarrow Y$ este **trivială** dacă $Y \subseteq X$.

Dependențe funcționale

În momentul proiectării și modelării unei baze de date relaționale pot să apară anomalii. Procesul de optimizare a relațiilor conceptuale poartă numele de **normalizare**. Pentru a depista eventualele redundanțe se utilizează **dependențele funcționale**.

Fie $R(A_1, A_2, \dots, A_n)$ o schema relationala.

De exemplu:

EMPLOYEES(employee_id, first_name, salary, job_id, job_title)

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Tipuri de anomalii:

- 1) **Redundanța** – stocarea repetată a unor valori (în exemplul anterior IT_PROG și PROGRAMATOR)
- 2) **Anomalii de actualizare** – modificarea valorii redundante de la 1) poate duce la modificarea valorii doar pentru unele înregistrări și nu pentru toate
- 3) **Anomalii de inserare** – nu se poate insera un job (job_id, job_title) decât dacă se inserează în baza de date și un angajat asociat
- 4) **Anomalii de ștergere** – dacă se șterge un angajat se pierde automat și datele despre job

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Apar aceste anomalii deoarece în relația **EMPLOYEES** sunt attribute care aparțin unor mulțimi diferite X și Y și deci unor relații diferite.

$X \rightarrow Y$ - **X determină funcțional pe Y sau Y depinde funcțional de X**
dacă pentru orice relație **nu există** două tupluri care să aibă aceleași valori pentru attributele lui X și să aibă valori diferite pentru cel puțin un atribut din Y. Cu alte cuvinte, o valoare a lui X, **determină unic** o valoare a lui Y (definiția de pe slide-ul 28)

De exemplu:

100 KING 24000 **IT_PROG PROGRAMATOR**
101 JOHN 9500 **MGR MANAGER**

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

De exemplu:

100 KING 24000 **IT_PROG PROGRAMATOR**

101 JOHN 9500 **MGR MANAGER**

IT_PROG determină funcțional **PROGRAMATOR**

Deci o valoare a lui X determină unic o valoare a lui Y. Astfel, se obțin următoarele submulțimi de attribute ale relației EMPLOYEES:

{job_id} -> {job_title} – job_id determina functional pe job_title

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Se observa urmatoarele submultimi de attribute:

$X' = \{\text{employee_id}, \text{first_name}, \text{salary}, \text{job_id}\}$

$Y' = \{\text{job_id}, \text{job_title}\}$

Din submulțimile X' și Y' de attribute, se obțin următoarele scheme relaționale:

EMPLOYEES(employee_id, first_name, salary, job_id)

JOBS(job_id, job_title)

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Se observă că informația din relația **EMPLOYEES** trebuie de fapt stocată în două relații, astfel:

EMPLOYEES

emp_id	first_name	salary	Job_id
100	KING	24000	IT_PROG
101	JOHN	9500	MGR
102	HUNOLD	17000	IT_PROG

JOB

job_id	Job_title
IT_PROG	PROGRAMATOR
MGR	MANAGER

Dependențe funcționale

Acest procedeu se numeste **descompunerea relațiilor**.

Descompunerile se realizează pe baza formelor normale pe care le vom studia în cursul următor, iar definirea formelor normale se bazează pe noțiunea de ***dependență funcțională***.

Dependențe funcționale

EMPLOYEES

emp_id	first_name	salary
100	KING	24000
101	JOHN	9500
102	HUNOLD	17000

JOBS

job_id	Job_title
IT_PROG	PROGRAMATOR
MGR	MANAGER

In exemplul anterior exista urmatoarele dependente functionale:

employee_id -> first_name, salary (se citeste: employee_id **determina functional** first_name si salary sau first_name si salary **depind functional** de employee_id)

job_id -> job_title

Dependențe funcționale

Pornind de la aceste dependente putem afirma ca:

- Dacă doi angajati ar avea același `employee_id` înseamnă ca este același angajat
- Dacă doi angajati au același `job_id` atunci rezulta ca au implicit și același `job_title`

Dependențe funcționale

- Comparând toate submulțimile de attribute ale unei relații și determinând legăturile dintre ele, se pot obține toate dependențele funcționale pe care o relație le satisface.
 - Această abordare **nu** este eficientă, consumând mult timp;
- Există posibilitatea ca, știind **anumite dependențe** funcționale și utilizând **reguli de deducție**, să fie obținute **toate** dependențele funcționale.

Dependențe funcționale

Fie X, Y, Z, W mulțimi de attribute ale unei scheme relaționale R și fie următoarele axiome:

- **Ax1 – reflexivitate.** $X \rightarrow X$. Mai general, dacă $Y \subseteq X$, atunci $X \rightarrow Y$.
- **Ax2 – creșterea determinantului (augmentare).** Pot fi considerate următoarele formulări echivalente pentru această axiomă.
 - Dacă $X \rightarrow Y$ și $X \subseteq Z$, atunci $Z \rightarrow Y$. ($\subseteq \rightarrow$ fiecare element din X este și element în Z ; X este inclus în Z)
 - Dacă $X \rightarrow Y$ și $W \subseteq Z$, atunci $X \cup Z \rightarrow Y \cup W$.
 - Dacă $X \rightarrow Y$ atunci $X \cup Z \rightarrow Y \cup Z$.
 - Dacă $X \rightarrow Y$ atunci $X \cup Z \rightarrow Y$.

În urma reuniunii se obțin dependențe funcționale valide

- **Ax3 – tranzitivitate.** Dacă $X \rightarrow Y$ și $Y \rightarrow Z$, atunci $X \rightarrow Z$.

Dependențe funcționale

- O mulțime de axiome este **completă** dacă și numai dacă plecând de la o mulțime de dependențe E se pot obține toate dependențele închiderii lui E , utilizând axiomele mulțimii.
- O mulțime de axiome este **închisă** dacă și numai dacă plecând de la o mulțime de dependențe E , nu poate fi dedusă cu ajutorul axiomelor o dependență care nu aparține închiderii lui E .
- **Ullman** a demonstrat că axiomele Ax1 – Ax3, numite **axiomele lui Armstrong**, reprezintă o mulțime închisă și completă de axiome.

Dependențe funcționale

- Nu toate dependențele funcționale sunt folositoare pentru modelarea relațională.
- O dependență funcțională $X \rightarrow Y$ se numește **dependență funcțională totală** (FT), dacă și numai dacă nu există nicio submulțime proprie $X' \subset X$, astfel încât $X' \rightarrow Y$.
 - Dacă există o submulțime proprie $X' \subset X$, astfel încât $X' \rightarrow Y$, atunci dependența funcțională $X \rightarrow Y$ este parțială. În axioma Ax2, dependența $Z \rightarrow Y$ este o dependență funcțională parțială;

Dependențe funcționale

- În cazul dependenței funcționale totale, axiomele lui Armstrong se reduc la o axiomă unică și anume **pseudo-tranzitivitatea**:
 - dacă $X \rightarrow Y$ și $W \cup Y \rightarrow Z$, atunci $W \cup X \rightarrow Z$
- Această axiomă este o regulă de deducție completă pentru total dependențe:
 - pseudo-tranzitivitatea implică tranzitivitatea (daca $W = \emptyset$);
 - reflexivitatea nu poate fi utilizată pentru a obține dependențe totale;

Dependențe funcționale

- Dacă F este o mulțime de dependențe funcționale totale, atunci **închiderea pseudo-tranzitivă** F^+ a acestei mulțimi este reuniunea mulțimilor dependențelor funcționale totale care pot fi obținute din F folosind axioma de pseudo-tranzitivitate. Multimea tuturor dependențelor funcționale care se pot deduce din F se numeste închiderea mulțimii de dependente F , notata cu F^+
- Două mulțimi de dependențe funcționale totale sunt **echivalente** dacă au **închideri pseudo-tranzitive identice**

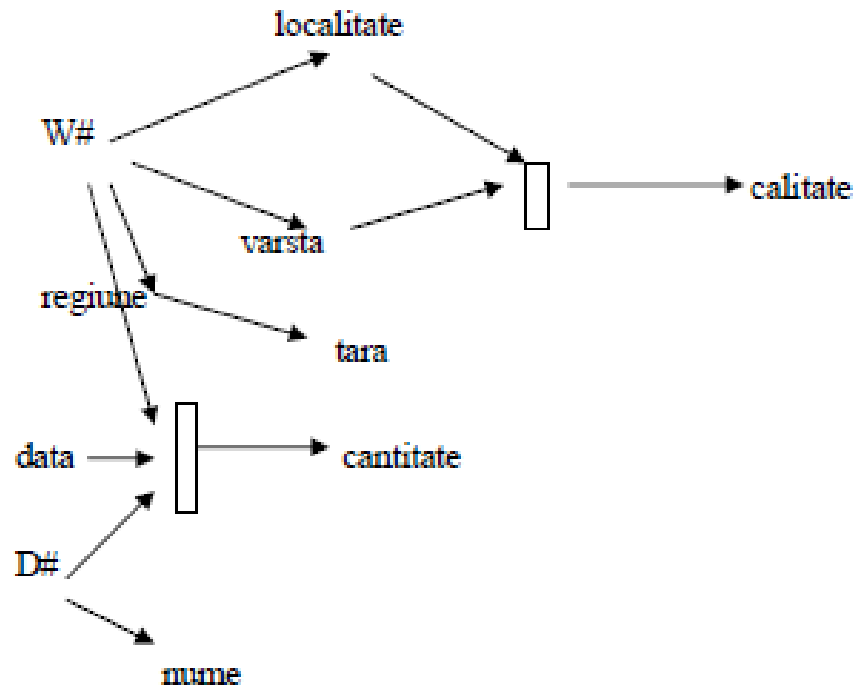
Dependențe funcționale

- Dependențele funcționale între atributele bazei pot fi reprezentate grafic
 - Fie $A = \{A_1, A_2, \dots, A_n\}$ o mulțime de atribute și fie o mulțime de dependențe funcționale $\{X_i \rightarrow A_j\}$, unde X_i este o submulțime a lui A .
- **Graful dependențelor funcționale** este un **graf direcționat bipartit**, definit astfel:
 1. pentru fiecare atribut A_j există un singur **nod** având eticheta A_j ;
 2. pentru fiecare dependență funcțională de forma $A_i \rightarrow A_j$, există un **arc** de la A_i la A_j ;
 3. pentru fiecare dependență funcțională de forma $X_i \rightarrow A_j$, unde mulțimea X_i este definită de $X_i = \{A_{i_1}, \dots, A_{i_p}\}$ cu $p > 1$, există un **nod auxiliar** etichetat prin X_i și **o mulțime de arce** plecând de la $A_{i_1}, \dots, A_{i_p}$ pentru a obține pe X_i și **printr-un arc adițional** de la X_i la A_j . Nodurile X_i se reprezintă prin dreptunghiuri.

Dependențe funcționale

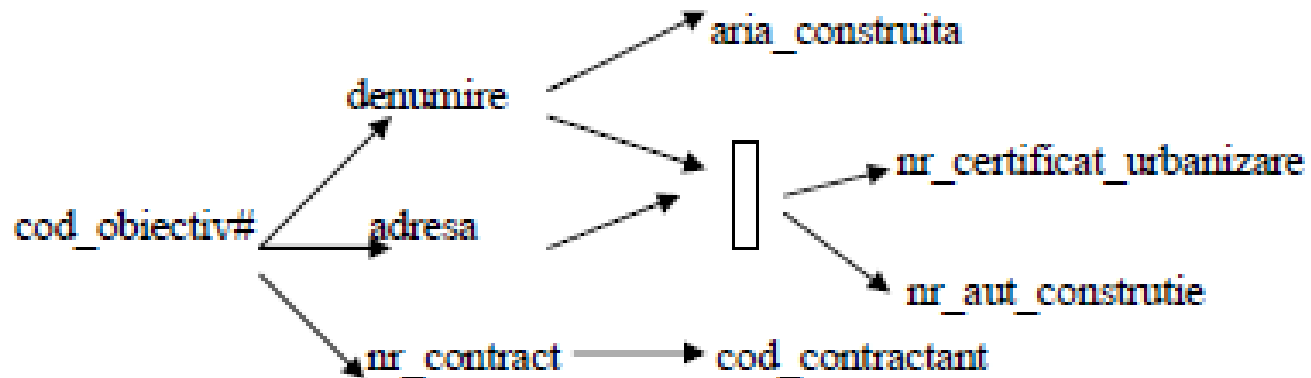
Exemple:

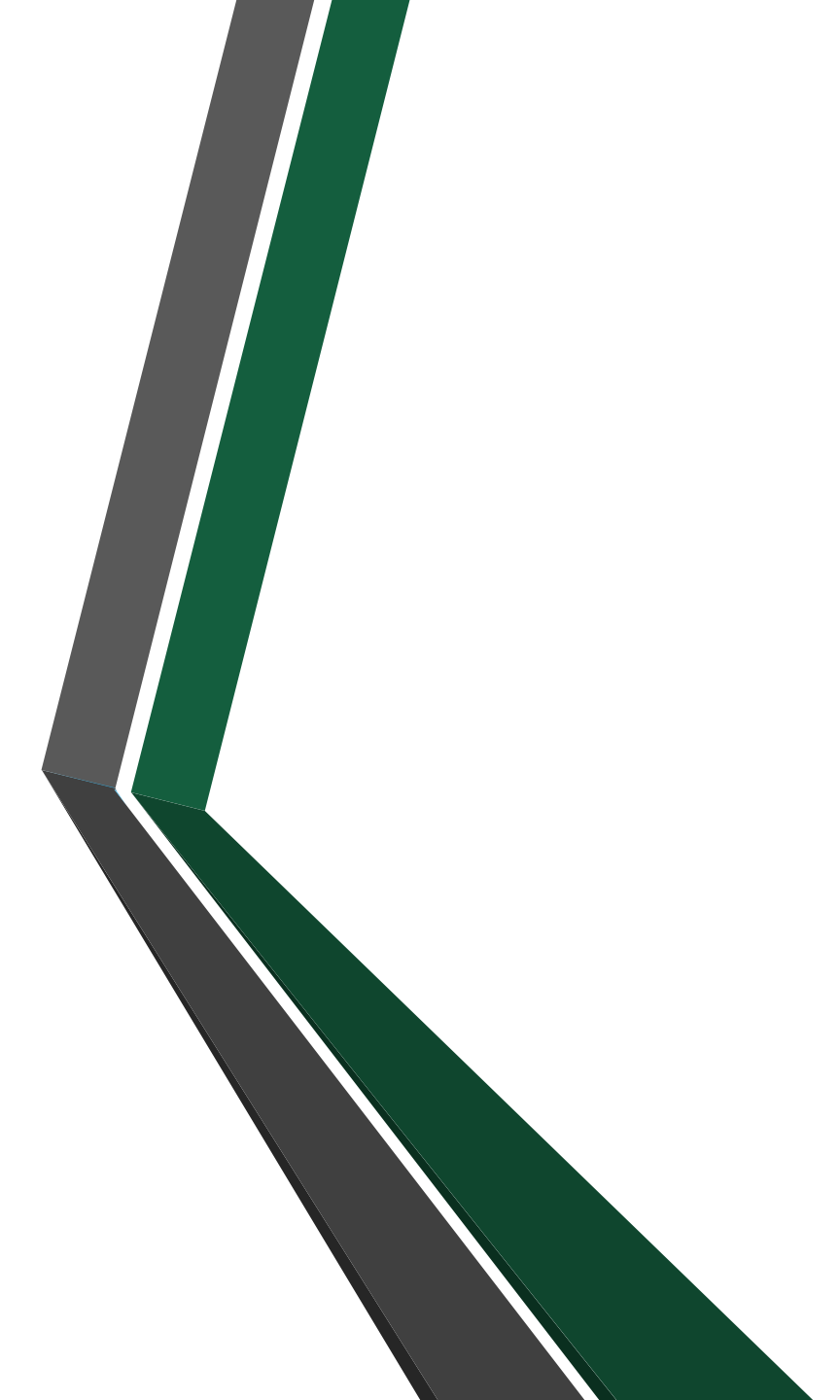
1. Graful dependențelor funcționale pentru schema relațională **CONSUMATOR_DE_VIN**(W#, localitate, varsta, calitate, regiune, tara, D#, nume, data, cantitate).



Dependențe funcționale

2. Graful dependențelor funcționale pentru schema relațională **OBIECTIV_INVESTITIE**. Dependentele sunt deduse din regulile impuse de beneficiar.





BAZE DE DATE

CURS 9

Partea I

Necesitatea normalizării

- În momentul proiectării și modelării unei baze de date relaționale pot să apară anomalii. Procesul de optimizare a relațiilor conceptuale poartă numele de **normalizare**. Pentru a depista eventualele redundanțe se utilizează **dependențele funcționale** (**Vezi Curs 8**)
- **X → Y** - X determină funcțional pe Y sau Y depinde funcțional de X
- Anomaliile care apar în lucrul cu baza de date se produc din cauza dependențelor care există între datele din cadrul relațiilor bazei de date
 - Dependențele sunt plasate greșit în tabele!!!
- O dependență poate provoca:
 - **anomalii** la inserare, modificare sau ștergere (**Vezi Curs 8**)
 - **redundanță** în date

Necesitatea normalizării

- **Normalizarea** are drept scop:
 - suprimarea redundanței logice;
 - evitarea anomaliilor la reactualizare;
 - rezolvarea problemei reconexiunii;
- Există o **teorie matematică a normalizării** al cărei autor este E.F. Codd
 - **Soluția**: construirea unor tabele standard (forme normale);
- Normalizarea este procesul reversibil de transformare a unei relații, în mai multe relații cu structură mai simplă
 - Procesul este reversibil în sensul că nicio informație nu este pierdută în timpul transformării;
 - O relație este într-o formă normală particulară dacă ea satisface o mulțime specificată de constrângeri;

Necesitatea normalizării

➤ **Relație universală + mulțime de anomalii**

- Orice formă normală se obține aplicând o **schemă de descompunere**

Există două tipuri de descompuneri:

➤ **Descompuneri ce conservă dependențele.**

- descompunerea relației R în proiecțiile $R_1, R_2, \dots, R_k$, a.î. dependențele lui R sunt echivalente (au închideri pseudo-tranzitive identice) cu reuniunea dependențelor lui $R_1, R_2, \dots, R_k$.

➤ **Descompuneri fără pierderi de informație (*L-join*).**

- descompunerea relației R într-o mulțime de proiecții $R_1, R_2, \dots, R_j$, a.î. pentru orice instanță a lui R este adevărată relația:

$$R = \text{JOIN}(\Pi_{B_1}(R), \Pi_{B_2}(R), \dots, \Pi_{B_j}(R))$$

- Relația inițială = compunerea naturală a relațiilor obținute prin descompunere;

Necesitatea normalizării

- Formele normale sunt obținute prin descompuneri fără pierderi de informație
- O descompunere fără pierdere de informație, utilizată în procesul normalizării, este dată de **regula Casey-Delobel**:

- Fie $R(A)$ o schemă relațională și fie α, β, γ o partiție a lui A . Presupunem că α determină funcțional pe β . Atunci:

$$R(A) = \text{JOIN}(\Pi_{\alpha \cup \beta}(R), \Pi_{\alpha \cup \gamma}(R)).$$

- $\alpha \cup \beta \rightarrow$ mulțimea atributelor care intervin în dependențele funcționale;
- $\alpha \cup \gamma \rightarrow$ reprezintă reuniunea determinantului cu restul atributelor lui A .

Exemplu regula Casey-Delobel

► Se considera relatia:

AVION(A#, nume, capacitate, localitate)

$\alpha = \{\text{nume}\}$

$\beta = \{\text{capacitate}\}$

$\gamma = \{A\#, \text{localitate}\}$

Avion

A#	nume	capacitate	localitate
1	AIRBUS	250	PARIS
2	AIRBUS	250	PARIS
3	AIRBUS	250	LONDRA
4	CAR	100	PARIS
5	B707	150	LONDRA
6	B707	150	LONDRA

Regula Casey-Delobel: Fie $R(A)$ o schemă relațională și fie α , β , γ o partiție a lui A . Presupunem că α determină funcțional pe β . Atunci:

$$R(A) = \text{JOIN}(\Pi_{\alpha \cup \beta}(R), \Pi_{\alpha \cup \gamma}(R))$$

$\left\{ \begin{array}{l} \alpha \rightarrow \beta \\ \text{nume} \rightarrow \text{capacitate} \Rightarrow \text{AVION1} = (\text{nume}\#, \text{capacitate}) \end{array} \right.$

► Avem $\Pi_{\alpha \cup \beta}(R)$, unde $\alpha \cup \beta$ – este multimea atributelor care intervin in dependentele functionale

► $\Pi_{\alpha \cup \gamma}(R)$, unde $\alpha \cup \gamma$ – reprezinta reuniunea determinantului cu restul atributelor relatiei AVION

In final relatia AVION se va descompune in urmatoarele scheme relationale:

AVION1 = (nume#, capacitate)
AVION2 = (A#, nume, localitate)

Exemplu regula Casey-Delobel

Avion

A#	nume	capacitate	localitate
1	AIRBUS	250	PARIS
2	AIRBUS	250	PARIS
3	AIRBUS	250	LONDRA
4	CAR	100	PARIS
5	B707	150	LONDRA
6	B707	150	LONDRA



AVION1

Nume	Capacitate
AIRBUS	250
CAR	100
B707	150

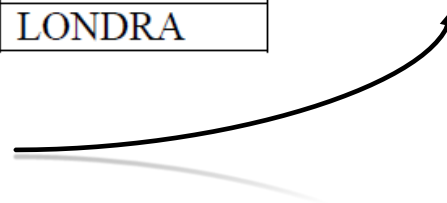
AVION2

A#	Nume	Localitate
1	AIRBUS	PARIS
2	AIRBUS	PARIS
3	AIRBUS	LONDRA
4	CAR	PARIS
5	B707	LONDRA
6	B707	LONDRA

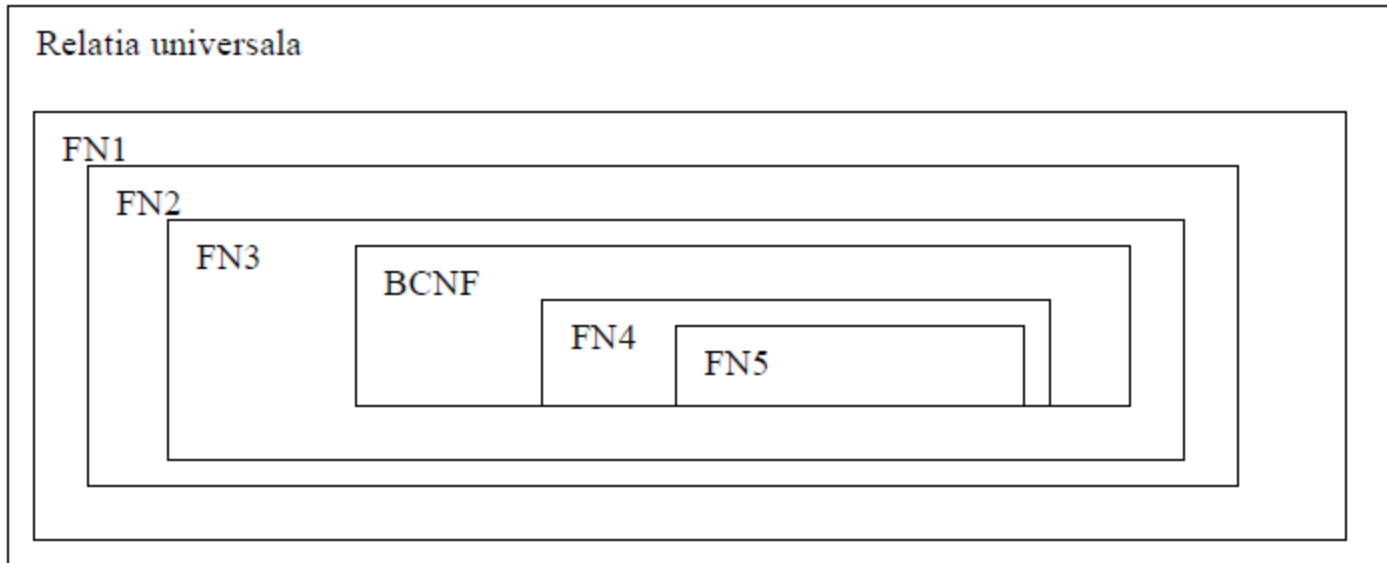
In final relatia AVION se va descompune in urmatoarele scheme relationale:

AVION1 = (nume#, capacitate)

AVION2 = (A#, nume, localitate)



Necesitatea normalizării



Forma normală 1 (FN1)

- O relație este în **prima formă normală** dacă fiecărui atribut care o compune îi corespunde o valoare indivizibilă (atomică).
- Exemplu:

VEHICUL (Non FN1)

cod_persoana#	vehicule
P1	DL, RC, FF
P2	RM, VW
P3	DL

Forma normală 1 (FN1)

Varianta 1:

VEHICUL (FN1)

cod_persoana#	vehicul#
P1	DL
P1	RC
P1	FF
P2	RM
P2	VW
P3	DL

Varianta 2:

VEHICUL (FN1)

cod_persoana#	vehicul1	vehicul2	vehicul3
P1	DL	RC	FF
P2	RM	VW	null
P3	DL	null	null

Forma normală 2 (FN2)

- O relație R este în **a doua formă normală** dacă și numai dacă:
 - relația R este în **FN1**;
 - fiecare atribut care nu este cheie (nu participă la cheia primară) este dependent de întreaga cheie primară;
- FN2 interzice manifestarea unor dependențe funcționale parțiale în cadrul relației R

Forma normală 2 (FN2)

atasat_la

Cod_salariat#	Job_cod	Nr_proiect#	Funcția	Suma
S1	Programator	P1	Supervizor	60
S1	Programator	P2	Cercetator	25
S1	Programator	P3	Auxiliar	10
S3	Vanzator	P3	Supervizor	60
S5	Inginer	P3	Supervizor	60

- Un salariat are mai multe functii si poate lucra la mai multe proiecte
- O relație *R* este în a doua formă normală dacă și numai dacă:
 - relația *R* este în FN1; -> **relatia atasat_la se afla in FN1 deoarece avem identificator unic pentru toate intrarile din tabel**
 - fiecare atribut care nu este cheie (nu participă la cheia primară) este dependent de întreaga cheie primară – în cazul nostru attributele **job_cod, funcția, suma** nu sunt chei si trebuie sa depinda direct de întreaga cheie primara **cod_salariat#** si **nr_proiect#** -> aceste attribute nu depind direct de întreaga cheie primara deoarece se observa dependenta directa dintre **job_cod** si **cod_salariat**, insemnand ca job_cod depinde direct doar de o parte a cheii primare, si anume doar de cod_salariat -> **relatia atasat_la nu se afla in FN2**

Forma normală 2 (FN2)

atasat_la

Cod_salariat#	Job_cod	Nr_proiect#	Funcția	Suma
S1	Programator	P1	Supervizor	60
S1	Programator	P2	Cercetator	25
S1	Programator	P3	Auxiliar	10
S3	Vanzator	P3	Supervizor	60
S5	Inginer	P3	Supervizor	60

- Fiecare atribut care nu este cheie (nu participă la cheia primară) este dependent de întreaga cheie primară – în cazul nostru attributele **job_cod**, **funcția**, **suma** nu sunt chei și trebuie să depindă direct de întreaga cheie primară **cod_salariat#** și **nr_proiect#** -> aceste attribute nu depind direct de întreaga cheie primară deoarece se observă dependența directă dintre **job_cod** și **cod_salariat**, însemnând că **job_cod** depinde direct doar de o parte a cheii primare, și anume doar de **cod_salariat** -> **relația atasar la nu se afla în FN2**
- Astfel avem:
 - {cod_salariat#} -> {job_cod} – cod_salariat determină funcțional job_cod
 - {cod_salariat#, nr_proiect#} -> {funcția, suma}

Forma normală 2 (FN2)

atasat_la

Cod_salariat#	Job_cod	Nr_proiect#	Funcția	Suma
S1	Programator	P1	Supervizor	60
S1	Programator	P2	Cercetator	25
S1	Programator	P3	Auxiliar	10
S3	Vanzator	P3	Supervizor	60
S5	Inginer	P3	Supervizor	60

➤ Astfel avem:

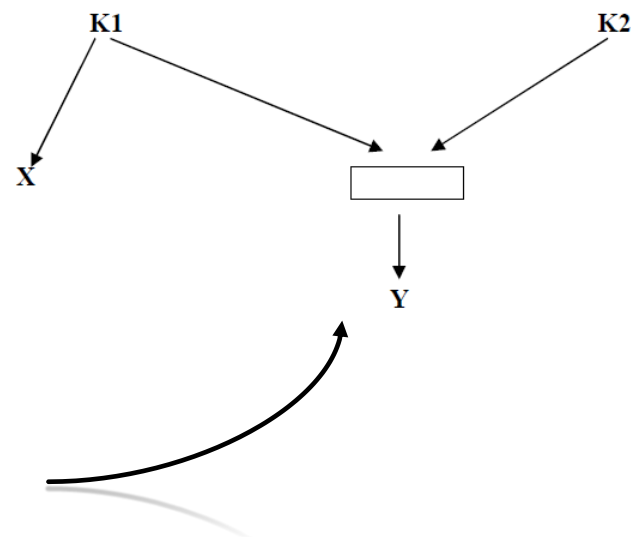
- {cod_salariat#} → {job_cod} – cod_salariat determina functional job_cod
- {cod_salariat#, nr_proiect#} → {funcția, suma}

Aplicarea regulii Casey-Delobel pentru FN2:

Fie relația $R(K1, K2, X, Y)$, unde $K1$ și $K2$ definesc cheia primară, iar X și Y sunt mulțimi de attribute, astfel încât $K1 \rightarrow X$.

Din cauza dependenței funcționale $K1 \rightarrow X$ care arată că R nu este în FN2, se înlocuiește R (fără pierdere de informație) cu două proiecții:

$R1(K1, K2, Y)$ și $R2(K1, X)$.



Forma normală 2 (FN2)

atasat_la

Cod_salariat#	Job_cod	Nr_proiect#	Funcția	Suma
S1	Programator	P1	Supervizor	60
S1	Programator	P2	Cercetator	25
S1	Programator	P3	Auxiliar	10
S3	Vanzator	P3	Supervizor	60
S5	Inginer	P3	Supervizor	60

Transformarea in FN2:

atasat_2a

Cod_salariat#	Nr_proiect#	Funcția	Suma
S1	P1	Supervizor	60
S1	P2	Cercetator	25
S1	P3	Auxiliar	10
S3	P3	Supervizor	60
S5	P3	Supervizor	60

atasat_2b

Cod_salariat#	Job_cod
S1	Programator
S3	Vanzator
S5	Inginer

Astfel avem:

{cod_salariat#} -> {job_cod}

– cod_salariat determina functional job_cod

{cod_salariat#, nr_proiect#} -> {funcția, suma}

Forma normală 2 (FN2)

Exemplu:

- Presupunem că **un șantier poate executa mai multe lucrări de bază** și că **o lucrare poate fi executată de mai multe șantiere**, iar o lucrare este definită de intervalul de timp în care este executată.

LUCRARE(cod_obiectiv#, cod_lucrare#, nume);

SANTIER(nr_santier#, specialitate, sef);

EXECUTA(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator, data_inceput, data_sfarsit).

- Pentru relația **EXECUTA** sunt evidente dependențele:

$\{\text{cod_obiectiv\#, cod_lucrare\#}\} \rightarrow \{\text{data_inceput, data_sfarsit}\},$

$\{\text{cod_obiectiv\#, cod_lucrare\#, nr_santier\#}\} \rightarrow \{\text{descriere, functie, conducator}\}.$

- Relația **EXECUTA** este în FN1, dar nu este în FN2. **Se aplică regula Casey Delobel:**
EXECUTA_1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator).
EXECUTA_2(cod_obiectiv#, cod_lucrare#, data_inceput, data_sfarsit).

Forma normală 3 (FN3)

- Intuitiv, o relație R este în **a treia formă normală** dacă și numai dacă:
 - relația R este în FN2;
 - fiecare atribut care nu este cheie (nu participă la o cheie) depinde direct de cheia primară;
- Fie R o relație, X o submulțime de attribute ale lui R și A un atribut al relației R . A este **dependent tranzitiv** de X dacă există Y astfel încât $X \rightarrow Y$ și $Y \rightarrow A$
 - De exemplu, dacă $K_1, K_2, K_3 \rightarrow A_1$ și dacă $K_1, K_2, A_1 \rightarrow A_2$.
 - Rezulta ca A_2 este **dependent tranzitiv** de K_1, K_2, K_3 . (A_2 este dependent de K_1, K_2, K_3 prin A_1)

Forma normală 3 (FN3)

atasat_2a

Cod_salariat#	Nr_proiect#	Functia	Suma
S1	P1	Supervizor	60
S1	P2	Cercetator	25
S1	P3	Auxiliar	10
S3	P3	Supervizor	60
S5	P3	Supervizor	60

Formal, o relație R este în **a treia formă normală** dacă și numai dacă:

- relația R este în FN2; -> **este in FN2** deoarece este in FN1 si indeplineste conditia ca fiecare atribut care nu este cheie (nu participă la cheia primară) sa fie dependent de întreaga cheie primară
- fiecare atribut care nu este cheie (nu participă la o cheie) nu este dependent tranzitiv de nicio cheie a relatiei R. **Cu alte cuvinte o relație este în FN3 dacă și numai dacă fiecare atribut (coloană) care nu este cheie, depinde de cheie, de întreaga cheie și numai de cheie.**

In exemplul **atasat_2a** se observa ca atributul **suma** depinde tranzitiv de cheia primara compusa **cod_salariat** si **nr_proiect** prin intermediul atributului **functia**.

Forma normală 3 (FN3)

atasat_2a

Cod_salariat#	Nr_proiect#	Funcția	Suma
S1	P1	Supervizor	60
S1	P2	Cercetator	25
S1	P3	Auxiliar	10
S3	P3	Supervizor	60
S5	P3	Supervizor	60

- În exemplul **atasat_2a** se observă că atributul **suma** depinde tranzitiv de cheia primară compusă **cod_salariat** și **nr_proiect** prin intermediul atributului **funcția**.
- Astfel avem:
 - {cod_salariat#, nr_proiect#} → {funcția} – cod_salariat și nr_proiect determină funcțional atributul funcția
 - {cod_salariat#, nr_proiect#} → {funcția} → {suma}

Pentru a aduce relația **atasat_2a** în FN3 se aplică **regula Casey-Delobel**. Relația se descompune, prin eliminarea dependențelor funcționale tranzitive, în proiecțiile:

ATASAT_3A(cod_salariat#, nr_proiect#, funcția)

ATASAT_3B(funcția, suma)

Forma normală 3 (FN3)

atasat_2a

Cod_salariat#	Nr_proiect#	Funcția	Suma
S1	P1	Supervizor	60
S1	P2	Cercetator	25
S1	P3	Auxiliar	10
S3	P3	Supervizor	60
S5	P3	Supervizor	60

- Pentru a aduce relația **atasat_2a** în FN3 se aplică **regula Casey-Delobel**. Relația se descompune, prin eliminarea dependențelor funcționale tranzitive, în proiecțiile:

ATASAT_3A(cod_salariat#, nr_proiect#, funcția)

ATASAT_3B(funcția, suma)

- Transformarea în FN3

atasat_3a

Cod_salariat#	Nr_proiect#	Funcția
S1	P1	Supervizor
S1	P2	Cercetator
S1	P3	Auxiliar
S3	P3	Supervizor
S5	P3	Supervizor

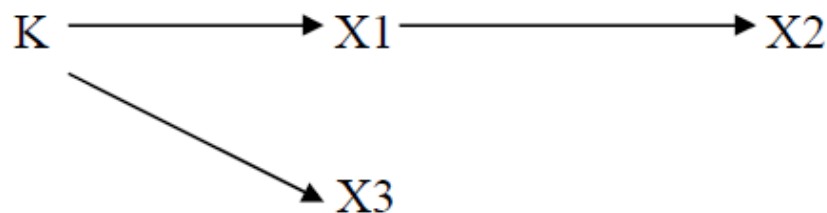
atasat_3b

Funcția	Suma
Supervizor	60
Cercetator	25
Auxiliar	10

Forma normală 3 (FN3)

Aplicarea regulii Casey-Delobel pentru FN3:

- Fie relația $R(K, X_1, X_2, X_3)$, unde atributul X_2 depinde tranzitiv de K , iar K este cheia primară a lui R . Presupunem că $K \rightarrow X_1 \rightarrow X_2$
- Din cauza dependenței funcționale $X_1 \rightarrow X_2$ care arată că R nu este în FN3, se înlocuiește R (fără pierdere de informație) prin două proiecții $R1(K, X_1, X_3)$ și $R2(X_1, X_2)$



Forma normală 3 (FN3)

Exemplu:

- În tabelul **EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)** continuă să existe redundanță în date;
- Atributul **conducator** depinde indirect de cheia primară prin intermediul atributului **functie**. Între attributele relației există dependențele:

{cod_obiectiv#, cod_lucrare#, nr_santier#} → {descriere},

{cod_obiectiv#, cod_lucrare#, nr_santier#} → {functie} → {conducator}.

- Se aplică **regula Casey-Delobel**. Relația se descompune, prin eliminarea dependențelor funcționale tranzitive, în proiecțiile:

EXECUTA1_1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie)

EXECUTA1_2(functie, conducator).

Schema de sinteză pentru obținerea lui FN3

- **Algoritmul de sinteză** construiește o acoperire minimală F a dependențelor funcționale totale.
 - Se elimină atributele și dependențele funcționale redundante;
 - Mulțimea F este partiționată în grupuri F_i , astfel încât în fiecare grup F_i sunt dependențe funcționale care au același membru stâng și nu există două grupuri având același membru stâng;
 - Fiecare grup F_i produce o schemă FN3;
- Algoritmul realizează o descompunere ce conservă dependențele.

Schema de sinteză pentru obținerea lui FN3

Algoritm SNF3 (aducerea unei relații în FN3 prin utilizarea unei scheme de sinteză):

1. Se determină **F o acoperire minimală a lui E** (mulțimea dependențelor funcționale);
2. Se descompune mulțimea F în **grupuri notate F_i** , astfel încât în cadrul fiecărui grup să existe dependențe funcționale **având aceeași parte stângă**;
3. Se determină **perechile de chei echivalente (X, Y)** în raport cu F (două mulțimi de attribute X, Y sunt chei echivalente dacă în mulțimea de dependențe E există atât dependența $X \rightarrow Y$, cât și dependența $Y \rightarrow X$);

Schema de sinteză pentru obținerea lui FN3

4. Pentru fiecare pereche de chei echivalente: se identifică grupurile F_i și F_j care conțin dependențele funcționale cu partea stângă X și respectiv Y ; se formează **un nou grup de dependențe F_{ij}** , care va conține dependențele funcționale având membrul stâng (X, Y) ; se elimină grupurile F_i și F_j , iar locul lor va fi luat de grupul F_{ij} .
5. Se determină o **acoperire minimală a lui F** , care va include toate dependențele $X \rightarrow Y$, unde X și Y sunt chei echivalente (celelalte dependențe sunt redundante).
6. Se construiesc relații FN3 (câte o relație pentru fiecare grup de dependențe funcționale).

Schema de sinteză pentru obținerea lui FN3

- Se observă că algoritmul solicită:
 - determinarea unei **acoperiri minimale** (algoritmii **EAR** și **EDF**);
 - determinarea **închiderii (A^+) unei mulțimi de attribute A** în raport cu mulțimea de dependențe funcționale E (algoritm **AIDF**).
- Determinarea acoperirii minimale presupune eliminarea atributelor și dependențelor redundante;
- Acoperirea minimală **nu este unică** și depinde de ordinea în care sunt eliminate aceste attribute și dependențe redundante;

Schema de sinteză pentru obținerea lui FN3

- **Algoritm EAR** (elimină attributele redundante din determinantul dependențelor funcționale)
- Pentru fiecare dependență funcțională din E și pentru fiecare atribut din partea stângă a unei dependențe funcționale:
 - **Pas1.** Se elimină atributul considerat.
 - **Pas2.** Se calculează închiderea părții stângi reduse.
 - **Pas3.** Dacă închiderea conține toate attributele din determinantul dependenței, atunci atributul eliminat la pasul 1 este redundant și rămâne eliminat. În caz contrar, atributul nu este redundant și se reintroduce în partea stângă a dependenței funcționale.

Schema de sinteză pentru obținerea lui FN3

- **Algoritm EDF** (elimină dependențele funcționale redundante din E)
- Pentru fiecare dependență funcțională $X \rightarrow Y$ din E:
 - **Pas1.** Se elimină dependența din E.
 - **Pas2.** Se calculează închiderea X^+ , în raport cu mulțimea redusă de dependențe.
 - **Pas3.** Dacă Y este inclus în X^+ , atunci dependența $X \rightarrow Y$ este redundantă și rămâne eliminată. În caz contrar, se reintroduce în E.

Schema de sinteză pentru obținerea lui FN3

➤ **Algoritm AIDF** (determină închiderea lui A)

- **Pas1.** Se caută dacă există în E dependențe $X \rightarrow Y$ pentru care determinantul X este o submulțime a lui A, iar determinatul Y nu este inclus în A.
- **Pas2.** Pentru fiecare astfel de dependență funcțională se adaugă mulțimii A attributele care constituie determinatul dependenței.
- **Pas3.** Dacă nu mai există dependențe funcționale de tipul de la pasul 1, atunci $A^+ = A$.

Exemplu

Fie relația **R(F, N, P, U, C, T)**. Se aplică **algoritmul SNF3** (aducerea unei relații în FN3 prin utilizarea schemei de sinteză).

1. Se determină **F o acoperire minimală a lui E** (mulțimea dependențelor funcționale). Aceste dependențe se aplică știind regulile de proiectare ale modelului. Se pornește de la descrierea modelului real.

$F \rightarrow N; F \rightarrow P; P, F, N \rightarrow U; P \rightarrow C; P \rightarrow T; C \rightarrow T; N \rightarrow F$

Exemplu

2. Se descompune mulțimea F în **grupuri notate F_i** , astfel încât în cadrul fiecărui grup să existe dependențe funcționale **având aceeași parte stângă**;

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

Exemplu

Se elimină **atributele** și dependențele funcționale redundante.

3. Se determină **perechile de chei echivalente (X, Y)** în raport cu F (două mulțimi de atribute X, Y sunt chei echivalente dacă în mulțimea de dependențe E există atât dependența $X \rightarrow Y$, cât și dependența $Y \rightarrow X$);

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

➤ **Atributele redundante** se identifică în raport cu dependențele pe care le au cu alte atribute.

În **F3** se observă dependența $P \rightarrow U$
- se poate elimina astfel atributul P deoarece avem:

➤ **F2:** $F \rightarrow P$ și

➤ **F3:** $P \rightarrow U$

➤ rezultând $F \rightarrow U$, dependență pe care o avem deja tot în **F3**.

Exemplu

Se elimină **atributele** și dependențele funcționale redundante.

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

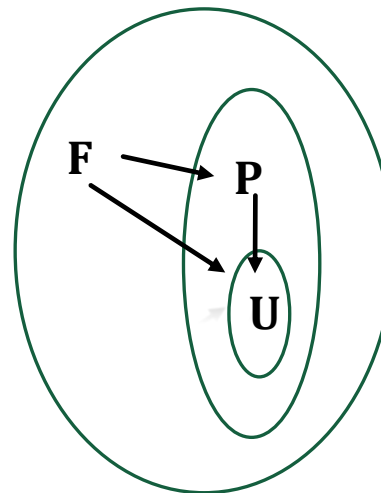
F7: $N \rightarrow F$

Atributul se elimină utilizand pseudo-tranzitivitatea astfel:

dacă $X \rightarrow Y$ și $W \cup Y \rightarrow Z$, atunci $W \cup X \rightarrow Z$

$F \rightarrow P$ și $P, F \rightarrow U$, atunci rezultă $F \rightarrow U$

SAU conform reprezentării grafice de mai jos:



Exemplu

Se elimină **atributele** și dependențele funcționale redundante.

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

La fel se procedează și în cazul atributului N

$F \rightarrow N$

$F, N \rightarrow U$

=> se elimină N și se păstrează în F3 doar dependența $F \rightarrow U$

Exemplu

Se elimină **atributele** și dependențele funcționale redundante.

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

În final F3 o să aibă dependența funcțională
 $F \rightarrow U$

Exemplu

Se elimină attributele și **dependențele funcționale** redundante.

3. Se determină **perechile de chei echivalente (X, Y)** în raport cu F (două mulțimi de attribute X, Y sunt chei echivalente dacă în mulțimea de dependențe E există atât dependența $X \rightarrow Y$, cât și dependența $Y \rightarrow X$);

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $F \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

În acest caz se identifică perechi echivalente analizând dependențe funcționale marcate cu roșu.

Din dependențele $P \rightarrow C$ și $C \rightarrow T$ rezultă că $P \rightarrow T$.

Deci **F5: $P \rightarrow T$** este o dependență funcțională care poate fi eliminată.

Exemplu

Se elimină attributele și **dependențele funcționale** redundante.

3. Se determină **perechile de chei echivalente (X, Y)** în raport cu F (două mulțimi de attribute X, Y sunt chei echivalente dacă în mulțimea de dependențe E există atât dependența $X \rightarrow Y$, cât și dependența $Y \rightarrow X$);

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $F \rightarrow U$

F4: $P \rightarrow C$

~~**F5:** $P \rightarrow T$~~

F6: $C \rightarrow T$

F7: $N \rightarrow F$

Pentru eliminarea dependențelor funcționale se identifică perechi echivalente. În acest caz se analizează dependențele funcționale marcate cu roșu.

Din dependențele **$P \rightarrow C$** și **$C \rightarrow T$** rezultă că **$P \rightarrow T$** .

Deci **$F5: P \rightarrow T$** este o dependență funcțională care poate fi eliminată.

Exemplu

Dupa eliminarea atributelor si dependentelor functionale se aplica urmatoarea regula de sinteza:

Mulțimea F este partiționată în grupuri F_i , astfel încât în fiecare grup F_i sunt dependențe funcționale care au același membru stâng și nu există două grupuri având același membru stâng.

Am renumerotat dependențele rămase în urma eliminărilor efectuate la pași anteriori, după care vom aplica regula de mai sus.

G1: F -> N

G2: F -> P

G3: F -> U

G4: P -> C

G5: C -> T

G6: N -> F

Se observă dependențele care au același membru stâng. Pe acestea le vom regrupa.

Exemplu

Mulțimea F este partiționată în grupuri F_i , astfel încât în fiecare grup F_i sunt dependențe funcționale care au același membru stâng și nu există două grupuri având același membru stâng.

G1: **F** $\rightarrow$ N

G2: **F** $\rightarrow$ P

G3: **F** $\rightarrow$ U

G4: **P** $\rightarrow$ C

G5: **C** $\rightarrow$ T

G6: **N** $\rightarrow$ F

F1 = {G1, G2, G3}

F2 = {G4}

F3 = {G5}

F4 = {G6}

Exemplu

4. Pentru fiecare pereche de chei echivalente: se identifică grupurile F_i și F_j care conțin dependențele funcționale cu partea stângă X și respectiv Y ; se formează **un nou grup de dependențe F_{ij}** , care va conține dependențele funcționale având membrul stâng (X, Y) ; se elimină grupurile F_i și F_j , iar locul lor va fi luat de grupul F_{ij} .

G1: **F** -> N

G2: **F** -> P

G3: **F** -> U

G4: **P** -> C

G5: **C** -> T

G6: **N** -> F

F1 = {G1, G2, G3}

F2 = {G4}

F3 = {G5}

F4 = {G6}



F1 = {G1, G2, G3, **G6**}

F2 = {G4}

F3 = {G5}

~~F4 = {G6}~~

G1 și G6 sunt
echivalente

Exemplu

5. Se determină o **acoperire minimală a lui F** , care va include toate dependențele $X \rightarrow Y$, unde X și Y sunt chei echivalente (celelalte dependențe sunt redundante).
6. Se construiesc relații FN3 (câte o relație pentru fiecare grup de dependențe funcționale).

F1 = {G1, G2, G3, G6}

F2 = {G4}

F3 = {G5}



R1 = {F#, N, P, U}

R2 = {P#, C}

R3 = {C#, T}

G1: F -> N

G2: F -> P

G3: F -> U

G4: P -> C

G5: C -> T

G6: N -> F

Exemplul 2

Se dă următoarea schemă relațională. Stiind că **descrierea** se realizează în funcție de **obiectiv**, de **lucrarea executată** și de **șantierul** pe care aceasta are loc, iar **conducătorul** depinde de **funcția** pe care o are, aplicați **algoritmul de sinteză FN3** (SNF3) și aduceți relația în forma normală 3.

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

Se notează attributele relației **EXECUTA1**:

cod_obiectiv = A

cod_lucrare = B

nr_santier = C

descriere = D

functie = E

conducator = F

O relație este în **FN3** dacă și numai dacă fiecare atribut (coloană) care nu este cheie, depinde de cheie, de întreaga cheie și numai de cheie.

Exemplul 2

Se dă următoarea schemă relațională. Stiind că **descrierea** se realizează în funcție de **obiectiv**, de **lucrarea executată** și de **șantierul** pe care aceasta are loc, iar **conducătorul** depinde de **funcția** pe care o are, aplicați **algoritmul de sinteză FN3** (SNF3) și aduceți relația în forma normală 3.

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

1. Se determină **F o acoperire minimală a lui E** (mulțimea dependențelor funcționale). Aceste dependențe se aplică știind regulile de proiectare ale modelului. Se pornește de la descrierea modelului real.
2. Se descompune mulțimea F în **grupuri notate F_i** , astfel încât în cadrul fiecărui grup să existe dependențe funcționale **având aceeași parte stângă**;

Exemplul 2

Se dă următoarea schemă relațională. Stiind că **descrierea** se realizează în funcție de **obiectiv**, de **lucrarea executată** și de **șantierul** pe care aceasta are loc, iar **conducătorul** depinde de **funcția** pe care o are, aplicați **algoritmul de sinteză FN3** (SNF3) și aduceți relația în forma normală 3.

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

F1: A, B, C -> D

F2: A, B, C -> E

F3: E -> F

F1: cod_obiectiv, cod_lucrare, nr_santier -> descriere

F2: cod_obiectiv, cod_lucrare, nr_santier -> functie

F3: functie -> conducator

Exemplul 2

Se elimină attributele și dependențele funcționale redundante.

În exemplul curent nu există attribute sau dependențe funcționale redundante. Așadar, se trece la pașii următori.

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

F1: A, B, C -> D

F1: cod_obiectiv, cod_lucrare, nr_santier -> descriere

F2: A, B, C -> E

F2: cod_obiectiv, cod_lucrare, nr_santier -> functie

F3: E -> F

F3: functie -> conducator

Exemplul 2

Mulțimea F este partiționată în grupuri F_i , astfel încât în fiecare grup F_i sunt dependențe funcționale care au același membru stâng și nu există două grupuri având același membru stâng.

4. Pentru fiecare pereche de chei echivalente: se identifică grupurile F_i și F_j care conțin dependențele funcționale cu partea stângă X și respectiv Y ; se formează **un nou grup de dependențe F_{ij}** , care va conține dependențele funcționale având membrul stâng (X, Y) ; se elimină grupurile F_i și F_j , iar locul lor va fi luat de grupul F_{ij} .

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

F1: A, B, C -> D

F2: A, B, C -> E

F3: E -> F



$F_{1,2} = \{A, B, C, D, E\}$

$F3 = \{E, F\}$



$F_{1,2} = \{\text{cod_obiectiv}, \text{cod_lucrare}, \text{nr_santier}, \text{descriere}, \text{functie}\}$

$F3 = \{\text{functie}, \text{conducator}\}$

Exemplul 2

5. Se determină o **acoperire minimală a lui F** , care va include toate dependențele $X \rightarrow Y$, unde X și Y sunt chei echivalente (celelalte dependențe sunt redundante).
6. Se construiesc relații FN3 (câte o relație pentru fiecare grup de dependențe funcționale).

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

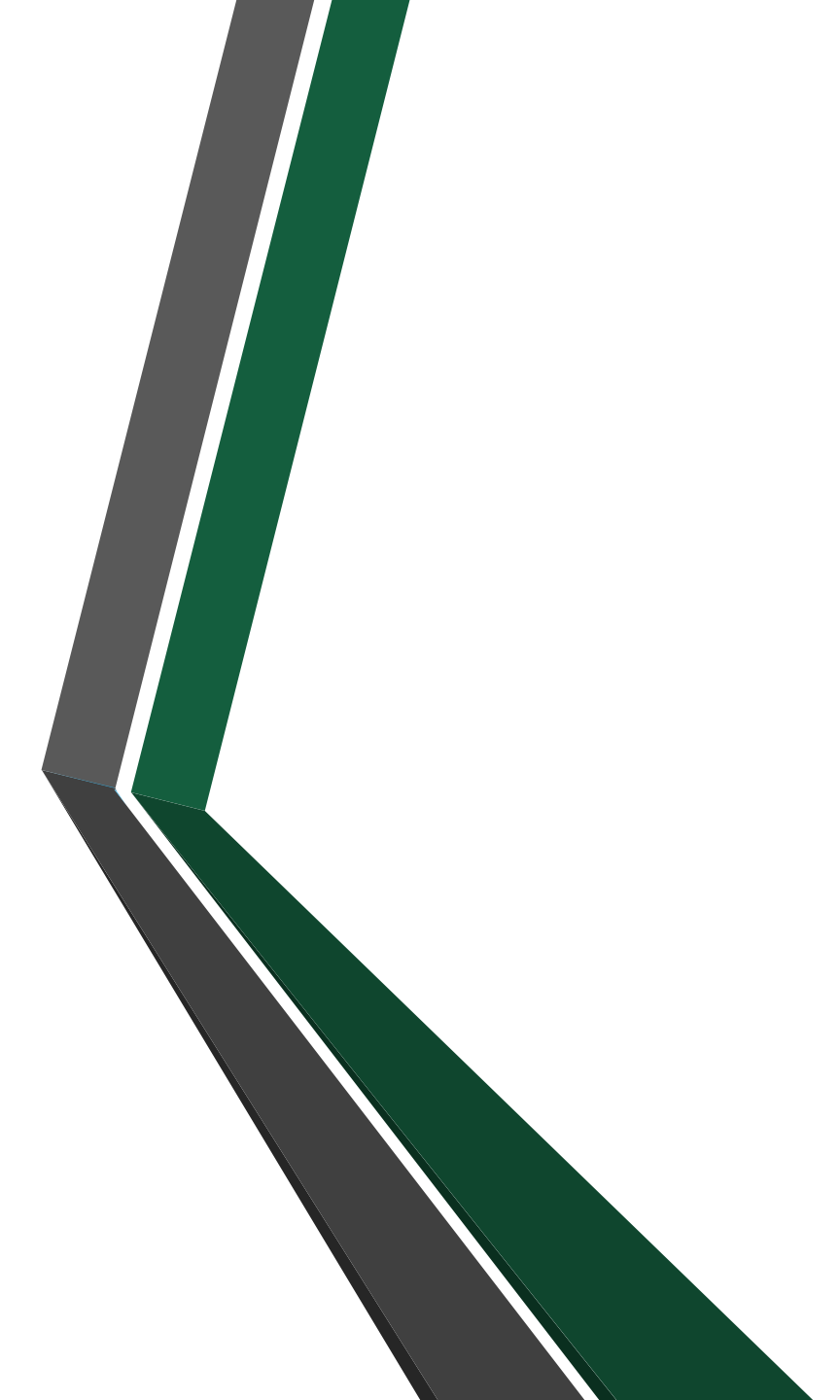
$F_{1,2} = \{\text{cod_obiectiv}, \text{cod_lucrare}, \text{nr_santier}, \text{descriere}, \text{functie}\}$

$F_3 = \{\text{functie}, \text{conducator}\}$



EXECUTA1_1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie)

EXECUTA1_2(functie#, conducator).



BAZE DE DATE

CURS 9

Partea II

Forma normală Boyce-Codd (BCNF)

- Determinantul este un atribut sau o mulțime de attribute neredundante, care constituie un identificator unic pentru alt atribut sau altă mulțime de attribute ale unei relații date.
- Intuitiv, o relație R este în **forma normală Boyce-Codd** dacă și numai dacă **fiecare determinant este o cheie candidat**.
- Formal, o relație R este în forma normală Boyce-Codd dacă și numai dacă pentru orice dependență funcțională totală $X \rightarrow A$, X este o cheie (candidat) a lui R .

Forma normală Boyce-Codd (BCNF)

- O relație R este în **forma normală Boyce-Codd** dacă și numai dacă **fiecare determinant este o cheie candidat**.
- **Regula Casey Delobel** pentru $R(K1\#, K2\#, X)$ presupunând că există dependența: $X \rightarrow K2$ (X determina functional pe K2)

→ $R1(K1\#, X)$ și $R2(X\#, K2)$

BCNF este o versiune puțin mai restrictivă a lui FN3. În FN3 attributele depind de cheia primară, doar de cheia primară și de nimic altceva în afara de cheie. Deci:

$\{K1\#, K2\# \} \rightarrow \{X\}$ (în FN3, X depinde de cheia K1#, K2#)

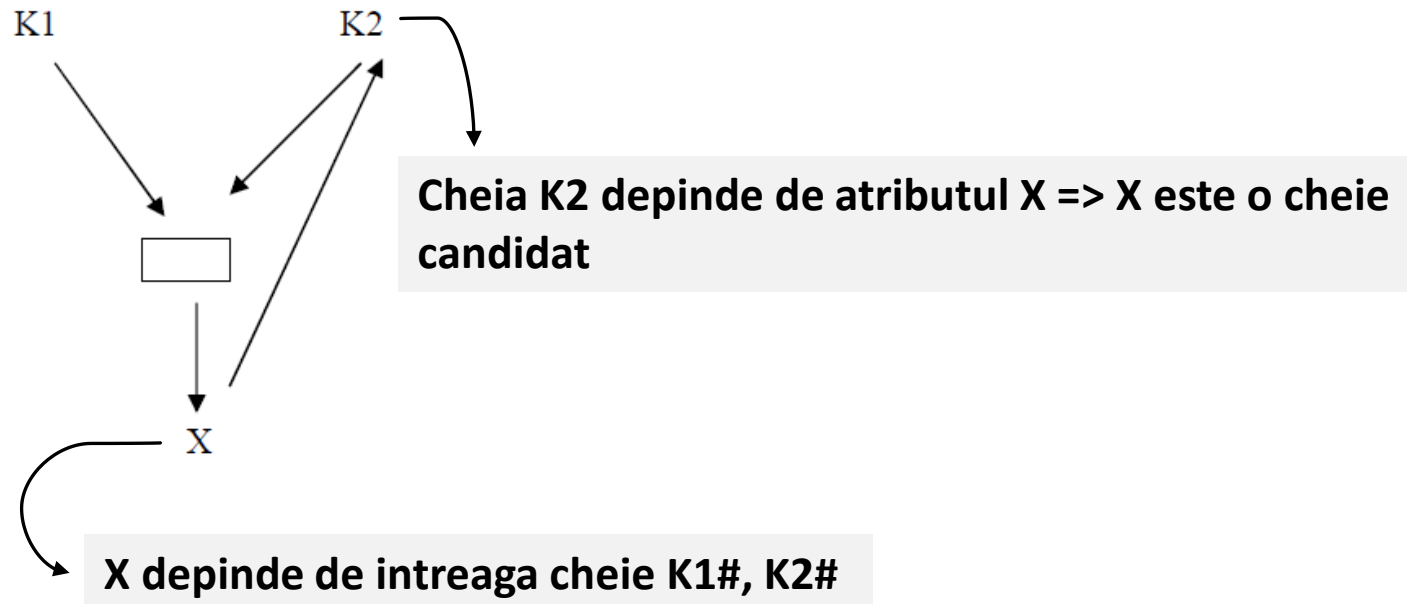
În BCNF fiecare determinant (în cazul exemplului curent - X) este o cheie candidat (adică poate fi cheia primară).

Asadar, $X \rightarrow K2$ (cheia K2 depinde de atributul X, deci X este o cheie candidat pentru relația R).

Forma normală Boyce-Codd (BCNF)

- O relație R este în **forma normală Boyce-Codd** dacă și numai dacă **fiecare determinant este o cheie candidat**.
- **Regula Casey Delobel** pentru $R(K1\#, K2\#, X)$ presupunând că există dependența: $X \rightarrow K2$ (X determina functional pe K2)

➔ $R1(K1\#, X)$ și $R2(X\#, K2)$ -> X este cheia candidat



Forma normală Boyce-Codd (BCNF)

Exemplul 1:

Relația **INVESTESE_IN** leagă entitățile **INVESTITOR** și **OBIECTIV_INVESTITIE**. Ea are schema relațională:

INVESTESE_IN (cod_contractant#, cod_obiectiv#, nr_contract, cota_parte)

- Între atributele relației există dependențele:

$\{\text{cod_contractant\#}, \text{cod_obiectiv\#}\} \rightarrow \{\text{nr_contract}, \text{cota_parte}\},$

$\{\text{nr_contract}\} \rightarrow \{\text{cod_obiectiv}\}$ (se observa ca nr_contract este o cheie candidat, cheia cod_obiectiv depinzand de atributul nr_contract)

- Se aplică regula Casey-Delobel și **se aduce relația în BCNF.**

INVESTESE_IN_1 (cod_obiectiv, nr_contract#); (nr_contract fiind cheie candidat devine cheie primara)

INVESTESE_IN_2 (cod_contractant#, nr_contract, cota_parte); (nr_contract este cheie externa)

Forma normală Boyce-Codd (BCNF)

INVESTESE_IN (cod_contractant#, cod_obiectiv#, nr_contract, cota_parte).

Se descompune in:

INVESTESE_IN_1 (cod_obiectiv, nr_contract#); (nr_contract fiind cheie candidat devine cheie primara)

INVESTESE_IN_2 (cod_contractant#, nr_contract, cota_parte).
(nr_contract este cheie externa)

cod_contractant#	cod_obiectiv#	nr_contract	cota_parte
C1	01	NR1	20%
C1	02	NR2	20%
C2	02	NR2	30%
C2	01	NR3	30%
C3	01	NR1	35%

Se poate observa cheia candidat **nr_contract**

Atributul nr_contract devine cheie primara, iar relatia se va descompune astfel:

Forma normală Boyce-Codd (BCNF)

INVESTESTE_IN_1 (cod_obiectiv, nr_contract#); (nr_contract fiind cheie candidat devine cheie primara)

INVESTESTE_IN_2 (cod_contractant#, nr_contract#, cota_parte). (nr_contract este cheie externa)

cod_contractant#	cod_obiectiv#	nr_contract	cota_parte
C1	01	NR1	20%
C1	02	NR2	20%
C2	02	NR2	30%
C2	01	NR3	30%
C3	01	NR1	35%

Atributul nr_contract devine cheie primara, iar relatia se va descompune astfel:

cod_obiectiv	nr_contract#
01	NR1
02	NR2
01	NR3

cod_contractant#	nr_contract#	cota_parte
C1	Nr1	20%
C1	Nr2	20%
C2	Nr2	30%
C2	Nr3	30%
C3	Nr1	35%

Forma normală Boyce-Codd (BCNF)

Exemplul 2:

Avem următoarea relație: **R (Student#, Materie#, Profesor)** – un student este înscris la mai multe materii, urmând cursurile mai multor profesori, iar un profesor predă o singură materie.

{Student#, Materie#} -> {Profesor}

{Profesor} -> {Materie} (atributul profesor este o cheie candidat deoarece un profesor predă o singură materie, deci apare o singură dată alături de materia lui)

- Un student este înscris la mai multe materii (avem nevoie de cheia primară compusă astfel încât studentul să poată participa în cadrul diferitelor materii). Un profesor poate predă o singură materie.

S1	M1	P1
S2	M1	P1
S3	M1	P1
S1	M2	P2

Se observă că studentul S1 este înscris la materiile M1 și M2, predate de profesorii P1 și P2. De asemenea, materia M1 are mai mulți studenți înscrși.

Forma normală Boyce-Codd (BCNF)

$\{\text{Student\#}, \text{Materie\#}\} \rightarrow \{\text{Profesor}\}$

$\{\text{Profesor}\} \rightarrow \{\text{Materie}\}$ (atributul profesor este o cheie candidat deoarece un profesor predă o singură materie, deci apare o singură dată alături de materia lui)

S1	M1	P1
S2	M1	P1
S3	M1	P1
S1	M2	P2

Aplicăm regula Casey-Delobel:

R1 (Profesor#, Materie#) – cheia candidat **Profesor** a devenit cheie primară

R2 (Student#, Profesor#) – atributul Profesor nu trebuie să fie neapărat cheie externă în R2. Poate fi și cheie primară compusă (student#, profesor# - este cheie primară compusă, dar atributul profesor individual este cheie externă). **În acest caz este utilă cheia primară compusă. De ce?**

Forma normală Boyce-Codd (BCNF)

- Pentru ca o relație să fie adusă în BCNF **nu trebuie în mod obligatoriu să fie în FN3**.
- Se pot aduce în BCNF și relații aflate în FN1 sau FN2.
 - Acest lucru este posibil întrucât dependențele funcționale parțiale și cele tranzitive sunt tot dependențe noncheie, adică dependențe ai căror determinanți nu sunt chei candidat.

Forma normală 4 (FN4)

- FN4 elimină redundanțele datorate relațiilor $m:n$, adică datorate **dependenței multiple**
- Intuitiv, o relație R este în **a patra formă normală** dacă și numai dacă relația este în BCNF (adică fiecare determinant trebuie să fie o cheie candidat) și nu conține relații $m:n$ independente
- Multidependentele se notează astfel:
 - $X \twoheadrightarrow Z$ (X multidetermină pe Z sau Z este multidependent de X)

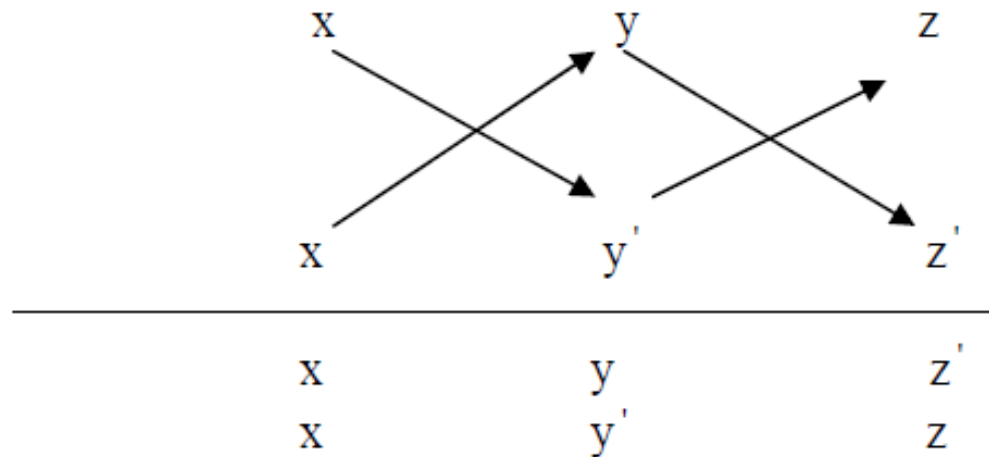
Forma normală 4 (FN4)

- Fie R o relație definită pe o mulțime de attribute $A = \{A_1, A_2, \dots, A_n\}$ și fie $X, Y, Z \subset A$. Se spune că X **multidetermină** pe Z sau că Z este multidependent de X :
 - dacă pentru fiecare valoare a lui Z în R există numai o valoare pentru perechea (X, Y) ;
 - dacă valoarea lui Z depinde numai de valoarea lui X .
- Acest tip de dependență, numită și multivaloare sau multidependență (MVD) se notează prin $X \twoheadrightarrow Z$
- Intuitiv, multidependența reprezintă situația în care valoarea unui atribut (sau a unei mulțimi de attribute) determină o mulțime de valori a altui atribut (sau mulțimi de attribute)

Forma normală 4 (FN4)

Multidependența $X \twoheadrightarrow Y$ poate fi gândită ca o **regulă de deducție**:

- dacă tuplurile $\langle x, y, z \rangle$ și $\langle x, y', z' \rangle$ sunt în relație la un moment r , atunci la momentul r sunt în relație și tuplurile $\langle x, y, z' \rangle$ și $\langle x, y', z \rangle$.



Forma normală 4 (FN4)

FN4 elimină redundanțele datorate relațiilor $m:n$, adică datorate **dependenței multiple**.

Intuitiv, o relație R este **în a patra formă normală** dacă și numai dacă relația este în BCNF (adică fiecare determinant trebuie să fie o cheie candidat) și conține relații $m:n$ dependente (dependenta – materie și profesor / independenta – user și song)

Formal, relația R este **în a patra formă normală** dacă și numai dacă:

- R este în BCNF;
- orice dependență multivaloare este o dependență funcțională;

Cu alte cuvinte, o relație BCNF este în FN4 dacă pentru orice multidependență elementară de forma $X \twoheadrightarrow Y$, X este o **supercheie** a lui R .

Forma normală 4 (FN4)

Exemplul 1:

Un restaurant are mai multe tipuri de pizza si mai multe arii de distributie. Astfel, daca este necesara inserarea unui nou restaurant se poate realiza aceasta operatie doar daca se insereaza si un tip de pizza, dar si o arie de distributie.

Exista urmatoarea relatie: **R (FIRMA#, TIP_LIVRARE#, ZONA#)**

In acest exemplu avem urmatoarele dependente multiple:

firma $\rightarrow \rightarrow$ tip_livrare

firma $\rightarrow \rightarrow$ zona

Regula de descompunere în relații FN4:

- Fie $R(X, Y, Z)$ o schemă relațională care nu este în FN4 și fie $X \rightarrow \rightarrow Y$ o multidependență elementară care nu este de forma „CHEIE $\rightarrow \rightarrow$ atribut”. Această relație este descompusă prin proiecție în două relații:

$$R = \text{JOIN}(\Pi_{X \cup Y}(R), \Pi_{X \cup Z}(R)).$$

In cazul exemplului anterior, relatia R se va descompune astfel:

R1 (FIRMA#, TIP_LIVRARE#)

R2 (FIRMA#, ZONA#)

Forma normală 4 (FN4)

Exemplul 1:

O companie de curierat are mai multe tipuri de livrari si mai multe zone in care livreaza.

Exista urmatoarea relatie:

R (FIRMA#, TIP_LIVRARE#, ZONA#)

In acest exemplu avem urmatoarele dependente multiple:

FIRMA -> -> TIP_LIVRARE

FIRMA -> -> ZONA

Relatiile rezultate dupa aplicarea lui FN4:

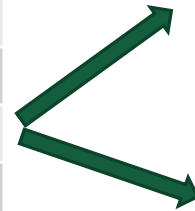
R1 (FIRMA#, TIP_LIVRARE#)

R2 (FIRMA#, ZONA#)

FIRMA	TIP_LIVRARE	ZONA
GLS	RAPID	ZONA1
GLS	RAPID	ZONA2
GLS	CLASIC	ZONA1
GLS	CLASIC	ZONA2
FAN	RAPID	ZONA3
FAN	CLASIC	ZONA3
CARGUS	CLASIC	ZONA1
CARGUS	CLASIC	ZONA2
CARGUS	CLASIC	ZONA4
CARGUS	RAPID	ZONA1
CARGUS	RAPID	ZONA2
CARGUS	RAPID	ZONA4

Forma normală 4 (FN4)

FIRMA	TIP_LIVRARE	ZONA
GLS	RAPID	ZONA1
GLS	RAPID	ZONA2
GLS	CLASIC	ZONA1
GLS	CLASIC	ZONA2
FAN	RAPID	ZONA3
FAN	CLASIC	ZONA3
CARGUS	CLASIC	ZONA1
CARGUS	CLASIC	ZONA2
CARGUS	CLASIC	ZONA4
CARGUS	RAPID	ZONA1
CARGUS	RAPID	ZONA2
CARGUS	RAPID	ZONA4



FIRMA	TIP_LIVRARE
GLS	RAPID
GLS	CLASIC
FAN	RAPID
FAN	CLASIC
CARGUS	CLASIC
CARGUS	RAPID

FIRMA	ZONA
GLS	ZONA1
GLS	ZONA2
FAN	ZONA3
CARGUS	ZONA1
CARGUS	ZONA2
CARGUS	ZONA4

Forma normală 4 (FN4)

Exemplul 2:

Avem următoarea relație **R(Curs#, Prof#, Carte#)** și regula – un curs este predat de mai mulți profesori și se afla în mai multe cărți.

Curs#	Prof#	Carte#
1	P1	Carte1
1	P1	Carte2
1	P2	Carte1
1	P2	Carte2

Relația nu se află în FN4 din cauza multidependențelor:

Curs $\twoheadrightarrow$ Prof

Curs $\twoheadrightarrow$ Carte

Aducem în FN4:

Curs1(Curs#, Carte#)

1	Carte1
1	Carte2

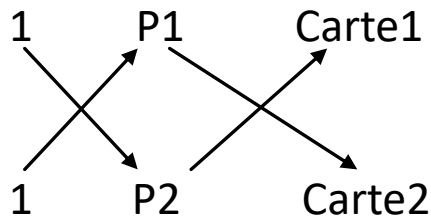
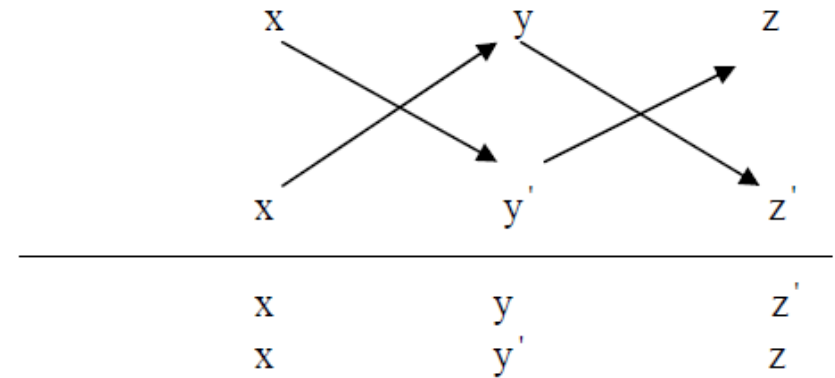
Curs2(Curs#, Prof#)

1	P1
1	P2

Forma normală 4 (FN4)

Cum se poate aplica regula de deducție în acest caz:

Curs#	Prof#	Carte#
1	P1	Carte1
1	P1	Carte2
1	P2	Carte1
1	P2	Carte2



Pornim de la cursul 1 situat în partea de sus, urmând săgețile =>

=> **1 P2 Carte1**

Pornim de la cursul 1 din partea de jos =>

=> **1 P1 Carte2**

Forma normală 4 (FN4)

Exemplul 3:

Fie relația:

INVESTITIE(cod_contractant#, cod_obiectiv#, telefon#) și ca regulă de proiectare -> un investitor poate avea mai multe numere de telefon și poate investi în mai multe obiective.

Între attributele relației există multidependențele:

cod_contractant# $\rightarrow\rightarrow$ cod_obiectiv;

cod_contractant# $\rightarrow\rightarrow$ telefon;

Relația **INVESTITIE** este în **BCNF**. Pentru a aduce relația în **FN4** o vom descompune prin proiecție în două relații:

INVESTITIE_1(cod_contractant#, cod_obiectiv#),

INVESTITIE_2(cod_contractant#, telefon#).

INVESTITIE = JOIN(INVESTITIE_1, INVESTITIE_2).

Forma normală 5 (FN5)

- **FN5** își propune eliminarea redundanțelor care apar în relații $m:n$ dependente.
 - În general, aceste relații nu pot fi descompuse.
 - S-a arătat că o relație de tip 3 este diferită de trei relații de tip 2. Există totuși o excepție, și anume, **dacă relația este ciclică**
- **Intuitiv**, o relație R este **în forma normală 5** dacă și numai dacă:
 - relația este în FN4;
 - **nu conține dependențe ciclice.**

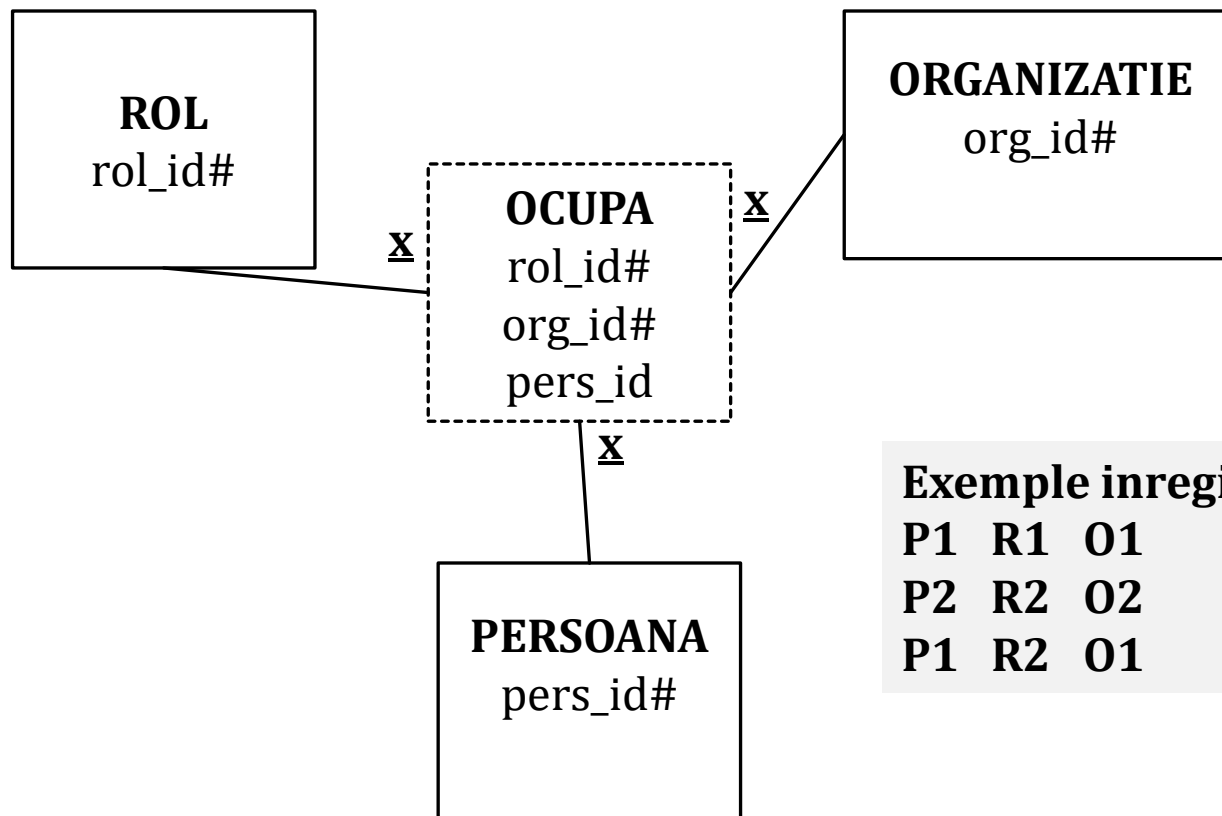
Forma normală 5 (FN5)

- Dependența funcțională și multidependența permit descompunerea prin proiecție, fără pierdere de informație, a unei relații în **două relații**;
- Regulile de descompunere (FN1 – FN4) nu dau toate descompunerile posibile prin proiecție ale unei relații;
- Există relații care nu pot fi descompuse în două relații, dar pot fi descompuse în trei, patru sau mai multe relații fără a pierde informații.
- Pentru a obține descompuneri *L-join* în trei sau mai multe relații, s-a introdus conceptul de **join-dependență sau dependență la compunere** (JD).

Forma normală 5 (FN5)

Exemplu:

O persoana poate ocupa mai multe roluri si poate face parte din mai multe organizatii. O organizatie contine mai multe roluri.

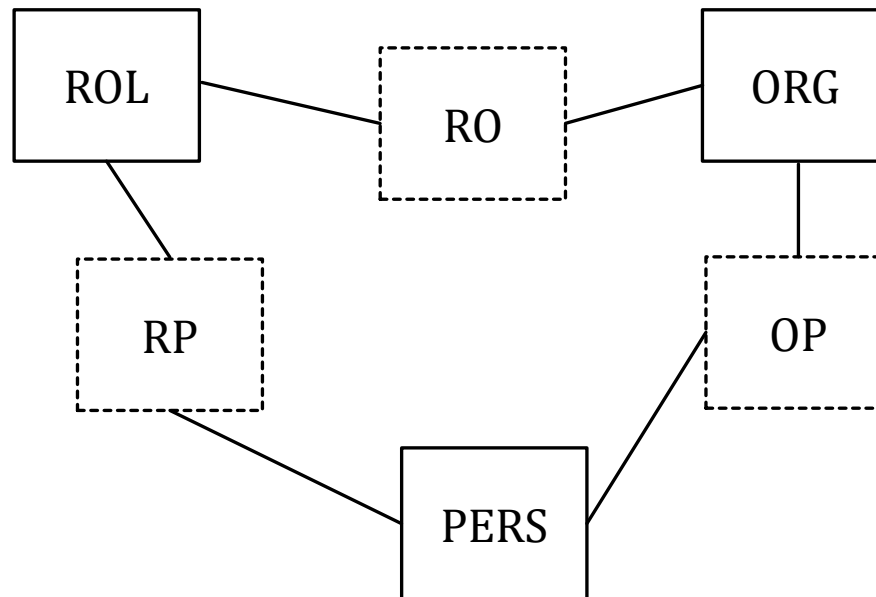


Exemple inregistrari:

P1	R1	01
P2	R2	02
P1	R2	01

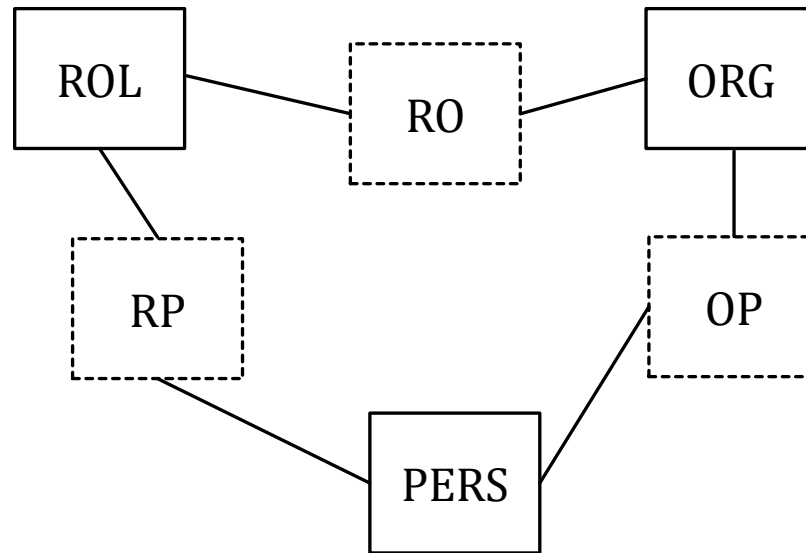
Forma normală 5 (FN5)

Relatia de tip 3 precedenta poate fi echivalenta cu 3 relatii de tip 2 doar daca aceste relatii de tip 2 sunt ciclice. In cazul de mai jos, relatiile de tip 2 existente sunt ciclice, conform regulilor modelului.



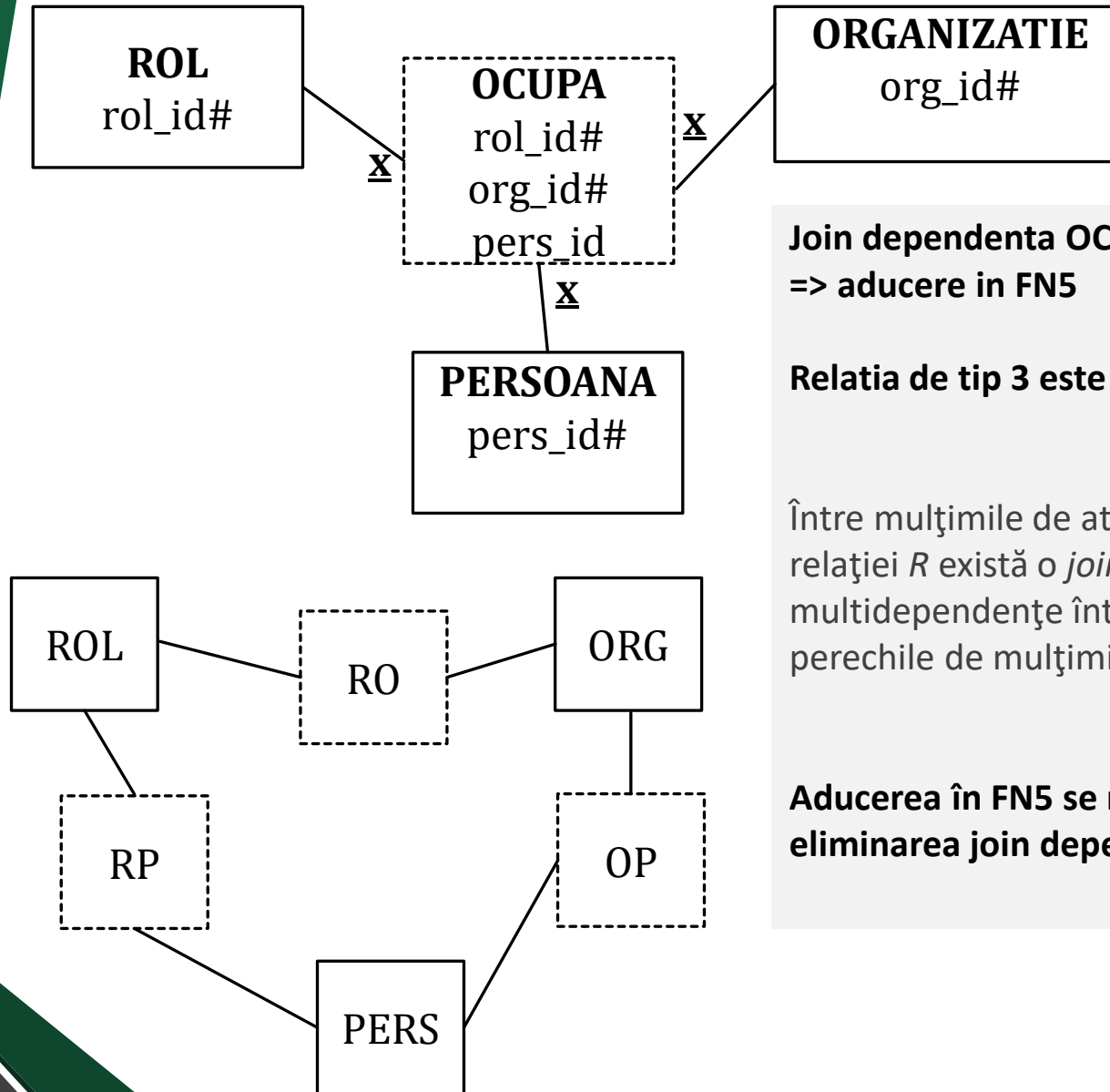
Forma normală 5 (FN5)

Relatia de tip 3 precedenta poate fi echivalenta cu 3 relatii de tip 2 doar daca aceste relatii de tip 2 sunt ciclice. In cazul de mai jos, relatiile de tip 2 existente sunt ciclice, conform regulilor modelului.



Relatia fiind ciclica, atunci cand facem toate join-urile vom obtine un rezultat echivalent cu cel obtinut din relatia de tip 3. O relatie pentru a fi in FN5 trebuie sa fie in FN4 si **sa nu contina dependente ciclice**. Se observa ca cele 3 relatii de tip 2 compun o diagrama care contine dependente ciclice, deci relatia de mai sus **nu se afla in FN5**.

Forma normală 5 (FN5)



Join dependenta **OCUPA** = JOIN (RO, RP, OP)
=> aducere în FN5

Relatia de tip 3 este în FN5.

Între mulțimile de attribute X , Y și Z din cadrul relației R există o *join* dependență dacă există multidependențe între fiecare dintre perechile de mulțimi (X, Y) , (Y, Z) și (X, Z) .

Aducerea în FN5 se realizează prin eliminarea join dependențelor.

Concluzii normalizare

1. **FN1 $\rightarrow$ FN2** elimină redundanțele datorate dependenței netotale a atributelor care nu participă la o cheie, față de cheile lui R . Se suprimă dependențele funcționale care nu sunt totale.
2. **FN2 $\rightarrow$ FN3** elimină redundanțele datorate dependenței tranzitive. Se suprimă dependențele funcționale tranzitive.
3. **FN3 $\rightarrow$ BCNF** elimină redundanțele datorate dependenței funcționale. Se suprimă dependențele în care partea stângă nu este o supercheie.
4. **BCNF $\rightarrow$ FN4** elimină redundanțele datorate multidependenței. Se suprimă toate multidependențele care nu sunt și dependențe funcționale.
5. **FN4 $\rightarrow$ FN5** elimină redundanțele datorate dependenței ciclice. Se suprimă toate *join*-dependențele care nu sunt implicate de o cheie.
6. **BCNF, FN4 și FN5** corespund regulii că orice determinant este o cheie, dar de fiecare dată dependența cu care se definește determinantul este alta și anume dependența funcțională, multidependența sau *join*-dependența.
7. Descompunerea unei relații FN2 în FN3 conservă datele și dependențele, pe când descompunerea unei relații FN3 în BCNF și, respectiv, a unei relații BCNF în FN4 conservă doar datele.

Denormalizare

- Procedura de normalizare elimină redundanțele prin efectuarea unor proiecții, DAR NU toate redundanțele pot fi eliminate în acest mod.

=> Uneori este necesară **denormalizarea** care presupune:

- Mărirea redundanței;
- Reducerea numărului de join-uri efectuate => micșorare timp de execuție;

Denormalizare

- Ideile normalizării sunt utile în proiectarea BD, dar **nu sunt obligatorii**
- Dependența și normalizarea sunt de natură semantică (cu alte cuvinte, se referă la ceea ce înseamnă datele)
- În schimb, algebra relațională și calculul relațional (limbajele SQL) se referă doar la valorile efective ale datelor și în multe cazuri nu necesită mai mult decât FN1

Denormalizare

- Obiectivul denormalizării constă în **reducerea numărului de join-uri** efectuate pentru rezolvarea unei interogări, prin realizarea unora dintre acestea în avans, ca făcând parte din proiectarea bazei de date;
- Conceptul de denormalizare suferă de un număr de probleme binecunoscute:
 - Odată începută denormalizarea, nu este clar unde trebuie să se oprească;
 - Nu se mai lucrează cu relații normalizate și astfel pot apărea anomaliile pe care normalizarea le corectează;

Denormalizare

Exemple:

- Avem un tabel în care sunt stocate produse, numit **PRODUSE**. Produsele au și un atribut **greutate**. Această coloană conține valori repetitive, aceeași greutate definind mai multe produse. În acest caz, dacă în baza de date există un tabel separat în care se află greutatea împreună cu id-ul produsului căruia îi corespunde, este necesar procesul de **denormalizare** în urma căruia atributul **greutate** se va plasa în tabelul **PRODUSE** deoarece nu este eficient ca acest atribut să se afle într-un tabel separat.
- Pe de o parte, este un avantaj ca atributul greutate să fie într-un tabel separat pentru cazurile în care **se modifică** greutatea produselor destul de des. Fiind într-un tabel separat, modificarea va avea loc o singură dată în acest tabel, fiind mai rapid și mai optim.
- Pe de alta parte, dacă se consideră că **accesarea** greutății produselor are loc frecvent, odată cu accesarea informațiilor despre produse, atunci denormalizarea este cea mai bună alegere.

Denormalizare

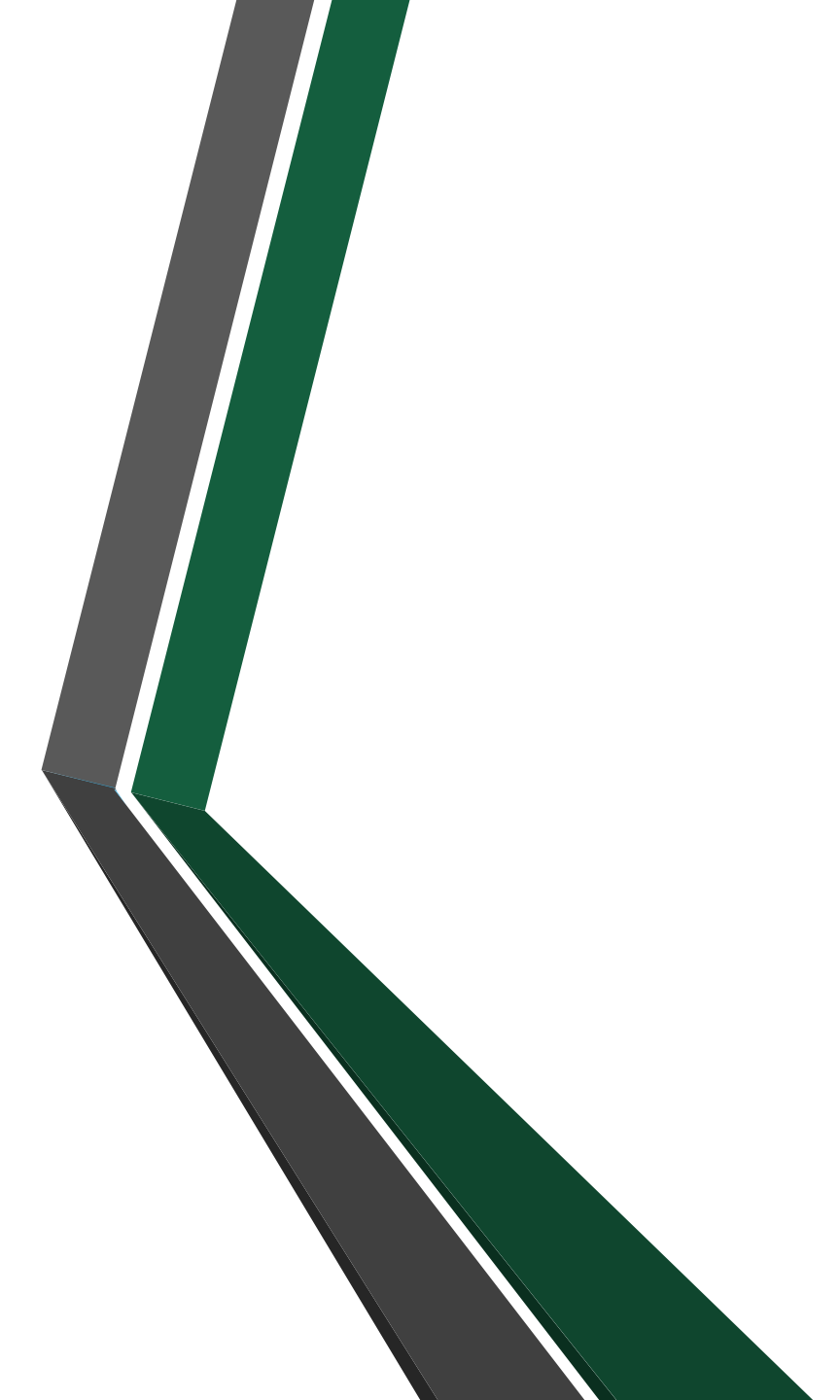
Exemple:

- În cazul în care avem **User** si **Profile** între care există relația one-to-many (un user are un singur profil), iar tabelul profil conține attributele **nume** și **prenume**, atunci se poate realiza o denormalizare în urma căreia cele doua attribute se vor plasa în tabelul User deoarece operațiile de join sunt costisitoare, fiind mult mai eficientă în acest caz mutarea coloanelor în tabelul User. Pe de altă parte, dacă tabelul Profile conține mai multe attribute, atunci se vor păstra ambele tabele în baza de date.

Denormalizare

Când este utilă denormalizarea?

- Ca o regulă empirică, se poate afirma că, dacă performanțele nu sunt satisfăcătoare și relația are o rată de reactualizare scăzută, dar o rată a interogarilor foarte ridicată, denormalizarea poate constitui o opțiune viabilă;
- Nu există reguli fixe pentru stabilirea situațiilor în care este indicată denormalizarea relațiilor;



BAZE DE DATE

CURS 10

SQL

Structured Query Language

- limbaj universal care poate fi utilizat pentru a **defini, interoga, reactualiza** și **gestiona** baze de date relaționale.
 - LDD + LMD + LCD + alte comenzi (de administrare etc.)
- SQL este un limbaj **neprocedural**, adică se specifică **CE** informație este solicitată, dar nu modul **CUM** se obține această informație.
- SQL poate fi utilizat autonom sau prin inserarea comenzilor sale într-un limbaj de programare.

SQL

3 familii de comenzi:

- Comenzi pentru definirea datelor (LDD) - permit **descrierea** (definirea) **structurii obiectelor** ce modelează sistemul studiat.
- Comenzi pentru prelucrarea datelor (LMD) - ce permit **consultarea, reactualizarea, suprimarea sau inserarea** datelor.
- Comenzi pentru controlul datelor (LCD) - permit asigurarea **confidențialității și integrității** datelor, **salvarea** informației, realizarea fizică a **modificărilor** în baza de date, rezolvarea unor probleme de **concurență**.

LDD

- O BD este alcătuită din scheme.
- O schemă este o mulțime de structuri logice de date, numite obiecte. Ea aparține unui utilizator al bazei de date și poartă numele său.
- Specificarea bazelor de date și a obiectelor care le compun se realizează prin intermediul limbajului de definire a datelor (LDD).
- Definirea unui obiect presupune crearea, modificarea și suprimarea sa. Limbajul de definire a datelor cuprinde instrucțiunile SQL care permit realizarea acestor operații (CREATE, ALTER, DROP).
- Instrucțiunile LDD au efect imediat asupra bazei de date și înregistrează informația în dicționarul datelor.
- LDD conține și instrucțiunile RENAME, TRUNCATE.



LDD

Tipuri de date

- Pentru memorarea datelor numerice, tipurile cele mai frecvent folosite sunt: **NUMBER, INTEGER, FLOAT, DECIMAL**.
- Pentru memorarea șirurilor de caractere, cele mai frecvent tipuri de date utilizate sunt: **CHAR, VARCHAR2** și **LONG** (inlocuit de **CLOB**).
- Informații relative la timp sau dată calendaristică se obțin utilizând tipul **DATE**.
 - anul, luna, ziua, ora, minutul, secunda
 - pentru o coloană de tip DATE sistemul rezervă 7 bytes

LDD

Modele de format

- Un model de format este un literal caracter care descrie formatul valorilor de tip DATE sau NUMBER stocate într-un șir de caractere.
- Atunci când se convertește un șir de caractere într-o dată calendaristică sau într-un număr, modelul de format indică sistemului cum să interpreteze șirul respectiv.
- În instrucțiunile SQL se poate folosi un model de format ca argument al funcțiilor **TO_CHAR** și **TO_DATE**.

LDD

Valoarea NULL

- Valoarea *null*, reprezentând lipsa datelor, nu este egală cu nicio altă valoare sau nu este diferită de nicio altă valoare, inclusiv *null*;
- Sistemul Oracle consideră două valori *null* ca fiind egale la evaluarea funcției DECODE

LDD

Pseudocoloane

- O pseudocoloană se comportă ca o coloană a unui tabel, dar nu este stocată efectiv într-un tabel.
- Se pot face interogări asupra pseudocoloanelor, dar nu se pot insera, actualiza sau șterge valorile acestora.
- LEVEL returnează nivelul liniilor rezultat ale unei cereri ierarhice.
- CURRVAL și NEXTVAL sunt pseudocoloane utile în lucrul cu secvențe și sunt tratate în secțiunea corespunzătoare acestora.
- ROWNUM returnează numărul de ordine al liniilor rezultate în urma execuției unei cereri. Pseudocoloana poate fi utilizată pentru a limita numărul de linii returnate.

Tabele

Crearea unui tabel

- constă din generarea **structurii** sale, adică atribuirea unui nume tabelului și definirea caracteristicilor sale (se definesc coloanele, se definesc constrângerile de integritate, se specifică parametrii de stocare, etc).
- Comanda CREATE TABLE permite crearea unui **tabel relațional** sau a unui tabel obiect. Tabelul relațional reprezintă **structura fundamentală pentru stocarea datelor utilizatorului**.

Tabele

- Pentru a crea un tabel, utilizatorul trebuie să aibă acest **privilegiu** și să dispună de **spațiul de memorie** în care să creeze obiectul.
- La nivelul schemei sale, un utilizator are toate privilegiile.

```
CREATE TABLE [<nume_schema>.] <nume_tabel> (  
    <nume_coloana_1> <tip_date> [DEFAULT <expresie>], ...  
    <nume_coloana_n> <tip_date> [DEFAULT <expresie>])  
    [CLUSTER <nume_cluster> (<coloana_1>, ..., <coloana_m>)][ENABLE |  
DISABLE <clause>];
```

- Comanda poate conține opțional clauza **TABLESPACE**, care specifică spațiul tabel în care va fi stocat tabelul.
- De asemenea, poate conține opțional clauza **STORAGE** care este folosită pentru setarea parametrilor de stocare prin intermediul cărora se specifică mărimea și modul de alocare a extinderilor segmentului tabel.

Constrângeri

- Constrângerea este un mecanism care asigură că valorile unei coloane sau ale unei mulțimi de coloane satisfac o **condiție declarată**.
- Unei constrângeri i se poate da un **nume unic** în cadrul schemei.
 - Dacă nu se specifică un nume explicit atunci sistemul automat îi atribuie un nume de forma **SYS_C<n>**, unde **n** reprezintă numărul constrângerii.
- Constrângerile pot fi **șterse**, pot fi **adăugate**, pot fi **activate sau dezactivate**, dar *nu pot fi modificate*.

Constrângeri

- **Constrângeri declarative:** constrângeri de domeniu, constrângerea de integritate a entităţii, constrângerea de integritate referenţială.
- **Constrângerile de domeniu** definesc valori luate de un atribut (DEFAULT, CHECK, UNIQUE, NOT NULL).
- **Constrângerea de integritate a entităţii** precizează cheia primară a unui tabel.
 - Când se creează cheia primară se generează automat un **index unic**.
 - **Valorile cheii primare sunt distincte şi diferite de valoarea *null*.**
- **Constrângerea de integritate referenţială** asigură coerenţa între cheile primare şi cheile externe corespunzătoare.
 - se verifică dacă a fost definită o cheie primară pentru tabelul referit de cheia externă;
 - dacă numărul coloanelor ce compun cheia externă corespunde numărului de coloane ale cheii primare;
 - dacă tipul şi lungimea fiecărei coloane a cheii externe corespunde cu tipul şi lungimea fiecărei coloane a cheii primare.

Constrângeri

- Definițiile și numele constrângerilor definite se pot fi consulta prin interogarea vizualizărilor **USER_CONSTRAINTS** și **ALL_CONSTRAINTS** din dicționarul datelor.
- Există posibilitatea ca o constrângere să fie amânată (**DEFERRABLE**).
 - În acest caz, mai multe comenzi SQL pot fi executate fără a se verifica restricția, aceasta fiind verificată numai la sfârșitul tranzacției, atunci când este executată comanda COMMIT.
 - Dacă vreuna din comenzile tranzacției încalcă restricția, atunci întreaga tranzacție este derulată înapoi și este returnată o eroare. Opțiunea implicită este **NOT DEFERRABLE**.

Modificarea structurii unui tabel

- Comanda care realizează modificarea structurii tabelului (la nivel de coloană sau la nivel de tabel), dar **nu modificarea conținutului acestuia**, este **ALTER TABLE**.
- Comanda **nu schimbă conținutul tabelului**.

Modificarea structurii unui tabel

Comanda **ALTER TABLE** permite:

- adăugarea (**ADD**) de coloane, chei (primare sau externe), constrângeri într-un tabel existent;
- modificarea (**MODIFY**) coloanelor unui tabel, inclusiv specificarea unei valori implicite pentru o coloană existentă;
- suprimarea unei coloane (**DROP COLUMN**);
- adăugarea de constrângeri (**ADD CONSTRAINT**);
- suprimarea (**DROP CONSTRAINT**) cheii primare, a cheii externe sau a altor constrângeri;
- activarea și dezactivarea (**ENABLE, DISABLE**) unor constrângeri;

Modificarea structurii unui tabel

Comanda ALTER TABLE are următoarea sintaxă simplificată:

```
ALTER TABLE [<nume_schema>.] <nume_tabel>  
[ADD(<nume_coloana> <tip_date>, <constrângere>)  
| MODIFY (<nume_coloana_1>, ..., <nume_coloana_n>)  
| DROP      <clauza_drop>,) [ENABLE | DISABLE  
<clause>];
```


Suprimarea unui tabel

- Pentru ștergerea unui tabel este utilizată comanda DROP TABLE:
DROP TABLE [nume_schema.]nume_tabel [CASCADE CONSTRAINTS] ;
- Clauza **CASCADE CONSTRAINTS** permite suprimarea tuturor constrângerilor de integritate referențială corespunzătoare cheilor primare și unice din tabelul supus ștergerii.
 - Altfel, sistemul returnează o eroare și nu suprimă tabelul.
- Suprimarea unui tabel presupune:
 - suprimarea definiției sale în dicționarul datelor;
 - suprimarea indecșilor asociați;
 - suprimarea privilegiilor conferite în legătură cu tabelul;
 - recuperarea spațiului ocupat de tabel;
 - permanentizarea tranzacțiilor în așteptare;
 - **invalidarea** (dar nu suprimarea) funcțiilor, procedurilor, vizualizărilor, secvențelor, sinonimelor referitoare la tabel.

Suprimarea unui tabel

- Odată executată, instrucțiunea DROP TABLE este **ireversibilă**.
- Ca și în cazul celorlalte instrucțiuni ale limbajului de definire a datelor, această comandă nu poate fi anulată (*ROLLBACK*).
- Oracle 10g a introdus o nouă manieră pentru suprimarea unui tabel.
 - Când se șterge un tabel, baza de date nu eliberează imediat spațiul asociat tabelului.
 - Ea redenumeste tabelul și acesta este plasat într-un *recycle bin* de unde poate fi eventual recuperat ulterior prin comanda **FLASHBACK TABLE**.

Suprimarea unui tabel

Exemplu:

```
DROP TABLE exemplu;
```

```
SELECT ...
```

```
INSERT...
```

```
FLASHBACK TABLE exemplu TO BEFORE DROP;
```

- Ștergerea unui tabel se poate face simultan cu eliberarea spațiului asociat tabelului, dacă este utilizată clauza **PURGE** în comanda **DROP TABLE**.
 - Nu este posibil un *rollback* pe o comanda **DROP TABLE** cu clauza **PURGE**.
- Pentru ștergerea întregului conținut al unui tabel și eliberarea spațiului de memorie ocupat de acesta, sistemul Oracle oferă instrucțiunea:

```
TRUNCATE TABLE nume_tabel;
```

Suprimarea unui tabel

TRUNCATE sau DELETE?

- Fiind o instrucțiune LDD, TRUNCATE nu poate fi anulată ulterior (prin ROLLBACK).
- Ea reprezintă o alternativă a comenzii DELETE din limbajul de prelucrare a datelor.
- DELETE nu eliberează spațiul de memorie.
- TRUNCATE este mai rapidă deoarece nu generează informație ROLLBACK și nu activează declanșatorii asociați operației de ștergere.
 - Dacă tabelul este „părintele” unei constrângeri de integritate referențială, el nu poate fi trunchiat.
 - Pentru a putea fi aplicată instrucțiunea TRUNCATE, constrângerea trebuie să fie mai întâi dezactivată.

Informații în DD

- În DD, informațiile despre tabele se găsesc în vizualizarea USER_TABLES. Dintre cele mai importante coloane ale acesteia, se remarcă:

TABLE_NAME	Numele tabelului
TABLESPACE_NAME	Spațiul tabel în care se află tabelul
CLUSTER_NAME	Numele cluster-ului din care face parte tabelul
PCT_FREE	Procentul de spațiu păstrat liber în interiorul fiecărui bloc
PCT_USED	Procentul de spațiu ce poate fi utilizat în fiecare bloc
INI_TRANS	Numărul inițial de tranzacții concurente în interiorul unui bloc
NUM_ROWS	Numărul de înregistrări din tabel
BLOCKS	Numărul de blocuri utilizate de tabel
EMPTY_BLOCKS	Numărul de blocuri ce nu conțin date
TEMPORARY	Y sau N, indică dacă tabelul este temporar (sau nu)

Informatii in DD

- Exemplu:

```
DESCRIBE USER_TABLES;  
SELECT    TABLE_NAME, NUM_ROWS, TEMPORARY  
FROM      USER_TABLES;
```

- Exemplu:

```
SELECT 'DROP TABLE' || OBJECT_NAME || ';'   
FROM USER_OBJECTS  
WHERE OBJECT_TYPE = 'TABLE';
```

Indecși

- Un index este un obiect al schemei unei baze de date care **crește viteza de execuție a cererilor**
- Server-ul Oracle utilizează identificatorul ROWID pentru regăsirea liniilor în structura fizică a bazei de date
- Indexul, din punct de vedere logic, este compus **dintr-o valoare cheie și din identificatorul adresă ROWID**
- Indecșii sunt utilizați și întreținuți automat de către server-ul Oracle. Odată creat indexul, el nu necesită o acțiune directă din partea utilizatorului

Indecși

- Indecșii pot fi creați în două moduri:
 - automat, de server-ul Oracle (**PRIMARY KEY, UNIQUE KEY**);
 - manual, de către utilizator (CREATE INDEX, CREATE TABLE).
- Server-ul Oracle creează automat un **index unic** atunci când se definește o constrângere PRIMARY KEY sau UNIQUE asupra unei coloane sau unui grup de coloane.
- Numele indexului va fi același cu numele constrângerii.
- Indecșii, fiind obiecte ale schemei bazei, beneficiază de procesul de definire a unui obiect.

Indecși

Crearea unui index (care nu este obligatoriu unic) pe una sau mai multe coloane ale unui tabel se face prin comanda:

```
CREATE [UNIQUE] INDEX <nume_index> ON [<nume_schema>.]  
<nume_tabel> (<nume_col> [ASC | DESC], <nume_col> [ASC | DESC], ...)  
| CLUSTER <nume_cluster>;
```

Indecși

Mai mulți indecși asupra unui tabel nu implică întotdeauna interogări mai rapide.

Fiecare operație LMD care este permanentizată asupra unui tabel cu indecși asociați presupune actualizarea indecșilor respectivi

Prin urmare, **este recomandată** crearea de indecși numai în anumite situații:

- coloana conține o varietate mare de valori;
- coloana conține un număr mare de valori *null*;
- una sau mai multe coloane sunt utilizate frecvent împreună într-o clauză WHERE sau într-o condiție de JOIN;
- tabelul este mare și este de așteptat ca majoritatea interogărilor asupra acestuia să regăsească mai puțin de 2-4% din linii;

Indecși

Crearea unui index **nu este recomandată** în următoarele cazuri:

- tabelul este mic;
- coloanele nu sunt utilizate des în cadrul unei condiții dintr-o cerere;
- majoritatea interogărilor regăsesc mai mult de 2-4% din liniile tabelului;
- tabelul **este actualizat frecvent**;
- coloanele indexate sunt referite în expresii;

Indecși

In concluzie, ce tabele sau ce coloane trebuie (sau nu) indexate?

- indexați tabelele pentru care interogările selectează **un număr redus de rânduri** (sub 5%);
- indexați tabelele care sunt interogate folosind **clauze SQL simple**;
- nu indexați tabelele ce conțin puține înregistrări (accesul secvențial este mai simplu);
- nu indexați tabelele care sunt frecvent actualizate, deoarece **ștergerile, inserările și modificările sunt îngreunate de indecși**;
- indexați coloanele folosite frecvent în clauza **WHERE** sau în clauza **ORDER BY**;
- nu indexați coloanele ce conțin date asemănătoare (puține valori distincte);

Indecși

Exemplu:

```
CREATE INDEX upper_num_idx ON artist (UPPER(num));
```

- Indexul creat prin instrucțiunea precedentă facilitează prelucrarea unor interogări precum:

```
SELECT * FROM artist  
WHERE UPPER(num) = 'GRIGORESCU';
```

- Pentru a asigura că server-ul Oracle **utilizează indexul** și nu efectuează o căutare asupra întregului tabel, valoarea funcției corespunzătoare expresiei indexate **trebuie să nu fie null** în interogările ulterioare creării indexului. Următoarea instrucțiune garantează utilizarea indexului dar, în absența clauzei WHERE, serverul Oracle ar putea cerceta întreg tabelul.

```
SELECT * FROM artist  
WHERE UPPER(num) IS NOT NULL  
ORDER BY UPPER(num);
```

Indecși

Exemplu:

Să se creeze tabelul *artist*, specificându-se indexul asociat cheii primare.

```
CREATE TABLE artist (cod_artist NUMBER PRIMARY KEY
                        USING INDEX
                        (CREATE INDEX artist_cod_idx
                         ON artist(cod_artist)
                        ),
                    nume VARCHAR2(30) NOT NULL,
                    prenume VARCHAR2(30),
                    ...);
```

- **Ștergerea** unui index se face prin comanda:

```
DROP INDEX nume_index [ON [nume_schema.] nume_tabel]
```

- Pentru a suprima indexul trebuie ca acesta să se găsească în schema personală sau să avem privilegiul de sistem DROP ANY INDEX.
- Pentru a **reconstrui un index** se pot folosi două metode:
 - se șterge indexul (DROP INDEX) și se recreează (CREATE INDEX);
 - se utilizează comanda ALTER INDEX cu opțiunea REBUILD.

Secvențe

- O secvență este un obiect în baza de date care servește pentru a genera **întregi unici** în sistemele multi-utilizator, evitând apariția conflictelor și a blocării.
- Secvențele sunt memorate și generate **independent de tabele**; aceeași secvență poate fi utilizată pentru mai multe tabele.
- O secvență poate fi creată de un utilizator și poate fi partajată de mai mulți utilizatori.
- Crearea unei secvențe se face cu ajutorul comenzii:

```
CREATE SEQUENCE [<nume_schema>.]<nume_secventa>  
[INCREMENT BY n] [START WITH m] [{MAXVALUE n |  
NOMAXVALUE}] [{MINVALUE n | NOMINVALUE}] [{CACHE k  
| NOCACHE}]
```

- **CACHE k | NOCACHE** specifică numărul de valori alocate de server-ul Oracle pe care le va păstra în memoria *cache* pentru a oferi utilizatorilor un acces rapid (implicit sunt alocate 20 de valori);

Secvențe

- O secvență este referită într-o comandă SQL cu ajutorul pseudo-coloanelor:
 - NEXTVAL – referă valoarea următoare a secvenței;
 - CURRVAL – referă valoarea curentă a secvenței.
- NEXTVAL și CURRVAL pot fi folosite în:
 - clauza VALUES a unei comenzi INSERT;
 - clauza SET a unei comenzi UPDATE;
 - lista SELECT a unei subcereri dintr-o comanda INSERT;
 - lista unei comenzi SELECT.

Secvențe

NEXTVAL și **CURRVAL** **nu pot fi folosite** în:

- subinterogare în SELECT, DELETE sau UPDATE;
- interogarea unei vizualizări;
- comandă SELECT cu operatorul DISTINCT;
- comandă SELECT cu clauza GROUP BY, HAVING sau ORDER BY;
- clauza WHERE a unei comenzi SELECT;
- condiția unei constrângeri CHECK;
- valoarea DEFAULT a unei coloane într-o comandă CREATE TABLE sau ALTER TABLE;
- comandă SELECT care este combinată cu altă comandă SELECT printr-un operator pe mulțimi (UNION, INTERSECT, MINUS).

Secvențe

Din dicționarul datelor pot fi obținute informații despre secvențe folosind vizualizarea USER_SEQUENCES.

Exemplu:

- Să se creeze o secvență *domeniuseq* care să fie utilizată pentru a insera noi domenii în tabelul domeniu și să se insereze un nou domeniu.
- Să se afișeze informațiile referitoare la secvența *domeniuseq*.

```
CREATE SEQUENCE domeniuseq  
START WITH 1  
INCREMENT BY 1;
```

```
INSERT INTO domeniu VALUES  
(domeniuseq.NEXTVAL, 'Informatica');
```

```
SELECT INCREMENT, START, MAXVALUE, MINVALUE  
FROM USER_SEQUENCES  
WHERE SEQUENCE_NAME = 'domeniuseq';
```

Vizualizări

- Vizualizarea (view) este un **tabel logic (virtual)** relativ la date din una sau mai multe tabele sau vizualizări.
- Vizualizarea este definită plecând de la o **cerere** a limbajului de interogare a datelor, moștenind caracteristicile obiectelor la care se referă.
- Vizualizarea, fiind virtuală, **nu solicită o alocare de memorie** pentru date.
- Textul cererii (SELECT) care definește vizualizarea este salvat în DD.
- Oracle transformă cererea referitoare la o vizualizare într-o cerere relativă la tabelele de bază.
 - Nucleul Oracle determină **fuzionarea** cererii relative la vizualizare cu comanda de definire a vizualizării, analizează rezultatul fuziunii în zona partajată și execută cererea.
 - Dacă sunt utilizate clauzele UNION, GROUP BY și CONNECT BY, atunci Oracle nu determină fuzionarea, el va rezolva vizualizarea și apoi va aplica cererea rezultatului obținut.

Vizualizări

- O vizualizare reflectă la orice moment **conținutul** exact al tabelelor de bază.
- Orice modificare efectuată asupra tabelelor se realizează instantaneu și asupra vizualizării.
- Ștergerea unui tabel implică **invalidarea vizualizărilor asociate** tabelului și nu ștergerea acestora.
- Vizualizările sunt definite pentru:
 - furnizarea unui nivel mai înalt de **securizare a bazei** (restricționarea accesului la date);
 - simplificarea formulării unei cereri;
 - mascarea complexității datelor;
 - afișarea datelor într-o altă reprezentare decât cea a tabelelor de bază;
 - asigurarea confidențialității anumitor informații;

Vizualizări

- Vizualizările pot fi **simple** și **complexe**
- O vizualizare **simplă**:
 - extrage date dintr-un singur tabel;
 - nu conține funcții sau grupări de date;
 - asupra ei pot fi efectuate operații LMD;
- O vizualizare este considerată **complexă** dacă:
 - extrage date din mai multe tabele;
 - conține funcții sau grupări de date;
 - nu permite întotdeauna (prin intermediul său) operații LMD asupra tabelelor de bază;

Vizualizări

- Operațiile LMD asupra vizualizărilor complexe sunt restricționate de următoarele **reguli**:
 - nu se poate insera, actualiza sau șterge o linie dintr-o vizualizare dacă aceasta conține **funcții grup, clauza GROUP BY, cuvântul cheie DISTINCT sau pseudocoloana ROWNUM**;
 - nu se poate adăuga sau modifica o linie dintr-o vizualizare, dacă aceasta conține **coloane definite prin expresii**;
- Nu pot fi adăugate linii printr-o vizualizare, dacă tabelul de bază conține coloane care au **constrângerea NOT NULL** și nu apar în **lista SELECT a vizualizării**

Vizualizări

Crearea unei vizualizări se realizează cu ajutorul comenzii:

```
CREATE [OR REPLACE] [FORCE | NOFORCE] VIEW  
[<nume_schema>.]<nume_view> [(<alias>[,<alias>]...)]  
AS <cerere_SELECT>  
[WITH {CHECK OPTION [CONSTRAINT <nume_constrangere>]  
| READ ONLY }];
```

- **OR REPLACE** recreează vizualizarea dacă aceasta deja există.
- **FORCE** creează vizualizarea chiar dacă tabelul de bază nu există sau chiar dacă vizualizarea face referință la obiecte care încă nu sunt create. Deși vizualizarea va fi creată, utilizatorul nu poate să o folosească.
- **NO FORCE** este implicită și se referă la faptul că vizualizarea este creată numai dacă tabelele de bază există.
- Cererea este o comandă SELECT care poate să conțină alias pentru coloane.
- **WITH CHECK OPTION** specifică faptul că reactualizarea datelor din tabele (inserare sau modificare) se poate face numai asupra datelor selectate de vizualizare (care apar în clauza WHERE).
- **WITH READ ONLY** asigură că nici o operație LMD nu poate fi executată asupra vizualizării.

Vizualizări

Reactualizarea tabelelor implică reactualizarea corespunzătoare a vizualizărilor. Reactualizarea vizualizărilor implică reactualizarea tabelelor de bază? NU! Există restricții care trebuie respectate.

- Nu pot fi modificate date din vizualizare sau adaugate date prin vizualizare, dacă aceasta conține coloane definite prin expresii.
- Nu pot fi inserate, șterse sau actualizate date din vizualizări ce conțin:
 - operatorul DISTINCT;
 - clauzele GROUP BY, HAVING, START WITH, CONNECT BY;
 - pseudo-coloana ROWNUM;
 - funcții grup;
 - operatori de mulțimi.
- Nu pot fi inserate sau actualizate date care ar încălca constrângerile din tabellele de bază.

Vizualizări

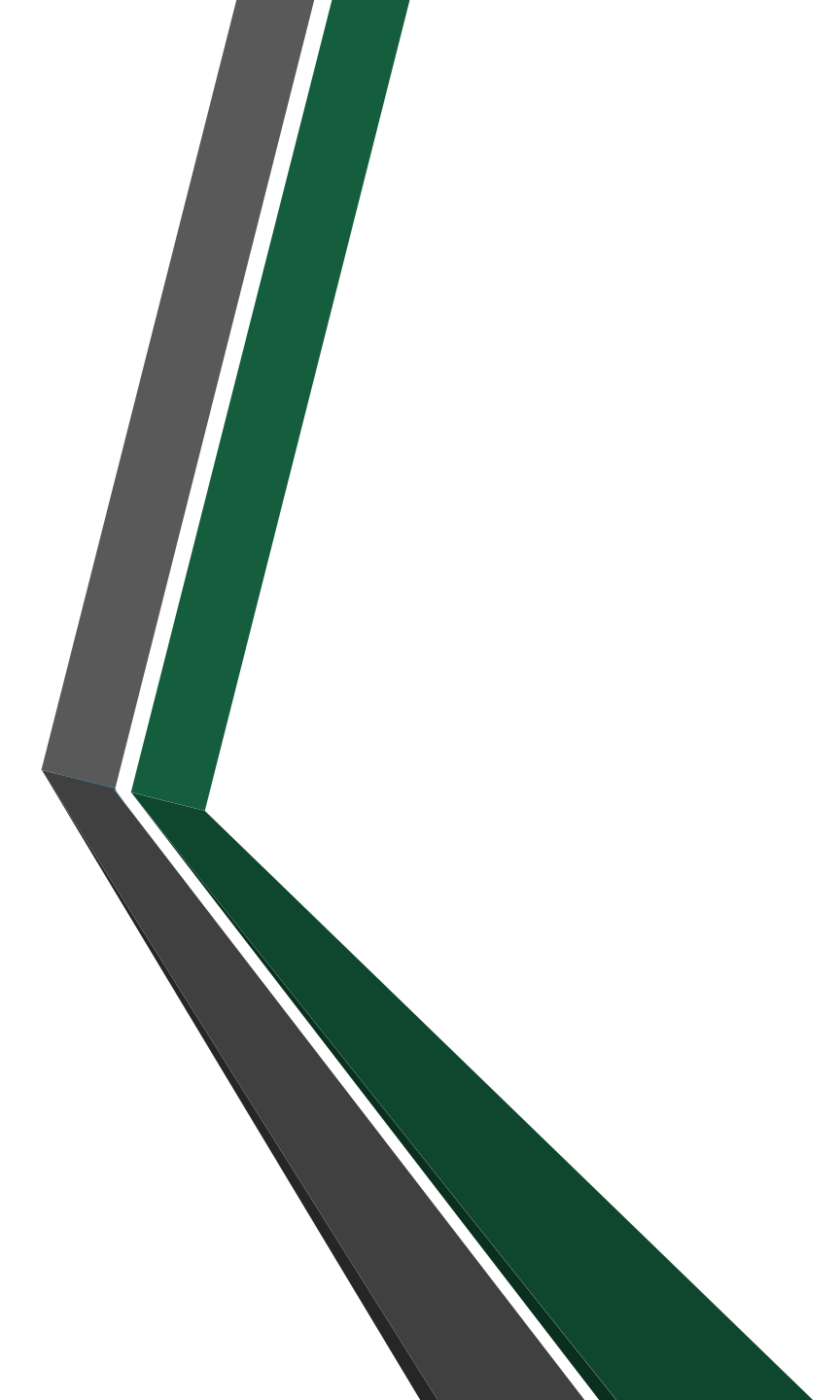
- Nu pot fi inserate sau actualizate valorile coloanelor care rezultă prin calcul
- Nu se pot face operații LMD asupra coloanelor calculate cu DECODE

Alături de restricțiile prezentate anterior, aplicabile tuturor vizualizărilor, există restricții specifice, aplicabile vizualizărilor bazate pe mai multe tabele.

- Regula fundamentală este că orice operație INSERT, UPDATE sau DELETE pe o vizualizare bazată pe mai multe tabele poate modifica datele doar din unul din tabelele de bază.
- Un tabel de bază al unei vizualizări este protejat prin cheie (key preserved table) dacă orice cheie selectată a tabelului este de asemenea și cheie a vizualizării.
 - Deci, un tabel protejat prin cheie este un tabel ale cărui chei se păstrează și la nivel de vizualizare.

Vizualizări

- Asupra unui *join view* pot fi aplicate instrucțiunile INSERT, UPDATE sau DELETE, doar dacă sunt îndeplinite următoarele condiții:
 - instrucțiunea LMD afectează numai unul dintre tabelele de bază;
 - în cazul instrucțiunii **UPDATE**, toate coloanele care pot fi reactualizate trebuie să corespundă coloanelor dintr-un tabel protejat prin cheie (în caz contrar, Oracle nu va putea identifica unic înregistrarea care trebuie reactualizată);
 - în cazul instrucțiunii **DELETE**, rândurile unei vizualizări pot fi șterse numai dacă există un tabel în join protejat prin cheie și numai unul (în caz contrar, Oracle nu ar ști din care tabel să șteargă);
 - în cazul instrucțiunii **INSERT**, toate coloanele în care sunt inserate valori trebuie să provină dintr-un tabel protejat prin cheie.



BAZE DE DATE

CURS 11

Limbajul de prelucrare a datelor

- *SQL* furnizează comenzi ce permit consultarea (*SELECT*) și actualizarea (*INSERT*, *UPDATE*, *DELETE*, *MERGE*) conținutului bazei de date.
- Aceste comenzi definesc limbajul de prelucrare a datelor (*LMD*).
- Comenzile limbajului *LMD* pot fi:
 - formulate direct;
 - utilizate în utilitare ale sistemului *Oracle*;
 - încapsulate într-un program *PL/SQL*;

Comanda *SELECT*

- Una dintre cele mai importante comenzi ale limbajului de prelucrare a datelor este *SELECT*;
- Cu ajutorul ei pot fi extrase submulțimi de valori atât pe verticală (coloane), cât și pe orizontală (linii) din unul sau mai multe tabele;
- Sintaxa comenzii este simplă, apropiată de limbajul natural;

Comanda *SELECT*

SELECT [ALL / DISTINCT] { * / listă de attribute selectate / expr AS alias }

FROM { [schema.]{tabel [PARTITION (partition_name)] /
(subquery)} [alias_tabel] }

[**WHERE** condiție]

[**START WITH** condiție]

[**CONNECT BY** condiție]

[**GROUP BY** listă de expresii [**HAVING** condiție]]

[**ORDER BY** {expresie / poziție / c_alias} [**ASC** / **DESC**]] [**FOR UPDATE**

[**OF** [schema.]{table / view}.coloană] [**NOWAIT**]

Comanda *SELECT*

- Clauzele *START WITH* și *CONNECT BY* sunt utile pentru a construi cereri ierarhizate.
 - *START WITH* -> înregistrarea rădăcină a arborelui
 - *CONNECT BY* -> relația dintre coloanele (*părinte* și *copil*)
 - Prin folosirea operatorului *PRIOR* se poate face referință la înregistrarea *părinte*.

Comanda *SELECT*

Operatorii utilizați sunt:

- operatori aritmetici (unari sau binari),
- operatorul de concatenare (||),
- operatorii de comparare (=, !=, ^=, < >, >, >=, <, <=, *IN* (echivalent cu =ANY, adică egal cu cel puțin una din valorile listei), *NOT IN* (echivalent cu !=ALL, adică diferit de toate elementele listei), *ALL*, [*NOT*] *BETWEEN* x *AND* y, [*NOT*] *EXISTS*, [*NOT*] *LIKE*, *IS* [*NOT*] *NULL*,
- operatori logici (*NOT*, *AND*, *OR*).

Comanda *SELECT*

- Limbajul permite prezența unor instrucțiuni *SELECT* **imbricate** în oricare din clauzele *SELECT*, *WHERE*, *HAVING* sau *FROM*.
- Instrucțiunile *SELECT* care apar în clauzele respective se numesc **subcereri**.
- În cazul folosirii subcererilor, pot fi utilizați:
 - operatorii *ALL*, *ANY*, *IN* (*=ANY*), *EXIST*, *NOT IN* (*!=ANY*), care sunt specifici cererilor ce returnează mai multe linii (*multiple-row subquery*)
 - operatorii de comparare *=*, *<*, *>*, *>=*, *<=*, *<>*, specifici cererilor care returnează o singură linie (*single-row subquery*).

Comanda *SELECT*

Executarea subcererilor se poate face:

- fie **cu sincronizare** (corelat -> evaluarea subcererii face referință la o coloană a cererii principale și cererea interioară se execută pentru fiecare linie a cererii principale care o conține);
- fie **fără sincronizare** (-> se execută mai întâi cererea interioară, iar rezultatul ei este transmis cererii de nivel imediat superior).

Comanda *INSERT*

INSERT INTO *nume_tabel* | *nume_view* [(*col1*[, *col2*[,...]])]

VALUES (*expresia1*[, *expresia2*[,...]]) | *subcerere*;

- *expresia1*, *expresia2* – reprezintă expresii a căror evaluare este atribuită coloanelor precizate (se inserează o linie);
- *subcerere* – reprezintă o interogare (se inserează una sau mai multe linii).

Comanda *INSERT*

- Dacă lipsește specificația coloanelor se consideră că sunt completate toate câmpurile tabelului sau vizualizării.
- Dacă se introduc date doar în anumite coloane, atunci aceste coloane trebuie specificate. În restul coloanelor se introduce automat *null* (daca nu exista *DEFAULT*).
- Specificarea cererii din comanda *INSERT* determină copierea unor date dintr-un tabel în altul pe atâtea linii câte au rezultat din cerere.
- Dacă se introduc numai anumite câmpuri într-o înregistrare, atunci printre acestea trebuie să se găsească câmpurile cheii primare.
- Pentru a putea executa comanda *INSERT* este necesar ca utilizatorul care execută această instrucțiune să aibă privilegiul de a insera înregistrări în tabel sau în vizualizare.

Comanda *DELETE*

DELETE

[*FROM*] *tablename / viewname* [*AS* *alias*] [*WHERE* *condiție*]
[*clauza_returning*]

- Comanda *DELETE* nu șterge **structura** tabelului.
- Pentru a se putea executa instrucțiunea *DELETE*, utilizatorul care o lansează în execuție trebuie să aibă acest privilegiu.
- În clauza *WHERE* pot fi folosite și subcereri.
- Comanda nu poate fi folosită pentru ștergerea valorilor unui câmp individual. Acest lucru se poate realiza cu ajutorul comenzii *UPDATE*.

Comanda *UPDATE*

UPDATE *tablename* | *viewname*

SET (*column1*[,*column2*[,...]]) = (*subquery*) | *column* = *expr* /
(*query*)

[*WHERE* *condition*]

- Pentru a se putea executa instrucțiunea *UPDATE*, utilizatorul care o lansează în execuție trebuie să aibă acest privilegiu.
- Dacă nu este specificată clauza *WHERE* se vor modifica toate liniile.
- Cererea trebuie să furnizeze un număr de valori corespunzător numărului de coloane din paranteza care precede caracterul de egalitate.

LIMBAJUL PENTRU CONTROLUL DATELOR

Controlul unei baze de date cu ajutorul *SQL*-ului se refera la:

- asigurarea confidentialitatii si securitatii datelor;
- organizarea fizica a datelor;
- realizarea unor performante;
- reluarea unor actiuni in cazul unei defectiuni;
- garantarea coerentei datelor in cazul prelucrarii concurente;

LIMBAJUL PENTRU CONTROLUL DATELOR

Sistemul de gestiune trebuie:

- să pună la dispoziția unui număr mare de utilizatori o mulțime **coerentă** de date;
- să garanteze **coerența** datelor în cazul manipulării simultane de către diferiți utilizatori;

LIMBAJUL PENTRU CONTROLUL DATELOR

- Coerența este asigurată cu ajutorul conceptului de **tranzacție**.
- Tranzacția este unitatea logică de lucru constând din una sau mai multe instrucțiuni *SQL*

LIMBAJUL PENTRU CONTROLUL DATELOR

- Limbajul pentru controlul datelor (*LCD*) permite salvarea informației, realizarea fizică a modificărilor în baza de date, rezolvarea unor probleme de concurență.
- Limbajul conține următoarele instrucțiuni:
 - *COMMIT* - folosită pentru permanentizarea modificărilor executate asupra BD (modificările sunt înregistrate și sunt vizibile tuturor utilizatorilor);
 - *ROLLBACK* - folosită pentru refacerea stării anterioare a BD (sunt anulate toate reactualizările efectuate de la începutul tranzacției);
 - *SAVEPOINT* - folosită în conjuncție cu instrucțiunea *ROLLBACK*, pentru definirea unor puncte de salvare în fluxul programului;

LIMBAJUL PENTRU CONTROLUL DATELOR

O **tranzacție** constă:

- dintr-o singură instrucțiune *LDD*;
- dintr-o singură instrucțiune *LCD*;
- din instrucțiuni *LMD* care fac schimbări consistente în date;

LIMBAJUL PENTRU CONTROLUL DATELOR

- Tranzacția **începe**:
 - după o comandă *COMMIT*;
 - după o comandă *ROLLBACK*;
 - după conectarea inițială la Oracle;
 - când este executată prima instrucțiune *SQL*;
- Tranzacția se **termină**:
 - dacă sistemul nu mai functioneaza;
 - dacă utilizatorul se deconectează;
 - dacă se dau comenzile *COMMIT* sau *ROLLBACK* ;
 - dacă se execută o comandă *LDD*;
- După ce se termină o tranzacție, prima instrucțiune *SQL* executabilă va genera automat începutul unei noi tranzacții.

LIMBAJUL PENTRU CONTROLUL DATELOR

- Din momentul în care s-a executat instrucțiunea **COMMIT**, BD s-a modificat (permanent) în conformitate cu instrucțiunile SQL executate în cadrul tranzacției care tocmai s-a terminat. Din acest punct începe o nouă tranzacție.
- Dacă se folosește utilitarul *SQL*Plus*, există posibilitatea ca după fiecare comandă *LMD* să aibă loc o permanentizare automată a datelor (un *COMMIT* implicit). Acest lucru se poate realiza folosind comanda:

SET AUTO[COMMIT] {ON | OFF}

- Comanda **ROLLBACK** permite restaurarea unei stări anterioare a BD.

ROLLBACK [TO [SAVEPOINT] savepoint];

LIMBAJUL PENTRU CONTROLUL DATELOR

- Dacă nu se specifică nici un *savepoint*, toate modificările făcute în tranzacția curentă sunt anulate;
- Dacă se specifică un anumit *savepoint*, atunci doar modificările de la acel *savepoint* până în momentul respectiv sunt anulate;
- Executarea unei instrucțiuni *ROLLBACK* presupune terminarea tranzacției curente și începerea unei noi tranzacții;

LIMBAJUL PENTRU CONTROLUL DATELOR

- Punctele de salvare pot fi considerate ca niște etichete care referă o submulțime a schimbărilor dintr-o tranzacție, marcând efectiv un punct de salvare pentru tranzacția curentă. Punctele de salvare **NU sunt obiecte ale schemei**. Prin urmare, nu sunt referite în DD.
- *Server-ul Oracle* implementează **un punct de salvare implicit** pe care îl mută automat după ultima comandă *LMD* executată.
- Dacă este creat un punct de salvare având același nume cu unul creat anterior, cel definit anterior este șters automat.

SAVEPOINT savepoint;

LIMBAJUL PENTRU CONTROLUL DATELOR

Starea datelor înainte de *COMMIT* sau *ROLLBACK* este următoarea:

- starea anterioară a datelor poate fi recuperată;
- utilizatorul curent poate vizualiza rezultatele operațiilor *LMD* prin interogări asupra tabelelor;
- alți utilizatori nu pot vizualiza rezultatele comenzilor *LMD* făcute de utilizatorul curent (***read consistency***);
- înregistrările (liniile) afectate sunt blocate și, prin urmare, alți utilizatori nu pot face schimbări în datele acestor înregistrări.

LIMBAJUL PENTRU CONTROLUL DATELOR

Execuția unei comenzi *COMMIT* implică anumite modificări.

- Toate schimbările (*INSERT*, *DELETE*, *UPDATE*) din baza de date făcute după anterioara comandă *COMMIT* sau *ROLLBACK* sunt **definitive**. Comanda se referă numai la schimbările făcute de utilizatorul care dă comanda *COMMIT*.
- Toate punctele de salvare vor fi șterse.
- Starea anterioară a datelor este pierdută definitiv.
- Toți utilizatorii pot vizualiza rezultatele.
- Blocările asupra liniilor afectate sunt eliberate; liniile pot fi folosite de alți utilizatori pentru a face schimbări în date.

LIMBAJUL PENTRU CONTROLUL DATELOR

Execuția unei comenzi *ROLLBACK* implică anumite modificări.

- **Anulează** tranzacția în curs și toate modificările de date făcute după ultima comandă *COMMIT*.
- Sunt **eliberate** blocările liniilor implicate.